MAANG SPEC-DRIVEN ENGINEERING & AI UNIFIED PROCESS (BILINGUAL)

Aptala Anlatır Gibi AI Çağında Use Case

Writing Use Cases for AI: How to Write for Stakeholders, Engineers, and AI Agents

10 Bölüm • 60 Ders • Mimari Şemalar • Simon Martinelli / Ivar Jacobson / Alistair Cockburn Temelleri • Java 25 LTS

🔄 The AI Inversion

Kod ucuzladı, niyet kıymete bindi. Vibe Coding çöküşü ve Spec-Driven Development

🎯 The Three Readers

Paydaş doğrular, Mühendis inceler, Yapay Zekâ doğrudan kod ve test üretir

🛡️ 3 Zones & Blocklist

Gözlemlenebilir davranış net, mekanizma sessiz. Test-Protected dilimleme mimarisi

Yayın Sürümü: 1.0.0 • AI Unified Process • Spec-Driven Development • 2026

Önsöz ve Başlarken / Preface & Getting Started

🇹🇷 Bu Eğitim Hakkında

Bu kitap; Simon Martinelli'nin **"Writing Use Cases for AI: How to write use cases for stakeholders, engineers, and AI agents"** kitabındaki devrim niteliğindeki ilkeler (AI Unified Process / Spec-Driven Development) ve MAANG seviyesi sistem mimarisi mühendisliği temel alınarak hazırlanmıştır.

Üretken yapay zekâ yazılım dünyasının ekonomisini tersine çevirdi (The Inversion): Artık kod yazmak ucuz, hızlı ve anında tekrarlanabilir hale geldi. Asıl kıt ve paha biçilemez olan şey **ifade edilmiş niyettir (expressed intent)**. Bir use case şartnamesi yalnızca insan paydaşın doğrulayacağı veya yazılımcının kodlayacağı bir belge değildir; yapay zekâ ajanının doğrudan çalışan kod ve testler ürettiği yaşayan bir **Sözleşmedir (Contract)**.

60 derslik bu kapsamlı müfredatta; Ivar Jacobson ve Alistair Cockburn'ün 40 yıllık klasik temellerinden 2026'nın AIUP ilkelerine (Üç Okuyucu, 3 Bölge, Kara Liste, Dilimleme ve Taban Çizgisi Kurtarma) uzanan yolculuk, **bir aptala anlatır gibi** analojiler, zengin ASCII süreç şemaları ve modern **Java 25 LTS** mimarisiyle sunulmaktadır.

🇬🇧 About This Masterclass

This masterclass is founded upon Simon Martinelli's seminal work **"Writing Use Cases for AI: How to write use cases for stakeholders, engineers, and AI agents"** within the AI Unified Process (AIUP) and Spec-Driven Development paradigm.

Generative AI has inverted software economics: implementation is now cheap, fast, and repeatable. The scarce resource is precisely expressed intent. In this new era, a use case serves three distinct readers: the stakeholder who validates it, the software engineer who reviews it, and the tireless, literal AI agent that implements code directly from it.

Across 60 structured lessons organized into 10 modules, we demystify the complete specification discipline —from sea-level goals and the 24-point audit checklist to the Zone 3 blocklist (banning SQL, JWT, regex in specs) and brownfield recovery—using intuitive Explain-Like-I'm-5 (ELI5) metaphors, architectural ASCII schematics, and clean Java 25 LTS reference implementations.

İçindekiler / Table of Contents

🇹🇷 Bu Eğitim Hakkında 2
🇬🇧 About This Masterclass 2

BÖLÜM 1 / MODULE 1

Büyük Tersine Dönüş (The Inversion) & AI Çağında Use Case Rönesansı

The AI Inversion & Why Use Cases Rule the GenAI Era

Yazılım mühendisliği tarihinin en büyük ekonomik kırılması: Kodlama ucuzladı, niyet (intent) paha biçilmez oldu. Vibe coding'in çöküşü, determinizm yanılgısı, 1987'den günümüze Use Case evrimi ve Spec-Driven Development (SDD) paradigması.

The greatest economic shift in software history: Implementation is cheap, intent is scarce. The failure of vibe coding, the determinism fallacy, the evolution of use cases from 1987 to 2026, and the Spec-Driven Development paradigm.

1

BÖLÜM: BÜYÜK TERSINE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Büyük Tersine Dönüş (The Inversion): Kod Ucuzladı, İfade Edilmiş Niyet Kıymete Bindi

The Great Inversion: Implementation is Cheap, Intent is Scarce

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────────┐
│                YAZILIM EKONOMİSİNDE BÜYÜK KIRILMA (THE INVERSION)                │
└──────────────────────────────────────────────────────────────────┘

  1. GELENEKSEL DÜNYA (1970 - 2022):
     [ Niyet / Spesifikasyon: UCUZ LAF ] ──► [ Kodlama: ÇOK PAHALI, YAVAŞ, KIT ]
     * Geliştiriciler koridor sohbetlerinde boşlukları sezgileriyle doldururdu.
     * "Kod tek gerçektir" (Code is the truth) demek bir sinizm değil, ekonomik zorunluluktu.

  2. ÜRETKEN YAPAY ZEKA ÇAĞI (2023 - 2026+):
     [ Niyet / Spesifikasyon: KIT & PAHA BİÇİLMEZ ] ──► [ Kodlama: UCUZ, IŞIK HIZINDA ]
     * Kod artık bir kaynak değil; derleyici çıktısı gibi üretilip atılabilen bir ara üründür!
     * AI her niyet boşluğunu kendi uydurduğu (halüsinasyon) sahte bir özellikle doldurur!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eskiden bir heykeltıraşa mermerden heykel yaptırmak aylar sürerdi ve çok pahalıydı; ona kabataslak 'Güzel bir at yap' desen bile adam oturup günlerce yontar, eksikleri zanaatiyle tamamlardı. Bugün ise elinde saniyede 10.000 heykel basabilen sihirli bir 3D yazıcı (Yapay Zekâ) var! Ama yazıcıya sadece 'at yap' dersen, sana kanatlı, 6 bacaklı, yeşil bir canavar basar ve 'Sen tam olarak ne istediğini söylemedin ki!' der. Artık heykel yontmak bedava; paha biçilmez olan şey heykelin milimetrik çizimidir!

🇬🇧 ENGLISH (ELI5)

Historically, commissioning a stone sculptor was exorbitant and slow. If you gave vague instructions like 'carve a horse', the master craftsman filled the gaps with personal aesthetic judgment. Today, generative AI is a hyper-speed 3D printer that manufactures statues in 3 seconds for pennies. But if you lack precise blueprints, the printer invents a 6-legged neon unicorn and presents it with total confidence. Carving stone is now free; the rare, precious asset is the architectural blueprint of your intent.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Simon Martinelli'nin 'Writing Use Cases for AI' kitabında ortaya koyduğu 'The Inversion' tezi: Yazılım tarihi boyunca kod yazmak (implementation) pahalı, şartname hazırlamak (specification) ucuz bir formaliteydi. Ekipler eksik şartname yazmayı göze alabiliyordu çünkü kıdemli mühendisler eksik iş kurallarını kahve molalarında paydaşlarla konuşup sezgileriyle kapatıyordu. LLM'lerin (Claude Code, Cursor, Antigravity) gelmesiyle denklem tersyüz oldu. Yapay zekâ ajanı belirsiz bir prompt aldığında soru sormaz; boşluğu en olası gördüğü varsayımla sessizce doldurur. Sonuç: İlk bakışta çalışan ama asla sürdürülemeyen, yeniden üretilemeyen (regenerate edilemeyen) 'vibe coding spagettisi'. Çözüm: Şartnameyi birincil varlık (asset), kodu ise derlenebilir bir çıktı (output) kabul etmektir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The core thesis of 'The Inversion': Throughout software engineering history, implementation was bottlenecked by expensive human labor, while specifications were treated as disposable scaffolding. Teams could afford under-specification because human engineers exercised contextual judgment during hallway discussions. Generative AI fundamentally inverts this economic equation: code generation is near-zero marginal cost, deterministic, and instantaneous. What is scarce is expressed, testable intent. When prompted with ambiguous requirements, an AI agent does not pause to ask for clarification; it hallucinates a plausible feature set that compiles but diverges from business reality.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Niyetin Açıkça Belirtildiği Use Case Arayüzü (Contract First)
package com.maang.aiup.banking;

public class LoanApplicationUseCase {
    // Niyet: Tam ve kesin girdi modeli (Records ile değişmezlik)
    public record ApplyLoanCommand(String memberId, double amountUsd, int durationMonths) {
        public ApplyLoanCommand {
            if (amountUsd <= 0) throw new IllegalArgumentException("Kredi tutarı pozitif olmalıdır.");
            if (durationMonths < 1 || durationMonths > 60) throw new IllegalArgumentException("Vade
1-60 ay arasında olmalıdır.");
        }
    }

    public record LoanDecision(boolean approved, String loanId, double interestRate, String
rejectionReason) {}

    // Sistem Sözleşmesi: AI'ın kafasına göre kural uydurmasını engelleyen kesin metod
    public interface LoanService {
        LoanDecision applyForLoan(ApplyLoanCommand command);
    }

    void main() {
        var cmd = new ApplyLoanCommand("MEM-8921", 50000.0, 24);
        System.out.println("Niyet açıkça modellendi: " + cmd);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Vibe coding tuzağı: Prompt ile hızlıca ayağa kaldırılan kodlar ilk 2 gün harika görünür; ancak 3. hafta bir iş kuralı değiştiğinde kodun neden öyle yazıldığını kimse bilemez ve yeniden üretilemez. Şartnamesiz üretilen kod teknik borç değil, teknik iflastır!

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

The Vibe Coding Trap: Generating software from loose prompts produces code that works on day one but is unmaintainable by day ten. Without a versioned specification, the business intent lives only in ephemeral chat history, rendering automated regeneration impossible.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Kodlama artık ucuz ve tekrarlanabilir bir emtiadır; asıl kıymetli varlık açıkça ifade edilmiş niyettir.
- AI belirsizlik gördüğünde durup soru sormaz; boşluğu uydurma (halüsinasyon) özelliklerle doldurur.
- AI Unified Process'te şartname ana varlıktır, kod ise onun derlenebilir bir çıktısıdır.

🎯 Key Architectural Takeaways

- Implementation is now a cheap commodity; precisely expressed intent is the primary asset.
- AI never pauses at ambiguities; it fills gaps with plausible hallucinations.
- In AIUP, the specification is the asset, and code is merely its generated output.

2

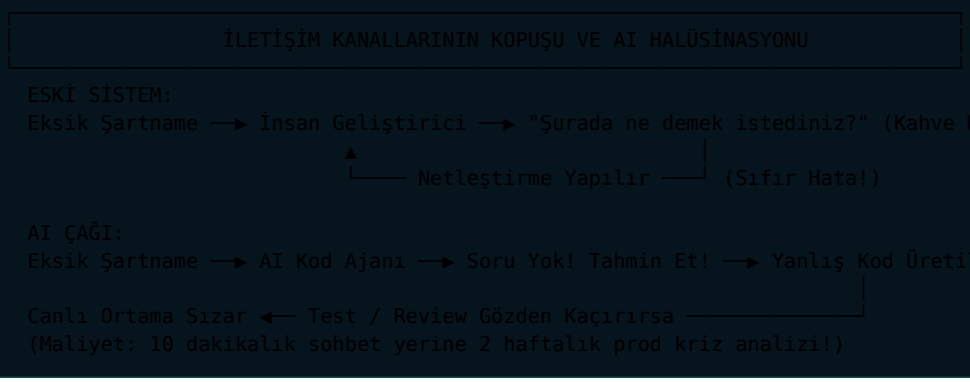
BÖLÜM: BÜYÜK TERSİNE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Koridor Sohbeti Öldü: İletişim Boşluklarının ve AI Halüsinasyonunun Gerçek Maliyeti

The Hallway Conversation is Dead: The True Cost of Code Hallucination

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski iş yerlerinde stajyer bir çocuğa 'Müşterilere indirim yap' dediğinde, çocuk masana gelip 'Abi kime yüzde kaç indirim yapacağız?' diye sorardı. Yapay zekâ ise soru sormayan, aşırı özgüvenli bir robottur. Ona 'indirim yap' dersen, kafasına göre şirketin en pahalı ürününü herkese %90 indirimle satar ve sana 'İndirim yaptım, harika oldu!' diye gülümser. Soru soracak insan kalmadığı için kuralları en baştan taşa kazımak zorundasın!

🇬🇧 ENGLISH (ELI5)

In a traditional office, if you told an apprentice engineer to 'discount the shopping cart', they would walk to your desk and clarify the percentage rules. An AI coding agent is a hyper-confident worker that never asks questions. Tell it to 'discount carts', and it might quietly slash prices by 90% across the entire inventory, presenting the pull request with a cheerful smile. Because hallway questions are gone, precision must be baked into the spec upfront.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Geleneksel yazılım süreçlerinde 'Eksik Belgeleme' (Under-specification) bir felaket değildi çünkü organizasyonun içinde 'Sosyal Netleştirme Ağı' (Social Disambiguation Network) vardı: Slack mesajları, standup toplantıları ve koridor sohbetleri. Bir junior mühendis şartnamede bir çelişki gördüğünde işi durdurup sorardı; bu soru ücretsiz bir 'belirsizlik dedektörü' görevi görürdü. AI ajanları (Claude, Copilot, Antigravity) ise bu dedektörü tamamen yok etti. Bir ajan, boş bırakılan her alanı istatistiksel ağırlıklarına göre en mantıklı gelen kod parçasıyla tamamlar. Martinelli'nin belirttiği gibi: Eskiden belirsizliğin maliyeti 5 dakikalık bir sohbetti; bugün ise canlıya çıkan ve haftalarca fark edilmeyen sessiz veri bozulmalarıdır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

In classical workflows, under-specification was buffered by organizational social networks: hallway chats, quick Slack pings, and standup debates. An observant human developer served as a free ambiguity detector: encountering an unhandled edge case, they halted execution and sought clarification. LLM-based agents eradicate this feedback loop. They suffer from zero hesitation, synthesizing plausible implementations across unstated boundaries. The cost of ambiguity has migrated from a 5-minute conversation to production data corruption that escapes unnoticed until an audit.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Boşluk Bırakmayan Domain Doğrulama Mekanizması
package com.maang.aiup.discount;

import java.math.BigDecimal;

public class DiscountPolicy {
    // İş Kuralı BR-001: İndirim sadece VIP üyelere ve maksimum %20 uygulanabilir.
    public record DiscountRequest(String customerTier, BigDecimal cartTotal, BigDecimal
requestedDiscountPercent) {}

    public record ValidatedDiscount(BigDecimal finalDiscountAmount, String appliedRule) {}

    // AI'ın kafasına göre indirim uydurmasını engelleyen deterministik kural motoru
    public static ValidatedDiscount calculate(DiscountRequest req) {
        if (!"VIP".equalsIgnoreCase(req.customerTier())) {
            return new ValidatedDiscount(BigDecimal.ZERO, "BR-001: Standart üyelere indirim
uygulanamaz.");
        }
        // AI'ın %90 yapmasını engelleyen sınır
        BigDecimal maxAllowed = new BigDecimal("20.00");
        BigDecimal effectivePercent = req.requestedDiscountPercent().min(maxAllowed);
        BigDecimal discount = req.cartTotal().multiply(effectivePercent).divide(new
BigDecimal("100"));

        return new ValidatedDiscount(discount, "BR-001: VIP indirimi başarıyla hesaplandı.");
    }

    void main() {
        var req = new DiscountRequest("VIP", new BigDecimal("1000.00"), new BigDecimal("35.00"));
        var result = calculate(req);
        System.out.println("Sonuç: " + result);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

AI ile çalışırken 'Sistem kullanıcıyı uygun şekilde bilgilendirir' veya 'Gereken indirim yapılır' gibi yuvarlak cümleler kurmayın! 'Uygun şekilde' kelimesi, sistemin mimarisini ve kurallarını tamamen yapay zekanın hayal gücüne terk etmek demektir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never write vague requirements like 'system notifies user appropriately' or 'applies a reasonable discount'. The phrase 'appropriately' delegates total architectural authority to an algorithm that cannot feel business consequences.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- İnsan geliştiricilerin sorduğu netleştirme soruları ücretsiz bir belirsizlik dedektörüydü; AI bu dedektörü yok etti.
- Belirsizlik artık bir soru olarak geri dönmez; yanlış ve sessizce çalışan bir kod olarak canlıya sızar.
- Tüm iş kuralları ve limitler şartnamede sayısal ve net olarak tanımlanmalıdır.

🎯 Key Architectural Takeaways

- Human clarifying questions were free ambiguity detectors; AI agents completely eliminate them.
- Ambiguity no longer surfaces as a question; it manifests as silent production bugs.
- All business invariants and limits must be specified with numerical precision in the spec.

3

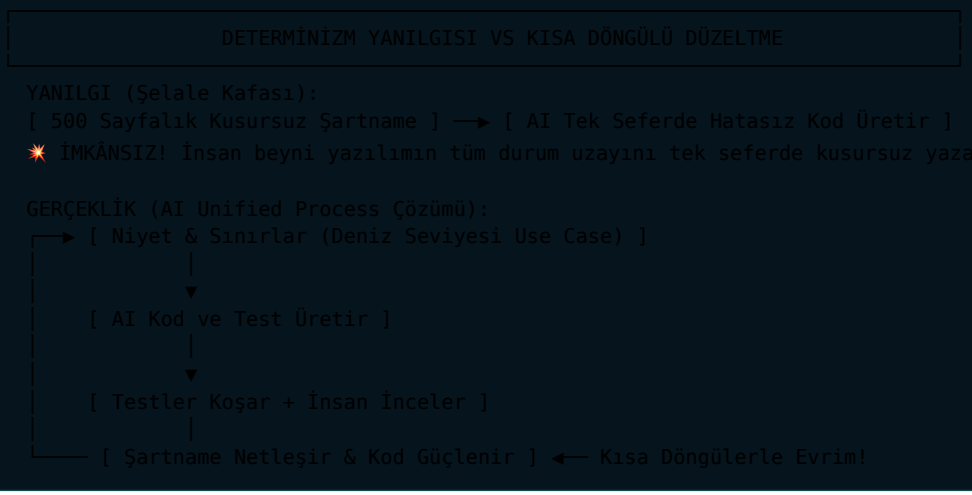
BÖLÜM: BÜYÜK TERSINE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Determinizm Yanılgısı: Mükemmel Şartname Diye Bir Şey Yoktur ve Gerek de Yoktur!

The Determinism Fallacy: Why Perfect Specs Don't Exist and Aren't Needed

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bazı insanlar 'Yapay zekanın doğru kod yazması için 10.000 sayfalık kusursuz bir kural kitabı yazmalıyız, bu da imkansız, demek ki AI bir işe yaramaz!' der. Bu şuna benzer: 'Bir şoföre gideceği yeri anlatmak için yoldaki her taşın yerini tarif edemem, o halde taksiye binmeyeyim!' Hayır! Şoföre sadece 'Kadıköy rıhtıma git, sahilden sür' dersin (sınırlar). Yanlış sokağa girerse 'Sağa dön' dersin (kısa döngülü test). Mükemmel tarif gerekmez; net hedef ve anında düzeltme yeterlidir!

🇬🇧 ENGLISH (ELI5)

Skeptics argue: 'AI generation only works if the spec is 100% mathematically complete to force deterministic code. Perfect specs are impossible, therefore AI coding is a failure.' This is like refusing to take a taxi because you cannot describe the molecular coordinates of every pebble on the highway. You don't need a perfect spec; you need clear boundary constraints and rapid automated tests that catch deviations within seconds.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Determinizm Yanılgısı (The Determinism Fallacy): Birçok mühendis, AI güdümlü yazılım geliştirmenin ancak 'tamamen deterministik ve sıfır boşluklu' devasa spesifikasyonlarla mümkün olabileceğini savunarak yöntemi baştan reddeder. Cockburn ve Martinelli bu tezi çürütür: 1) İlk taslak değerin %50'sini üretir; kelimeler üzerinde sonsuza kadar oynamak iletişimi artırmaz. 2) Amaç deterministik bir robot yaratmak değil, 'her okuyucunun (insan paydaş, mühendis ve AI) aynı niyeti anladığı' bir zemin kurmaktır. 3) Kalite, tek seferlik kusursuz yazımda değil; Şartname -> Üretim -> Test Doğrulaması -> Paydaş İncelemesi döngüsünün dakikalar içinde tekrarlanabilmesinde saklıdır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Determinism Fallacy deconstructed: The objection states that generative AI requires airtight, mathematically exhaustive specifications to prevent non-deterministic code branching. Because perfect specifications are impossible in human language, the approach is deemed unviable. Martinelli refutes this: the AI Unified Process never demands exhaustive determinism. It requires precision strictly about observable behavior, backed by immutable test assertions. Quality is achieved not through monolithic upfront perfection, but through hyper-short feedback loops: specify intent, generate, verify with unit/slice tests, refine.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kısa Döngülü Test Doğrulaması ile Niyet Denetimi (Test-Protected)
package com.maang.aiup.verification;

public class OrderValidationEngine {
    public record OrderItem(String sku, int quantity, double unitPrice) {}
    public record Order(String orderId, java.util.List<OrderItem> items) {}

    // Kural BR-010: Bir siparişte en fazla 10 adet ürün olabilir ve toplam tutar pozitif olmalıdır.
    public static boolean validateOrder(Order order) {
        if (order.items() == null || order.items().isEmpty()) return false;
        int totalQty = order.items().stream().mapToInt(OrderItem::quantity).sum();
        if (totalQty > 10) return false;
        double total = order.items().stream().mapToDouble(i -> i.quantity() * i.unitPrice()).sum();
        return total > 0.0;
    }

    void main() {
        // Hızlı doğrulama testi (Determinizmi koruyan emniyet kemeri)
        var validOrder = new Order("ORD-1", java.util.List.of(new OrderItem("BOOK-1", 2, 25.0)));
        var invalidOrder = new Order("ORD-2", java.util.List.of(new OrderItem("BOOK-2", 15, 10.0)));

        assert validateOrder(validOrder) : "Geçerli sipariş onaylanmalıydı!";
        assert !validateOrder(invalidOrder) : "10 adetten fazla ürün reddedilmeliydi!";

        System.out.println("✅ Determinizm testlerle koruma altına alındı!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnameyi kusursuzlaştırmak için haftalarca toplantı yapıp tek satır kod üretmemek (Analysis Paralysis) şelale modeline geri dönüştür. İlk gün deniz seviyesi taslağı çıkarın, AI'a ürettirin ve test sonuçlarına bakarak şartnameyi rafine edin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Spending weeks polishing a specification without running an AI generation pass is a regression into the waterfall trap. Draft the sea-level spec in hours, run `/implement`, inspect the generated test suite, and iterate.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Mükemmel şartname imkânsızdır ve gerekli de değildir; önemli olan şey gözlemlenebilir davranışın netliğidir.
- Determinizm tek seferlik yazımla değil, testlerle korunan kısa çevrimli geri bildirim döngüleriyle sağlanır.
- Şartname niyetin sınırlarını çizer, AI bu sınırlar içinde kodu doldurur, testler ise sınır ihlalini yakalar.

🎯 Key Architectural Takeaways

- Perfect specifications are impossible and unnecessary; observable clarity is what matters.
- Determinism is enforced by automated test harnesses in rapid iterative loops.
- The spec sets the boundaries, AI implements inside them, and tests catch boundary violations.

4

BÖLÜM: BÜYÜK TERSINE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Use Case'in 40 Yıllık Evrimi: Ericsson (1987), Cockburn (2001) ve Use-Case 3.0 (2024)

The Evolution of Use Cases: From Ericsson (1987) to Cockburn (2001) to Use-Case 3.0

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                    USE CASE'İN 40 YILLIK BÜYÜK YOLCULUĞU

1987: Ivar Jacobson (Ericsson)
└──► OOPSLA bildirisi: Telefon santralleri için kullanıcı odaklı senaryolar.
     "Sistem ne yapar?" sorusunun ilk yapısal cevabı.

2001: Alistair Cockburn ("Writing Effective Use Cases")
└──► Yazma sanatı: Hedef Seviyeleri (Deniz/Bulut/Balık), Paydaş Çıkarları,
     Ana Akış ve Alternatif Akış disiplini.

2000-2020: Çevik Yozlaşma (User Stories İstismarı)
└──► Post-it çılgınlığı! Parçalanmış, bağlamsız, uç durumları unutan yapışkan kağıtlar.

2024: Jacobson & Cockburn ("Use-Case 3.0")
└──► Dilimleme (Slices), Çevik ve Hafif Entegrasyon: "Use Cases are Essential!"

2026+: AI Unified Process & Simon Martinelli
└──► Ajanlar için yeniden doğuş: AI'ın anlayabileceği tek tam davranışsal kontrat!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

1980'lerde icat edilen çekiç ve tekerlek gibi bazı aletler o kadar mükemmeldir ki teknoloji ne kadar değişirse değişsin eskimez. 1987'de telefon santralleri için icat edilen 'Use Case', 2000'lerde 'Biz artık çeviğiz, post-it yapıştırıyoruz' denilerek çöpe atılmak istendi. Ama yapay zekâ gelince görüldü ki, mini minnacık post-it notlarıyla koca sistemleri AI'a anlatamazsın! 40 yıllık Use Case, yapay zekanın en iyi anlayabildiği süper modern bir mimari haritaya dönüştü!

🇬🇧 ENGLISH (ELI5)

Fundamental inventions like the wheel or the hammer never become obsolete. Invented in 1987 by Ivar Jacobson at Ericsson and formalized by Alistair Cockburn in 2001, use cases were temporarily sidelined by agile teams wielding single-sentence sticky notes. But generative AI cannot build complex systems from disconnected sticky notes. Today, Use-Case 3.0 and the AI Unified Process have restored use cases as the premier behavioral contract for autonomous agents.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Use Case'in tarihsel kırılma noktaları: 1) 1987 Jacobson OOPSLA: Nesne yönelimli programlamanın (OOP) içine 'davranışsal hikaye' kavramını soktu; Rational Unified Process'in (RUP) üç temel direğinden biri oldu (Use-Case Driven). 2) 2001 Cockburn: Şablon çılgınlığını sadeleştirdi; Deniz Seviyesi (Sea Level) hedefleri ve başarısızlık durumlarını garanti altına alan sözleşme (contract) modelini oturttu. 3) 2023-2024 ACM Queue & Use-Case 3.0: Jacobson ve Cockburn 'Use Cases are Essential' makalesiyle kullanım senaryolarının çevik dilimlerle (slices) Scrum ve Kanban'a entegre olabileceğini kanıtladı. 4) 2026 AIUP: Martinelli, üçüncü okuyucunun (AI Ajanı) gelişiyle use case'in kod ve test üreten birincil generatif varlık olduğunu ilan etti.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The 40-year architectural lineage: 1) 1987 Jacobson (Ericsson): Introduced use cases at OOPSLA to model telecommunication switching flows, establishing the core axiom: software must be driven by user goals. 2) 2001 Cockburn: Standardized the craft in 'Writing Effective Use Cases', defining goal altitude tiers (Cloud, Sea, Fish), stakeholder guarantees, and the 3-kind step hierarchy. 3) 2024 Use-Case 3.0: Reconciled use cases with lightweight agile delivery via 'use-case slices'—modular delivery units threading through the basic flow into edge paths. 4) 2026 AIUP (Martinelli): Positioned use cases as the central generative hub connecting system vision to autonomous AI coding agents.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Use-Case 3.0 Dilimleme (Slice) Mantığının Kodda Karşılığı
package com.maang.aiup.slices;

public class BookBorrowingSlice {
    // Dilim 1 (Slice 1): Sadece Temel Başarı Senaryosu (Happy Path)
    public record BorrowBasicSlice(String memberId, String bookIsbn) {
        public void execute() {
            System.out.println("Slice 1 Çalıştı: Kitap ödünç verildi, teslim tarihi 28 gün sonrası.");
        }
    }

    // Dilim 2 (Slice 2): Alternatif Akış Eklenmiş Gelişmiş Dilim
    public record BorrowWithLimitCheckSlice(String memberId, String bookIsbn, int activeLoanCount) {
        public boolean execute() {
            if (activeLoanCount >= 5) {
                System.out.println("Slice 2 (A2 Flow): Limit aşıldı! Ödünç engellendi.");
                return false;
            }
            System.out.println("Slice 2: Kontrol başarılı, kitap verildi.");
            return true;
        }
    }

    void main() {
        new BorrowBasicSlice("M101", "ISBN-9988").execute();
        new BorrowWithLimitCheckSlice("M101", "ISBN-9988", 5).execute();
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Geçmişte RUP döneminde yazılan 100 sayfalık bürokratik use case dökümanlarıyla modern Use-Case 3.0'ı karıştırmayın! Modern use case hafiftir, 3-9 adımlı senaryolardan oluşur ve doğrudan dilimler (slices) halinde kodlanır.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not conflate modern AIUP use cases with 1990s RUP bureaucratic monoliths. Modern use cases are lean, bounded to 3-9 steps, sliced iteratively, and engineered for direct machine parsing.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Use case 1987'de doğmuş, 2001'de Cockburn ile olgunlaşmış ve 2024'te Use-Case 3.0 ile çevikleşmiştir.
- Kullanıcı hikayeleri konuşma başlatır; Use Case ise varılan kararların ve kuralların yaşadığı yerdir.
- AI çağında use case, kodlama ajanlarına niyet aktarabilen tek eksiksiz davranış formatıdır.

🎯 Key Architectural Takeaways

- Use cases originated in 1987, matured via Cockburn in 2001, and became agile in 2024.
- User stories initiate conversations; use cases store the architectural decisions.
- In the AI era, use cases are the only established format that conveys complete behavioral context.

5

BÖLÜM: BÜYÜK TERSINE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Kullanıcı Hikayeleri vs Use Case: Neden Yapışkan Post-it Kağıtları LLM Çağında İflas Etti?

User Stories vs Use Cases: Why Agile Sticky Notes Broke in the Age of LLMs

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                USER STORY (POST-IT) VS USE CASE (DAVRANIŞSAL ANLATI)

1. USER STORY (Tek Cümlelik Parça):
   "Bir üye olarak, kitap ödünç almak istiyorum, böylece evde okuyabilirim."
   ❌ Tetikleyici nerede? Önkoşullar ne? Hangi hata durumları var?
   ❌ AI bu cümleyi görünce tüm doğrulama ve veritabanı kurallarını KAFASINA GÖRE UYDURUR!

2. USE CASE (Tam Davranışsal Sözleşme):
   Tetikleyici ⟶ Önkoşul ⟶ 1..9 Başarı Adımları ⟶ Alternatif Akışlar (A1, A2) ⟶ Sonkoşul
   ✅ AI ajanı neyin başarılı neyin başarısız olduğunu tam olarak bilir;
      tahmin etmez, uydurmaz, sözleşmeyi koda döker!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

User Story, buzdolabının üstüne yapıştırılan 'Akşama ekmek al' notu gibidir. Eve gelen eşin fırının kapalı olduğunu görürse ne yapacağını bilmez (eve boş döner veya pastaneden pasta alır). Use Case ise tam bir görev talimatıdır: 'Akşam fırından 2 tam buğday ekmeği al. Fırın kapalıysa yan bakkala bak. Bakkalda da yoksa beni ara, sakın beyaz ekmek alma!' Yapay zekâya post-it verirsen kafası karışır; Use Case verirsen harfiyen yerine getirir!

🇬🇧 ENGLISH (ELI5)

A user story is a fridge note that says: 'Buy bread on the way home.' If the bakery is locked, your partner is baffled: do they visit a grocery store, buy cake, or return empty-handed? A use case is the full protocol: 'Buy two sourdough loaves at Bakery A. If closed (A1), proceed to Market B. If sold out (A2), call home. Under no circumstances buy white bread.' AI agents flounder with fridge notes; they thrive on structured protocols.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Kullanıcı Hikayeleri (User Stories) ile Use Case arasındaki tamamlayıcı ilişki: Çevik ekipler 20 yıl boyunca use case'lerin yerini user story'lerin aldığını iddia etti. Oysa Cockburn'ün 2024'te vurguladığı gibi: User Story bir konuşma biletidir (token for a conversation); detay içermez. Ancak AI ile çalışırken karşınızda kahve içip sohbet edebileceğiniz bir insan yoktur! Bir LLM ajanına 'As a user I want to borrow a book' prompt'u verirseniz; kaç kitap limiti olduğunu, gecikmiş borç kontrolünü, kitap kopyası kalmadığında ne yapılacağını bilemez ve kafasına göre bir senaryo yazar. AI Unified Process'te `docs/requirements.md` içinde gereksinimler User Story formatında listelenir (FR-012); ancak bu hikayelerin gerçek davranışsal sözleşmesi Use Case dosyalarında (UC-002) yaşar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The dialectic of User Stories vs Use Cases: User stories ('As a… I want… So that…') were explicitly designed as conversation starters—placeholders for verbal negotiations between product owners and developers. But LLMs cannot engage in informal office banter; they generate code immediately from whatever text they are fed. Feeding an ambiguous user story into an AI coding engine forces the model to invent preconditions, error pathways, and state rollbacks. In the AI Unified Process, user stories express functional requirements in the catalog (`docs/requirements.md`), while use cases encapsulate the multi-path behavioral contract.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: User Story'nin Use Case Sözleşmesine Dönüştürülmesi
package com.maang.aiup.contract;

public class LibraryContract {
    // 1. User Story: "Bir üye olarak kitap ödünç almak istiyorum..." (Gereksinim Kataloğu)
    public record UserStoryRef(String frId, String summary) {}

    // 2. Use Case: Tüm alternatif akışları içeren somut davranış modeli
    public static class BorrowBookUseCase {
        public static final UserStoryRef TRACES_TO = new UserStoryRef("FR-012", "Book Borrowing
Flow");

        public record BorrowResult(boolean success, String message, String dueDate) {}

        public BorrowResult execute(String memberId, String isbn, boolean isOverdue, int currentLoans,
int copiesAvailable) {
            // Alternatif Akış A3: Gecikmiş borç kontrolü
            if (isOverdue) return new BorrowResult(false, "A3: Gecikmiş kitap borcunuz var!", null);
            // Alternatif Akış A2: Limit kontrolü
            if (currentLoans >= 5) return new BorrowResult(false, "A2: Maksimum 5 kitap limitine
ulaştınız!", null);
            // Alternatif Akış A1: Stok kontrolü
            if (copiesAvailable <= 0) return new BorrowResult(false, "A1: Uygun kopya kalmadı,
rezervasyon önerildi.", null);

            // Ana Başarı Akışı
            return new BorrowResult(true, "Kitap başarıyla ödünç verildi.", "2026-11-17");
        }
    }

    void main() {
        var uc = new BorrowBookUseCase();
        var res = uc.execute("M101", "ISBN-123", false, 2, 3);
        System.out.println("Use Case İcrası: " + res);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

User Story ile Use Case'i birbirine düşman sanmak büyük bir hatadır. User Story 'Neden ve Kimin için' olduğunu söyler; Use Case ise 'Sistemin adım adım ne yapacağını ve hangi hataları yöneteceğini' belirler.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not treat user stories and use cases as adversarial paradigms. User stories capture business intent and stakeholder value; use cases specify the exhaustive interaction contract and failure topologies.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- User Story konuşma başlatmak için bir jetondur; Use Case ise varılan kararların yaşadığı evdir.
- Tek satırlık kullanıcı hikayesiyle kod üreten AI, tüm köşe vakaları ve kuralları kafasından uydurur.
- AIUP'ta her Use Case en az bir kullanıcı hikayesine (FR-XXX) mekanik olarak izlenebilir şekilde bağlanır.

🎯 Key Architectural Takeaways

- User stories are tokens for conversation; use cases are where the negotiated details live.
- Feeding single-sentence user stories to an AI forces it to hallucinate business logic.
- In AIUP, every use case traces mechanically to at least one user story in the requirements catalog.

6

BÖLÜM: BÜYÜK TERSINE DÖNÜŞ (THE INVERSION) & AI ÇAĞINDA USE CASE RÖNESANSI | MODULE: THE AI INVERSION & WHY USE CASES RULE THE GENAI ERA

Spec-Driven Development (SDD): Şartnameyi Birincil Varlık Olarak Konumlandırma

Spec-Driven Development: Treating the Specification as the Primary Asset

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              SPEC-DRIVEN DEVELOPMENT (SDD) YAŞAM DÖNGÜSÜ │
└──────────────────────────────────────────────────────────┘

KURAL: Kod asla doğrudan elle değiştirilmez! Değişiklik daima şartnameden başlar!

1. Şartnameyi Güncelle (docs/use_cases/UC-002-borrow-book.md)
          |
          ▼
2. /spec-review ile Tutarlılık Denetimi Yap
          |
          ▼
3. /implement Komutunu Çalıştır ──► AI Kodu ve Testleri Yeniden Üretir!
          |
          ▼
4. Testler Yeşile Döner ──► Canlıya Al! (Sıfır Sapma, %100 İzlenebilirlik!)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir gökdelen inşa ederken elektrik tesisatını beğenmediğinde gidip duvarı kırıp kafana göre kablo çekmezsin. Önce mimari projeyi (şartnameyi) güncellersin, mühendisler onaylar, sonra ustalar gelip plana göre tesisatı döşer. Eğer projeyi güncellemeden duvara kablo çekersen, yarın başka bir usta gelip o kabloyu keser ve bina yanar! SDD tam olarak budur: Önce planı düzelt, işçiler (AI) plana göre binayı saniyeler içinde yenilesin!

🇬🇧 ENGLISH (ELI5)

When building a skyscraper, if you decide to add an electrical outlet, you don't secretly chisel into a load-bearing column and solder wires. You update the architectural blueprints first. Once approved, the construction crew executes the change. Modifying code directly while leaving the specification stale is architectural vandalism; the next automated regeneration will wipe out your undocumented hotfix.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Spec-Driven Development (SDD) ve Kodun Geçiciliği Paradigması: Simon Martinelli'nin 'Spec-Driven Development' kitabında altını çizdiği en temel disiplin: Kod artık kalıcı bir varlık değildir; şartnamenin bir derleme çıktısıdır. Geleneksel yaklaşımlarda kod değiştikçe döküman eskir ve çöp olurdu. SDD bu döngüyü kırar: 1) Tek Doğruluk Kaynağı (Single Source of Truth): Şartname ve kod çeliştiğinde her zaman kod hatalı kabul edilir. 2) Yeniden Üretilebilirlik (Regenerability): Yeni bir framework sürümü çıktığında (örneğin Spring Boot 3'ten 4'e veya Java 21'den 25'e), şartname hiç değişmeden tek bir `/implement` komutuyla tüm codebase modern kütüphanelerle baştan üretilebilir. 3) İzlenebilirlik (Traceability): Canlı sistemdeki her bir satır kod, şartnamedeki bir iş kuralına (BR-XXX) ve use case adımına doğrudan bağlanır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Spec-Driven Development (SDD) architectural discipline: SDD posits that software source code is no longer a durable asset; it is an ephemeral compilation artifact derived from formal specifications. The core tenets: 1) Specification Primacy: Whenever source code deviates from the use case specification, the code is definitively defective by definition. 2) Effortless Regeneration: Modernizing frameworks or porting stacks (e.g. Java to TypeScript, JPA to jOOQ) becomes mechanical: rerun generation commands against unchanged, human-verified specifications. 3) Strict Traceability: Every generated controller, entity validation, and integration test links back to a specific business rule identifier (BR-XXX).

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Spec-Driven Kod Üretiminin İdeal Çıktısı (Tam İzlenebilir Sınıf)
package com.maang.aiup.sdd;

/**
 * ⚠ DİKKAT: Bu sınıf Spec-Driven Development kapsamında AI tarafından üretilmiştir.
 * Bu dosyayı doğrudan elle DEĞİŞTİRMEYİN!
 * Değişiklikler için şartnameyi güncelleyin: docs/use_cases/UC-002-borrow-book.md
 *
 * Traces to:
 * - Use Case: UC-002 (Borrow Book)
 * - Requirements: FR-012, NFR-002
 * - Rules: BR-004 (Availability), BR-005 (Loan Limit), BR-006 (Overdue Block)
 */
public final class BorrowBookService {
    public record LoanConfirmation(String loanId, String memberId, String bookCopyId,
java.time.LocalDate dueDate) {}

    public LoanConfirmation processLoan(String memberId, String bookCopyId, int activeLoans, boolean
hasOverdue, int availableCopies) {
        // Step 4: System validates that a copy is available (BR-004)
        if (availableCopies <= 0) {
            throw new IllegalStateException("BR-004: Kitap kopyası mevcut değil.");
        }
        // Step 5: System validates member has not reached limit (BR-005)
        if (activeLoans >= 5) {
            throw new IllegalStateException("BR-005: Üye 5 olan ödünç limitine ulaştı.");
        }
        // Step 6: System validates member has no overdue loans (BR-006)
        if (hasOverdue) {
            throw new IllegalStateException("BR-006: Gecikmiş borçlar nedeniyle ödünç verilemez.");
        }

        // Step 7: System records loan with due date 28 days from today
        var dueDate = java.time.LocalDate.now().plusDays(28);
        return new LoanConfirmation("LN-" + System.currentTimeMillis(), memberId, bookCopyId,
dueDate);
    }

    void main() {
        var service = new BorrowBookService();
        var loan = service.processLoan("MEM-1", "CPY-99", 2, false, 3);
        System.out.println("✅ SDD ile üretilen ve şartnameye %100 uyan kayıt: " + loan);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Acil bir canlı sistem hatasında (hotfix) kodu doğrudan IDE'den düzeltip 'şartnameyi sonra güncellerim' demek metodolojinin ölüm fermanıdır. O şartname asla güncellenmez ve sonraki kod üretiminde hotfix'iniz tamamen silinir!

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Applying a hotfix directly to production code while promising to 'update the spec later' destroys the entire methodology. The specification rots immediately, and the next automated regeneration erases your hotfix without warning.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Spec-Driven Development'ta kod değil, şartname temel varlıktır; kod onun derlenmiş çıktısıdır.

• Kodu doğrudan elle değiştirmek yasaktır; tüm değişiklikler şartnameden başlayarak koda akar.

• Şartnameleri yaşayan birer sözleşme olarak korumak, projeyi gelecekte sıfır maliyetle modernize etmeyi sağlar.

🎯 Key Architectural Takeaways

• In Spec-Driven Development, the specification is the primary asset; code is its compiled output.

• Never modify code directly; all changes must cascade downstream from the specification.

• Living specifications allow entire software systems to be safely regenerated onto new tech stacks.

BÖLÜM 2 / MODULE 2

Üç Okuyucu Modeli & Yapay Zekâ Çağının 5 Temel İlkesi

The Three Readers & The 5 AI-Era Principles

Use case'in üç farklı kitlesi: Paydaş (Stakeholder), Yazılım Mühendisi ve Yapay Zekâ Ajanı. AI'ın lafı harfi harfine anlaması ve boşluk bırakamaması. Use-Case 3.0'ın 10 ilkesine eklenen 5 yeni AI çağı ilkesi (İlke 11-15).

The three readers of a use case: Stakeholder, Software Engineer, and AI Agent. How AI reads literally and cannot leave gaps unfilled. The five new principles for the AI era (Principles 11-15) extending Use-Case 3.0.

7

BÖLÜM: ÜÇ OKUYUCU MODELI & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESI | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

Üç Okuyucu Modeli: Paydaş Doğrular, Mühendis İnceler, AI Üretir

The Three Readers: Stakeholder Validates, Engineer Reviews, AI Generates

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                USE CASE'İN ÜÇ OKUYUCUSU VE OKUMA ŞEKİLLERİ

OKUYUCU 1: PAYDAŞ (STAKEHOLDER)
- Ne İçin Okur? "İstediğim şey bu mu? Teslim edildiğinde anlayabilecek miyim?"
- Nasıl Okur?   Hızlıca göz gezdirir (Skim). Hikayeye, isimlere, mutlu yola bakar. Çabuk sıkılır!

OKUYUCU 2: YAZILIM MÜHENDİSİ (SOFTWARE ENGINEER)
- Ne İçin Okur? "Bu inşa edilebilir mi? Üretilen kod ve testler bu şartnameye uyuyor mu?"
- Nasıl Okur?   Satır satır inceler (Review). Akışları kod ve testlerle çapraz doğrular.

OKUYUCU 3: YAPAY ZEKÂ AJANI (AI AGENT)
- Ne İçin Okur? "Ben tam olarak ne üreteceğim?"
- Nasıl Okur?   Harfi harfine ve eksiksizce (Literally & Exhaustively). Asla sıkılmaz,
                tüm referansları takip eder ve 'ne demek istediniz' diye ASLA sormaz!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoran düşünün: 1) Müşteri (Paydaş) menüye bakar, resimleri ve yemek isimlerini hızlıca süzer, lezzetli görünüyorsa 'Bunu istiyorum' der. 2) Restoran Müdürü (Mühendis) menüye bakar; 'Mutfakta malzeme var mı, maliyet kurtarır mı, hijyen kurallarına uygun mu?' diye kontrol eder. 3) Mutfaktaki Robot Aşçı (AI) ise tarifi harfi harfine okur: '1 gram tuz, 200 derece fırın'. Robot aşçı müşterinin gözünün içine bakıp 'Tuzlu mu seversin?' diye sormaz; tarifte ne yazıyorsa milimetrik olarak tabağa koyar!

🇬🇧 ENGLISH (ELI5)

Consider a restaurant menu: 1) The Customer (Stakeholder) skims the pictures and dish names. If it looks appetizing, they order. They have zero patience for ingredient chemical formulas. 2) The General Manager (Engineer) reviews the menu to confirm inventory feasibility and safety regulations. 3) The Robotic Cook (AI Agent) parses the recipe exhaustively down to the milligram. The robot cook will never look up from the stove to ask if you prefer extra garlic; it executes the written text with uncompromising literalism.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Klasik yazılım mühendisliğinde kullanım senaryolarının yalnızca iki okuyucusu vardı: Doğrulayan iş birimi (business) ve kodu yazan insan geliştirici (developer). Cockburn'ün en önemli kuralı 'okunabilirlik' (readability) idi: 'Okunabilirliği feda ederseniz paydaşlarınız use case okumayı bırakır.' AI Unified Process üçüncü bir okuyucuyu sahneye çıkardı: AI Kodlama Ajanı. Ancak yapılan en ölümcül hata, 'Madem AI okuyor, öyleyse use case'i prompt mühendisliğine çevirelim, JSON veya pseudo-code yazalım' yanılgısıdır. Bir use case pseudo-code haline getirilirse paydaş onu okumadan imzalar; okunmadan imzalanan bir şartname ise kontrat olmaktan çıkar! Çözüm: İnsan için okunabilir olan kuralların (etken çatı, net özneler, tekil adımlar) aynı zamanda AI için de en az belirsizlik içeren format olduğunu anlamaktır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Historically, use cases served a dual audience: the business stakeholder who verified functional intent, and the software engineer who transcribed it into code. Cockburn made readability the supreme heuristic: an unreadable spec is abandoned by its business sponsors. The AI Unified Process introduces a third constituent: the autonomous AI agent. A catastrophic misconception is attempting to optimize the specification format exclusively for the machine via pseudo-code or prompt-engineering templates. Doing so alienates stakeholders, who sign off unread, turning the contract into fiction. Crucially, syntactic clarity (active voice, explicit subjects, named validation rules) satisfies both human skimming and machine parsing without compromise.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Üç Okuyucunun Ortak Dili Olan 'Clean Use Case Interactor'
package com.maang.aiup.readers;

public class CancelSubscriptionUseCase {
    // 1. Paydaşın anladığı iş modeli (Gözlemlenebilir Niyet)
    public record CancellationRequest(String memberId, String reason, boolean immediate) {}
    public record CancellationSummary(String memberId, boolean isTerminated, String refundStatus) {}

    // 2. Mühendisin denetlediği mimari kontrat
    public interface SubscriptionRepository {
        boolean hasActiveSubscription(String memberId);
        void recordCancellation(String memberId, String reason);
    }

    // 3. AI'ın doğrudan ürettiği ve test ettiği icra sınıfı
    public static class CancelSubscriptionInteractor {
        private final SubscriptionRepository repository;

        public CancelSubscriptionInteractor(SubscriptionRepository repository) {
            this.repository = repository;
        }

        public CancellationSummary execute(CancellationRequest request) {
            // Step 2: System validates active subscription exists (BR-014)
            if (!repository.hasActiveSubscription(request.memberId())) {
                throw new IllegalStateException("BR-014: İptal edilecek aktif abonelik bulunamadı.");
            }
            // Step 3: System records cancellation and calculates refund
            repository.recordCancellation(request.memberId(), request.reason());
            String refund = request.immediate() ? "REFUND_PRORATED" :
"NO_REFUND_ACTIVE_UNTIL_CYCLE_END";

            return new CancellationSummary(request.memberId(), true, refund);
        }
    }

    void main() {
        System.out.println("Üç okuyucunun niyetini taşıyan Clean Use Case hazır.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Use case'i yapay zekaya hitap edeceğim diye Python/Java benzeri sözde kodlarla (pseudo-code) yazmayın! Bir dökümanı paydaş (iş analisti, ürün yöneticisi) okuyup gözüyle doğrulayamıyorsa, o döküman bir şartname değil teknik çöplüktür.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never degrade use cases into pseudo-code or machine-syntax prompts. If a business stakeholder cannot skim the narrative and validate whether it solves their problem, the contract is severed and the governance model collapses.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Use case'in 3 okuyucusu vardır: Paydaş doğrular, Mühendis inceler, AI ise doğrudan kod ve test üretir.

• Okunabilirlik her zaman ilk şarttır; makine için insanı devre dışı bırakan döküman şartname vasfını yitirir.

• İyi yazılmış bir senaryo hem meşgul bir paydaşı hem de harfi harfine çalışan AI'ı aynı anda tatmin eder.

🎯 Key Architectural Takeaways

• A use case has three readers: Stakeholder validates, Engineer reviews, AI generates.

• Readability remains paramount; a spec that humans cannot validate is no longer a contract.

• A disciplined, active-voice spec satisfies both the busy human stakeholder and the literal AI agent.

8

BÖLÜM: ÜÇ OKUYUCU MODELI & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESI | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

Lafı Harfi Harfine Anlayan Okuyucu: AI Neden Bir Boşluğu Asla Doldurmadan Bırakamaz?

The Literal Reader: Why an AI Cannot Leave a Gap Unfilled

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────┐
│          BOŞLUK DOLDURMA MEKANİZMASI: İNSAN VS YAPAY ZEKÂ          │
└──────────────────────────────────────────────────────┘

SENARYO: "Sistem başarısız giriş denemelerini yönetir." (Eksik / Muğlak Kural)

1. İNSAN MÜHENDİS:
   - Durur, düşünür.
   - "3 deneme mi 5 deneme mi? Kaç dakika kilitlenecek?" diye soru sorar.
   - BİLMEDİĞİNİN FARKINDADIR (Aware of Ignorance).

2. AI AJANI (LLM / Claude Code / Copilot):
   - Asla durmaz! Boşluğu hemen doldurur!
   - Kendi kafasından uydurur: "5 deneme olsun, 15 dakika kilitlensin, SMS atsın!"
   - TAM ÖZGÜVENLE SUNAR (Confident Hallucination).
   - Sonuç: Müşterinin hiç istemediği uydurma bir özellik artık canlı kodunuzda!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir ressama ‘Bana bir oturma odası çiz’ deyip koltuğun rengini söylemezsen, ressam durup ‘Koltuk ne renk olsun?’ diye sorar. Ama yapay zekâ fırçayı eline alır almaz boş gördüğü her yere pembe puantiyeli koltuklar, tavana disko topu ve köşeye bir zürafa çizer! Neden? Çünkü yapay zekâ boş tuval görünce dayanamaz, istatistiksel olarak ne denk gelirse doldurur. Boşluk bırakırsan, o boşluğu senin yerine AI’ın zevkleri doldurur!

🇬🇧 ENGLISH (ELI5)

Ask an interior designer to draw a living room without specifying the sofa color, and they pause to ask your preference. Generative AI cannot tolerate an empty canvas: it instantly paints neon-leopard sofas, hangs a disco ball, and places a giraffe in the corner with total conviction. AI has no capacity to say ‘I notice a gap here, what did you mean?’ It completes the statistical distribution. Any intent you omit becomes an invented feature.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Yapay zekanın olasılıksal doğası (Autoregressive Token Generation) ve niyet boşlukları: LLM'ler bir sonraki en olası token'ı tahmin etmek üzere eğitilmiştir. Bu mimari zorunluluk gereği bir LLM 'Burada bir iş kuralı eksik, durup bekleyeyim' diyemez (bunu ancak harici meta-prompting veya /spec-review gibi deterministik araçlarla sağlayabilirsiniz). Use case içinde bir sınır, sıralama veya hata mesajı belirtilmemişse, model eğitim verisindeki binlerce açık kaynak projeden öğrendiği ortalama davranışı uygular. Örneğin: 'Kullanıcı şifresini unuttuysa sistem şifreyi sıfırlar.' AI burada parola sıfırlama linkinin süresini 24 saat mi yapacak, 15 dakika mı? Tek kullanımlık token mı üretecek? Eğer şartname 'BR-021: Parola sıfırlama token'ı 15 dakika geçerlidir' demezse, AI her kod üretiminde farklı bir süre uydurur. Yeniden üretilebilirlik (reproducibility) yok olur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Autoregressive generation dynamics and the impossibility of omission: LLMs are mathematical prediction engines optimized to generate plausible continuations. By structural design, an LLM cannot autonomously halt on a requirements vacuum and say 'The author omitted a timeout parameter; I shall suspend output.' Instead, it samples from its latent training distribution, selecting standard web defaults. If a use case states 'System blocks abusive IPs', the model will arbitrarily invent an arbitrary threshold (e.g. 10 requests per minute vs 100). The downstream consequence is devastating: the system exhibits behavioral drift across regenerations because implicit rules fluctuate with each prompt run.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: AI Boşluklarını Kapatan Tip Güvenli ve Parametrik Güvenlik Kuralı
package com.maang.aiup.literal;

import java.time.Duration;

public class LoginPolicyEngine {
    // Şartnameden birebir gelen ve AI'a hiçbir uydurma payı bırakmayan kural record'u
    public record LockoutPolicy(
        int maxFailedAttempts,        // BR-008: Kesin sayı (örn: 3)
        Duration lockoutDuration,      // BR-008: Kesin süre (örn: 15 dakika)
        boolean notifyAdminByEmail    // BR-009: Kesin bayrak
    ) {}

    public static final LockoutPolicy PRODUCTION_POLICY =
        new LockoutPolicy(3, Duration.ofMinutes(15), true);

    public record LoginAttemptResult(boolean allowed, String statusMessage) {}

    public static LoginAttemptResult evaluate(int consecutiveFails, LockoutPolicy policy) {
        if (consecutiveFails >= policy.maxFailedAttempts()) {
            return new LoginAttemptResult(false,
                "BR-008: Hesap " + policy.lockoutDuration().toMinutes() + " dakika süreyle
kilitlendi.");
        }
        return new LoginAttemptResult(true, "Giriş denemesi geçerli.");
    }

    void main() {
        var result = evaluate(3, PRODUCTION_POLICY);
        System.out.println("Deterministik sonuç: " + result);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir use case yazarken 'vb.', 've benzeri', 'gerekirse', 'uygun şekilde' kelimelerini kullanmak, yapay zekaya açık çek vermektir. Bu kelimeleri gördüğünüz yerde şartnameyi silip somut sınırları yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Using words like 'etc.', 'and so on', 'as appropriate', or 'if necessary' is writing a blank check to an AI agent. Every ambiguous clause is an architectural decision delegated to an algorithm.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Yapay zekâ bir boşluğu asla boş bırakamaz; en olası gördüğü varsayımla sessizce doldurur.
- Yazılmayan gereksinim soru olarak dönmez; hatalı üretilmiş bir özellik olarak canlıya sızar.
- Tüm limitler, süreler ve hata akışları şartnamede eksiksiz ve somut olarak belirtilmelidir.

🎯 Key Architectural Takeaways

- An AI agent cannot leave a gap unfilled; it fills every omission with plausible invention.
- Unwritten requirements return as hallucinated features, not polite clarifying questions.
- All thresholds, durations, and failure paths must be explicitly bound within the specification.

9

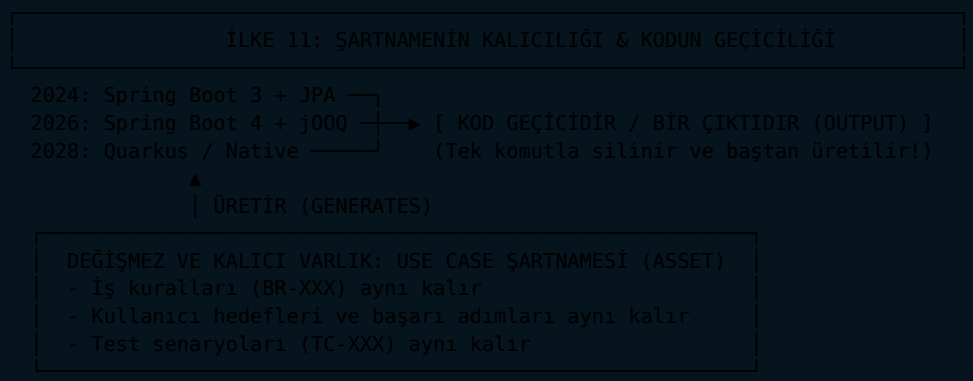
BÖLÜM: ÜÇ OKUYUCU MODELİ & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESİ | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

AI-Çağı İlkesi 11: Şartname Koddan Daha Uzun Yaşar (The Spec Outlives the Code)

AI-Era Principle 11: The Specification Outlives the Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir tiyatro oyununu düşünün: Hamlet oyunu 400 yıldır aynı senaryoyla oynanıyor. Ancak aktörler, kostümler, sahne ışıkları ve tiyatro binası yüzlerce kez değişti! Kod dediğin şey sahnedeki kostümdür; eskiyince çöpe atar yenisini giydirirsin. Use Case şartnamesi ise Shakespeare'in yazdığı ana tiyatro metnidir (Hamlet)! Kostüm değişir ama metin asla ölmez!

🇬🇧 ENGLISH (ELI5)

Consider Shakespeare's Hamlet: the play has endured for 400 years, while the actors, costumes, stages, and lighting rigs have been replaced thousands of times. Source code is merely the temporary stage costume—it wears out, gets refactored, and is ultimately discarded. The use case specification is the timeless script. Frameworks and syntax will fade, but the core business contract outlives them all.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Yazılım Mühendisliği 2.0 ve İlke 11'in Derinliği: Martinelli'nin Use-Case 3.0'a eklediği 11. ilke: 'Şartname koddan daha uzun yaşar.' Geleneksel yazılım geliştirmede kod en değerli varlıktı (asset); şartname ise geliştirme bitince bir köşeye atılan geçici bir iskeleydi (scaffolding). AI çağında bu durum 180 derece değişti. Bugün modern bir AI ajanı, elinde kaliteli bir use case ve veri modeli varsa, 10.000 satırlık bir mikroservisi 10 dakikada sıfırdan üretebilir. Ancak o 10.000 satırlık koda bakarak şirketin gerçek iş kurallarını (business intent) geri çıkarmak aylar sürer ve devasa hatalara gebedir. Bu nedenle bir projede kod ve şartname çeliştiğinde, varsayılan kural şudur: 'Şartname doğrudur, kod hatalıdır.' Kod her zaman şartnameye uydurulmak üzere yeniden üretilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 11 Architectural Mandate: 'The specification outlives the code.' In legacy paradigms, code was the ultimate store of organizational value; specifications were discarded scaffolding. In modern AI-augmented software engineering, implementation is fungible. Armed with a rigorous use case model and entity dictionary, an LLM agent generates an entire production-grade subsystem in minutes. Conversely, reverse-engineering unstated business invariants from legacy binary bytecodes is agonizing and prone to catastrophic data loss. Therefore, whenever code and specification disagree, the architectural presumption is uncompromising: the code is defective. Code is transient; intent is permanent.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 10 Yıl Sonra Bile Geçerliliğini Koruyan Değişmez İş Kuralı Modeli
package com.maang.aiup.principles;

public class Principle11Demo {
    // 2026'da Java 25 ile, 2030'da Java 30 ile derlense de bu kural DEĞİŞMEZ:
    // BR-005: Bir üye aynı anda en fazla 5 açık ödünç kitaba sahip olabilir.
    public static final int MAX_OPEN_LOANS_LIMIT = 5;

    public record MemberLoanProfile(String memberId, int activeLoansCount) {
        public boolean canBorrowMore() {
            return activeLoansCount < MAX_OPEN_LOANS_LIMIT;
        }
    }

    void main() {
        var profile = new MemberLoanProfile("M-77", 4);
        System.out.printf("Üye ID: %s | Yeni kitap alabilir mi? %s (Limit: %d)%n",
            profile.memberId(), profile.canBorrowMore(), MAX_OPEN_LOANS_LIMIT);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir kodlama hatasını düzeltmek için doğrudan koda girip 'if (count < 6)' yazmayın! Bu satırı gören biri kuralın 5 mi 6 mı olduğunu anlayamaz. Önce şartnamedeki BR-005 kuralını güncelleyin, ardından kodu yeniden ürettirin.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never patch an operational defect by directly tweaking conditional logic in source files without touching the specification. Unsynchronized code patches cause specification drift, guaranteeing that future automated rebuilds will overwrite your changes.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kod bir derleme çıktısıdır; atılabilir, değiştirilebilir ve yeniden üretilebilir.

• Use Case şartnamesi kurumun ana entelektüel sermayesidir (asset) ve sistem yaşadıkça korunur.

• Kod ile şartname çeliştiğinde her zaman kod hatalı kabul edilir.

🎯 Key Architectural Takeaways

• Code is now a build artifact: disposable, replaceable, and effortlessly regenerated.

• The use case specification is the enterprise asset, maintained for the life of the system.

• When specification and code disagree, the default axiom is that the code is wrong.

10

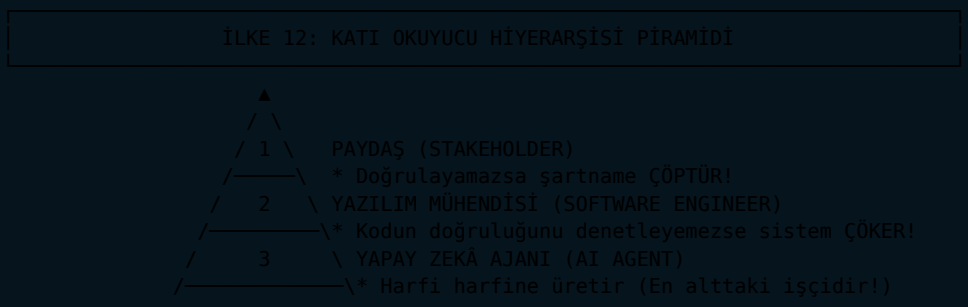
BÖLÜM: ÜÇ OKUYUCU MODELI & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESI | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

AI-Çağı İlkesi 12: Okuyucu Hiyerarşisi (Paydaş > Mühendis > Yapay Zekâ)

AI-Era Principle 12: Reader Hierarchy (Stakeholder > Engineer > AI)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette patron (Paydaş), başmimar (Mühendis) ve inşaat robotu (AI) vardır. Şirketin yeni binasının planı çizilirken kime göre çizilir? Elbette patrona ve mimara göre! Eğer robot daha rahat anlasın diye mimari planı '010101' şeklinde ikili bilgisayar kodlarıyla yazarsan, patron 'Ben bu binayı istemedim ki!' diyemez bile. İlk önce patron anlayacak, sonra mimar onaylayacak, en son robot tuğlayı koyacak!

🇬🇧 ENGLISH (ELI5)

In a corporate hierarchy, you have the Executive (Stakeholder), the Chief Architect (Engineer), and the Construction Drone (AI Agent). If you draft building blueprints in raw assembly bytecode so the drone can ingest them faster, the executive cannot verify whether the building has enough emergency exits. The blueprint must first be legible to the owner, verifiable by the engineer, and only then parsed by the drone.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İlke 12: Üç okuyucu için şu katı öncelik sırasıyla yaz: 1) Paydaş, 2) Mühendis, 3) Yapay Zekâ. Martinelli'nin uyarısı: Eğer bir paydaş şartnameyi okuyup doğrulayamıyorsa, o şartnamenin taşıdığı teknik hassasiyet tamamen değersizdir. Eğer bir mühendis o şartnameye bakarak üretilen kodun doğruluğunu teyit edemiyorsa, sistemin güvenliği sıfırdır. Ancak ilk iki okuyucu tatmin edildikten sonra yapay zekanın tüketebilirliği (AI-consumability) önem kazanır. Harika olan gerçek şudur: İlk iki okuyucuya mükemmel hizmet eden bir use case (etken çatı, açık aktör isimleri, 'kontrol eder' yerine 'doğrular', numaralı adımlar), neredeyse otomatik olarak üçüncü okuyucuya (AI) da kusursuz hizmet eder. Bu yüzden okuyucular arasında bir ödünleşim (trade-off) hissettiğinizde, neredeyse kesinlikle gereksiz bir uygulama detayına (implementation mechanism) sapmışsınız demektir!

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 12 Governance Formula: 'Write for three readers, in strict descending order: Stakeholder, Engineer, AI.' If a business stakeholder cannot comprehend the specification, its algorithmic precision is worthless. If a software engineer cannot cross-examine the generated code against the spec, correctness cannot be validated. Only after these human gates are established does machine consumability matter. Fortunately, these needs converge: active voice, single-responsibility steps, and explicit business rules satisfy humans and machines simultaneously. If you ever feel compelled to sacrifice human readability for AI parsing, you have invariably leaked implementation mechanisms into the specification.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Paydaşın, Mühendisin ve AI'ın Birlikte Doğrulayabildiği Domain Modeli
package com.maang.aiup.hierarchy;

public class ReaderHierarchyDemo {
    // Paydaş Okur: "Üye, Kurs ve Kayıt" kelimelerini görür.
    // Mühendis Okur: Tip güvenliğini ve değişmezliği (immutability) görür.
    // AI Okur: Hangi validation kurallarının işletileceğini eksiksiz anlar.
    public record CourseEnrollment(
        String enrollmentId,
        String memberId,
        String courseCode,
        boolean isPrerequisiteSatisfied,
        int availableSeats
    ) {
        public CourseEnrollment {
            // Step 4: System validates prerequisites (BR-020)
            if (!isPrerequisiteSatisfied) {
                throw new IllegalStateException("BR-020: Önkoşul dersleri tamamlanmamış.");
            }
            // Step 5: System validates seat availability (BR-021)
            if (availableSeats <= 0) {
                throw new IllegalStateException("BR-021: Kursta boş kontenjan kalmadı.");
            }
        }
    }

    void main() {
        var enrollment = new CourseEnrollment("ENR-10", "MEM-5", "CS-101", true, 15);
        System.out.println("✅ Üç okuyucunun da onayladığı kayıt: " + enrollment);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnameye JSON şemaları, SQL sorguları veya regex desenleri koyarak 'AI böyle daha iyi anlıyor' bahanesine sığınmayın. Bu tür detaylar paydaşın okumasını engeller ve İlke 12'yi doğrudan ihlal eder.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never inline regex formulas, SQL statements, or JSON schemas into use case steps under the pretext that 'the AI understands code better'. Doing so alienates stakeholders and destroys the foundational reader hierarchy.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Okuyucu sırası asla değişmez: 1. Paydaş, 2. Mühendis, 3. Yapay Zekâ.
- Paydaşın onaylamadığı şartnameyle üretilen kod, ne kadar hatasız çalışırsa çalışsın çöptür.
- İnsanın kolay okuduğu yapı (aktif dil, numaralı adımlar), AI için de en az yanılsama üreten yapıdır.

🎯 Key Architectural Takeaways

- Reader priority is immutable: 1. Stakeholder, 2. Software Engineer, 3. AI Agent.
- Code generated from an unvalidated spec is useless, regardless of algorithmic elegance.
- Structures that maximize human readability also minimize AI hallucination rates.

11

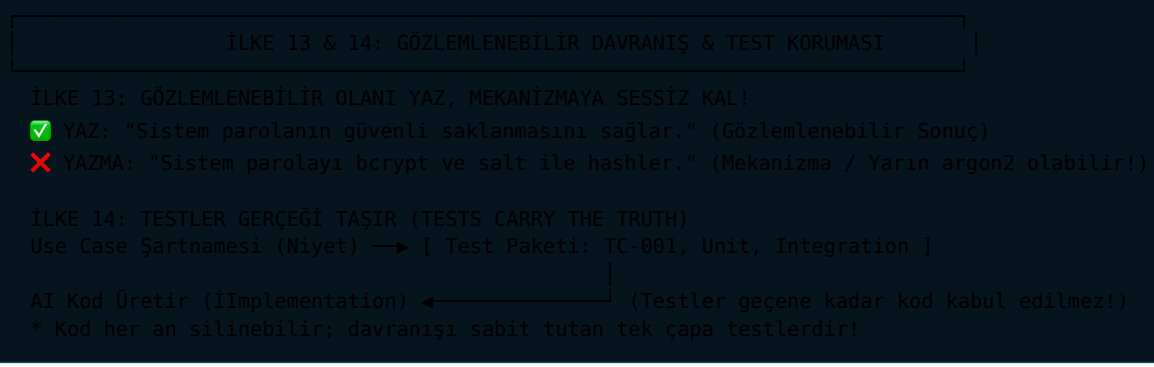
BÖLÜM: ÜÇ OKUYUCU MODELİ & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESİ | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

AI-Çağı İlkeleri 13 & 14: Gözlemlenebilir Davranış vs Testler Gerçeği Taşır

AI-Era Principles 13 & 14: Observable Behavior vs Tests Carry the Truth

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İlke 13: Bankaya gittiğinde veznedara 'Bana paramı ver ama parayı verirken sol elini 45 derece açıyla kaldır, kasayı anahtarla 3 kere çevir' demezsin. Sadece '1000 TL çekmek istiyorum' dersin (Sonuç!). Kasayı nasıl açtığı bankanın iç meselesidir. İlke 14: Veznedarın parayı doğru verip vermediğini anlamak için parayı eline alır sayarsın (Test!). Parayı saymazsan eksik mi verdi sahte mi verdi bilemezsin!

🇬🇧 ENGLISH (ELI5)

Principle 13: At an ATM, you request '$100 cash'. You never instruct the machine: 'Spin gear 4 at 120 RPM and trigger hydraulic valve B'. The mechanical implementation is the machine's private affair. Principle 14: When the bills emerge, you count them. Tests are the physical counting of the cash. A use case without automated tests is merely wishful thinking.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İlke 13 ve 14'ün Sistem Mimarisi Açısından Önemi: 1) İlke 13: Gözlemlenebilir davranış (observable behavior) ile mekanizma (mechanism) arasındaki ayrım, bir şartnamenin ömrünü belirler. Bir kullanıcının veya testin dışarıdan gözlemleyebileceği her şey (ekrandaki kolonlar, hata mesajları, 28 günlük vade gibi kesin sayılar) Zone 1'dir ve yazılmalıdır. Sistemin bunu nasıl başardığı (SQL query, JWT token, bcrypt, Redis cache) Zone 3'tür ve KESİNLİKLE YASAKTIR. Mekanizma sızarsa şartname mimariyi kilitler ve framework yükseltmelerinde çöker. 2) İlke 14: 'Testler gerçeği taşır' (Tests carry the truth). Use-Case 3.0'ın en çarpıcı tespiti: Test senaryoları bir use case'in anlatısından bile daha önemlidir. Çünkü AI çağında kodu yeniden üretmek (regeneration) ancak davranış dışarıdan bir test takımıyla sabitlenmişse güvenlidir. Kod değişir, silinir, baştan yazılır; ancak testler yeşil kaldığı sürece sistemin sözleşmesi korunmuş demektir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Deep dive on Principles 13 & 14: 1) Principle 13 (Observable vs Mechanism): Replaces decades of ambiguous advice regarding 'the right level of specification detail'. If a stakeholder can verify it or an automated test can assert it, it belongs in the specification (Zone 1). If it describes internal computational wiring (protocols, database schemas, hashing algorithms), it belongs to the implementation skill (Zone 3). Leaking Zone 3 mechanics guarantees that the spec will contradict future framework upgrades. 2) Principle 14 (Tests Carry the Truth): In an automated regeneration lifecycle, source code is volatile. What keeps the system's behavior invariant across LLM rewrites is the test suite. Every use case flow must yield an executable test case; without tests, automated code generation is reckless gambling.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Mekanizmadan Bağımsız Davranışsal Test (Tests Carry the Truth)
package com.maang.aiup.principles;

public class ObservableBehaviorTest {
    // Sözleşme: Parola formatı doğrulanır (Mekanizma: regex mi değil mi şartnameye yazılmaz!)
    public interface PasswordValidator {
        boolean isValid(String password);
    }

    // İlke 14: Gerçeği taşıyan test paketi
    public static void runTests(PasswordValidator validator) {
        // Test 1: En az 8 karakter olmalı
        assert !validator.isValid("short") : "8 karakterden kısa parola reddedilmeli!";
        // Test 2: Rakam içermeli
        assert !validator.isValid("NoDigitsHere!") : "Rakam içermeyen parola reddedilmeli!";
        // Test 3: Başarılı parola
        assert validator.isValid("SecureP@ss2026") : "Geçerli parola kabul edilmeli!";

        System.out.println("✅ İlke 14: Tüm davranışsal testler başarıyla geçti!");
    }

    void main() {
        // AI'ın ürettiği implementasyon (İster regex kullansın, ister döngü!)
        PasswordValidator aiGenerated = pwd -> pwd != null && pwd.length() >= 8 &&
pwd.matches(".*\d.*");
        runTests(aiGenerated);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnameye 'Parola bcrypt ile hashlenir' yazarsanız, yarın kuantum güvenli veya Argon2 algoritmasına geçtiğinizde şartname yalan söylemeye başlar. 'Parola güvenli şekilde saklanır' yazın; algoritma seçimini mimari yapılandırmaya bırakın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

If you write 'password is hashed with bcrypt' in a use case, the moment your team upgrades to Argon2id, the specification becomes a lie. Write 'system securely stores password' and delegate cryptography to the tech stack configuration.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Paydaşın görebildiği ve testin doğrulayabildiği her şey yazılır; mekanizma asla yazılmaz.
- Teknolojiler, kütüphaneler ve veri tabanı komutları şartnameye sızarsa sistem kilitlenir.
- Testler olmadan AI ile kod üretmek kumar oynamaktır; gerçeği kod değil testler taşır.

🎯 Key Architectural Takeaways

- Specify everything observable and testable; stay completely silent on internal mechanisms.
- Leaking protocols, SQL, and algorithms into specs makes future modernization impossible.
- Without automated tests, AI code generation is blind; tests carry the enduring truth.

12

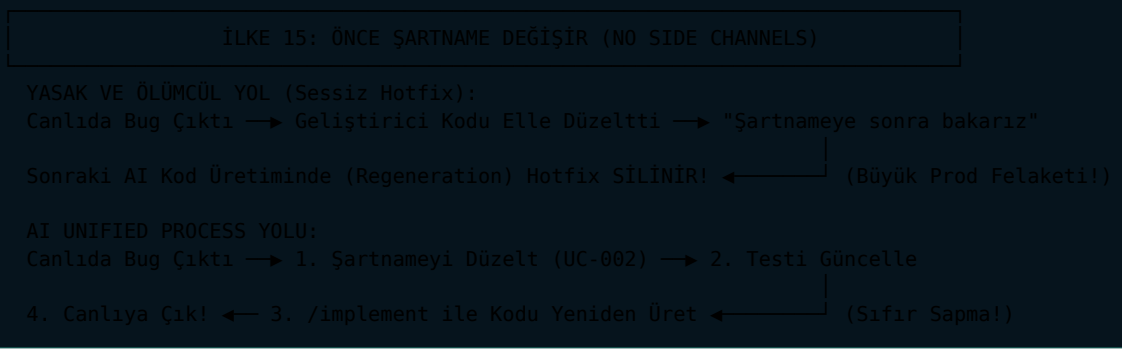
BÖLÜM: ÜÇ OKUYUCU MODELI & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESI | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

AI-Çağı İlkesi 15: Önce Şartname Değişir (Sessiz Hotfix Kültürüne Son!)

AI-Era Principle 15: Change the Specification First (Killing the Silent Hotfix)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir geminin rotasını değiştirmek istediğinde dümencinin eline gizlice vurup dümeni çevirmezsin. Önce kaptan köşküne gider haritayı güncellersin! Eğer haritayı güncellemeden dümene asılırsan, bir sonraki vardiyada gelen yeni dümenci (AI) haritaya bakar ve gemiyi tekrar eski rotaya kırıp kayalıklara çarpar! Hata çıksa da yeni özellik gelse de önce harita (şartname) değişir!

🇬🇧 ENGLISH (ELI5)

You don't alter a ship's heading by sneaking into the wheelhouse and wrestling the steering wheel away from the helmsman. You update the navigational nautical chart. If you yank the wheel without recording the new course on the chart, the next watchman (the AI generator) inspects the chart, swings the rudder back, and rams the vessel into an iceberg. The map changes first, always.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İlke 15: Önce şartnameyi değiştir (Change the specification first). Yeni bir özellik (feature), bir değişiklik talebi (change request) ve bir hata (bug)... AI Unified Process'te bu üç kavramın hepsi TEK VE AYNI ŞEYDİR: 'Bir use case'e yapılan değişiklik.' Sistemde /bug veya /hotfix adında özel bir komut yoktur! Hata çıktığında süreç şöyledir: 1) Hatanın niyetini sınıflandır (Kod mu saptı yoksa şartname mi yanlıştı?). 2) Şartnameyi güncelle. 3) /implement komutunu yeniden çalıştır. AI mevcut sınıfları bulur ve şartnamedeki değişikliğe göre yerinde günceller (in-place reconciliation). Eğer acil durum bahanesiyle kodu doğrudan IDE'den düzeltip şartnameyi güncellemezseniz, şartnamenin 'gerçeklik kaynağı' olma vasfı anında ölür. Bir sonraki sprint'te birisi `/implement` çalıştırdığında elle yazdığınız o hotfix tamamen silinir!

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 15 Operational Discipline: 'Change the specification first.' A new feature, a change request, and a production bug report are fundamentally identical abstractions: a delta applied to a use case specification. The AI Unified Process provides no special `/hotfix` or `/patch` command. The protocol is singular: 1) Classify intent (did code diverge from spec, or was the spec flawed?). 2) Update the use case markdown file. 3) Re-invoke `/implement`. The generative engine reconciles existing classes in-place against the updated specification. Bypassing this loop under production pressure instantly corrupts the artifact chain. The next automated regeneration cycle will treat your undocumented hotfix as dead code and purge it.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Şartname Değişikliğinin Koddaki Yansıması (Regeneration Friendly)
package com.maang.aiup.principles;

public class LoanInterestCalculation {
    // ESKİ KURAL BR-012: Faiz oranı sabit %5.0 idi.
    // YENİ KURAL BR-012 (Değişiklik Şartnamede Yapıldı): VIP üyeler için faiz %3.5, diğerleri %5.0.

    public record Customer(String id, boolean isVip) {}

    public static double calculateInterestRate(Customer customer) {
        // Şartnamedeki BR-012 güncellemesinden sonra AI tarafından üretilen yeni kod:
        if (customer.isVip()) {
            return 3.5; // BR-012 (Revize edilmiş şartname kuralı)
        }
        return 5.0;     // BR-012 Standart kural
    }

    void main() {
        var vip = new Customer("VIP-1", true);
        var standard = new Customer("STD-2", false);

        System.out.println("VIP Faiz: %" + calculateInterestRate(vip));
        System.out.println("Standart Faiz: %" + calculateInterestRate(standard));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Geliştiricilerin 'Hata çok acil, prod yanıyor, önce kodu pushlayalım şartnameyi pazartesi yazarız' tuzağına ASLA düşmeyin. O pazartesi hiçbir zaman gelmez. Şartnameyi değiştirmek 30 saniye, /implement çalıştırmak 10 saniyedir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Beware the fatal team rationalization: 'Production is down, let us hot-patch the code now and backfill the markdown spec on Monday.' That Monday never arrives. Updating the spec takes 30 seconds; letting it rot costs weeks of technical bankruptcy.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Yeni özellik, değişiklik ve bug: Hepsi aslında use case şartnamesine yapılan bir değişikliktir.
- Kodu şartnameden geçirmeden doğrudan düzeltmek yasaktır; yan kanallar şartnameyi çürütür.
- Şartname güncellendiğinde aynı /implement ve test komutları çalışarak kodu yerinde senkronize eder.

🎯 Key Architectural Takeaways

- Features, change requests, and bugs are all the exact same thing: a delta to a use case.
- Bypassing the specification via direct code hotfixes triggers immediate specification rot.
- When the spec changes, running the same commands reconciles existing code in place cleanly.

BÖLÜM 3 / MODULE 3

Temel Esaslar: Aktörler, Deniz Seviyesi Hedefler & Paydaş Sözleşmeleri

Core Fundamentals: Actors, Sea-Level Goals & Contracts

Birincil ve ikincil aktörler, Alistair Cockburn'ün 3 adımlı adli testi (Etkileşim, Doğrulama, Durum Değişimi), Hedef İrtifaları (Bulut, Deniz Seviyesi, Balık), Kapsam sınırları ve In/Out listesi disiplini, 'Manage X' birleştir-sonra-böl kuralı ve kullanıcı dilinde isimlendirme.

Primary and secondary actors, Cockburn's three-kind step test (Interaction, Validation, State Change), Goal Altitude tiers (Cloud, Sea Level, Fish), Scope boundaries via the In/Out list, the 'Manage X' aggregate-then-split rule, and naming in user-centric language.

13

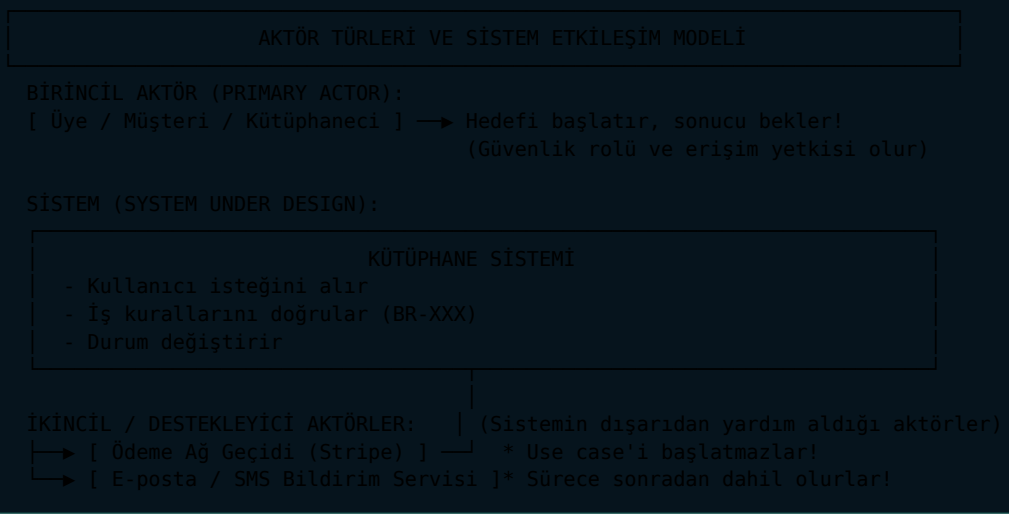
BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Aktörü Doğru Tanımlamak: Rol Odaklılık, Birincil Aktörler & İkincil Destekleyici Sistemler

Defining the Actor: Roles, Primary Actors & Supporting Systems

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir film çekimini düşünün: Birincil aktör filmin başrol oyuncusudur (Örn: James Bond). Olayları o başlatır, bombayı o imha eder, alkışı o toplar. İkincil aktör ise Bond'a telsizden bilgi veren veya arabasını getiren figüranlardır (Q veya araba kiralama şirketi). Eğer senaryoda başrolü arabayı kiralayan şirkete verirsen film saçma sapan bir hale gelir! Birincil aktör daima hedefin peşinden koşan kişidir!

🇬🇧 ENGLISH (ELI5)

In a spy thriller, the Primary Actor is James Bond. He initiates the mission, pursues the objective, and claims the victory. The Secondary Actors are the quartermaster (Q) who equips him with gadgets and the satellite network that relays coordinates. If you write the script from the perspective of the satellite instead of the secret agent, the narrative collapses into incoherence.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Aktör Tanımlamasının AIUP ve MAANG Mimarilerindeki Kritik Rolü: 1) İsim Değil Rol: Bir aktör asla 'Ahmet Bey' veya 'Muhasebe Departmanı' gibi kişisel veya departman bazlı tanımlanamaz; 'Üye' (Member), 'Kütüphaneci' (Librarian) gibi soyut roller olmalıdır. Çünkü bu roller doğrudan sistemin Güvenlik Modeline (RBAC - Role-Based Access Control) ve Spring Security rollerine dönüşür. AI kod üretirken `\@PreAuthorize("hasRole('MEMBER')")` annotasyonunu bu aktör tanımına göre koyar. 2) Birincil Aktör (Primary Actor): Senaryoyu tetikleyen ve use case'in hedefine ulaşmasından doğrudan fayda sağlayan aktördür. 3) İkincil / Destekleyici Aktörler (Secondary Actors): Senaryoyu başlatmazlar; sistem hedefe yürürken onların bir servisine ihtiyaç duyar (Ödeme Gateway'i, Fraud Kontrol Servisi vb.). Use case şablonunda 'Secondary Actors: Payment Gateway, Email Service' olarak açıkça listelenmelidir; aksi halde AI bu entegrasyonları mock'lamaz veya bağımlılıkları bağlamayı unutur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Actor semantics in enterprise systems: 1) Roles over Individuals: An actor must represent a systemic role ('Member', 'Underwriter'), never a persona or organizational hierarchy. This designation directly synthesizes the application's authorization boundary: an AI agent translates primary actor metadata into role-based security guards (`\@PreAuthorize`, JWT claims, and integration test credentials). 2) Primary Actor: Initiates the use case to satisfy a measurable goal. 3) Secondary / Supporting Actors: External systems or back-office actors enlisted by the system to complete sub-operations (e.g. Stripe, SendGrid, Fraud Detection APIs). Explicitly declaring secondary actors ensures downstream AI engines generate appropriate outbound ports and client interfaces within a Hexagonal architecture.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Aktör Rollerini Güvenlik ve Yetkilendirme Modeline Dönüştürme
package com.maang.aiup.actors;

public class ActorSecurityModel {
    // Şartnamede tanımlanan roller doğrudan Java 25 Enum/Record ile modellenir
    public enum SystemRole {
        MEMBER, LIBRARIAN, SYSTEM_ADMIN
    }

    public record SecurityContext(String userId, SystemRole role) {
        public void requireRole(SystemRole requiredRole) {
            if (this.role != requiredRole) {
                throw new SecurityException("Erişim reddedildi: Bu işlem için " + requiredRole + "
rolü zorunludur.");
            }
        }
    }

    // Birincil Aktör: Member
    public static void borrowBook(SecurityContext context, String bookId) {
        context.requireRole(SystemRole.MEMBER);
        System.out.println("Üye yetkisi doğrulandı, işlem başlıyor: " + bookId);
    }

    void main() {
        var memberCtx = new SecurityContext("user-42", SystemRole.MEMBER);
        borrowBook(memberCtx, "BOOK-1984");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Sistemin kendisini asla birincil aktör yapmayın! 'Zamanlanmış Görev (Cron)' veya 'Gece Batch İşlemi' gibi durumlarda aktör 'Sistem' değil, 'Zamanlayıcı Servisi' (Time / Scheduler) olmalıdır.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never designate the system itself as the primary actor. For automated background jobs, the primary actor is 'Time' or the 'Scheduler', which triggers the use case at a designated temporal threshold.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Aktör bir kişi veya departman değil, bir roldür; bu rol doğrudan güvenlik modelini besler.
- Birincil aktör hedefi başlatır ve faydayı alır; ikincil aktör ise sisteme destek verir.
- İkincil aktörleri açıkça listelemek, AI'ın dış servis entegrasyonlarını doğru modellemesini sağlar.

🎯 Key Architectural Takeaways

- An actor is a role, not a person or department; it directly feeds the security model.
- Primary actors pursue goals; secondary actors are external entities enlisted for support.
- Listing secondary actors forces AI to generate clean outbound API client boundaries.

14

BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Üç Adımlı Adli Test: Etkileşim, Doğrulama & Dahili Durum Değişimi

The Three-Step Test: Interaction, Validation & Internal State Change

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              COCKBURN'ÜN 3 ADIMLI USE CASE ADLİ TESTİ    │
└──────────────────────────────────────────────────────────┘

Bir use case senaryosundaki HER ADIM mutlaka şu 3 türden biri olmak ZORUNDADIR:

TÜR 1: ETKİLEŞİM (INTERACTION)
└──► İki aktör arasında bilgi veya talep aktarımı.
     Örnek: "Üye ödünç almak istediği kitabı seçer ve talepte bulunur."

TÜR 2: DOĞRULAMA (VALIDATION)
└──► Sistemin bir paydaş çıkarını korumak için yaptığı kural kontrolü.
     Örnek: "Sistem üyenin gecikmiş borcu olmadığını doğrular (BR-006)."

TÜR 3: DURUM DEĞİŞİMİ (STATE CHANGE)
└──► Sistemin kalıcı belleğinde meydana gelen iş değişikliği.
     Örnek: "Sistem ödünç kaydını 28 gün vadeli olarak veritabanına işler."

⚠ ALARM: Eğer bir senaryoda HİÇ doğrulama ve durum değişimi yoksa,
         o bir use case değil; gizli bir 'Ekran Gezintisi'dir (UI Walkthrough)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir dükkâna gidip alışveriş yapmayı düşünün: 1) Etkileşim: 'Bana şu gömleği ver' dersin (Konuşma). 2) Doğrulama: Satıcı cüzdanındaki parayı kontrol eder, sahte mi değil mi bakar (Kontrol). 3) Durum Değişimi: Parayı kasaya koyar, gömleği torbaya atar (Durum değişti!). Eğer satıcı sadece seninle sohbet edip gömleğin rengini övseydi ama ne paraya baksaydı ne de gömleği verseydi, ortada bir ticaret olmazdı! Yazılım da böyledir: Doğrulama ve durum değişimi yoksa orada gerçek bir iş yoktur!

🇬🇧 ENGLISH (ELI5)

Buying shoes involves three distinct actions: 1) Interaction: You ask the clerk for size 42. 2) Validation: The clerk inspects inventory to verify stock and checks your credit card for sufficient funds. 3) State Change: The card is charged, and the shoes are registered as sold. If the interaction consisted purely of browsing shoe colors without inventory verification or payment capture, no commercial transaction occurred. Software without validations and state changes is merely idle screen chatter.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Üç Adımlı Testin Mühendislik ve AI Açısından Önemi: Cockburn'ün en derin keşfi şudur: 'Kullanım senaryoları klavye başındaki ekran akışları değildir; paydaşlar arasındaki kontratın davranışsal koruyucusudur.' Sistem, klavye başında oturmayan paydaşların da çıkarlarını korumak zorundadır (örneğin şirketin parasını kaptırmaması, denetçinin log istemesi, veritabanının tutarlı kalması). Bir use case sadece Etkileşim adımlarından ('Kullanıcı butona tıklar, sistem ekranı açar') ibaretse, geliştiriciye ve AI'a hiçbir iş mantığı verilmemiş demektir! Doğrulamalar (Validation) yazılmazsa AI hiçbir güvenlik kontrolü ve kural motoru üretmez. Cockburn'ün vaka analizi: Bir şirket paydaş çıkarlarını ancak canlıya çıktıktan sonra listelediğinde, bu listenin ilk yıl gelen tüm 'Hata ve Değişiklik Talepleri' ile birebir aynı olduğu görüldü. AI çağında bu 1 yıllık gecikme ilk öğleden sonraya sıkışır: Yazılmayan her doğrulama, üretilmeyen bir güvenlik açığıdır!

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Cockburn's Three-Kind Step Axiom: Every sentence in a scenario must classify into one of three structural types: 1) Interaction between two actors (transmitting intent or payloads). 2) Validation protecting a stakeholder's interest ('System validates that member has no overdue loans'). 3) Internal State Change ('System records loan and decrements inventory'). The three-kind test is the supreme defense against pseudo-specifications that disguise UI navigation as functional requirements. When a spec omits validations and state mutations, an AI agent generates hollow presentation mockups devoid of transactional integrity. Cockburn demonstrated that omitted stakeholder interests accurately forecast a system's entire first-year change request backlog. In the AI era, this backlog manifests instantly on day one.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Üç Adımlı Testin Mimarideki Birebir Karşılığı
package com.maang.aiup.stepkinds;

public class ThreeStepTestService {
    // Domain State
    public record BookCopy(String id, boolean isAvailable) {}
    public record LoanRecord(String loanId, String memberId, String copyId, java.time.LocalDate
dueDate) {}

    public LoanRecord executeBorrowProcess(String memberId, BookCopy copy) {
        // TÜR 1: Etkileşim (Girdi parametreleri olarak geldi: memberId ve copy)

        // TÜR 2: DOĞRULAMA (Validation - Stakeholder Interest)
        if (!copy.isAvailable()) {
            throw new IllegalStateException("BR-004: Kitap kopyası ödünç verilebilir durumda değil!");
        }

        // TÜR 3: DURUM DEĞİŞİMİ (Internal State Change)
        var updatedCopy = new BookCopy(copy.id(), false); // Stok azaldı
        var loan = new LoanRecord("L-99", memberId, updatedCopy.id(),
java.time.LocalDate.now().plusDays(28));

        System.out.println("✅ 3 Adımlı Test Tamamlandı: " + loan);
        return loan;
    }

    void main() {
        var service = new ThreeStepTestService();
        service.executeBorrowProcess("MEM-1", new BookCopy("CPY-1", true));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir use case adımı 'Kullanıcı Kaydet butonuna basar' dedikten hemen sonra 'Sistem kullanıcıya ana sayfayı gösterir' diyorsa DURUN! Aradaki doğrulama ve veri tabanına kayıt (durum değişimi) adımları unutulmuştur. AI bu adımları görmezse hiçbir şey kaydetmeyen bomboş bir sayfa üretir!

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

If a step states 'User clicks Submit' followed immediately by 'System displays Success screen', HALT! The validation step and the state mutation step were completely omitted. The AI will generate a hollow screen transition that persists nothing.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Her senaryo adımı 3 türden biri olmalıdır: Etkileşim, Doğrulama veya Durum Değişimi.
- Sadece etkileşim içeren senaryolar use case değildir; ekran gezintisidir ve iş kurallarını kaybeder.
- Yazılmayan her doğrulama adımı, yapay zekanın üretmeyi unuttuğu bir güvenlik zafiyetidir.

🎯 Key Architectural Takeaways

- Every step must strictly be an Interaction, a Validation, or an Internal State Change.
- Specs containing only interactions are UI walkthroughs that forfeit all business rules.
- Every unstated validation step becomes a vulnerability omitted by the AI generator.

15

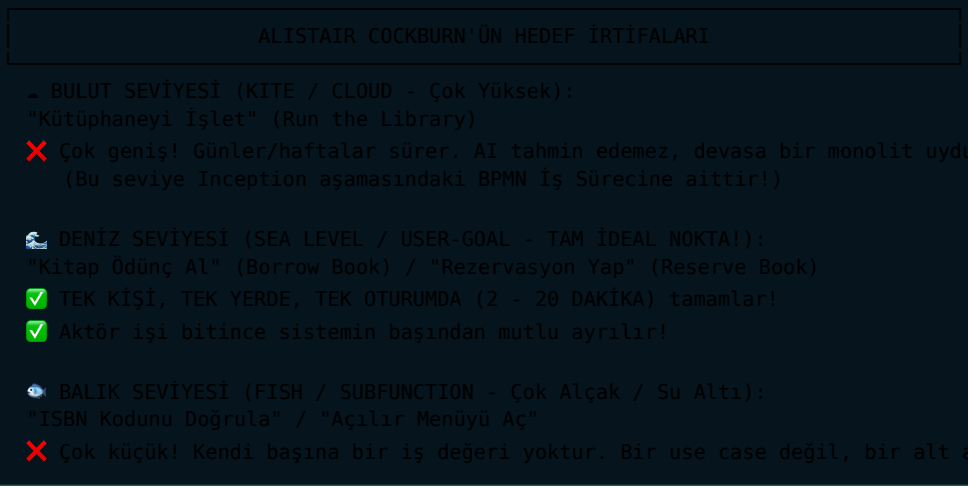
BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Hedef İrtifası: Neden Deniz Seviyesinde (Sea Level) Yazıyoruz? (Bulut ve Balık Tuzakları)

Goal Altitude: Why We Write at Sea Level (Avoiding Kite & Fish Traps)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restorana gittiğini hayal et: 1) Bulut Seviyesi: 'Beni doyur ve ömrümün sonuna kadar besle' demek gibidir (Çok genel, garson ne yapacağını şaşırır). 2) Balık Seviyesi: 'Garsona çatalı uzatmasını söyle, çatalı sol eline al, ağzını 3 santim aç' demektir (Gereksiz detayda boğulursun). 3) Deniz Seviyesi: 'Bir porsiyon mantı getir' demektir! Siparişini verirsin, yersin, hesabı öder mutlu bir şekilde masadan kalkarsın (20 dakika, tek oturum!). Use case'ler daima deniz seviyesinde yazılır!

🇬🇧 ENGLISH (ELI5)

At a restaurant: 1) Cloud Level: Demanding 'Nourish my biological existence for the next forty years.' (Absurdly broad, paralyzes the waiter). 2) Fish Level: 'Extend index finger, wrap around fork tine, elevate elbow 15 degrees.' (Absurdly granular, micromanages physical mechanics). 3) Sea Level: 'Order the ribeye steak.' You order, dine, settle the bill, and leave satisfied in 45 minutes. Use cases exist strictly at sea level.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Hedef Seviyeleri (Goal Levels) ve Mühendislik Dengesi: Cockburn use case yazımındaki en zor zanaatin doğru irtifayı bulmak olduğunu söyler: 1) Su Altı / Balık Seviyesi (Subfunctions): 'Kullanıcı Adı Kontrolü' veya 'Kredi Kartı Format Doğrulama' birer use case değildir; bir ana hedefin içindeki basit birer adımdır. Eğer bunları use case yaparsanız, sistem 500 tane anlamsız, bağlamsız, başı sonu olmayan dosya parçasına bölünür (Fragment Monolith). AI bunlardan kod üretirken bağımlılıkları bağlayamaz. 2) Bulut Seviyesi (Summary Goals): 'Hastaneyi Yönet' veya 'Tedarik Zincirini Çalıştır'. Bunlar birden fazla kişinin günlerce süren oturumlarını kapsar. Bu süreçler use case değil, BPMN iş süreçleridir. 3) Deniz Seviyesi Testi: Bir hedefin deniz seviyesinde olduğunu anlamanın iki turnusol kağıdı vardır: a) 'Neden?' sorusu bir üst seviyeyi (bulut), 'Nasıl?' sorusu bir alt seviyeyi (balık) verir. b) Tek bir aktör, tek bir yerde, tek bir oturumda (2 ila 20 dakika içinde) bitirip sistemin başından mutlu ayrılıyor mu? AIUP'ta tüm use case'ler istisnasız deniz seviyesinde yazılır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Goal Altitude Taxonomy: 1) Fish Level (Subfunction): Underwater activities like 'Validate ISBN' or 'Query Customer Record'. They deliver zero standalone business utility. Elevating them to use case artifacts fractures the architecture into hundreds of disconnected fragments that confuse LLMs. 2) Cloud Level (Summary Goal): Multi-day, multi-party business workflows ('Manage Inventory', 'Run Logistics'). These belong in Inception BPMN diagrams, not functional specs. 3) Sea Level (User-Goal Level): The sweet spot. Cockburn's litmus test: executed by one person, in one location, in one sitting (2 to 20 minutes), concluding with the actor walking away with completed business value. LLMs generate cohesive services and comprehensive test harnesses when bounded strictly to sea-level scope.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Deniz Seviyesi Use Case Sınırı (Ne Çok Büyük Ne Çok Küçük)
package com.maang.aiup.altitude;

public class SeaLevelUseCaseDemo {
    // ❌ BALIK SEVİYESİ (Ayrı bir use case DEĞİL, sadece bir metod/kuraldır):
    public static boolean validateIsbn(String isbn) {
        return isbn != null && isbn.startsWith("978");
    }

    // ✅ DENİZ SEVİYESİ (Gerçek Use Case: Tek Oturum, Tek Hedef, Tam Değer):
    public record BorrowBookRequest(String memberId, String isbn) {}
    public record BorrowBookResponse(boolean success, String receiptId) {}

    public static class BorrowBookService {
        public BorrowBookResponse execute(BorrowBookRequest request) {
            // Balık seviyesi kontroller içeride çağrılır:
            if (!validateIsbn(request.isbn())) {
                throw new IllegalArgumentException("Geçersiz ISBN!");
            }
            // Deniz seviyesi iş tamamlanır:
            return new BorrowBookResponse(true, "RCPT-" + System.currentTimeMillis());
        }
    }

    void main() {
        var service = new BorrowBookService();
        var resp = service.execute(new BorrowBookRequest("M-1", "978-0134685991"));
        System.out.println("Deniz seviyesi use case tamamlandı: " + resp);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

'Müşteri Ara' bir use case değildir! Müşteri ara butonuna basıp ekranı kapatıp evine giden kimse mutlu olmaz. Müşteri 'Siparişi İptal Etmek' için aranır; asıl use case sipariş iptalidir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

'Search Customer' is rarely a sea-level use case. Nobody logs into a banking app simply to gaze at a customer profile and log out satisfied. They search in order to transfer funds or update an address. The user goal is the transfer.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Use case'ler daima deniz seviyesinde yazılır: Tek kişi, tek yerde, tek oturumda (2-20 dk).
- Balık seviyesi hedefler (Validate ISBN) use case değil, bir use case'in içindeki adımlardır.
- Bulut seviyesi hedefler (Run Company) use case değil, Inception aşamasındaki BPMN süreçleridir.

🎯 Key Architectural Takeaways

- Use cases must be drafted at sea level: one person, in one place, in one sitting (2-20 min).
- Fish-level tasks are implementation steps or validations, not independent use cases.
- Cloud-level workflows belong in high-level BPMN business process diagrams.

16

BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Kapsam Denetimi & In/Out Listesi: Kapsam Kaymasını ve AI Halüsinasyonunu Engelleme

Scope Control & The In/Out List: Preventing Scope Creep & AI Hallucinations

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              COCKBURN'ÜN IN / OUT LİSTESİ İLE KAPSAM KİLİTLEME │
└──────────────────────────────────────────────────────────────┘

SİSTEM: KÜTÜPHANE YÖNETİM SİSTEMİ (LIBRARY SYSTEM)

KAPSAM İÇİNDE (IN)                    | KAPSAM DIŞINDA (OUT) - ASLA ÜRETME!
--------------------------------------|-----------------------------------
- Katalogda kitap arama               | - Çevrimiçi kredi kartı ile ödeme
- Kitap ödünç alma & iade             | - Sesli kitap (Audiobook) streaming
- Gecikme cezası hesaplama            | - Kargo ile eve kitap teslimatı
- E-posta ile iade hatırlatması       | - Kafeterya entegrasyonu ve kahve siparişi

🛡️ GÖREVİ: AI ajanının 'Out' kolonundaki özellikleri kafasına göre
          koda eklemesini (Scope Hallucination) %100 engeller!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir marangoza ‘Bana bir çalışma masası yap’ dediğinde, marangoz kendi kendine gaza gelip masanın altına mini buzdolabı, üstüne disko ışığı ve kenarına masaj aleti takmaya kalkabilir. Cockburn’ün In/Out listesi masanın üstüne koyduğun kırmızı listedir: ‘Masa yapılacak: EVET. Çekmece olacak: EVET. Buzdolabı: KESİNLİKLE HAYIR! Disko topu: HAYIR!’ Marangoz (veya AI) bu kırmızı listeyi görünce haddini bilir, sadece senden isteneni yapar!

🇬🇧 ENGLISH (ELI5)

Commissioning a carpenter to build a desk without boundary limits might inspire them to mount a mini-fridge, cup warmer, and neon strip lights to the wood. An In/Out list is the contract pinned to their workbench: ‘Oak tabletop: IN. Two pencil drawers: IN. Refrigeration unit: OUT. Ambient LED lighting: OUT.’ AI agents possess expansive latent knowledge; without an explicit OUT boundary, they aggressively generate out-of-scope fluff.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Tasarım Kapsamı (Design Scope) ve In/Out Disiplini: Cockburn'ün geliştirdiği 'In/Out Listesi', yazılım tarihindeki en ucuz ve en etkili kapsam yönetimi aracıdır. Masaya oturup 10 dakika içinde neyin sistemin içinde neyin kesinlikle dışında olduğunu bir tabloya yazmak, haftalar süren kapsam kaymasını (Scope Creep) önler. AI Unified Process'te bu liste iki kat daha hayatidir: LLM'ler eğitim verilerindeki kalıpları tekrarlamaya meyillidir. Örneğin bir 'Kütüphane Ödünç Alma' sistemi tasarlarken, AI internette gördüğü Amazon veya e-ticaret sitelerinden esinlenerek 'Kredi Kartıyla Ödeme' veya 'Ev Adresine Kargo Takibi' gibi devasa modülleri koda eklemeye kalkar. Şartnamenin ve vizyon dökümanının (`docs/vision.md`) başındaki In/Out tablosu, AI ajanının bu gereksiz özellikleri üretmesini kesin olarak bloke eder. Sistem bir kara kutu (black box) olarak sınırlandırılır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Architectural Scope Boundaries via the In/Out List: Design scope defines the black-box perimeter of the target system. Cockburn's In/Out list remains the most cost-effective governance instrument in engineering: spending 10 minutes debating boundary edge cases prevents months of scope creep. In the AI Unified Process, the In/Out list acts as a hard negative-constraint prompt for autonomous coders. LLMs inherently suffer from feature inflation: prompt an agent to build an inventory tool, and it instinctively hallucinates Stripe checkout flows or third-party courier dispatchers because modern public repos frequently couple them. Explicitly documenting the OUT column creates a structural barrier against generative overreach.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kapsam Dışına Çıkmayı Engelleyen Sınırlandırılmış Servis Modeli
package com.maang.aiup.scope;

public class BoundedLibraryService {
    public record BorrowRequest(String memberId, String bookId) {}

    // IN Kapsamı: Sadece kütüphane içi ödünç alma işlemi
    public void processBorrow(BorrowRequest req) {
        System.out.println("IN Kapsamı: Kitap ödünç işlemi yapıldı: " + req.bookId());
    }

    // OUT Kapsamı Uyarısı: AI bu metodları uydurmaya kalkarsa derleme/mimari testleri engeller
    // public void processOnlineCreditCardPayment(...) -> YASAK (docs/vision.md OUT listesi!)
    // public void dispatchHomeDeliveryCargo(...)     -> YASAK (docs/vision.md OUT listesi!)

    void main() {
        var service = new BoundedLibraryService();
        service.processBorrow(new BorrowRequest("M-10", "BK-99"));
        System.out.println("✅ Kapsam sınırları başarıyla korundu.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

In/Out listesi yaparken OUT kolonunu boş bırakmayın! Bir sistemde neyin yapılmayacağını söylemek, neyin yapılacağını söylemek kadar önemlidir. AI 'yapılmayacaklar' listesinden güç alır.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never leave the OUT column empty. Declaring what the system will NOT do is just as critical as defining its features. Negative constraints narrow the AI's generation space exponentially.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- In/Out listesi 10 dakikada hazırlanır ve haftalar sürecek kapsam kaymasını engeller.
- AI geniş eğitim verisi yüzünden kapsam dışı e-ticaret/ödeme özellikleri uydurmaya meyillidir.
- OUT kolonu, yapay zekanın tasarım uzayını daraltan en güçlü negatif kısıttır.

🎯 Key Architectural Takeaways

- An In/Out list takes 10 minutes to draft and prevents weeks of costly scope drift.
- AI is prone to hallucinating out-of-scope enterprise features from its training data.
- The OUT column provides the strongest negative prompt boundary for code generation.

17

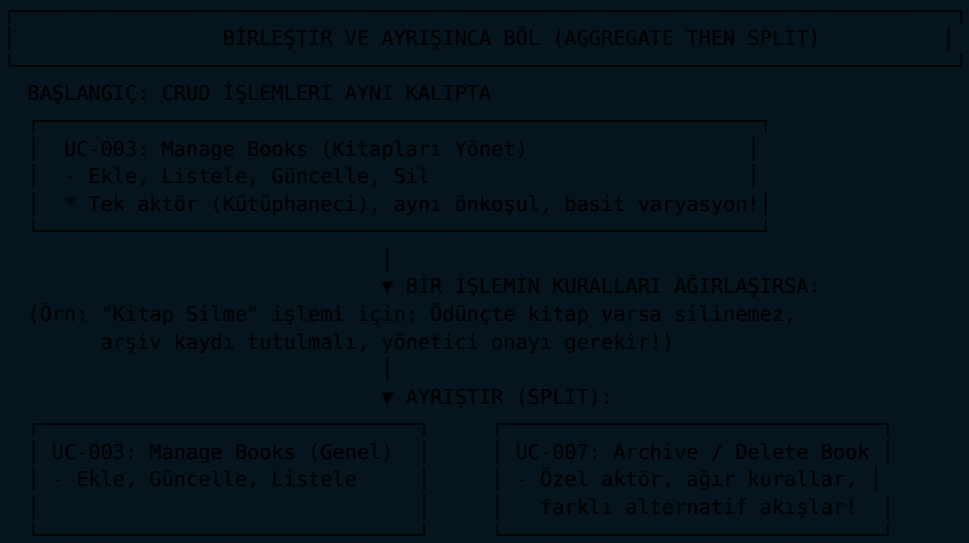
BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Önce Birleştir, Akışlar Ayrışınca Böl: 'Manage X' Kuralı ve CRUD Birleştirmesi

Aggregate First, Split When Flows Diverge: The 'Manage X' Rule

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda 'İçecekler' menüsü vardır; çay, kahve, su ve kola hepsi aynı sayfadadır çünkü hepsi bardağa dökülüp servis edilir (Basit CRUD). Ama müşteri 'Özel İsviçre Sıcak Çikolatası' istediğinde, aşçı özel tencere çıkarır, 15 dakika çikolata eritir, içine marshmallow koyar. İşte o zaman o içecek genel menüden çıkarılıp 'Özel Tatlılar' başlığına taşınır! Başta her şeyi bir sepete koy, kuralları karmaşıklaşan şeyi kendi özel kutusuna ayır!

🇬🇧 ENGLISH (ELI5)

On a café menu, standard drinks (tea, drip coffee, water) share a single section because they require identical preparation steps. But when the café introduces traditional siphon-brewed pour-overs involving specialized burners, timing scales, and origin certifications, that beverage earns its own dedicated specialty menu card. Start standard CRUD operations in one consolidated container; split them only when operational rules diverge.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Use Case Sayısı Optimizasyonu ve 'Manage X' Kuralı: Yeni başlayan mimarların en büyük hatası, 4 tabloluk bir veritabanı için 25 tane use case yazmaktır (Create Book, Read Book, Update Book, Delete Book, Search Book…). Bu durum şartname enflasyonuna yol açar ve AIUP'un 'küçük ve okunabilir model' felsefesini yok eder. Cockburn ve Martinelli'nin altın kuralı: 'Aggregate first, split when the flows diverge' (Önce birleştir, akışlar ayrışınca böl). Bir varlığın CRUD işlemleri aynı aktör (Kütüphaneci) tarafından yapılıyor ve temel bir başarı akışını paylaşıyorsa, başlangıçta tek bir 'UC-003: Manage Books' use case'i olarak yazılır. Ancak bu işlemlerden biri (örneğin Block Customer veya Archive Book) kendine has ağır iş kuralları, farklı önkoşullar veya farklı alternatif akışlar geliştirmeye başladığı an, o operasyon derhal 'UC-008: Block Customer' olarak bağımsız bir use case'e dönüştürülür. Bu sayede küçük bir mikroservis 40 değil, sadece 4 ila 8 use case ile kusursuz şekilde modellenir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Managing Use Case Cardinality via Aggregate-then-Split: Novice analysts routinely generate twenty distinct use cases for basic entity management (Create User, Edit User, View User, Delete User, Search User), creating specification bloat that overwhelms engineering reviews. The Martinelli-Cockburn heuristic dictates: start every CRUD resource as a single consolidated 'Manage X' use case. The aggregate holds cleanly as long as the operations share a single primary actor, identical preconditions, and standard data persistence steps. However, the precise moment an operation diverges—e.g. 'Block Account' requiring fraud checks, compliance event logging, and secondary approval flows—it must be excised into an action-level use case. This discipline bounds standard microservices to 4 to 8 use cases total.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Manage X' Çoklu İşlem Yöneticisinin Ayrışma Noktası
package com.maang.aiup.aggregate;

public class ManageBooksService {
    public record Book(String id, String title, String author, boolean isArchived) {}

    // Standart CRUD: Tek bir servis/use case içinde birleşik yaşar
    public void createBook(Book b) { System.out.println("Kitap eklendi: " + b.title()); }
    public void updateBook(Book b) { System.out.println("Kitap güncellendi: " + b.title()); }

    // AYRIŞMA NOKTASI (Ayrı bir Use Case adayı: ArchiveBookUseCase):
    // Ağır iş kuralları: Aktif ödünç kontrolü, denetim kaydı, bildirim!
    public void archiveBook(String bookId, int activeLoanCount) {
        if (activeLoanCount > 0) {
            throw new IllegalStateException("BR-030: Ödünçte kopyası bulunan kitap arşivlenemez!");
        }
        System.out.println("Kitap arşivlendi ve denetim logu üretildi: " + bookId);
    }

    void main() {
        var service = new ManageBooksService();
        service.createBook(new Book("B1", "Clean Code", "Robert Martin", false));
        service.archiveBook("B1", 0);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir aktör sisteme asla 'Müşterileri yönetmek' amacıyla oturmaz. Birleştirilmiş 'Manage Customers' use case'i sadece benzer CRUD operasyonlarını bir arada tutan bir başlangıç konteyneridir; kurallar ağırlaştığında bölmekten çekinmeyin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

An end-user never logs in with the generic goal to 'manage customers'; they come to add or block someone. The 'Manage X' pattern is a structural convenience for uniform CRUD. Split it immediately when business logic diverges.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Başlangıçta CRUD işlemlerini 'Manage X' altında tek bir use case olarak toplayın.
- Bir operasyon kendi özel iş kurallarını ve hata akışlarını geliştirdiğinde hemen ayrı bir use case'e bölün.
- Bu kural sayesinde tipik bir mikroservis 40 dağınık use case yerine 4-8 güçlü use case ile modellenir.

🎯 Key Architectural Takeaways

- Start CRUD operations consolidated under a single 'Manage X' use case.
- Split out an action-level use case the moment preconditions or business rules diverge.
- This aggregation keeps a microservice clean at 4 to 8 comprehensive use cases instead of forty.

18

BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Kullanıcının Diliyle İsimlendirme: Neden 'Kitap Ödünç Al', 'Ödünç Kaydı Oluştur'dan İyidir?

Naming in the User's Language: Why 'Borrow Book' Beats 'Create Loan Record'

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                İSİMLENDİRME FELSEFESİ: KULLANICI DİLİ VS TEKNİK JARGON

❌ GELİŞTİRİCİ DİLİ (TEKNİK / VERİTABANI ODAKLI):
- "Create Loan Record" (Ödünç Kaydı Oluştur)
- "Insert User Entity" (Kullanıcı Varlığı Ekle)
- "Update Account Balance Table" (Bakiye Tablosunu Güncelle)
💥 Hata: Paydaşın kafasını karıştırır, sistemi veritabanı CRUD operasyonuna indirger!

✅ KULLANICI DİLİ (İŞ HEDEFİ ODAKLI):
- "Borrow Book" (Kitap Ödünç Al)
- "Register Member" (Üye Kaydet)
- "Transfer Funds" (Para Transfer Et)
🎯 Başarı: Aktif fiil öbeğidir, gerçek dünya hedefini anlatır, her 3 okuyucu da anlar!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kütüphaneye gittiğinde görevliye 'Affedersiniz, veritabanınızdaki ödünç tablosuna yeni bir satır ekleyip benim ID'mi o satırla ilişkilendirmek istiyorum' demezsin! Görevliye sadece 'Şu kitabı ödünç almak istiyorum' dersin. Bir use case'in adı, kullanıcının ağzından çıkan o doğal cümle olmalıdır. Teknik kelimeleri arkadaki motora bırak; tabelaya herkesin anladığı ismi yaz!

🇬🇧 ENGLISH (ELI5)

When visiting a library, you never announce to the desk clerk: 'Greetings, I wish to initiate an INSERT transaction on your LOANS table linking my foreign key to this asset ID.' You say: 'I would like to borrow this book.' The title of a use case must always match the natural words spoken by the human pursuing the goal. Keep technical jargon inside the engine; keep the billboard clean.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İsimlendirme Semantiği ve Arayüz Olarak İsim Kuralı: Cockburn ve Martinelli use case isimlerinin yalnızca birer başlık değil, sistemin 'dış arayüzü' (interface) olduğunu vurgular: 1) Aktif Fiil Öbeği (Active Verb Phrase): İsim daima bir eylemle başlamalıdır: 'Borrow Book', 'Enroll in Course', 'Checkout Cart'. 'Kitap İşlemleri' veya 'Kullanıcı Modülü' gibi isimler use case olamaz çünkü bir hedef belirtmez. 2) Kullanıcı Dili (User Domain Language): 'Create Loan Record' ismi bir hedefin değil, veritabanındaki bir yazma operasyonunun adıdır. Eğer bir use case'i bir hedef olarak adlandıramıyorsanız, henüz elinizde bir use case yok demektir! 3) Birebir Eşleşme (Verbatim Interface): Use case diyagramındaki (PlantUML) isim ile şartname dosyasının adı (`UC-002-borrow-book.md`) ve dökümanın içindeki `# Use Case: Borrow Book` başlığı noktası virgülüne kadar aynı olmalıdır. Bu kural downstream çalışan kod üretim araçlarının (`/implement`) dökümanları hatasız bulmasını sağlar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Naming as an Architectural Interface: 1) Active Verb-Noun Syntax: Use case names must strictly form active verb phrases representing user objectives: 'Borrow Book', 'Register Account', 'Submit Expense Claim'. Generic labels like 'Book Processing' or 'User Maintenance' fail because they denote static categories rather than goal-directed behaviors. 2) User Vocabulary vs Schema Projections: 'Create Loan Record' describes an internal database mutation; 'Borrow Book' captures the real-world business transaction. If you cannot articulate the name as a tangible user goal, the requirements boundaries are premature. 3) Verbatim Traceability Interface: The label in the PlantUML diagram (`UC-002\nBorrow Book`), the filename (`UC-002-borrow-book.md`), and the title header must match identically. Downstream AI tools rely on strict lexical matching to bridge architectural models with source code generation.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kullanıcı Dilini Doğrudan Kod Semantiğine Taşıma (Ubiquitous Language)
package com.maang.aiup.naming;

public class DomainNamingDemo {
    // ❌ KÖTÜ TEKNİK İSİMLENDİRME:
    // public class LoanRecordCreator { public void insertRow(...) {} }

    // ✅ TEMİZ VE KULLANICI ODAKLI İSİMLENDİRME (Use Case ile Birebir):
    public static class BorrowBookUseCase {
        public record Command(String memberId, String bookIsbn) {}
        public record Result(String loanId, String dueDate) {}

        public Result borrowBook(Command cmd) {
            // Kod şartnamenin dilini konuşur!
            System.out.printf("Kitap ödünç alındı: Üye %s -> Kitap %s%n", cmd.memberId(),
cmd.bookIsbn());
            return new Result("L-101", "2026-11-17");
        }
    }

    void main() {
        var uc = new BorrowBookUseCase();
        uc.borrowBook(new BorrowBookUseCase.Command("MEM-9", "ISBN-456"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Use case adında 'Veritabanı', 'Kayıt', 'Tablo', 'API', 'Servis' gibi yazılımcı kelimeleri geçiyorsa o isim yanlıştır. Hemen o kelimeleri silin ve kullanıcının gerçek hayattaki eylemini yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

If words like 'Database', 'Record', 'Table', 'Entity', or 'API' appear in your use case title, the name is corrupted by implementation bias. Strip away technical nouns and state the actor's real-world goal.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Use case ismi daima kullanıcının dilinde kısa ve aktif bir fiil öbeğidir (Borrow Book).
- Veritabanı jargonu ('Create Loan Record') hedefi değil, uygulama mekanizmasını anlatır ve yanlıştır.
- Diyagramdaki isim, dosya adı ve başlık noktası virgülüne kadar birbirini tutmalıdır.

🎯 Key Architectural Takeaways

- Use case titles must be short active verb phrases in user language (e.g. Borrow Book).
- Database jargon describes technical mutations, failing to capture user goals.
- The diagram name, markdown file name, and header title must match verbatim.

BÖLÜM 4 / MODULE 4

Eser Zinciri (Artifact Chain) & PlantUML Mimarisi

The Artifact Chain & PlantUML Architecture

AI Unified Process'in 6 halkalı eser zinciri: Vizyondan koda tasarım uzayının daraltılması, Göbek ve Parmaklıklar (Hub-and-Spokes) modeli, PlantUML kullanım senaryosu diyagramı, <<include>> ve <<extend>> tuzakları, mekanik arayüz olarak kararlı kimlikler (FR, UC, BR, TC) ve /spec-review otomatik denetim mimarisi.

The 6-link AIUP artifact chain narrowing design space from vision to code, the Hub-and-Spokes architecture, PlantUML diagram as a table of contents, the truth about <<include>> and <<extend>>, identifiers as mechanical interfaces, and the two-pass /spec-review engine.

19

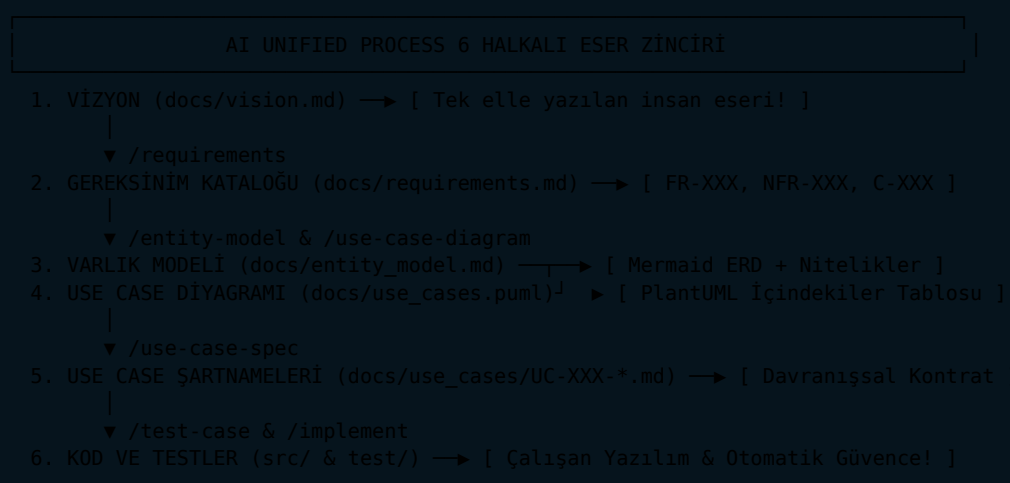
BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

6 Halkalı Eser Zinciri: Vizyondan Koda Tasarım Uzayını Daraltma

The 6-Link Artifact Chain: Narrowing Design Space from Vision to Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir roket fırlatırken hedefe ulaşmak için 6 aşamalı bir kilit sistemi kullanırsın: 1) Haritada Mars'ı gösterirsin (Vizyon). 2) İhtiyaçları yazarsın: 'Oksijen, yakıt, 3 astronot' (Gereksinimler). 3) Roketin parçalarını çizersin (Varlık Modeli). 4) Görev adımlarını listelersin (Diyagram). 5) Her düğmenin ne zaman basılacağını kâğıda dökersin (Use Case). 6) Motorları ateşlersin (Kod!). Eğer direkt 1. adımdan 6. adıma atlarsan roket havada patlar! Zincirdeki her halka roketin rotasını milimetrik daraltır.

🇬🇧 ENGLISH (ELI5)

Launching an interplanetary rocket requires a six-stage telemetry sequence: 1) Pointing at Mars on the celestial map (Vision). 2) Sizing fuel and payload specifications (Requirements Catalog). 3) Blueprinting module geometries (Entity Model). 4) Indexing operational phases (Diagram). 5) Authoring step-by-step launch protocols (Use Case Specs). 6) Igniting the thrusters (Code & Tests). Skipping directly from Vision to Ignition ensures catastrophic atmospheric explosion. Each artifact progressively restricts variance until implementation becomes deterministic.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Eser Zinciri (The Artifact Chain) ve Tasarım Uzayı Mekaniği: Bir LLM'e tek satırlık bir prompt verip 'Bana bir e-ticaret sistemi yaz' dediğinizde tasarım uzayı (design space) sonsuzdur; model milyonlarca farklı mimari, tablo ve API rotası seçebilir ve bu da determinizmi sıfırlar. AI Unified Process bu sorunu 'Aşamalı Kısıtlama Zinciri' ile çözer: 1) docs/vision.md: Projede tek elle yazılan insan eseridir. Sistemin sınırlarını, aktörlerini ve alan adlarını belirler. 2) docs/requirements.md: Niyeti FR (Kullanıcı Hikayeleri), NFR ve Kısıtlar (Constraints) olarak kodlar. 3) docs/entity_model.md: Domain terimlerini ve ilişkilerini tek bir yerde tanımlar. 4) docs/use_cases.puml: Sistemin tüm use case haritasını çizer. 5) docs/use_cases/UC-XXX.md: Davranışı adım adım kilitler. `/implement` komutu çalıştığında yapay zekanın elinde vizyon, veri modeli, gereksinim satırı ve use case zaten mevcuttur; bu sayede üretilen kod tesadüfi değil, matematiksel bir kesinlik taşır. Hiçbir tekil dökümanın kusursuz olması gerekmez; bu katmanlı yapı sistemi sağlam kılar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The mechanics of Design Space Constriction across the Artifact Chain: Prompting an LLM with 'Generate an e-commerce platform' presents an unconstrained, near-infinite degrees-of-freedom search space. The resulting architecture is chaotic, unprincipled, and un-reproducible. The AI Unified Process resolves this via a deterministic pipeline: each artifact progressively constricts the degrees of freedom for the next command. The vision document anchors boundary scope; the requirements catalog maps user stories to alphanumeric IDs; the entity model binds database nouns; the PlantUML diagram indexes the interaction portfolio; use case specifications define the step contracts. When `/implement` executes, the AI operates inside a tightly fenced sandbox. Reliable generation emerges from structural layering, not magical prompt engineering.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Eser Zincirinin Tamamını Yansıtan Temiz Mimari Yapısı
package com.maang.aiup.chain;

public class ArtifactChainDemonstration {
    // 1. docs/entity_model.md'den türeyen Domain Modeli
    public record BookCopyEntity(String copyId, String isbn, boolean isAvailable) {}

    // 2. docs/use_cases/UC-002-borrow-book.md'den türeyen Use Case Arayüzü
    public interface BorrowBookPort {
        void execute(String memberId, String copyId);
    }

    // 3. /implement tarafından üretilen temiz servis uygulaması
    public static class BorrowBookServiceImpl implements BorrowBookPort {
        @Override
        public void execute(String memberId, String copyId) {
            System.out.printf("Zincir tamamlandı: Üye %s -> Kopya %s başarıyla bağlandı.%n", memberId,
copyId);
        }
    }

    void main() {
        var service = new BorrowBookServiceImpl();
        service.execute("MEM-101", "CPY-99");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Zincirdeki halkaları atlayıp doğrudan /use-case-spec veya /implement çalıştırmaya kalkmayın! Elinde bir varlık modeli (entity model) olmayan AI, her use case'de aynı tablo için farklı kolon isimleri uydurur (örn: bir yerde memberId, diğerinde user_id) ve veritabanı kilitlenir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never short-circuit the artifact chain by executing `/implement` without an entity model. Deprived of a canonical entity dictionary, the AI will invent conflicting field names across specifications (e.g. `memberId` vs `user_id` vs `account_num`), triggering schema drift.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Eser zinciri 6 halkadan oluşur: Vizyon, Gereksinimler, Varlık Modeli, Diyagram, Şartnameler, Kod/Test.
- Tek elle yazılan döküman vizyondur; diğer tüm eserler komutlarla AI tarafından taslaklanır ve insan tarafından onaylanır.
- Her halka bir sonrakinin tasarım uzayını daraltarak rastgele kod üretimini deterministik kılar.

🎯 Key Architectural Takeaways

- The artifact chain spans 6 links: Vision, Requirements, Entity Model, Diagram, Specs, Code/Tests.
- Vision is the sole hand-written artifact; downstream artifacts are generated and human-refined.
- Each artifact narrows the design space, making AI code synthesis deterministic and predictable.

20

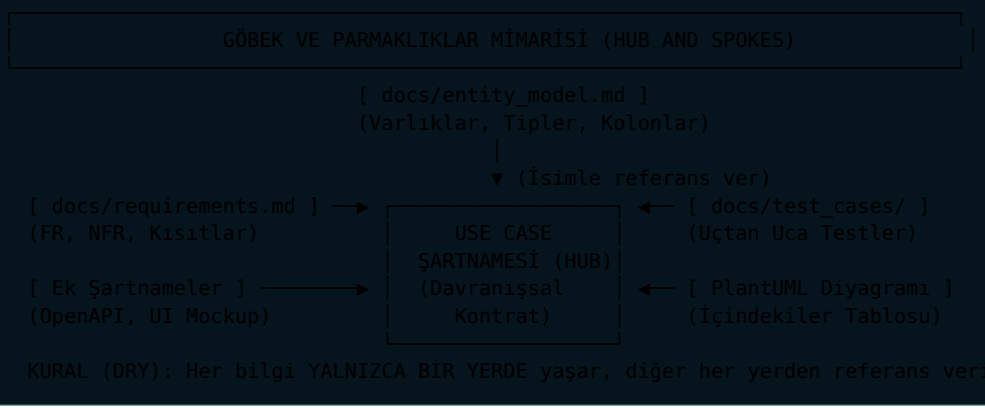
BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

Göbek ve Parmaklıklar Modeli (Hub & Spokes): Use Case Neden Sadece 'Bölüm 3'tür?

The Hub-and-Spokes Model: Why Use Cases Are Chapter 3 of the Requirements

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bisiklet tekerleğini düşünün: Ortada sağlam bir tekerlek göbeği (Hub) vardır ve bu göbekten dışarıya doğru onlarca metal parmaklık (Spokes) uzanır. Use Case bu tekerleğin tam merkezidir; ama tek başına gidemez! Bir parmaklık sözlüğe (Entity Model), diğer parmaklık hız sınırına (NFR), öbürü testlere gider. Eğer göbeğin içine lastiği, freni ve pedalları tıkıştırmaya çalışırsan tekerlek dönmez! Use case sadece 'kim ne yaptı' hikayesini anlatır; teknik detaylar parmaklıkların ucundaki kendi evlerinde yaşar.

🇬🇧 ENGLISH (ELI5)

Visualize a bicycle wheel: at the center sits the structural hub, with metallic spokes radiating outward to support the rim. The use case specification is the central hub. It governs behavioral flow, but delegates non-behavioral concerns out along its spokes: data definitions point to the Entity Model, performance budgets point to the NFR table, and user journeys point to Test Cases. Jamming database tables and UI wireframes directly into the hub fractures the wheel.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Cockburn'ün 'Bölüm 3' Metaforu ve DRY Prensibi: Cockburn, kullanım senaryolarının gereksinim dökümanının tamamı olmadığını, sadece 'Bölüm 3: Davranışsal Gereksinimler' olduğunu söyler. Diğer tüm konular use case'in dışındaki özel dosyalarda yaşar: 1) Veri Yapısı ve Alan Tipleri: `docs/entity_model.md` içinde tanımlanır; use case sadece 'Üye' veya 'Ödünç' ismini kullanır, asla içinde tablo şeması veya kolon listesi barındırmaz. 2) NFR ve Kısıtlar: `docs/requirements.md` içindeki NFR tablosunda yaşar; use case sadece `Requirements: [FR-012, NFR-002]` olarak linkler. 3) UI Mockup'ları ve OpenAPI: Ek dökümanlar olarak bağlanır. 4) Bilgi Tekrarı Yasağı (DRY): İnsan geliştiriciler için çift yazım sadece bir bakım zorluğuydu. Ancak yapay zekâ için çift yazım ÖLÜMCÜLDÜR! İki döküman zamanla birbirinden farklılaşır ve aralarında çelişki doğar. Bir AI ajanı çelişki gördüğünde bunu bir soru olarak sormaz; kafasına göre bir tarafı seçerek kod üretir. Bu yüzden her bilgi tam olarak TEK bir yerde tanımlanır ve sadece referans verilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Cockburn's 'Chapter 3' Doctrine and Requirements-Level DRY: Cockburn famously noted that use cases constitute merely Chapter 3 of a complete requirements document: the behavioral slice. Everything that resists the narrative structure is spun off into dedicated satellites. In the AI Unified Process, these spokes are mechanically formalized: 1) Data Schemas live in `docs/entity_model.md`. The spec cites domain nouns by business name and never inlines attribute tables. 2) Non-functional constraints reside in `docs/requirements.md` and are referenced via metadata tags. 3) Contract interfaces (OpenAPI, UI wireframes) are hyperlinked supplementary files. 4) The AI-Era DRY Imperative: For human teams, duplicated requirements created synchronization debt. For LLMs, duplication produces catastrophic failure: when two documents diverge by a single field, the AI resolves the contradiction silently, picking an arbitrary side. DRY is no longer merely a coding aesthetic; it is an existential requirements invariant.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Hub & Spokes Prensibine Göre Ayrıştırılmış Temiz Domain Mimarisi
package com.maang.aiup.hubspokes;

// SPOKE 1: Domain Modeli (docs/entity_model.md karşılığı)
record Member(String id, String name, String email) {}
record BookCopy(String copyId, String isbn) {}

// SPOKE 2: İş Kuralı Portu (docs/requirements.md karşılığı)
interface LoanPolicyGate {
    boolean canBorrow(Member member);
}

// HUB: Use Case İcra Sınıfı (docs/use_cases/UC-002 karşılığı - Sadece Akışı Yönetir!)
public class BorrowBookHubUseCase {
    private final LoanPolicyGate policyGate;

    public BorrowBookHubUseCase(LoanPolicyGate policyGate) {
        this.policyGate = policyGate;
    }

    public void execute(Member member, BookCopy copy) {
        // Hub sadece orkestrasyon yapar, veritabanı veya karmaşık kural detaylarını spokelara delege
eder
        if (!policyGate.canBorrow(member)) {
            throw new IllegalStateException("Ödünç alma kuralı ihlal edildi!");
        }
        System.out.printf("Hub tamamlandı: %s üyesine %s kitabı tahsis edildi.%n", member.name(),
copy.isbn());
    }

    void main() {
        var hub = new BorrowBookHubUseCase(_ -> true);
        hub.execute(new Member("M1", "Ada Lovelace", "ada@lovelace.org"), new BookCopy("C1",
"978-01"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Use case şartnamesinin içine SQL 'CREATE TABLE' komutları veya ekranın piksel genişliklerini ASLA yazmayın! Bu detaylar parmaklıkların ucundaki varlık modeline ve UI mockup dosyalarına aittir. Göbeği şişirmek AI'ın aklını karıştırır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never embed DDL statements (`CREATE TABLE`), column data types, or pixel coordinates directly inside a use case scenario. Inlining schema definitions bloats the hub and invites silent synchronization drift.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Use Case tekerleğin göbeğidir; sadece davranışsal akışı yönetir, detayları spokelara devreder.

• Veri yapıları entity_model.md'de, performans NFR tablosunda, testler ise test_cases klasöründe yaşar.

• DRY artık sadece kodlama değil, şartname kuralıdır: Çelişkili iki bilgi AI'ın sessizce yanlış kod yazmasına yol açar.

🎯 Key Architectural Takeaways

• The use case is the structural hub; it orchestrates behavior while delegating details to spokes.

• Data schemas live in the entity model, NFRs in the catalog, and journeys in test cases.

• DRY is a requirements imperative: contradictions across duplicated specs cause silent AI bugs.

21

BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

PlantUML Use Case Diyagramı: Sistemin İçindekiler Tablosu (Modelin Kendisi Değil!)

The PlantUML Use Case Diagram: Table of Contents, Not the Model

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
PLANTUML DİYAGRAMI VE VERBATIM DOSYA EŞLEŞMESİ

PlantUML Kodu (docs/use_cases.puml):
@startuml
actor "Member" as member
rectangle "Library System" {
  usecase "UC-001\nSearch Catalog" as UC001
  usecase "UC-002\nBorrow Book" as UC002
}
member --> UC001
member --> UC002
@enduml

BİREBİR DOSYA SİSTEMİ EŞLEŞMESİ (VERBATIM FILE SYSTEM):
docs/use_cases/
├── UC-001-search-catalog.md  ◄── (UC-001\nSearch Catalog ile noktası virgülüne aynı!)
└── UC-002-borrow-book.md     ◄── (UC-002\nBorrow Book ile noktası virgülüne aynı!)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kitaptaki 'İçindekiler' sayfasını düşünün: Sayfada sadece 'Bölüm 1: Giriş - Sayfa 15' yazar. İçindekiler sayfasına oturup tüm romanı sığdırmaya çalışmazsın; sadece hangi konunun nerede olduğunu tek bir bakışta gösterir. PlantUML diyagramı da yazılımın İçindekiler tablosudur! Yuvarlakların içine tüm detayları yazamazsın; yuvarlak sadece kapıdaki tabeladır, asıl oda arkasındaki markdown dosyasındadır!

🇬🇧 ENGLISH (ELI5)

A book's Table of Contents does not contain the complete novel; it indexes chapter titles and page numbers so readers can navigate the landscape at a glance. A PlantUML use case diagram plays the exact same role. The ellipses are not the system architecture; they are visual signposts. Attempting to cram complex algorithms into diagram bubbles ruins both the diagram and the specification.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

PlantUML Diyagramının Mühendislik Sınırları ve Kuralları: Cockburn use case diyagramlarının modelin kendisi olmadığını, sadece bir 'Büyük Resim / İçindekiler Tablosu' olduğunu ısrarla vurgulamıştır. `docs/use_cases.puml` dosyası downstream çalışan AI araçları için bir haritadır. Bu haritanın kusursuz çalışması için Martinelli 4 katı kural koyar: 1) İki Satırlı Etiket: Her use case yuvarlağı ID ve İsmi iki satırda taşımalıdır: `UC-001\nSearch Catalog`. 2) Gerçek Sistem İsmi: `rectangle` etiketinde soyut isimler değil, projenin gerçek sistem adı yazmalıdır (`rectangle "Library System"`). 3) Her Aktör Bağlı Olmalıdır: Diyagramdaki her aktör en az bir use case'e, her use case de en az bir aktöre bağlanmalıdır; yetim yuvarlak kalamaz. 4) Verbatim Eşleşme: Diyagramdaki isim kebab-case yapılarak dosya adına dönüşür: `UC-002\nBorrow Book` -> `docs/use_cases/UC-002-borrow-book.md`. `/spec-review` komutu bu eşleşmeyi otomatik doğrular; uyuşmazlık varsa CI derlemesini kırar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

PlantUML Diagram Semantics and Verification Protocol: Cockburn continually warned against treating use case diagrams as architectural models. In the AI Unified Process, `docs/use_cases.puml` serves strictly as the system's navigable index. Downstream parsing engines enforce four non-negotiable conventions: 1) Two-Line Labeling: Every use case node must declare its alphanumeric identifier and human name across two lines: `UC-001\nSearch Catalog`. 2) System Boundary: The enclosing bounding box must declare the authentic bounded-context system name. 3) Completeness Invariant: Every defined actor must link to at least one use case; isolated nodes fail verification. 4) Verbatim Filename Parity: The string literal in the diagram maps deterministically to the file system: `UC-002\nBorrow Book` maps exclusively to `docs/use_cases/UC-002-borrow-book.md`. Automated tools parse this contract to guarantee zero orphan specifications.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: PlantUML Eşleşmesini Otomatik Denetleyen Doğrulama Kodu
package com.maang.aiup.diagram;

import java.nio.file.Files;
import java.nio.file.Path;

public class DiagramParityChecker {
    // PlantUML dosyasındaki 'UC-002\nBorrow Book' tanımı:
    public record UseCaseEntry(String id, String rawName) {
        public String expectedFileName() {
            String sanitized = rawName.toLowerCase().replace(" ", "-");
            return id + "-" + sanitized + ".md";
        }
    }

    public static boolean verifyFileExists(Path baseDir, UseCaseEntry entry) {
        Path expectedPath = baseDir.resolve(entry.expectedFileName());
        boolean exists = Files.exists(expectedPath);
        System.out.printf("Denetim: %s -> %s (Mevcut: %s)%n", entry.rawName(),
expectedPath.getFileName(), exists);
        return exists;
    }

    void main() {
        var entry = new UseCaseEntry("UC-002", "Borrow Book");
        System.out.println("Beklenen dosya: " + entry.expectedFileName());
        assert entry.expectedFileName().equals("UC-002-borrow-book.md") : "İsim eşleşme hatası!";
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

PlantUML diyagramını oklarla, koşullarla ve if-else mantıklarıyla boğmayın! Diyagram bir akış şeması (flowchart) değildir. Adımların sırası diyagramda değil, use case şartnamesinin içindeki numaralı adımlarda yaşar.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not treat a use case diagram as an execution flowchart. Avoid drawing conditional branching arrows between ellipses. The chronological flow belongs exclusively within the numbered steps of the markdown specification.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- PlantUML diyagramı sistemin içindekiler tablosudur; modelin kendisi değil, haritasıdır.
- Diyagramdaki etiket iki satırdan oluşur: ID ve İsim (UC-001\nSearch Catalog).
- Diyagramdaki isim ile dosya adı (UC-001-search-catalog.md) noktası virgülüne eşleşmelidir.

🎯 Key Architectural Takeaways

- The PlantUML diagram is the system's table of contents, not the architectural model.
- Diagram nodes must declare ID and Name on two lines (e.g. UC-001\nSearch Catalog).
- The diagram name and the markdown filename must maintain 1:1 verbatim parity.

22

BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

<<include>> ve <<extend>> Hakkındaki Gerçek: Neden Neredeyse Hiç İhtiyacınız Yok?

The Truth About <<include>> and <<extend>>: Why You Almost Never Need Them

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              <<INCLUDE>> VE <<EXTEND>> TUZAĞI VE MODERN ÇÖZÜM │
└──────────────────────────────────────────────────────────────┘

1. ESKİ YANLIŞ KULLANIM (AŞIRI MÜHENDİSLİK):
   [ Kitap Ödünç Al ] ──<<include>>──▶ [ Kullanıcıyı Giriş Yaptır ] (YANLIŞ!)
   [ Kitap Ödünç Al ] ──<<include>>──▶ [ Veritabanına Bağlan ]      (FACİA!)
   * Kimlik doğrulama bir include DEĞİLDİR; o bir ÖNKOŞULDUR (Precondition)!

2. MODERN AIUP KURALI:
   - <<include>>: SADECE gerçekten 3'ten fazla use case tarafından paylaşılan
                  ve bağımsız değeri olan ağır bir alt akış varsa kullanılır (Nadir!).
   - <<extend>>:  Yazılımı anlamayı imkansızlaştırır; AIUP'ta NEREDEYSE HİÇ KULLANILMAZ!
   - Kural: Basit tutun! Bir akış diğerini çağıracaksa referans verin: (UC-005 Reserve Book).
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski modelciler yemek yaparken her hareket için ayrı bir döküman açmaya bayılırdı: 'Yemek Pişir' use case'i, 'Tuzluğu Tut' use case'ini `\<\<include>>` eder; 'Tuzluğu Tut' ise 'Kapağı Aç'ı `\<\<include>>` ederdi. Sonunda kimse yemek pişiremez hale gelirdi! 'Giriş Yap' bir include değildir; mutfağa girmeden önce kapının anahtarla açılmış olması bir önkoşuldur (Precondition). Akıllı bir usta gibi davranın: Birbirine bağlı gereksiz oklar çizmeyi bırakın, sadece ana işi yazın!

🇬🇧 ENGLISH (ELI5)

Bureaucratic modelers loved fracturing simple recipes: 'Cook Omelet' would `\<\<include>>` 'Grasp Salt Shaker', which would `\<\<include>>` 'Unscrew Cap'. The kitchen ground to a halt under paper overload. Authentication is not an included sub-routine; having unlocked the kitchen door is a static Precondition. In the AI era, procedural spaghetti arrows between ellipses confuse AI parsers. Omit them.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

<<include>> ve <<extend>> İlişkilerinin Çöküşü: UML tarihindeki en büyük kafa karışıklığı ve zaman israfı bu iki anahtar kelimeden doğmuştur. Cockburn 2001'de açıkça uyarmıştı: 'Alt use case'ler pahalıdır; yalnızca yeniden kullanılabilirlik veya aşırı okunabilirlik gerektiren durumlarda mazur görülebilir. Pratikte bu kural şuna indirgenir: Neredeyse hiçbir zaman!' AI Unified Process'te bu kural daha da katılaşır: 1) Kimlik Doğrulama Bir Include Değildir: 'Kullanıcı Girişi' (Log In) asla bir include olarak her senaryoya bağlanamaz; o bir Önkoşuldur (Preconditions: The member is signed in). Önkoşul olarak yazıldığında bir rota korumasına (Route Guard) dönüşür ve akışın içinde tekrar tekrar doğrulanmaz. 2) <<extend>> Yasağı: Extend ilişkisi akışın dışarıdan bir şarta bağlı olarak use case'e enjekte edilmesidir; akışın nerede dallandığını gizler ve hem insanı hem AI'ı şaşırtır. AIUP'ta extend yerine Alternatif Akışlar (A1, A2) kullanılır. 3) Handoff by Reference: Eğer bir use case başka bir use case'i tetikleyecekse, ok çizmek yerine alternatif akışın içine parantez içinde link verilir: `3. If the member accepts, system records reservation (UC-005 Reserve Book).` Hafif, temiz ve sıfır karmaşıklık!

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Deconstruction of <<include>> and <<extend>>: UML literature generated decades of futile debate over relationship arrows. Cockburn's original rule of thumb: sub-use cases carry immense overhead; they are justified strictly by cross-functional reuse. In practice, this heuristic collapses to: almost never. In the AI Unified Process: 1) Authentication is never an Include: Injecting 'Log In' via `\<\<include>>` into 20 different use cases is an architectural anti-pattern. Authentication is a Precondition ('The member is signed in'). Preconditions translate cleanly into Spring Security filters or gateway route guards, bypassing in-flow redundant validations. 2) The Banishment of <<extend>>: Extension points fragment execution context across disjointed files, baffling human reviewers and confusing AI generators. All conditional deviations belong in named Alternative Flows (A1, A2). 3) Handoff by Reference: When a use case legitimately triggers another, it invokes it inline via alphanumeric citation: `(UC-005 Reserve Book)` without drawing complex UML dependency webs.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: <<include>> Yerine Önkoşul (Route Guard) ve Referans ile Çağrı Modeli
package com.maang.aiup.relationships;

public class RelationshipModernPatterns {
    // ❌ YANLIŞ: Her use case içinde kimlik doğrulama include'u çalıştırmak.
    // ✅ DOĞRU: Önkoşul (Precondition) güvenlik filtresinde çözülür:
    public record AuthenticatedUser(String id, String role) {}

    public static class CleanBorrowService {
        // Önkoşul sistem tarafından garantilenmiştir (Guarded Entry)
        public void borrow(AuthenticatedUser user, String bookId) {
            // Include yok! Doğrudan iş kuralı ve referans var:
            boolean bookAvailable = false;

            if (!bookAvailable) {
                // Handoff by reference: UC-005 Reserve Book tetiklenir!
                triggerReserveBookUseCase(user, bookId);
                return;
            }
            System.out.println("Ödünç tamamlandı.");
        }

        private void triggerReserveBookUseCase(AuthenticatedUser user, String bookId) {
            System.out.printf("UC-005 Reserve Book devreye girdi: %s için rezervasyon yapıldı.%n",
user.id());
        }
    }

    void main() {
        var service = new CleanBorrowService();
        service.borrow(new AuthenticatedUser("U1", "MEMBER"), "BK-44");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir PlantUML diyagramında 10'dan fazla <<include>> ve <<extend>> oku görüyorsanız o diyagram çöp olmuştur. Hepsini silin; kimlik doğrulamayı önkoşul yapın, dallanmaları ise alternatif akışlara taşıyın.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

If a use case diagram resembles a spiderweb of \<\<include>> and \<\<extend>> arrows, the model is deeply corrupted. Purge them. Move authentication to Preconditions and push deviations into local Alternative Flows.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kimlik doğrulama bir <<include>> değildir; senaryo öncesi bir Önkoşuldur (Precondition).

• <<extend>> ilişkisi akışı gizler ve karmaşıklaştırır; AIUP'ta bunun yerine Alternatif Akışlar kullanılır.

• Başka bir use case'e geçiş, senaryo adımı içinde referans verilerek (UC-005 Reserve Book) hafifçe çözülür.

🎯 Key Architectural Takeaways

• Authentication is never an include; it is an established Precondition.

• `<<extend>>` obscures control flow; use explicit Alternative Flows instead.

• Hand off between use cases lightly by inline reference: (UC-005 Reserve Book).

23

BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

Arayüz Olarak Kararlı Kimlikler: FR-001'den Koda Mekanik İzlenebilirlik Zinciri

Identifiers as Interfaces: Mechanical Traceability from FR-001 to Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              MEKANİK İZLENEBİLİRLİK ZİNCİRİ (TRACEABILITY CHAIN)          │
└──────────────────────────────────────────────────────────┘

FR-005 (Kullanıcı Hikayesi)
   │
   ▼ realizes
UC-003 (Manage Books Use Case)
   │
   ▼ diverges at
A2 (Alternatif Akış: Stok Yetersiz)
   │
   ▼ enforces
UC-003 BR-007 (İş Kuralı: Asgari Rezervasyon Limiti)
   │
   ▼ generates
BookBorrowService.java (Kaynak Kod Satırı)
   │
   ▼ verifies
TC-003_A2_Test.java (Otomatik Test Senaryosu)

🎯 MÜKEMMEL DÜZEN: Bir müşteri "Bu sipariş niye reddedildi?" diye sorduğunda,
kimse kodun içinde kaybolmaz; kimlik zincirini (FR -> UC -> A2 -> BR) gösterir!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir otomobil fabrikasında her somunun ve her civatanın bir parça kodu (Örn: C-104) vardır. Araba arıza yaptığında tamirci 'Şu vidayı değiştir' demez; 'C-104 parçasını tak' der. Yazılımda da isimler değişkendir ama kimlikler (FR-001, UC-002, BR-004, TC-001) çelikten dökülmüş mühürlerdir! Bu mühürler sayesinde ilk müşteri talebinden en alttaki Java test koduna kadar her şey birbirine mekanik bir zincirle bağlanır.

🇬🇧 ENGLISH (ELI5)

In aeronautical engineering, every bolt and gasket carries a stamped serial code (e.g. PART-881). When an aircraft malfunctions, technicians don't say 'swap that little silver thing'; they reference the exact component code. In software, natural language drifts, but alphanumeric identifiers (FR-001, UC-002, BR-005, TC-003) are immutable interfaces. They bind client intent directly to automated tests with mechanical traceability.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Kararlı Kimlikler (Stable Identifiers) ve Mekanik Arayüz Mantığı: Geleneksel yazılım süreçlerinde isimlendirme kuralları (naming conventions) sadece bir 'temizlik ve düzen' (hygiene) meselesiydi. AI Unified Process'te ise bu kimlikler birer 'makine arayüzüdür' (interfaces). İnsanlar anlamı kelimelerden çıkarır; yapay zekâ araçları ise ilişkileri kimliklerle kurar. 1) Standart Kimlik Formatı: FR-XXX (Fonksiyonel Gereksinim), NFR-XXX (Fonksiyonel Olmayan), C-XXX (Kısıt), UC-XXX (Use Case), BR-XXX (İş Kuralı), TC-XXX (Test Senaryosu). 2) Yerel Numaralandırma ve Nitelikli Referans (Qualified Citation): BR numaralandırması her use case'in kendi içinde yereldir (`UC-001` dosyasında `BR-001..003`, `UC-002` dosyasında yine `BR-001..003` olabilir). Ancak bir use case dışarıdaki bir kurala atıfta bulunacaksa MUTLAKA nitelikli referans (qualified reference) vermek zorundadır: `UC-001 BR-002`. Başına use case kodu konulmayan çıplak bir `BR-002` sadece kendi dosyasının içinde geçerlidir. 3) Kesintisiz Zincir: Müşteri bir kuralı sorduğunda cevap kodun içinde aranmaz; `FR-005 -> UC-003 -> A2 -> UC-003 BR-007 -> Service.java -> Test.java` kimlik zinciri tek saniyede çıkarılır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Identifiers as Mechanical Interfaces: In legacy development, naming standards were mere aesthetic hygiene. In AIUP, identifiers are machine-parseable interfaces. While humans parse semantics from prose, LLM agent toolchains resolve relational graphs via alphanumeric coordinates. 1) Taxonomy: FR-XXX (Functional Requirement), NFR-XXX (Non-Functional), C-XXX (Constraint), UC-XXX (Use Case), BR-XXX (Business Rule), TC-XXX (Test Case). 2) Local Scope and Qualified Scoping: Business rule identifiers are local to their declaring use case (UC-001 has BR-001 through BR-003; UC-002 also starts at BR-001). Citing a rule across use case boundaries requires strict qualification: `UC-001 BR-002`. An unqualified `BR-002` is only valid within its host file. 3) Mechanical Audit Trails: When compliance audits require explaining an order rejection, the team provides an unbroken chain of identifiers: FR-005 -> UC-003 -> A2 -> UC-003 BR-007 -> code -> test. Nobody decompiles bytecode to discover business logic.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kimlikleri Kod ve Test Açıklamalarında Taşıyan İzlenebilirlik
package com.maang.aiup.traceability;

/**
 * Traces to:
 * - Requirement: FR-012 (Borrow Book Flow)
 * - Use Case: UC-002 (Borrow Book)
 * - Flow: A2 (Loan Limit Reached)
 * - Rule: UC-002 BR-005 (Max 5 open loans)
 */
public class LoanLimitEnforcer {
    public static final String TRACE_RULE = "UC-002 BR-005";
    public static final int MAX_LOANS = 5;

    public static void verifyLimit(int currentLoans) {
        if (currentLoans >= MAX_LOANS) {
            // İstisna mesajı doğrudan kural kimliğini taşır!
            throw new IllegalStateException(TRACE_RULE + ": Maksimum ödünç limitine ulaşıldı.");
        }
    }

    void main() {
        try {
            verifyLimit(5);
        } catch (IllegalStateException e) {
            System.out.println("✅ Mekanik hata yakalandı: " + e.getMessage());
        }
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

İş kurallarının metnini başka bir use case'in içine kopyalayıp yapıştırmak KESİNLİKLE YASAKTIR! Kural kopyalanırsa zamanla biri güncellenir diğeri unutulur (drift). Kuralı daima ait olduğu use case'den kimliğiyle çağırın: 'UC-002 BR-004'.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Copying and pasting business rule text across specifications is strictly forbidden. Duplicated rules inevitably drift. Always cite external rules using their qualified identifier: `UC-002 BR-004`.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Kimlikler (FR, UC, BR, TC) birer hijyen değil, makinenin sistemi bağladığı arayüzlerdir.
- BR-XXX numaraları use case içinde yereldir; dışarıdan çağrılırken nitelikli yazılır (UC-001 BR-002).
- İzlenebilirlik zinciri sayesinde bir iş kuralı şartnameden koda ve teste kadar tek bakışta izlenir.

🎯 Key Architectural Takeaways

- Identifiers are not stylistic hygiene; they are mechanical interfaces connecting the chain.
- Business rule IDs are local to a spec; cross-spec citations must be qualified (UC-001 BR-002).
- The traceability chain allows instantaneous traversal from requirement to code and tests.

24

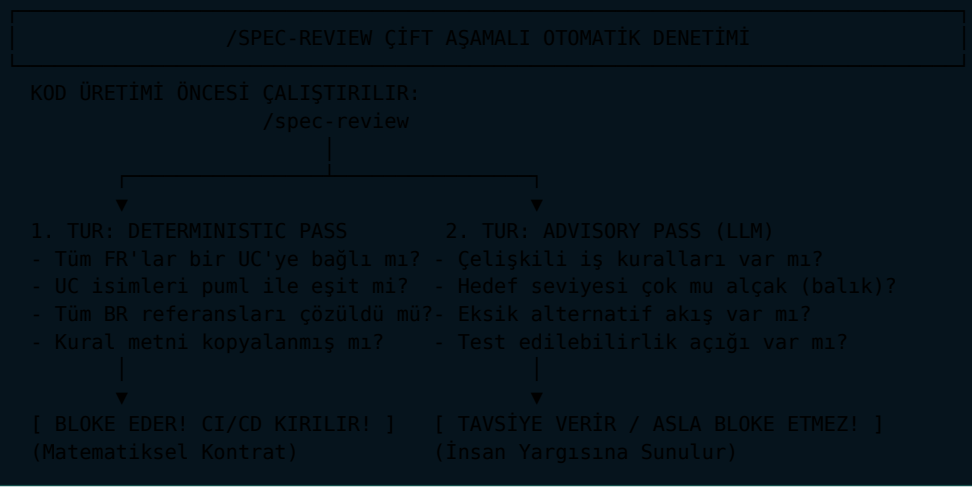
BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

Otomatik Doğrulama: /spec-review Komutunun Deterministik ve Danışmanlık Turları

Automated Validation with /spec-review: Deterministic vs Advisory Passes

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Uçağa binmeden önce iki güvenlik kontrolünden geçersin: 1) Deterministik Geçiş (Turnike): Biletini makineye okutursun. Biletin sahte veya başkasının adıneysa turnike KÜT diye kapanır, geçemezsin (Bloke eder!). 2) Danışmanlık Geçişi (Hostes): Hostes sana gülümser ve 'Beyefendi üzerinizdeki ceket bu havada sizi biraz üşütebilir' der (Tavsiye!). Karar senindir, uçağa binmeni engellemez. `/spec-review` aracı da böyledir: Bozuk referans varsa kod üretimini durdurur; kural eksikliği hissederse kulağına fısıldar!

🇬🇧 ENGLISH (ELI5)

Airport security operates in two phases: 1) Deterministic Gate: The automated e-gate scans your passport barcode. If the checksum fails or the name mismatches, the metal barrier locks shut instantly (Blocking). 2) Advisory Gate: A flight attendant observes you boarding and notes: 'Sir, it is raining heavily at our destination; you may want a coat.' (Advisory). `/spec-review` implements this exact dual-pass protocol: syntax and reference mismatches halt the build; heuristic ambiguities are flagged for human review.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

/spec-review Mimarisinin Çift Aşamalı Güvencesi: Herhangi bir use case'i `/implement` komutuyla yapay zekaya kodlattırmadan önce, şartname zincirinin tutarlı olduğunu doğrulamak şarttır. AIUP bu görevi `/spec-review` becerisine (skill) verir. Bu denetim iki bağımsız turda çalışır: 1) Tur 1 - Deterministik Geçiş (Deterministic Pass): Tamamen deterministik kurallarla kod tabanını ve markdown dosyalarını tarar. Her FR-XXX en az bir UC'ye bağlı mı? Diyagramdaki her dosya diskte var mı? Atıfta bulunulan tüm `UC-001` `BR-002` referansları gerçekten mevcut mu? Kurallar kopyalanmış mı yoksa linklenmiş mi? Bu aşamada tek bir hata bulunursa CI derlemesi kırılır (Build Failure); AI kod üretemez. 2) Tur 2 - Danışmanlık Geçişi (Advisory Pass): LLM devreye girer ve anlamsal (semantic) analiz yapar. İş kuralları arasında mantıksal bir çelişki var mı? Hedef seviyesi deniz seviyesinden balık seviyesine düşmüş mü? Eksik bir hata akışı (alternative flow) kalmış mı? Bu bulgular asla CI'ı bloke etmez; şartname sahibine (Requirements Engineer) bir tavsiye raporu olarak sunulur. Bu iki aşamalı filtre, AI'a teslim edilen şartnamenin %100 kusursuz ve çelişkisiz olmasını garantiler.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Architecture of the /spec-review Verification Engine: Prior to dispatching specifications to `/implement`, the artifact chain must be audited for referential integrity. The `/spec-review` command executes in two decoupled passes: 1) Pass 1 (Deterministic Structural Audit): An algorithmic static analyzer parses markdown trees without LLM invocation. It verifies that every functional user story maps to at least one use case; that every use case node in `use_cases.puml` possesses a corresponding `.md` file; that all cross-spec citations (`UC-XXX` `BR-YYY`) resolve to actual declarations; and that no rule text was cloned. A failure here blocks the CI pipeline unconditionally. 2) Pass 2 (Advisory Semantic Audit): An LLM auditor inspects the narrative for cognitive defects: conflicting business logic, underwater goal altitudes, unaddressed failure paths, and untestable assertions. Findings from Pass 2 are advisory: they prompt human engineers to exercise judgment and never halt execution automatically.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Deterministik /spec-review Doğrulayıcısının Çekirdek Mantığı
package com.maang.aiup.specreview;

import java.util.Set;

public class SpecReviewDeterministicEngine {
    // Kayıtlı gereksinimler ve bunları gerçekleştiren Use Case'ler
    public record VerificationReport(boolean passed, String errorMessage) {}

    public static VerificationReport verifyTraceability(
        Set<String> catalogFrIds,
        Set<String> realizedFrIds,
        Set<String> declaredRules,
        Set<String> referencedRules
    ) {
        // 1. Kural: Her FR en az bir Use Case tarafından gerçekleştirilmelidir!
        for (String fr : catalogFrIds) {
            if (!realizedFrIds.contains(fr)) {
                return new VerificationReport(false, "BLOKE EDİCİ HATA: " + fr + " hiçbir use case'e
bağlanmamış!");
            }
        }

        // 2. Kural: Referans verilen tüm BR'lar gerçekten var olmalıdır!
        for (String ref : referencedRules) {
            if (!declaredRules.contains(ref)) {
                return new VerificationReport(false, "BLOKE EDİCİ HATA: Çözülemeyen kural referansı ->
" + ref);
            }
        }

        return new VerificationReport(true, "✅ Deterministik denetim başarıyla tamamlandı.");
    }

    void main() {
        var frs = Set.of("FR-001", "FR-002");
        var realized = Set.of("FR-001", "FR-002");
        var declared = Set.of("UC-001 BR-001", "UC-002 BR-004");
        var refs = Set.of("UC-001 BR-001");

        var report = verifyTraceability(frs, realized, declared, refs);
        System.out.println(report.errorMessage());
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

/spec-review raporundaki 'Advisory' (Danışmanlık) uyarılarını görmezden gelip doğrudan koda geçmeyin. 'Bu akışta hata durumu eksik' uyarısı, yarın müşterinin sistemde yaşayacağı gerçek bir patlamanın habercisidir.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not dismiss Advisory findings from `/spec-review` casually. A warning regarding 'missing alternative flows' highlights an unhandled business condition that will manifest as unexpected behavior in production.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• /spec-review çift turludur: 1. Tur deterministik ve bloke edicidir, 2. Tur anlamsal tavsiyedir.

• Referansı çözülemeyen bir kural veya eksik bağlı bir FR varsa CI derlemesi kırılır.

• /spec-review asla dökümanları otomatik düzenlemez; sadece raporlar, karar insana aittir.

🎯 Key Architectural Takeaways

• /spec-review operates in two passes: Pass 1 is deterministic/blocking; Pass 2 is advisory.

• Dangling rule citations or orphaned requirements break the CI build deterministically.

• /spec-review reports findings without mutating files; decisions remain human-owned.

BÖLÜM 5 / MODULE 5

Ana Şartname Şablonu & Alan Alan Derinlemesine İnceleme

The Master Specification Template & Field Breakdown

AI Unified Process use case şablonunun anatomisi: Genel Bakış (Overview), Önkoşullar ile Tetikleyicilerin ayrımı, 3-9 adımlı Ana Başarı Senaryosu, Başarı ve Başarısızlık Sonkoşulları (Postconditions / Guarantees), İş Kuralları (BR-XXX) ve örnek 'Borrow Book' master şartnamesinin adli analizi.

Anatomy of the AIUP specification template: Overview metadata, the strict boundary between Preconditions and Triggers, the 3-to-9 step Main Success Scenario, transactional Postconditions (Success vs Failure guarantees), Business Rules (BR-XXX), and forensic deconstruction of the canonical 'Borrow Book' spec.

25

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

AIUP Use Case Şablonu: Makinenin ve İnsanın Kusursuz Anladığı Format

The AIUP Use Case Template: Anatomy of a Machine-Readable Document

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
AI UNIFIED PROCESS MASTER USE CASE ŞABLONU
```

```
# Use Case: [Use Case Name]
## Overview
- Use Case ID: UC-XXX  |  Status: Draft / Approved / Implemented
- Primary Actor / Secondary Actors
- Goal: Tek cümlelik gözlemlenebilir hedef ve gerekçesi
- Trigger: Süreci başlatan tetikleyici olay (Durum değil, geçiş!)
- Requirements: [FR-XXX, NFR-XXX, C-XXX](../requirements.md)

## Preconditions
- [Sistemin zaten doğru olduğunu varsaydığı kapı şartları]

## Main Success Scenario
1. [Etken çatılı, tek özneli, numaralı adımlar (3 - 9 adım)]

## Alternative Flows
### A1: [Hata veya İstisna Durumu] (step N)

## Postconditions (Success & Failure Garantileri)
## Business Rules (BR-XXX)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir pasaport başvuru formunu düşünün. Dünyanın her yerinde pasaport formu aynı standart kutucuklara sahiptir: Ad, Soyad, Doğum Yeri, Anne Adı. Eğer herkes kendi kafasına göre mektup şeklinde başvuru yazsaydı memurlar çıldırırdı, pasaportlar basılamazdı! AIUP şablonu da böyledir: Her use case aynı tek tip üniformayı giyer. İnsan gözü aradığı şeyi saniyesinde bulur, yapay zekâ ise metni okurken asla bocalamadan her alanı ilgili kod katmanına yerleştirir!

🇬🇧 ENGLISH (ELI5)

Consider an international passport application form: standardized boxes for Name, Date of Birth, Nationality, and Photo. If citizens submitted free-form handwritten essays, border control would collapse into chaos. Uniformity pays a double dividend: humans skim familiar structures in seconds, while AI generators parse labeled fields deterministically without guessing where the business logic begins.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Şablon Bütünlüğü ve Cockburn Şablonunun Evrimi: Cockburn 2001'de 'Töreni (ceremony) anlaşmazlığın maliyetine göre ayarlayın' demişti. Aynı odada çalışan küçük bir ekip günlük dilde use case yazabilirdi; ancak dağıtık ekipler 'fully dressed' (tam teşekküllü) formata muhtaçtı. AI çağında yapay zekâ, insan geliştiriciden katbekat daha uzaktır: Ne aynı koridoru paylaşır ne de kahve içer! Bu nedenle AIUP'ta her şartname istisnasız tam teşekküllü şablonla yazılır. Ancak bu şablon Cockburn'ün eski ağır şablonundan daha sadedir: 1) Kapsam (Scope) ve İrtifa (Level) satırları kaldırılmıştır; çünkü AIUP'ta kapsam daima Sistem Kapsamı, irtifa ise daima Deniz Seviyesidir (sabit kurallar tekrar edilmez). 2) Teknoloji ve Veri Varyasyonları kaldırılmıştır; çünkü teknoloji koda aittir. 3) Frekans ve Öncelik gereksinim kataloğuna (requirements.md) taşınmıştır. Geriye kalan alanlar saf, damıtılmış ve mekanik birer kontrattır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Structural Uniformity and the Modernization of Cockburn's Template: Cockburn advised calibrating documentation ceremony against the cost of misunderstanding: colocated engineers could survive on informal bullet points, whereas distributed enterprises required fully dressed forms. In generative development, the implementer (the LLM) is maximally distant. It possesses no shared culture and cannot ask for clarification. Consequently, every AIUP specification adopts a fully dressed structure. Yet the template is radically streamlined compared to classical RUP: 1) Scope and Altitude headers are omitted because they are invariant constants (System Scope, Sea Level). 2) Technology Variations are purged because technical mechanisms belong to stack plugins. 3) Prioritization and SLAs are delegated to the requirements catalog. The surviving schema is an unadulterated behavioral contract.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: AIUP Şablonunun Domain Mimarisine Birebir İzdüşümü (Clean Ports)
package com.maang.aiup.template;

public class UseCaseTemplateStructure {
    // 1. Overview Metadata: Hedef ve Niyet
    public record Metadata(String id, String name, String goal, String trigger) {}

    // 2. Preconditions: Route Guard veya Güvenlik Ön Kontrolü
    public record Preconditions(boolean isAuthenticated, boolean isAccountActive) {
        public void verify() {
            if (!isAuthenticated || !isAccountActive) {
                throw new SecurityException("Önkoşul ihlali: Kullanıcı oturum açmamış veya hesap aktif değil!");
            }
        }
    }

    // 3. Postconditions: Başarı ve Başarısızlık Çıktı Garantileri
    public record ExecutionOutcome(boolean success, String payload, String auditLog) {}

    void main() {
        var meta = new Metadata("UC-002", "Borrow Book", "Member borrows a book", "User requests borrow");
        var pre = new Preconditions(true, true);
        pre.verify();
        System.out.println("Şablon doğrulaması başarılı: " + meta.name());
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şablondaki başlıkların isimlerini kafanıza göre değiştirmeyin! 'Overview' yerine 'Genel Bilgiler', 'Main Success Scenario' yerine 'Adımlar' yazarsanız, downstream çalışan otomasyon ve /spec-review araçları şartnameyi parse edemez ve hata verir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not alter the markdown section headers arbitrarily. Renaming 'Main Success Scenario' to 'Steps' breaks the regex parsers in `/spec-review` and halts the automated build pipeline.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Tek tip standart şablon, insanların hızlı göz gezdirmesini ve AI'ın hatasız kod üretmesini sağlar.
- Gereksiz alanlar (Teknoloji, Kapsam) atılmıştır; sadece saf davranışsal kontrat korunmuştur.
- Başlıklar ve alan isimleri otomasyon araçları tarafından denetlenen katı arayüzlerdir.

🎯 Key Architectural Takeaways

- A uniform template accelerates human skimming and guarantees deterministic machine parsing.
- Obsolete fields are discarded; only the pure behavioral and transactional contract remains.
- Markdown headings are strict interfaces audited by automated quality tooling.

26

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

Önkoşul vs Tetikleyici (Preconditions vs Triggers): Asla Akışta Yeniden Doğrulanmayan Bekçiler

Preconditions vs Triggers: The Gatekeepers That AI Must Never Re-Validate

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                TETİKLEYİCİ (TRIGGER) VS ÖNKOŞUL (PRECONDITION)
└──────────────────────────────────────────────────────────────┘

 TETİKLEYİCİ (TRIGGER):
 - Bir DURUM değil, bir GEÇİŞTİR (Transition from Waiting to Running)!
 - "Üye kitabı seçer ve 'Ödünç Al' butonuna tıklar." (Olay anı!)
 - Sistemin uyku halinden uyanmasını sağlayan kıvılcımdır.

 ÖNKOŞUL (PRECONDITION):
 - Senaryo başlamadan önce ZATEN DOĞRU olduğu garanti edilen durumdur.
 - "Üye oturum açmıştır" & "Üyenin hesabı aktiftir."
 - ⚠ SİSTEM BUNU ASLA SENARYO ADIMLARINDA YENİDEN KONTROL ETMEZ!
 - Kodda bir if-else değil; bir ROTA KORUYUCUSU (Route Guard / Filter) olur!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir uçağa binmeyi düşünün: 1) Önkoşul: Pasaportunun olması ve güvenlik kontrolünden geçmiş olmandır (Sen daha uçağın kapısına gelmeden çok önce zaten tamamlanmıştır). 2) Tetikleyici: Hostesin 'Uçağa biniş başladı, lütfen biletinizi okutun' anonsu yapmasıdır (Hareketi başlatan an!). Uçağın içine girdikten sonra koltuğuna otururken hostes sana 'Dur bir daha pasaportunu göster, güvenlikten geçtin mi kontrol edeyim' demez (Önkoşul akış içinde tekrar doğrulanmaz!).

🇬🇧 ENGLISH (ELI5)

Boarding a commercial flight: 1) Precondition: Possessing a valid passport and having cleared airport security. This state was established long before reaching the boarding gate. 2) Trigger: The gate agent announcing 'Group A may now board.' (The catalyst transitioning you from waiting to boarding). Once you step down the jetway, flight attendants do not halt you in the aisle to re-screen your luggage for liquids. A precondition is never re-validated inside the execution flow.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Önkoşul Semantiği ve AI Kod Üretimindeki Önemi: Martinelli ve Cockburn önkoşul ile tetikleyici arasındaki farkı şöyle netleştirir: 1) Tetikleyici (Trigger): 'Bekleme' durumundan 'Çalışma' durumuna geçiştir. Kullanıcının bir isteği, zamana bağlı bir alarm veya harici sistemden gelen bir web-hook mesajıdır. Zaten doğru olan bir durum (ör: 'Üye kitap ister') tetikleyici olamaz; tetikleyici olayı anlatmalıdır. 2) Önkoşul (Precondition): Sistemin daha use case başlamadan önce kesinlikle doğru olduğunu garanti ettiği durumdur. Bu kuralın iki altın testi vardır: a) Sistem bunu zorlayabilir (enforce) mi? 'Üye kitabı geri getirmek niyetindedir' bir önkoşul olamaz çünkü sistem niyeti bilemez. b) Bu önkoşul daha önce başka bir use case tarafından kuruldu mu? 'Üye oturum açmıştır' önkoşulu, daha önce çalışmış olan 'UC-001 Log In' use case'i tarafından kurulmuştur. Eğer hiçbir use case kurmadıysa, o bir önkoşul değil; senaryonun 1. adımıdır! 3) AI Kuralı: Önkoşul, use case akışının içinde bir if-else kontrolü olarak YAZILMAZ; HTTP filtrelerinde veya Spring Controller `\@PreAuthorize` katmanında çözülür.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Precondition and Trigger Mechanics in Generated Architectures: 1) Trigger: The catalyst event that transitions the use case from idle to active. It must describe an occurrence (an actor action, an inbound webhook, or a temporal scheduler tick), never a persistent state. 2) Preconditions: The state guarantees enforced by the system prior to execution. Cockburn established two definitive tests: First, the condition must be programmatically enforceable (an actor's emotional intent fails). Second, it must have been established by an upstream use case (e.g. 'Member is signed in' was established by `UC-001 Log In`). If no prior use case established it, it is a runtime validation for Step 1, not a precondition. 3) Architectural Translation: Preconditions compile into ingress boundary assertions or route guards. Crucially, they are never re-checked inside the scenario steps; asserting preconditions within the happy path violates the additive flow model.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Önkoşulun (Guard) Akıştan Ayrılması (Spring Security / Gateway Modeli)
package com.maang.aiup.preconditions;

public class PreconditionEnforcementDemo {
    public record UserSession(String userId, boolean active) {}

    // ÖNKOŞUL KORUYUCUSU (Route Guard / Interceptor):
    // Use Case başlamadan önce çalışır; akışın içinde if-else kirliliği yapmaz!
    public static class PreconditionGuard {
        public static void ensurePreconditions(UserSession session) {
            if (session == null || !session.active()) {
                throw new SecurityException("Önkoşul İhlali: Oturum geçersiz veya hesap kilitli!");
            }
        }
    }

    // USE CASE: Önkoşulun sağlandığını bilir ve sadece asıl hedefe odaklanır
    public static class BorrowBookFlow {
        public void execute(UserSession session, String bookId) {
            PreconditionGuard.ensurePreconditions(session); // Kapı kontrolü
            // Akış içinde artık 'Oturum açık mı?' diye if-else yapılmaz!
            System.out.println("Ana akış çalışıyor: Kitap ödünç verildi -> " + bookId);
        }
    }

    void main() {
        var validSession = new UserSession("USR-99", true);
        new BorrowBookFlow().execute(validSession, "B-12");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Senaryo adımlarının 1. maddesine 'Sistem kullanıcının oturum açtığını kontrol eder' YAZMAYIN! Eğer oturum açmak bir önkoşulsa zaten Önkoşullar bölümüne yazılmıştır. Senaryonun içinde tekrarlamak AI'ın gereksiz çift kontroller üretmesine yol açar.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never write 'Step 1: System verifies user is logged in'. If authentication is a precondition, it is already guaranteed. Duplicating preconditions inside scenario steps causes AI generators to produce redundant defensive boilerplate.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Tetikleyici sistemi uyandıran olaydır (eylem); Önkoşul ise zaten var olan durumdur.
- Önkoşul sistem tarafından denetlenebilir olmalı ve daha önceki bir use case tarafından kurulmuş olmalıdır.
- Önkoşullar kodda rota koruyucusu (route guard) olur; senaryo adımlarında tekrar kontrol edilmez.

🎯 Key Architectural Takeaways

- A trigger is the event waking the system; a precondition is an already established state.
- Preconditions must be enforceable and established by an upstream use case.
- Preconditions translate into route guards; they are never re-checked within scenario steps.

27

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

Ana Başarı Senaryosu (Happy Path): 3 ila 9 Adımlı Altın Ritim ve Denge

Main Success Scenario: The 3-to-9 Step Golden Rhythm

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                ANA BAŞARI SENARYOSUNUN 3-9 ADIMLI ALTIN RİTMİ

  1. ADIM: Aktör talebini ve verilerini gönderir. (Etkileşim)
  2. ADIM: Sistem varlığı ve seçenekleri gösterir. (Etkileşim)
  3. ADIM: Aktör onaylar / işlemi talep eder. (Etkileşim)
  4. ADIM: Sistem 1. kuralı doğrular (BR-001). (Doğrulama)
  5. ADIM: Sistem 2. kuralı doğrular (BR-002). (Doğrulama)
  6. ADIM: Sistem durumu kaydeder ve stoğu düşer. (Durum Değişimi)
  7. ADIM: Sistem kullanıcıya onay ve makbuz gösterir. (Hedef Teslim Edildi!)

  ⚠ KURAL:
  - 3 adımdan azsa: Muhtemelen doğrulama ve durum değişimi adımları unutulmuştur!
  - 9 adımdan çoksa: Araya buton tıklamaları ve UI mekanikleri sızmıştır; birleştirin!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Mükemmel bir fıkra veya masal 3 ila 9 cümleden oluşur: Giriş, gelişme ve mutlu son. Eğer masalı 2 cümlede bitirirsen ('Keloğlan yola çıktı, kral oldu') kimse bir şey anlamaz (Çok kısa!). Eğer 25 cümle yapıp Keloğlan'ın her adımda ayağını nasıl kaldırdığını anlatırsan herkes uyuyakalır (Çok uzun!). Use case'in ana senaryosu da 3 ila 9 adımdır: Ne eksik ne fazla; hedefe dümdüz giden en temiz yoldur!

🇬🇧 ENGLISH (ELI5)

A classic fairy tale spans 3 to 9 narrative beats: setup, rising action, and resolution. A 2-sentence story ('The prince rode forth and rescued the kingdom') is too abrupt to convey meaning. A 30-sentence story micromanaging how the prince buttoned his left boot puts the audience to sleep. The main success scenario requires 3 to 9 steps: lean, purposeful, and marching straight toward the goal.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Ana Başarı Senaryosunun (Main Success Scenario) Mühendislik Dinamikleri: Cockburn 20 yılı aşkın tecrübesinde 11 adımdan uzun olup da kısaltıldığında daha iyi hale gelmeyen TEK BİR use case senaryosu bile görmediğini belirtir. 1) 3-9 Adım Kuralı: İnsan beyninin çalışma belleği (working memory) Miller Kanununa göre 7±2 birim bilgiyi tutabilir. Bir paydaş veya mühendis 9 adımdan uzun bir senaryoyu kafasında canlandıramaz; doğrulama zafiyetleri başlar. 2) Adımların Şekli: Senaryoda ASLA if-else veya dallanma olamaz. Her adım MUTLAKA başarılı olur. 'Sistem şifre doğru mu bakar' denmez; 'Sistem şifrenin doğruluğunu onaylar' denir. Başarısızlıklar alternatif akışlara fırlatılır. 3) Adım Birleştirme (Collapsing): Birbiri ardına gelen mikro hareketler ('Kullanıcı adı girer', 'Kullanıcı soyadı girer', 'Kullanıcı tıklar') tek bir adıma indirgenir: '1. Kullanıcı kişisel bilgilerini girer.' Bu kural arayüz mekaniklerinin şartnameyi kirletmesini engeller.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Mechanics of the Main Success Scenario: Cockburn observed that he had never seen a scenario longer than 11 steps that did not dramatically improve when shortened. 1) The 3-to-9 Step Heuristic: Rooted in cognitive science (Miller's $7 pm 2$ working memory limit). Both stakeholders validating the workflow and engineers auditing the generated architecture must hold the sequence in mind simultaneously. If a scenario has fewer than 3 steps, state mutations and validations were omitted. If it exceeds 9 steps, UI mechanical clicks have invariably infiltrated the text. 2) The Uncompromising Happy Path: The main scenario contains zero conditional branching. Every step succeeds unconditionally. Failures are decoupled into named alternative flows. 3) Step Aggregation: Unify fragmented micro-interactions ('Enters first name', 'Enters last name', 'Clicks Next') into an intent-driven bundle: 'User enters personal information.'

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 3-9 Adımlı Ana Senaryoyu Yürüten Temiz Orkestrasyon Metodu
package com.maang.aiup.scenario;

public class MainScenarioOrchestrator {
    public record BorrowCommand(String memberId, String bookId) {}
    public record BorrowReceipt(String transactionId, String dueDate) {}

    // Ana Başarı Senaryosunun 3-9 adımlı temiz icrası:
    public BorrowReceipt executeHappyPath(BorrowCommand cmd) {
        // Step 4: System validates copy availability (BR-004)
        validateAvailability(cmd.bookId());

        // Step 5: System validates loan limit (BR-005)
        validateLoanLimit(cmd.memberId());

        // Step 6: System records loan
        String txnId = persistLoanRecord(cmd.memberId(), cmd.bookId());

        // Step 7: System decrements copy count
        decrementInventory(cmd.bookId());

        // Step 8: System confirms and delivers goal
        return new BorrowReceipt(txnId, "2026-11-17");
    }

    private void validateAvailability(String b) {}
    private void validateLoanLimit(String m) {}
    private String persistLoanRecord(String m, String b) { return "TXN-101"; }
    private void decrementInventory(String b) {}

    void main() {
        var receipt = new MainScenarioOrchestrator().executeHappyPath(new BorrowCommand("M1", "B9"));
        System.out.println("✅ Ana senaryo başarıyla icra edildi: " + receipt);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Ana senaryo adımlarında ‘Eğer stok yoksa…’ veya ‘Kullanıcı dilerse…’ gibi koşul cümleleri (if-statements) KESİNLİKLE YASAKTIR! Koşullu her durum Alternatif Akışlar (Alternative Flows) bölümüne aittir.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never introduce ‘If the item is out of stock…’ inside the main success scenario. The main scenario asserts absolute success. All deviations and conditional forks belong exclusively in the Alternative Flows section.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Ana senaryo 3 ila 9 adımdan oluşur; dallanma içermez, her adım başarıyla tamamlanır.

• 9 adımdan uzun senaryolarda ekran detayları sızmıştır; adımları niyet bazında birleştirin.

• Son adım mutlaka kullanıcının hedefine ulaştığını ve değerin teslim edildiğini gösterir.

🎯 Key Architectural Takeaways

• The main scenario has 3 to 9 steps, contains zero branches, and succeeds unconditionally.

• Scenarios exceeding 9 steps suffer from UI mechanical leakage; collapse them into intent.

• The final step must unambiguously deliver the user's primary goal.

28

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

Sonkoşullar Kontrattır: Başarı Garantileri vs Asgari Başarısızlık Garantileri

Postconditions as Contracts: Success Guarantees vs Failure Minimums

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│          SONKOŞULLAR (POSTCONDITIONS): İKİ TARAFLI GARANTİ          │
└──────────────────────────────────────────────────────────┘

BAŞARI SONKOŞULLARI (SUCCESS POSTCONDITIONS / SUCCESS GUARANTEES):
- Hedef tamamlandığında dünyanın aldığı YENİ DURUM:
  * "Üye ile kitap arasında 28 gün vadeli bir ödünç kaydı mevcuttur."
  * "Kitabın mevcut kopya sayısı 1 azaltılmıştır."
  * "Ödünç kaydı üyenin aktif listesinde görüntülenir."

BAŞARISIZLIK SONKOŞULLARI (FAILURE POSTCONDITIONS / MINIMAL GUARANTEES):
- Hangi hata olursa olsun (A1, A2, A3) sistemin verdiği ASGARİ TAAHHÜT:
  * "Hiçbir kısmi kayıt veritabanına YAZILMAMIŞTIR (Rollback garantisi!)."
  * "Kitabın mevcut kopya sayısı DEĞİŞMEMİŞTİR."
  * "Başarısızlık denemesi denetim loguna (audit log) işlenmiştir."

🎯 AI İÇİN ÖNEMİ: Sonkoşullar veritabanı @Transactional sınırını ve testlerin
                  assert (doğrulama) bloklarını doğrudan belirler!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir banka ATM'sinden para çekmeyi düşünün: 1) Başarı Sonkoşulu: Cebinde 1000 TL nakit para vardır, banka hesabından 1000 TL düşmüştür (Hedef tamam!). 2) Başarısızlık Sonkoşulu: ATM tam para sayarken elektrik kesilirse sistemin sana verdiği asgari sözdür: 'Merak etme! Para vermediysem hesabından da 1 kuruş bile DÜŞMEDİM, kartını geri verdim ve log tuttum!' (Kısmi işlem yok!). Sonkoşul, sistemin ne olursa olsun arkasında çöp bırakmayacağının garantisidir.

🇬🇧 ENGLISH (ELI5)

Withdrawing money from an ATM: 1) Success Postcondition: You hold $100 cash in hand, and your account balance is debited by exactly $100. 2) Failure Postcondition: If the power cuts out while gears are turning, the bank provides an ironclad minimum guarantee: 'No partial withdrawal occurred, your bank balance is untouched, your card is returned, and an audit incident is logged.' Failure postconditions enforce transactional atomicity.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Sonkoşullar ve İşlemsel Bütünlük (Transactional Contract): Cockburn'ün en güçlü kavramlarından biri olan 'Guarantees' (Garantiler), AIUP şablonunda 'Success Postconditions' ve 'Failure Postconditions' olarak ikiye ayrılmıştır. 1) Başarı Sonkoşulları: Sistemin mutlu yolda yarattığı tüm kalıcı durum değişikliklerini listeler. 2) Başarısızlık Sonkoşulları (Minimal Guarantees): Sistem başarısız olduğunda neyin ASLA bozulmayacağını taahhüt eder. 'Hiçbir kısmi veri kaydedilmez' garantisi, AI'ın veritabanı işlemlerini `\@Transactional` bloğu içine almasını zorunlu kılar. 'Deneme denetim günlüğüne kaydedilir' garantisi ise AI'ın güvenlik loglarını üretmesini sağlar. 3) Testlerin Tohumu: Her iki sonkoşul bölümü de boş bırakılamaz! Çünkü üretilen otomatik testlerin `assert` blokları doğrudan bu sonkoşullardan türetilir: `assertEquals(oldBalance, newBalance)` başarısızlık sonkoşulunun doğrudan koddaki yansımasıdır. Cockburn'ün altın taktiği: Senaryoyu yazmadan ÖNCE sonkoşulları yazın ve 'Başarılı bir akışın sonunda paydaşı ne mutsuz ederdi?' diye sorun; çıkan cevaplar unuttuğunuz doğrulamaları yakalamanızı sağlar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Postconditions as Transactional Contracts: Cockburn's concept of 'Guarantees' maps directly to the dual postconditions of the AIUP template: 1) Success Postconditions: Explicitly describe the state of the universe when the goal is fulfilled: records created, inventories adjusted, notifications queued. 2) Failure Postconditions (Minimal Guarantees): Express what remains true even when disaster strikes. They are not descriptions of the failure; they are the baseline invariants the system promises under catastrophic exit (e.g. 'No partial loan is persisted', 'Pre-use-case state is restored', 'The failed attempt is recorded in the security log'). 3) Generative Significance: For an AI agent, failure postconditions dictate database transaction boundaries (`\@Transactional(rollbackFor = Exception.class)`). Furthermore, postconditions seed the assertion blocks of generated test suites. Cockburn's heuristic: draft postconditions before writing the steps, asking 'What would make stakeholders furious after an apparent success?'

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Başarı ve Başarısızlık Sonkoşullarını Garanti Eden Transactional Kod
package com.maang.aiup.postconditions;

public class TransactionalContractService {
    public record Account(String id, double balance) {}

    // Başarısızlık Sonkoşulu: Hata anında bakiye ASLA değişmez (Rollback garantisi!)
    public static Account transferMoney(Account from, Account to, double amount) {
        double originalBalance = from.balance();
        try {
            if (from.balance() < amount) {
                throw new IllegalStateException("Yetersiz bakiye!");
            }
            // Başarı Sonkoşulu: Para düşüldü
            return new Account(from.id(), from.balance() - amount);
        } catch (Exception e) {
            // Başarısızlık Garantisi Assert Edilir:
            assert from.balance() == originalBalance : "KRİTİK HATA: Bakiye bozuldu!";
            System.out.println("🛡️ Failure Postcondition devrede: Bakiye korundu, log yazıldı.");
            throw e;
        }
    }

    void main() {
        var acc = new Account("ACC-1", 500.0);
        try {
            transferMoney(acc, new Account("ACC-2", 0), 1000.0);
        } catch (Exception _) {
            System.out.println("İşlem beklendiği gibi güvenle iptal edildi.");
        }
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Başarısızlık sonkoşullarına 'Hata mesajı gösterilir' YAZMAYIN! Hata mesajı bir sonkoşul değil, alternatif akışın bir adımıdır. Sonkoşul sistemin kalıcı durumunu anlatır: 'Hiçbir veri kaydedilmedi ve hesap durumu değişmedi.'

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not write 'Error message is displayed' as a failure postcondition. Error notifications are transient UI steps within an alternative flow. A failure postcondition defines persistent state invariants: 'No partial loan was created and inventory remains unchanged.'

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Başarı sonkoşulları hedefe ulaşıldığında dünyanın yeni kalıcı durumunu belirtir.
- Başarısızlık sonkoşulları hata anında verilen asgari garantidir (Örn: Kısmi veri yazılmaz).
- Her iki sonkoşul bölümü de zorunludur; otomatik testlerin assert blokları buradan beslenir.

🎯 Key Architectural Takeaways

- Success postconditions define the persistent state of the world when the goal is achieved.
- Failure postconditions define minimal guarantees (e.g. no partial state is stored).
- Both postcondition sections are mandatory; they directly seed the assertion blocks of tests.

29

BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

İş Kuralları (BR-XXX): Yerel Numaralandırma, Nitelikli Çağrı & DRY İlkesi

Business Rules (BR-XXX): Local Numbering, Cross-Spec Referencing & DRY

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              İŞ KURALLARI (BUSINESS RULES) REFERANS VE KAPSAM MODELİ      │
└──────────────────────────────────────────────────────────┘

USE CASE 1 (docs/use_cases/UC-001-search-catalog.md):
## Business Rules
### BR-001: Minimum Search Characters
Arama terimi en az 3 karakter olmalıdır.

USE CASE 2 (docs/use_cases/UC-002-borrow-book.md):
## Main Success Scenario
4. System validates that the search term is valid (UC-001 BR-001). ◄── [ Nitelikli Referans! ]
5. System validates member has no overdue loans (BR-006).          ◄── [ Yerel Referans! ]

## Business Rules
### BR-006: Overdue Block
Gecikmiş borcu olan üyeler borç kapanana kadar yeni kitap alamaz.

🚫 KURAL: UC-001'deki kural metnini UC-002'ye ASLA kopyalama! Link ver!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kanun kitabını düşünün: Trafik Kanunu'nda 'Kırmızı ışıkta geçmek yasaktır (Madde 48)' yazar. Belediye otobüsleri yönetmeliğinde bu kural tekrar uzun uzun yazılmaz; 'Otobüs şoförleri Trafik Kanunu Madde 48'e uymak zorundadır' diye tek satır referans verilir. Eğer kuralı her yönetmeliğe baştan yazarsan, yarın kırmızı ışık cezası değiştiğinde bazı kitapçıklarda eski ceza kalır ve ortalık karışır! Kuralı bir kere yaz, her yerden maddesiyle (kimliğiyle) çağır!

🇬🇧 ENGLISH (ELI5)

In statutory law, the penal code defines traffic violations once (e.g. Article 48: Running Red Lights). Municipal bus transit regulations do not duplicate the full legal text of Article 48; they state: 'Bus operators are subject to Penal Code Article 48.' If rules are copy-pasted across fifty municipal pamphlets, an amendment to the primary statute creates immediate legal contradictions across stale copies. Define the rule once; cite it by identifier everywhere.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İş Kuralları (Business Rules) Mimarisi ve DRY Kuralı: Cockburn, kullanım senaryoları ile iş kuralları arasındaki derin ilişkiyi şu sözle özetler: 'Alternatif akışlar ve kurallar, sistemin en ilginç ve en değerli gereksinimlerinin yaşadığı yerdir.' Martinelli bu ilişkiyi katı bir mimariye oturtur: 1) Ayrı Bölüm (Dedicated Section): İş kuralları senaryo adımlarının içine satır içi (inline) gömülmez; şartnamenin sonundaki `## Business Rules` başlığı altında toplanır. Bu sayede 3 farklı akış aynı kurala atıfta bulunabilir. 2) Yerel Numaralandırma (Local Numbering): BR numaraları her use case'in kendi içinde yereldir (`BR-001`, `BR-002`...). Bu kural merkezi bir excel tablosundaki numara çakışması krizini çözer. 3) Nitelikli Çapraz Çağrı (Qualified Cross-Spec Citation): Başka bir use case'in kuralı çağrılacaksa mutlaka use case ID'si ile nitelenir: `(UC-001 BR-002)`. Başına use case kodu konmayan çıplak bir `(BR-005)` sadece o dosyanın içinde aranır. 4) Kopyalama Yasağı: Kural metnini kopyalayıp başka dökümana yapıştırmak `/spec-review` tarafından derhal yakalanır ve engellenir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Business Rule Architecture and Qualified Scoping: Cockburn recognized that the core business value of software concentrates within rule extensions. The AI Unified Process formalizes this via rigorous conventions: 1) Decoupled Rule Section: Rules are isolated in a dedicated `## Business Rules` section rather than embedded inline within step prose. This allows multiple steps and alternative flows to link against a singular rule definition. 2) Local Identifier Namespacing: Rule numbering is strictly local per use case specification (`UC-001` owns `BR-001..003`; `UC-002` owns `BR-001..006`). This eliminates global coordination bottlenecks across distributed squads. 3) Qualified Cross-Referencing: When a specification cites a rule declared elsewhere, it must qualify the identifier: `UC-001 BR-002`. An unqualified citation (`BR-004`) refers exclusively to the host file. 4) Absolute Prohibition of Rule Cloning: Duplicating rule prose across files triggers deterministic build failure during `/spec-review` audits.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: İş Kurallarını Merkezi Enum/Sözlük ile İzlenebilir Kılma
package com.maang.aiup.rules;

public class BusinessRuleCatalog {
    // İş kurallarının Java tarafındaki değişmez temsili
    public enum Rule {
        UC001_BR001("UC-001 BR-001", "Arama terimi en az 3 karakter olmalıdır."),
        UC002_BR005("UC-002 BR-005", "Üye aynı anda en fazla 5 açık ödünç kitaba sahip olabilir."),
        UC002_BR006("UC-002 BR-006", "Gecikmiş borcu olan üyeler ödünç alamaz.");

        public final String id;
        public final String description;
        Rule(String id, String desc) { this.id = id; this.description = desc; }
    }

    public static void validateMemberEligibility(int openLoans, boolean hasOverdue) {
        if (openLoans >= 5) {
            throw new IllegalStateException(Rule.UC002_BR005.id + ": " +
Rule.UC002_BR005.description);
        }
        if (hasOverdue) {
            throw new IllegalStateException(Rule.UC002_BR006.id + ": " +
Rule.UC002_BR006.description);
        }
    }

    void main() {
        validateMemberEligibility(3, false);
        System.out.println("✅ Tüm iş kuralları başarıyla doğrulandı.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir iş kuralına 'BR-001: Hızlı olmalıdır' veya 'BR-002: Güvenli olmalıdır' gibi ölçülemeyen temenniler yazmayın! İş kuralları net politikalardır: 'Üyenin en fazla 5 açık ödüncü olabilir.'

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never frame an NFR as a business rule (e.g. 'BR-001: Must respond quickly'). Business rules state definitive enterprise policy: 'A member may hold at most 5 concurrent loans.'

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- İş kuralları senaryo içine gömülmez; şartnamenin sonundaki Business Rules bölümünde listelenir.
- BR numaraları use case içinde yereldir; dışarıdan çağrılırken nitelikli yazılır (UC-001 BR-002).
- Kural metinlerini dosyalar arasında kopyalamak kesinlikle yasaktır; her kural bir kere yazılır.

🎯 Key Architectural Takeaways

- Business rules are declared in their own dedicated section, not embedded in step text.
- BR numbering is local per spec; cross-spec citations must be qualified (UC-001 BR-002).
- Cloning rule text across specifications is forbidden; define once and cite by identifier.

30

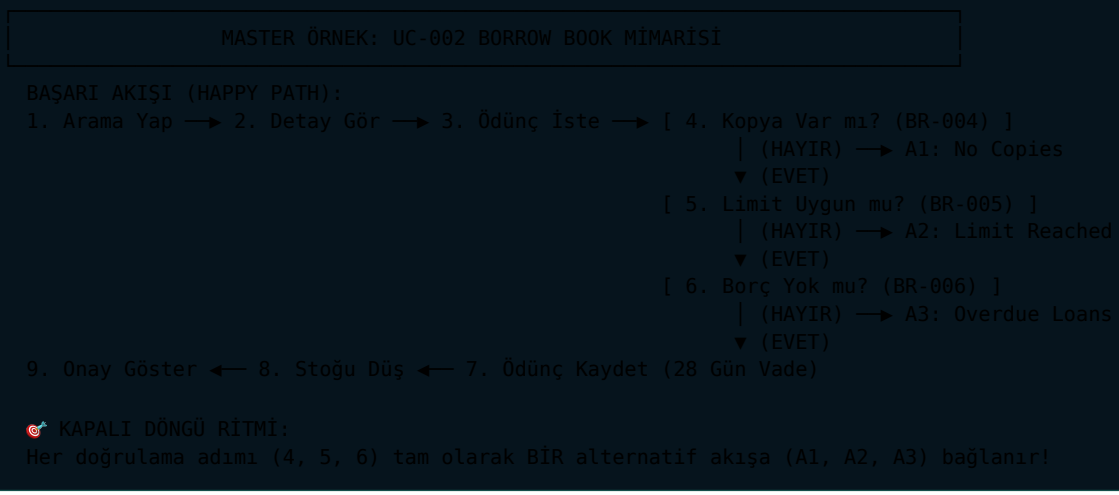
BÖLÜM: ANA ŞARTNAME ŞABLONU & ALAN ALAN DERINLEMESINE İNCELEME | MODULE: THE MASTER SPECIFICATION TEMPLATE & FIELD BREAKDOWN

Baştan Sona Örnek Analizi: 'Borrow Book' (Kitap Ödünç Al) Master Şartnamesi

End-to-End Walkthrough: Deconstructing the 'Borrow Book' Master Spec

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bu ders, bir yarış arabasının motorunu tamamen söküp parçalarını yere dizmeye benzer. 'Kitap Ödünç Al' şartnamesi, bir use case'in nasıl kusursuz yazılacağının şaheseridir: 1. adımda başlar, 9. adımda biter. 4, 5 ve 6. adımlarda 3 tane bekçi köpeği (doğrulama) oturur. Bir sorun çıkarsa köpek havlar ve seni A1, A2 veya A3 odasına yönlendirir. Sorun yoksa 28 gün vadeli kitabını alır, mutlu bir şekilde evine gidersin. Hiçbir adım havada kalmaz, hiçbir ipin ucu açıkta bırakılmaz!

🇬🇧 ENGLISH (ELI5)

This lesson disassembles a Formula-1 engine bolt by bolt. The canonical 'Borrow Book' specification is the gold standard of modern requirements engineering. Steps 1 to 9 march forward rhythmically. Steps 4, 5, and 6 serve as security checkpoints. If a checkpoint fails, the flow diverts cleanly to alternative branches A1, A2, or A3. If clear, the loan is committed for exactly 28 days. Every branch closes cleanly; zero dangling logic.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Borrow Book Şartnamesinin 5 Kusursuz Mimari Özelliği: Kitapta yer alan 'UC-002: Borrow Book' şartnamesi incelendiğinde her satırın arkasındaki derin mantık görülür: 1) Doğrulama ve Akış Simetrisi: 4, 5 ve 6. adımlar birer 'System validates' adımıdır. Her doğrulama adımı arkasında bir iş kuralına (BR-004, BR-005, BR-006) dayanır ve başarısızlık durumunda tam olarak bir alternatif akışa (A1, A2, A3) dallanır. 'Peki ya olmazsa?' sorusunun cevabı anında bulunur. 2) Varlık Modeli Uyumu: Metindeki 'member', 'book', 'copy', 'loan', 'reservation' kelimelerinin tamamı `docs/entity_model.md` dosyasında önceden tanımlanmıştır; use case içinde kolon tipi tartışılmaz. 3) Somut Rakamlar: '28 days' ve '5 open loans' ifadeleri mevcuttur; çünkü bunlar paydaşın onaylayacağı kesin iş rakamlarıdır. 4) Referansla Devretme (Handoff by Reference): A1 akışında kitap yoksa rezervasyon adımı için diğer use case'e link verilir: `(UC-005 Reserve Book)`. 5) Asgari Garanti: Başarısızlık sonkoşulu 'Hiçbir ödünç kaydı açılmaz ve stok değişmez' diyerek veritabanı rollback şartını belirler.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Architectural Forensics of the 'Borrow Book' Specification: Deconstructing the canonical spec reveals five structural virtues: 1) Validation-to-Flow Symmetry: Steps 4 through 6 are assertions of business rules (BR-004, BR-005, BR-006). Each assertion pairs with exactly one alternative flow (A1, A2, A3) detailing systemic recovery. 2) Entity Vocabulary Alignment: The nouns 'member', 'book', 'copy', 'loan', and 'reservation' exist verbatim in `docs/entity_model.md`. The spec never defines column schemas. 3) Concrete Business Quantities: Observable constraints like '28 days' and 'at most 5 loans' are stated numerically for stakeholder sign-off. Internal date-math algorithms are omitted. 4) Handoff by Reference: In flow A1, reservation creation delegates to `(UC-005 Reserve Book)` rather than embedding child flows inline. 5) Transactional Postconditions: The failure guarantee explicitly asserts that no partial loan is persisted and inventory counts remain unchanged, dictating `\@Transactional` boundaries to the AI generator.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Borrow Book' Master Şartnamesinin Birebir Java 25 Uygulaması
package com.maang.aiup.master;

import java.time.LocalDate;

public class MasterBorrowBookService {
    public record BorrowRequest(String memberId, String bookCopyId) {}
    public record LoanReceipt(String loanId, String memberId, String copyId, LocalDate dueDate) {}

    // Dışarıdan enjekte edilen bağımlılıklar (Hexagonal Outbound Ports)
    public interface BookInventoryPort {
        boolean isAvailable(String copyId); // BR-004
        void decrementStock(String copyId);
    }

    public interface MemberPort {
        int getActiveLoanCount(String memberId); // BR-005
        boolean hasOverdueLoans(String memberId); // BR-006
    }

    public interface LoanPersistencePort {
        String recordLoan(String memberId, String copyId, LocalDate dueDate);
    }

    private final BookInventoryPort inventory;
    private final MemberPort members;
    private final LoanPersistencePort loans;

    public MasterBorrowBookService(BookInventoryPort inv, MemberPort mem, LoanPersistencePort lns) {
        this.inventory = inv;
        this.members = mem;
        this.loans = lns;
    }

    public LoanReceipt borrowBook(BorrowRequest req) {
        // Step 4: System validates copy is available (BR-004 / A1)
        if (!inventory.isAvailable(req.bookCopyId())) {
            throw new IllegalStateException("A1: Kitap kopyası mevcut değil.");
        }
        // Step 5: System validates loan limit (BR-005 / A2)
        if (members.getActiveLoanCount(req.memberId()) >= 5) {
            throw new IllegalStateException("A2: Maksimum 5 ödünç limitine ulaşıldı.");
        }
        // Step 6: System validates no overdue loans (BR-006 / A3)
        if (members.hasOverdueLoans(req.memberId())) {
            throw new IllegalStateException("A3: Gecikmiş borçlar varken ödünç alınamaz.");
        }

        // Step 7: System records loan with due date 28 days from today
        LocalDate dueDate = LocalDate.now().plusDays(28);
        String loanId = loans.recordLoan(req.memberId(), req.bookCopyId(), dueDate);

        // Step 8: System decreases available copies by one
        inventory.decrementStock(req.bookCopyId());

        // Step 9: System confirms loan and delivers goal
        return new LoanReceipt(loanId, req.memberId(), req.bookCopyId(), dueDate);
    }

    void main() {
        System.out.println("Master Use Case (Borrow Book) mimari servisi hazır.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Borrow Book şartnamesinde '28 gün' yazmak yerine 'Sistem kütüphane kurallarına göre vade belirler' yazarsanız yapay zekayı belirsizliğe itersiniz. AI gider 14 gün yapar, testler patlar. Rakamları net yazın!

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Replacing 'due date 28 days from today' with 'system calculates appropriate due date' invites chaos. The AI might generate a 14-day default, breaking downstream acceptance assertions. State business quantities concretely.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Her 'validates' adımı arkasında bir iş kuralı taşır ve bir alternatif akışa dallanır.
- Kullanılan tüm isimler varlık modelinde mevcuttur; şema ve kolon detayları şartnamede yazılmaz.
- 28 gün ve 5 kitap gibi gözlemlenebilir sayılar şartnamede kesin olarak belirtilir.

🎯 Key Architectural Takeaways

- Every validation step rests on a business rule and links to an alternative flow.
- All nouns exist in the entity model; database schema details are never inlined.
- Observable quantities (28 days, 5 loans) are stated with numerical precision.

BÖLÜM 6 / MODULE 6

Senaryolar ve Alternatif Akışlar Yazma Sanatı

Writing Scenarios and Alternative Flows

Bir senaryo adımının anatomisi: Özne-Yüklem-Nesne, topun kimde olduğu kuralı, 'validates' vs 'checks whether' tercihi, Cockburn'ün 3 turlu alternatif akış beyin fırtınası, numaralı adıma bağlanan tetikleyiciler, açık uçlu akış yasağı ve kapalı döngü (closed-loop) ritmi.

The anatomy of a step: Subject-Verb-Object, who has the ball, 'validates' vs 'checks whether', Cockburn's 3-pass alternative flow brainstorming, anchored triggers, banned open-ended flows, and the closed-loop specification rhythm.

31

BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Bir Adımın Anatomisi: Özne-Yüklem-Nesne, Kuşbakışı Bakış & 'Top Kimde?' Kuralı

The Shape of a Step: Subject-Verb-Object & 'Who Has the Ball'

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              BİR ADIMIN ANATOMİSİ VE 'TOP KİMDE?' KURALI     │
└──────────────────────────────────────────────────────────────┘

❌ HATALI ADIM (ÖZNESİZ / EDİLGEN / İÇERİDEN BAKIŞ):
"Tutar bakiyeden düşülür." (Edilgen!)
"Düş bakiyeden 100 TL."   (Öznesiz!)
💥 AI Alarmı: Top kimde belli değil! UI mu yapacak, servis mi yapacak, banka mı?

✅ DOĞRU ADIM (ETKEN ÇATI / TEK ÖZNE / DIŞARIDAN KUŞBAKIŞI BAKIŞ):
[ ÖZNE (Topun Sahibi) ] + [ EYLEM / FİİL ] + [ NESNE ] + [ TÜMLEÇ ]
"Sistem"               + "düşer"          + "tutarı"  + "hesap bakiyesinden."
"Üye"                  + "seçer"          + "kitabı"  + "katalog listesinden."

🎯 KURAL: Her adımın İLK KELİMESİ o an hareket yapan aktörün adı olmalıdır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir basketbol maçını anlatan spikeri dinleyin: Spiker asla 'Top potaya girdi ve sayı oldu' deyip susmaz! 'Curry topu sürdü, pası LeBron'a attı, LeBron smacı bastı!' der. Her saniye topun kimin elinde olduğu bellidir. Eğer use case adımında 'Para çekildi' dersen spiker susmuş gibi olur; parayı hırsız mı çekti, veznedar mı verdi, ATM mi saydı bilemezsin! İlk kelime daima topu elinde tutan oyuncunun adı olmalıdır!

🇬🇧 ENGLISH (ELI5)

Listen to a live sports commentator: they never mutter passively 'The ball was hurled toward the hoop.' They say: 'Curry drives the lane, kicks the ball to LeBron, and LeBron slams it down!' At every fraction of a second, you know who holds the ball. If you write 'Money is debited', the ball-holder is invisible. Did a pickpocket steal it, or did the ledger process it? The first word of every step must name the actor holding the ball.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Bir Adımın Dilbilgisel ve Mimari Anatomisi: Cockburn use case yazımını 'temelde bir düzyazı denemesi egzersizi' olarak tanımlar. Ancak bu edebi bir üslup değil, katı bir mühendislik kuralıdır: 1) Basit Gramer: Özne, Yüklem, Nesne ve Edat öbeği. Şimdiki zaman ve etken çatı (Active Voice). 2) 'Top Kimde?' (Who Has the Ball): Her adımda tam olarak tek bir aktör hareket eder ve o aktör cümlenin ilk kelimesidir. Edilgen cümleler ('Para hesaba aktarılır') topun sahibini gizler. Oysa yapay zekâ için topun sahibi hayati önem taşır; çünkü o adımın bir Controller/UI kodu mu, bir Domain Servis kodu mu yoksa harici bir RestClient çağrısı mı olacağına bu özne karar verir! 3) Kuşbakışı Bakış (Bird's Eye View): Senaryo sistemin içinden dışarıya bakarak ('Kartı al, PIN doğrula, parayı say') yazılmaz; tüm etkileşimi yukarıdan izleyen tarafsız bir gözlemci gibi yazılır. İçeriden yazılan adımlarda kullanıcı kaybolur ve AI kullanıcı arayüzü üretmeyi unutur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Linguistic and Architectural Shape of a Step: 1) Grammatical Strictness: One sentence per step. Present tense, active voice. Subject - Verb - Direct Object - Prepositional Phrase ('System deducts amount from account balance'). Omitted subjects ('Deduct amount') force LLMs to infer the actor, frequently assigning server logic to client components. 2) 'Who Has the Ball': Exactly one actor acts per step, declared as the very first word. Passive voice ('The loan is recorded') conceals the ball-holder. The ball-holder is the primary signal an AI uses to partition responsibilities across the architecture: actor actions become UI events; system actions become domain service methods; secondary actor actions become outbound API adapters. 3) Bird's Eye Perspective: Frame interactions from above the entire ecosystem rather than from inside the server looking out ('Read card, dispense cash' loses the human). Retaining the actor guarantees that frontend views and forms are synthesized accurately.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Top Kimde' Kuralının Temiz Mimari Katmanlarına Dağılımı
package com.maang.aiup.stepshape;

public class BallHolderArchitecture {
    // 1. ÖZNE: Üye (UI / Presentation Layer) -> Top Üyede!
    public record MemberUiAction(String memberId, String bookId) {
        public void userClicksBorrowButton() {
            System.out.println("Üye butona tıkladı (Top Üyede): " + bookId);
        }
    }

    // 2. ÖZNE: Sistem (Domain Service Layer) -> Top Sistemde!
    public static class SystemDomainService {
        public void systemDeductsInventory(String bookId) {
            System.out.println("Sistem envanteri 1 azalttı (Top Sistemde): " + bookId);
        }
    }

    void main() {
        var ui = new MemberUiAction("M1", "BK-42");
        var service = new SystemDomainService();

        ui.userClicksBorrowButton();
        service.systemDeductsInventory("BK-42");
        System.out.println("✅ Adımların sorumluluğu katmanlara başarıyla yansıdı.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir adıma birden fazla aktörün hareketini sıkıştırmayın: 'Kullanıcı formu doldurur ve sistem bilgileri onaylar' YAZMAYIN! Bu iki ayrı adımdır. İlk adımda top kullanıcıda, ikinci adımda sistemdedir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never combine two actor actions into a single compound step (e.g. 'User submits form and system validates data'). Split them. Step 1: User submits form. Step 2: System validates data.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Her adım geniş zamanlı, etken çatılı ve tek bir cümledir; ilk kelime topun sahibidir.
- Edilgen cümleler ('Para aktarılır') aktörü gizler ve AI'ın kod katmanını şaşırmasına yol açar.
- Etkileşimi sistemin içinden değil, yukarıdan kuşbakışı izleyen tarafsız bir gözle yazın.

🎯 Key Architectural Takeaways

- A step is one active-voice sentence in present tense; the first word is the ball-holder.
- Passive voice obscures responsibilities, causing AI to misallocate logic across layers.
- Write from a bird's-eye view above the interaction, preserving the user's presence.

32

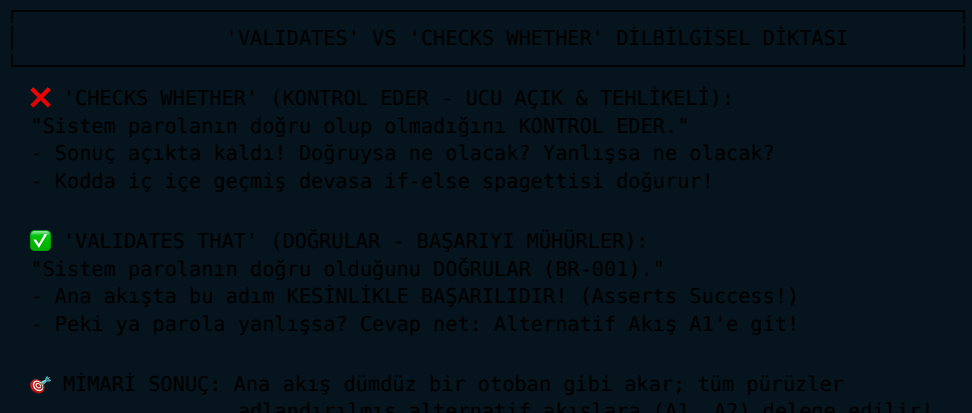
BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

'Validates' vs 'Checks Whether': Ana Akışta Başarıyı Mühürleme Sanatı

The 'Validates' vs 'Checks Whether' Dictum: Asserting Success in Main Flow

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Havalimanında pasaport polisine gittiğinde spiker şöyle demez: 'Polis pasaportun sahte olup olmadığına bakıyor ve düşünüyor.' Hayır! Ana senaryoda 'Polis pasaportun geçerli olduğunu doğrular' denir ve yürür geçersin. Eğer pasaport sahteyse, o zaman polis düğmeye basar ve seni yan odaya (Alternatif Akış A1) çekerler! 'Doğrular' kelimesi ana yolda yürümeyi sağlar; soru işaretlerini çöpe atar!

🇬🇧 ENGLISH (ELI5)

At passport control, the script does not read: 'Officer ponders whether the visa is forged.' The main scenario asserts: 'Border officer validates that the visa is authentic.' You proceed immediately to the gate. If forged, that failure diverts cleanly into Alternative Flow A1 (Detention Room). Using 'validates' asserts victory in the happy path and pushes failure into an explicit branch.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

'Validates' Kelimesinin Eklemeli (Additive) Mimari Gücü: Cockburn ve Martinelli'nin en radikal kuralı: 'Validates, never checks whether.' (Doğrular deyin, kontrol eder demeyin!). 'Checks whether' ifadesi iki ucu açık bir olasılıktır ve if-else şartlı dallanmalarını tetikler. Use case'in ana senaryosuna if-else sokulamaz! 'Sistem üyenin gecikmiş borcu olmadığını doğrular (BR-006)' dediğinizde, ana akışta bu adımın %100 başarılı olduğu mühürlenmiş olur. Bu tek bir kelime tercihi sisteme şu muhteşem mimari faydaları sağlar: 1) Eklemeli Yapı (Additive Structure): Ana senaryo baştan sona pürüzsüz akar; her bir başarısızlık ihtimali ise aşağıda bağımsız bir alternatif akış (A1, A2) olarak tanımlanır. 2) AI'a Hata Şablonu Verme: AI ajanı bu yapıyı gördüğünde kodun içinde spagetti if-else blokları kurmaz; doğrudan `if (!valid) throw new ValidationException()` veya Result/Either monadı üreterek akışı ilgili alternatif metoda delege eder.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Additive Architecture of 'Validates' vs 'Checks Whether': 'Checks whether' is an open-ended predicate that invites inline branching and nested conditionals within the main scenario. The main scenario must remain strictly linear and unconditionally successful. Writing 'System validates that password matches hash (BR-001)' asserts success in the primary flow. This single verb choice constructs the additive architecture of the specification: 1) The main scenario is an unbranching highway where every validation passes. 2) Deviations are strictly quarantined into numbered alternative flows. 3) For generative AI, this enforces clean architectural patterns: guard-clause rejections (`if (!valid) throw new DomainException()`) rather than tangled if-else pyramids.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Validates' Kuralının Guard-Clause Mimarisiyle Temiz İcrası
package com.maang.aiup.validates;

public class ValidationGuardArchitecture {
    // ❌ CHECKS WHETHER (Spagetti if-else):
    // if (checkPassword()) { if (checkLimit()) { doSomething(); } else { error(); } }

    // ✅ VALIDATES (Guard Clauses - Şartnamedeki her 'validates' bir guard'dır):
    public static void processBorrowWithGuards(boolean isCopyAvailable, boolean isLimitOk) {
        // Step 4: System validates copy is available (A1)
        if (!isCopyAvailable) {
            handleAlternativeA1();
            return;
        }

        // Step 5: System validates loan limit (A2)
        if (!isLimitOk) {
            handleAlternativeA2();
            return;
        }

        // Ana akış dümdüz devam eder!
        System.out.println("Tüm doğrulamalar geçti, ana akış başarıyla bitti.");
    }

    private static void handleAlternativeA1() { System.out.println("Dallanma A1: Kopya yok."); }
    private static void handleAlternativeA2() { System.out.println("Dallanma A2: Limit aşıldı."); }

    void main() {
        processBorrowWithGuards(true, true);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnamede 'Sistem kullanıcının bakiyesini kontrol eder' yazıp bırakırsanız AI bakiye yetersiz olduğunda ne yapacağını bilemez ve kafasına göre kullanıcıya eksi bakiye tanımlayabilir! 'Sistem bakiyenin yeterli olduğunu doğrular (BR-002)' yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Writing 'System checks customer balance' without asserting success leaves the failure path undefined. An AI agent might hallucinate overdraft credit. Write 'System validates that balance is sufficient (BR-002)' and bind failure to flow A1.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- 'Validates' ana senaryoda kesin başarıyı mühürler; 'checks whether' ise ucu açık if-else yaratır.
- Her 'validates' adımı başarısızlık ihtimalini isimlendirilmiş bir Alternatif Akışa devreder.
- Bu kural sayesinde ana akış dümdüz bir başarı otobanı gibi tertemiz kalır.

🎯 Key Architectural Takeaways

- 'Validates' asserts success in the main flow; 'checks whether' creates ambiguous branching.
- Every validation step delegates its failure outcome to a named alternative flow.
- This rule preserves the happy path as a linear, unbranching highway to the goal.

33

BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Alternatif Akışları Keşfetme Sanatı: Cockburn'ün 3 Turlu Beyin Fırtınası Yöntemi

Finding Alternative Flows: Cockburn's 3-Pass Brainstorming Method

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
COCKBURN'ÜN 3 TURLU ALTERNATİF AKIŞ BULMA YÖNTEMİ

TUR 1: BEYİN FIRTINASI (BRAINSTORM - Asla Çözüm Yazma!)
- Senaryodaki her 'validates' adımını al: "Bu doğrulama patlarsa ne olur?"
- Aktör hataları: "Yanlış girdi, zaman aşımı (timeout), vazgeçme?"
- Dış sistem hataları: "Ödeme gateway yanıt vermezse, SMS servisi çökerse?"
- İç donanım arızaları: "Para sayma haznesi sıkışırsa?"

TUR 2: RASYONALİZE ET & BİRLEŞTİR (RATIONALIZE)
- Sistemin tespit edemeyeceği şeyleri sil! ("Üye şifresini unuttu" -> tespit edilemez!)
  Bunu "3 kere yanlış şifre girildi" olarak yeniden yaz!
- Aynı sonucu veren arızaları birleştir: "Kart çizik, okuyucu bozuk" -> "Kart Okunamadı."

TUR 3: ÇÖZÜMÜ YAZ (WRITE THE HANDLING)
- Hayatta kalan her duruma A1, A2, A3 diye temiz akış adımlarını yaz!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir dedektif gibi suç mahallini inceliyorsun: 1. Turda aklına gelen her çılgın şüpheyi tahtaya yazarsın: 'Hırsız bacadan mı girdi, helikopterle mi indi, içerideki biri mi yardım etti?' (Asla hemen karar verme!). 2. Turda saçma olanları elersin: 'Evde baca yok, helikopter sesi duyulmadı, demek ki içeriden biri yardım etti' (Sadeleştir!). 3. Turda ise hayatta kalan tek ihtimalin peşine düşüp tutuklama planını yazarsın. Üçünü aynı anda yapmaya çalışırsan beynin yanar!

🇬🇧 ENGLISH (ELI5)

Think of a forensic detective: in Pass 1, brainstorm every wild hypothesis without filtering: 'Did the intruder rappel from a blimp? Tunnel through the basement? Use a master key?' In Pass 2, rationalize: cross off the blimp because radar was clear; collapse lock scratches into 'Forced Entry'. In Pass 3, write the formal arrest warrant. Trying to invent the solution while brainstorming the failure conditions drains cognitive energy.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Cockburn'ün 3 Turlu Yöntemi ve Eksik Akışların Maliyeti: Yazılım sistemlerinin kod karmaşıklığının %80'i mutlu yolda değil; alternatif akışlarda ve hata yönetiminde yaşar. Bir insan projesinde unutulan bir hata akışı, test aşamasında bir mühendisin 'Peki ya kartta bakiye yoksa ne olacak?' sorusuyla ortaya çıkabilirdi. AI çağında ise unutulan her alternatif akış, yapay zekanın kendi kafasından uydurduğu sahte bir davranışa dönüşür! Bu yüzden beyin fırtınası opsiyonel bir lüks değil, zorunlu bir güvenlik adımıdır: 1) Tur 1: Çözümü düşünmeden sadece tüm başarısızlık ihtimallerini listele. 2) Tur 2: Sistemin algılayamayacağı insan niyetlerini sistemsel olaylara dönüştür. Örneğin 'Üye PIN kodunu unuttu' sistem tarafından bilinemez; sistem sadece 'PIN girme süresi aşıldı (Timeout)' durumunu algılayabilir. Birbirinin aynı olan 5 hata sebebini tek bir başlıkta topla ('Kaydetme Başarısız'). 3) Tur 3: Sadece bu süzgeçten geçen gerçek durumlara A1, A2 akışlarını yaz.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Cockburn's 3-Pass Brainstorming Protocol: Eighty percent of enterprise code complexity resides in failure topologies. In legacy shops, an unstated failure mode surfaced during manual testing when an observant engineer raised an edge-case question. With autonomous LLM coders, an omitted failure mode produces invented behavior. Brainstorming is therefore mandatory: 1) Pass 1 (Exhaustive Brainstorming): Enumerate every conceivable failure without designing remedies. Sweep through every 'validates' step, actor input errors, timeout events, and secondary system outages. 2) Pass 2 (Rationalization): Discard undetectable phenomena ('User forgot password' is undetectable; reframe as 'Max failed attempts exceeded'). Merge identical outcomes ('Magnetic stripe scratched', 'Chip corrupted' merge into 'Card Unreadable'). 3) Pass 3 (Handling): Author the formal steps and termination semantics for the surviving conditions.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 3 Turlu Analizin Koddaki Karşılığı Olan Hata Hiyerarşisi
package com.maang.aiup.brainstorm;

public class AlternativeFlowHandling {
    // Tur 2'de sadeleştirilen ve sisteme mal edilen hata tipleri
    public sealed interface PaymentFailure permits CardUnreadable, InsufficientFunds, GatewayTimeout
{}

    public record CardUnreadable(String details) implements PaymentFailure {}
    public record InsufficientFunds(double required, double current) implements PaymentFailure {}
    public record GatewayTimeout(String gatewayName) implements PaymentFailure {}

    // Tur 3'te yazılan çözüm akışları (Pattern Matching ile kusursuz dallanma):
    public static void handleFailure(PaymentFailure failure) {
        switch (failure) {
            case CardUnreadable _ -> System.out.println("A1 Flow: Kart okunamadı, başka bir kart talep
edildi.");
            case InsufficientFunds f -> System.out.printf("A2 Flow: Bakiye yetersiz (Gereken: %.2f),
alternatif ödeme istendi.%n", f.required());
            case GatewayTimeout _ -> System.out.println("A3 Flow: Banka zaman aşımı, işlem kuyruğa
alındı.");
        }
    }

    void main() {
        handleFailure(new InsufficientFunds(100.0, 45.0));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Alternatif akışları düşünürken hemen çözüm yazmaya kalkışmayın. Hatanın nasıl çözüleceğini düşünmek, yeni hata durumlarını keşfetme enerjinizi tüketir. Önce tüm arızaları listeleyin, sonra çözümleri yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never attempt to write the handling logic while brainstorming failure conditions. Designing the solution exhausts the cognitive bandwidth required to uncover subtle system failure modes.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Alternatif akışlar 3 turda bulunur: 1. Beyin fırtınası yap, 2. Sadeleştir, 3. Çözümü yaz.
- Sistemin algılayamayacağı insan durumları ('şifreyi unuttu') sistemsel olaylara ('zaman aşımı') çevrilir.
- Aynı sonucu doğuran mikro hatalar tek bir genel akış başlığı altında birleştirilmelidir.

🎯 Key Architectural Takeaways

- Alternative flows are discovered in three passes: 1. Brainstorm, 2. Rationalize, 3. Author handling.
- Undetectable human states must be translated into detectable system events.
- Micro-failures yielding identical systemic outcomes must be rolled up into a unified flow.

34

BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Bir Alternatif Akışın Anatomisi: Adıma Bağlı Tetikleyici & Açık Uçlu Akış Yasağı

Anatomy of an Alternative Flow: Anchored Triggers & Explicit Endings

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│                BİR ALTERNATİF AKIŞIN KESİN ŞABLONU                │
└──────────────────────────────────────────────────────────┘

### A1: No Copies Available  ◄── [ Koşulun İsmi Başlık Olur ]

**Trigger:** No copy of the book is available (step 4) ◄── [ Hangi adımda koptu? ]
**Flow:**
1. System informs member that all copies are on loan.
2. System offers to place a reservation.
3. If member accepts, system records reservation (UC-005 Reserve Book).
4. Use case ends.  ◄── [ KESİN BİTİŞ! ASLA UCU AÇIK BIRAKILAMAZ! ]

🚫 YASAK: Bir akışın son satırında "Use case continues at step N." veya
         "Use case ends." yazmıyorsa, bu bir şartname hatasıdır (Spec Bug)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir tren yolculuğundasın ve makas değiştirip yan raya girdin. Makasın başında tabela yazar: 'Makas 4. kilometrede ayrıldı (Tetikleyici)'. Yan rayda birkaç yüz metre gidersin. Rayın sonunda ise iki ihtimal vardır: Ya ray tekrar ana hatta bağlanır ('Yolculuk 5. kilometreden devam eder') ya da ray bir takozla son bulur ('Yolculuk burada biter!'). Eğer rayı boşlukta bırakırsan tren uçurumdan yuvarlanır! Alternatif akışların ucu asla açık bırakılamaz!

🇬🇧 ENGLISH (ELI5)

Riding a train onto a siding track: the switch is explicitly marked ('Diverted at Kilometer 4'). You travel down the spur line for 500 meters. At the end of the spur, only two geometric outcomes are physically possible: either the track merges back into the main line ('Use case continues at step 5'), or it strikes a hydraulic buffer stop ('Use case ends'). Leaving a track hanging in mid-air derails the train. Open-ended flows are specification bugs.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Alternatif Akış Anatomisi ve Açık Uçlu Akış Yasağı: Cockburn bir alternatif akışı 'minyatür bir use case' olarak adlandırır. Bir alternatif akışın 3 zorunlu bileşeni vardır: 1) İsimlendirilmiş Başlık: Koşulun adını taşır (`### A1: No Copies Available`). 2) Adıma Çapalanmış Tetikleyici (Anchored Trigger): Sistemin neyi tespit ettiğini ve ana senaryonun HANGİ adımından koptuğunu kesin olarak belirtir: `**Trigger:** No copy of the book is available (step 4)`. Bu çapa hem insan denetçinin dalı bulmasını sağlar hem de AI'ın hata yakalama kodunu ilgili metodun içine gömmesini garantiler. Eğer hata bir aralıkta çıkabiliyorsa 'at any time between steps 2 and 6' yazılır. 3) Açık Bitiş Kuralı (Explicit Ending): Bir alternatif akış tam olarak iki şekilde bitebilir: a) `Use case continues at step N.` (Düzeltme yapıldı, ana yola geri dönüldü). b) `Use case ends.` (Hedefe ulaşılamadı, işlem iptal edildi). Sonu belirtilmemiş açık uçlu bir akış şartname hatasıdır (Spec Bug); insan geliştirici bunu omuz silkerek geçiştirebilir ama yapay zekâ kendi kafasından uyduruk bir son seçer.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Anatomy of an Alternative Flow and the Prohibition of Open-Endedness: An alternative flow is a miniature use case nested within the specification. It mandates three non-negotiable structural components: 1) Condition-Based Header: Named precisely after the observed system condition (`### A2: Loan Limit Reached`). 2) Step-Anchored Trigger: Declares what the system detected and anchors explicitly to a numbered main-scenario step: `**Trigger:** Member already has maximum loans (step 5)`. The step anchor enables AI code generators to attach the exception interceptor to the precise functional invocation. 3) Deterministic Termination: Exactly two mechanical endings exist: `Use case continues at step N.` or `Use case ends.` An alternative flow lacking an explicit terminal statement is a severe specification bug. While a human engineer might infer an ending, an LLM will invent one arbitrarily.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Adıma Çapalanmış ve Kesin Bitişli Alternatif Akış Metodu
package com.maang.aiup.flows;

public class AlternativeFlowAnatomyDemo {
    public record ReservationResult(boolean completed, String nextStep) {}

    // A1: No Copies Available (step 4)
    public static ReservationResult handleAlternativeA1(String bookId, boolean userAcceptsReservation)
{
        System.out.println("Step 4'ten A1 akışına girildi: Kopya bulunamadı.");

        // Flow Step 1: System informs member
        System.out.println("A1.1: Üyeye tüm kopyaların ödünçte olduğu bildirildi.");

        // Flow Step 2 & 3: Offer & Record Reservation
        if (userAcceptsReservation) {
            System.out.println("A1.3: Rezervasyon kaydedildi (UC-005 Reserve Book).");
        }

        // Flow Step 4: KESİN BİTİŞ (Use case ends!)
        System.out.println("A1.4: Use case ends.");
        return new ReservationResult(true, "USE_CASE_ENDS");
    }

    void main() {
        handleAlternativeA1("BK-101", true);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir alternatif akışın içine bir alternatif akış daha açıp (iç içe 3 katmanlı dallanma) şartnameyi labirente çevirmeyin! 2. seviyeden daha derin bir dallanma ihtiyacı doğduysa, o akış use case olmaktan çıkmış demektir; onu ayrı bir use case olarak sisteme ekleyin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never nest alternative flows more than one level deep. If an alternative flow branches into secondary and tertiary sub-alternatives, the abstraction is broken: promote the flow into its own standalone sea-level use case.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Her alternatif akışın tetikleyicisi ana senaryodaki numaralı bir adıma çapalanır (step N).
- Bir akış yalnızca iki şekilde bitebilir: 'Use case continues at step N.' veya 'Use case ends.'
- Ucu açık bırakılmış akışlar birer şartname hatasıdır ve AI'ın uydurma kararlar almasına yol açar.

🎯 Key Architectural Takeaways

- Every alternative flow trigger must anchor to a numbered main step (step N).
- Flows terminate strictly via 'Use case continues at step N.' or 'Use case ends.'
- Open-ended flows are specification defects that provoke arbitrary AI hallucinations.

35

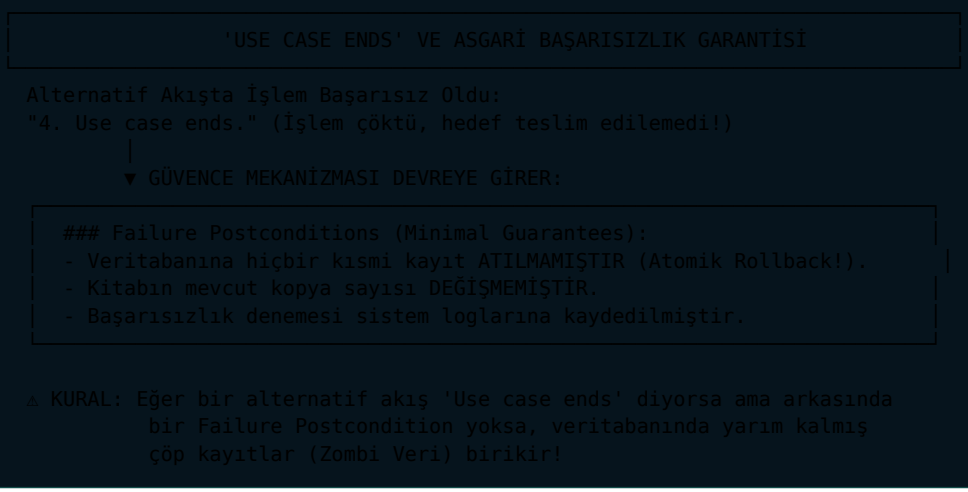
BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Toplam Başarısızlığı Yönetmek: 'Use Case Ends' Neden Başarısızlık Sonkoşulunu Şart Koşar?

Handling Total Failures: Why 'Use Case Ends' Demands Failure Postconditions

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir ameliyat sırasında anestezi uzmanı 'Tansiyon çok düştü, ameliyatı hemen durduruyoruz (Use case ends!)' dediğinde doktorlar hastayı öylece masada bırakıp eve gitmez! Yarayı hemen dikerler, kanamayı durdururlar, hastayı yoğun bakıma alıp rapor yazarlar (Başarısızlık Sonkoşulu!). İşte 'Use case ends' dendiğinde sistemin arkasında yaralı hasta veya açık yara (yarım kaydedilmiş bozuk veri) bırakmaması bu garantiler sayesinde sağlanır.

🇬🇧 ENGLISH (ELI5)

If a surgical team encounters an emergency and aborts an operation ('Use case ends'), the surgeons do not drop their scalpels and stroll out of the hospital. They immediately suture the incision, stabilize vitals, and transfer the patient to intensive care (Failure Postconditions). 'Use case ends' is only tolerable because the minimal guarantees promise that the patient survives clean and uncorrupted.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

'Use Case Ends' ve İşlemsel Geri Alma (Rollback) Mimarisi: Bir alternatif akış 'Use case ends' dediğinde, birincil aktör hedefine ulaşamadan sistemden ayrılmış demektir. Cockburn bu durumun sistem açısından kabul edilebilir olmasının TEK BİR şartı olduğunu belirtir: 'Failure Postconditions' (Asgari Başarısızlık Garantileri) bölümünün eksiksiz tanımlanmış olması. Eğer bir akış başarısızlıkla sonuçlandığında arkasında silinmemiş bir geçici tablo kaydı, düşülmüş ama iade edilmemiş bir bakiye veya kilitli kalmış bir stok bırakıyorsa, sistem zehirlenir. AI açısından bu durum hayati bir mimari ayrıntıdır: LLM ajanı bir kod üretirken metottaki `try-catch` ve `\@Transactional` bloklarının içine ne yazacağını bu başarısızlık sonkoşullarına bakarak belirler. Eğer şartnamede 'Failure Postconditions: No partial data is stored' yazıyorsa, AI veritabanı işlemlerini atomik bir transaction ile sarar ve hata anında rollback tetikler.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Transactional Mechanics of Total Failure: Declaring 'Use case ends' signifies that the primary actor departs without achieving their goal. Cockburn asserts that this total failure is acceptable if and only if the system's Minimal Guarantees hold true. If an aborted flow leaves partial records, decremented inventory without a corresponding order, or orphaned connection handles, the system suffers state corruption. For AI code generators, the Failure Postconditions block acts as the specification for transactional exception handling. Stating 'No loan is recorded and book copy count is unchanged' instructs the AI to generate atomic `\@Transactional(rollbackFor = Exception.class)` boundaries with zero manual intervention.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Failure Postconditions Sayesinde Otomatik Rollback Yapan İşlem Modeli
package com.maang.aiup.failure;

public class TransactionalFailureGuaranteeDemo {
    public static class InventoryRepository {
        private int stock = 10;
        public void decrement() { stock--; }
        public void rollback() { stock++; }
        public int getStock() { return stock; }
    }

    // Failure Postcondition Garantisi: Hata çıkarsa stok ASLA değişmez!
    public static void executeWithMinimalGuarantee(InventoryRepository repo, boolean forceFail) {
        int initialStock = repo.getStock();
        repo.decrement(); // Kısmi işlem yapıldı

        try {
            if (forceFail) {
                throw new RuntimeException("Ödeme başarısız oldu (Use case ends)!");
            }
        } catch (Exception e) {
            // Failure Postcondition gereği sistem durumu geri alır:
            repo.rollback();
            assert repo.getStock() == initialStock : "KRİTİK HATA: Stok tutarsız kaldı!";
            System.out.println("🛡️ Failure Postcondition doğrulandı: Stok eski haline döndürüldü -> "
+ repo.getStock());
        }
    }

    void main() {
        var repo = new InventoryRepository();
        executeWithMinimalGuarantee(repo, true);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir alternatif akışın sonunda 'Use case ends' yazdıktan sonra sistemin ne halde kalacağını Failure Postconditions'a yazmayı unutursanız, AI hata anında hiçbir veriyi geri almayan ve veritabanını bozan bir kod üretecektir.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Omitting Failure Postconditions while invoking 'Use case ends' guarantees that the generated code will lack transactional rollback handlers, resulting in corrupted database state under error conditions.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• 'Use case ends' ancak ve ancak Başarısızlık Sonkoşulları (Minimal Guarantees) sağlandığında geçerlidir.

• Başarısızlık garantisi hiçbir kısmi verinin kaydedilmediğini ve sistemin temiz kaldığını taahhüt eder.

• Yapay zekâ veritabanı rollback ve loglama mekanizmasını bu sonkoşullara bakarak inşa eder.

🎯 Key Architectural Takeaways

• 'Use case ends' is valid only when backed by explicit Failure Postconditions.

• Failure guarantees promise that no partial state is persisted and pre-state is restored.

• AI synthesizes transactional rollback and audit handlers directly from failure postconditions.

36

BÖLÜM: SENARYOLAR VE ALTERNATIF AKIŞLAR YAZMA SANATI | MODULE: WRITING SCENARIOS AND ALTERNATIVE FLOWS

Tamamlanmış Şartnamenin Ritmi: Sıfır Açık Uç ve Kapalı Döngü (Closed Loop) Mimarisi

The Rhythm of a Finished Spec: Closed Loops with Zero Dangling Threads

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              KUSURSUZ ŞARTNAMENİN KAPALI DÖNGÜ (CLOSED LOOP) RİTMİ        │
└──────────────────────────────────────────────────────────┘

 1. HER ANA SENARYO ADIMI KESİNLİKLE BAŞARILIDIR. (Dallanma yok!)
    │
    ▼
 2. HER 'VALIDATES' ADIMININ TAM OLARAK BİR ALTERNATİF AKIŞ KARŞILIĞI VARDIR.
    (Step 4 Validates ──► A1 Flow)
    │
    ▼
 3. HER ALTERNATİF AKIŞ NUMARALI BİR ADIMDA BAŞLAR, KESİN BİR ÇIKIŞLA BİTER.
    (Trigger: Step 4 ──► Ends / Continues at Step N)
    │
    ▼
 4. HER 'ENDS' BİR BAŞARISIZLIK SONKOŞULUYLA (FAILURE POSTCONDITION) DESTEKLENİR.
    │
    ▼
 5. ATIFTA BULUNULAN HER (BR-XXX) İŞ KURALI LİSTESİNDE TANIMLIDIR.

 🔒 SIFIR AÇIK UÇ! Bu kapalı döngü yapay zekanın uydurma yapmasını İMKÂNSIZ kılar!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir elektrik devresini düşünün: Pilden çıkan kablo ampule gider, ampulden anahtara, anahtardan tekrar pile döner. Eğer devrede tek bir kablonun ucu havada sallanıyorsa ampul yanmaz (Açık devre!). Kusursuz bir use case şartnamesi kapalı bir elektrik devresi gibidir: Adım başlar, kurala bağlanır, hataya bağlanır, sonkoşula bağlanır ve döngü tamamlanır. Tek bir açık uç kalmadığında sistem ışık saçar!

🇬🇧 ENGLISH (ELI5)

Consider an electrical circuit: current flows from the battery to the switch, through the lamp, and returns cleanly to ground. If a single wire dangles disconnected, no current flows. A finished use case specification is a closed electrical circuit: every validation links to a rule, every rule links to a flow, and every flow terminates cleanly. When there are zero dangling threads, the AI generates software that works on the first run.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Kapalı Döngü Şartname Ritim ve Denetim Kuralları: Martinelli, kaliteli bir use case dökümanının 'kendi içinde kapalı bir döngü' (closed loop) oluşturması gerektiğini belirtir. Bu ritim review (inceleme) sırasında gözle veya `/spec-review` ile kolayca doğrulanabilir: 1) Kural 1: Ana senaryodaki her adım başarıyla biter; if-else veya dallanma barındırmaz. 2) Kural 2: Ana senaryoda kaç tane 'validates' kelimesi geçiyorsa, alternatif akışlarda tam olarak o sayıda A1, A2 akışı bulunmalıdır; ne eksik ne fazla! 3) Kural 3: Her alternatif akışın tetikleyicisi ana senaryodaki numaralı bir adıma çapalanır ve ya 'Use case continues at step N' ya da 'Use case ends' ile biter. 4) Kural 4: Senaryodaki her 'Use case ends' ifadesi, Failure Postconditions bölümündeki en az bir garantiyle örtüşür. 5) Kural 5: Metin içinde parantezle referans verilen her `(BR-XXX)` kuralı, dökümanın altındaki Business Rules listesinde eksiksiz mevcuttur. Bu 5 kural sağlandığında şartnamede tek bir 'açıkta kalmış ip' (dangling thread) kalmaz. İşte bu kapalı döngü ritmi, şartnameyi generatif (AI tarafından %100 kodlanabilir) kılan mucizevi sırdır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Closed-Loop Specification Rhythm: A production-ready use case forms a closed relational loop where no logical thread dangles unmoored. Reviewers audit this rhythm via five invariants: 1) Invariant 1: Every main-scenario step succeeds unconditionally; no branches exist. 2) Invariant 2: Every step containing the verb 'validates' is matched by exactly one alternative flow resolving its failure. 3) Invariant 3: Every alternative flow anchors to a numbered step and terminates deterministically. 4) Invariant 4: Every 'Use case ends' terminal state is insured by a corresponding Failure Postcondition assertion. 5) Invariant 5: Every rule cited as `(BR-XXX)` exists in the Business Rules section, and every business rule is cited at least once. This closed loop provides total architectural containment: an AI agent cannot execute an unhandled path without directly violating a line of written text.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Kapalı Döngü (Closed Loop) Ritmine Sahip Kusursuz Use Case Denetçisi
package com.maang.aiup.closedloop;

public class ClosedLoopAuditor {
    public record SpecAuditMetrics(int validationSteps, int alternativeFlows, boolean
hasFailurePostconditions) {
        public boolean isClosedLoop() {
            // Ritim 2 & 4: Doğrulama sayısı akış sayısına eşit olmalı ve sonkoşul bulunmalıdır!
            return validationSteps == alternativeFlows && hasFailurePostconditions;
        }
    }

    void main() {
        // Borrow Book Şartnamesinin Ritim Sayımı: 3 Doğrulama (Adım 4,5,6) = 3 Akış (A1,A2,A3)
        var borrowBookSpec = new SpecAuditMetrics(3, 3, true);

        if (borrowBookSpec.isClosedLoop()) {
            System.out.println("✅ Şartname Kapalı Döngü (Closed Loop) ritmine sahip: Sıfır açık
uç!");
        } else {
            System.err.println("❌ Şartname ritmi bozuk: Eksik akış veya sonkoşul tespit edildi!");
        }
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnamenizde bir 'System validates' adımı olup da altında o adımı karşılayan bir alternatif akış (A1, A2) yoksa bu ölümcül bir açıktır. AI o doğrulamayı kodlar ama başarısız olduğunda sistemin çökeceği bir RuntimeException fırlatıp kullanıcıyı ekranda dondurur.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Leaving a 'System validates' step without a corresponding Alternative Flow creates a critical flaw. The AI will generate a validation check that throws an unhandled server error, locking the frontend UI.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Kapalı döngü şartnamede her 'validates' bir akışa, her akış ise bir sonkoşula bağlanır.
- Hiçbir açık uç bırakılmadığında yapay zekanın kendi kafasından kural uydurması imkansızlaşır.
- Bu kapalı döngü ritmi hem insan incelemesini kolaylaştırır hem de otomatik test üretimini besler.

🎯 Key Architectural Takeaways

- A closed loop connects every validation to a flow, and every flow to a postcondition.
- Zero dangling threads makes it mathematically impossible for an AI to hallucinate logic.
- This closed-loop rhythm accelerates human reviews and powers automated test generation.

BÖLÜM 7 / MODULE 7

Yapay Zekâ İçin Hassasiyet: Ne Söylenmeli, Ne Dışarıda Bırakılmalı?

Precision for AI – What to Say and What to Leave Out

Klasik Cockburn kuralı ile AI çağının çatışması: Gözlemlenebilir davranış (Zone 1), alan modeli referansı (Zone 2) ve yasaklı mekanizma detayları (Zone 3). Kara liste (JWT, SQL, regex, SMTP), UI tasarımcısının üçlü bölünmesi ve 3 soruluk revizyon filtresi.

The collision between classical restraint and AI demands: Observable behavior (Zone 1), domain references (Zone 2), and forbidden mechanisms (Zone 3). The banned blocklist (JWT, SQL, regex, SMTP), the three-way UI split, and the 3-question revision pass.

37

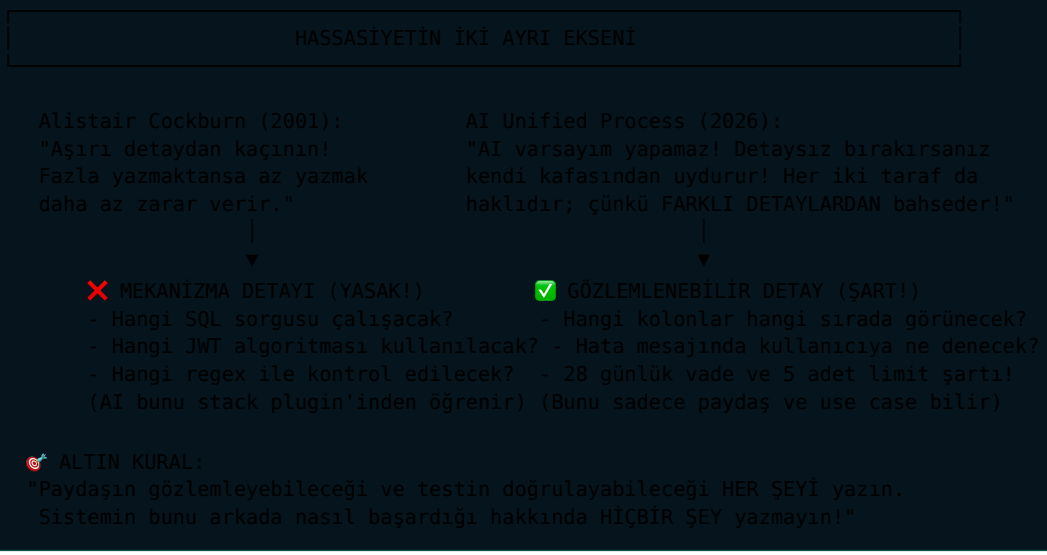
BÖLÜM: YAPAY ZEKÂ İÇİN HASSASIYET: NE SÖYLENMELI, NE DIŞARIDA BIRAKILMALI? | MODULE: PRECISION FOR AI – WHAT TO SAY AND WHAT TO LEAVE OUT

Hassasiyet Paradoksu: Gözlemlenebilir Davranış vs Mekanizma Detayı

The Precision Paradox: Observable Behavior vs Mechanism Detail

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restorana gidip şefe sipariş verdiğinizi düşünün: Şefe 'Bana orta pişmiş antrikot, yanında kuşkonmaz ve patates püresi ile saat 20:00'de gelsin' dersiniz (Gözlemlenebilir istek!). Ama şefe 'Fırını 230 dereceye ayarla, tavayı 3 dakika ısıt, tuzu sol elle at' derseniz şef sizi mutfaktan kovar (Mekanizma detayı!). İşte use case şefe ne istediğinizi söyler, mutfak aletlerinin kullanımını şefe bırakır!

🇬🇧 ENGLISH (ELI5)

Imagine ordering at a fine restaurant: You tell the chef 'Bring me medium-rare ribeye steak with grilled asparagus and mashed potatoes at 8:00 PM' (Observable requirement!). But if you instruct the chef: 'Set the stove to 230 degrees, preheat the pan for 3 minutes, and sprinkle salt with your left hand', the chef kicks you out (Mechanism detail!). The use case defines what lands on the table; it lets the kitchen execute the technique.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Hassasiyet Paradoksunun Çözümü: Klasik use case literatüründe Cockburn, detay yazmanın tehlikeleri konusunda onlarca sayfa uyarıda bulunmuştur. Detay belgeyi kırılganlaştırır, bakım maliyetini artırır ve yetenekli geliştiricilerin yaratıcılığını çalar. Ancak günümüz AI destekli dünyasında Martinelli, 'Kitap listesini gösterir' gibi gevşek adımların AI tarafından tamamen rastgele kolonlarla üretilmesine sebep olduğunu gösterir. Çözüm, detayı tek bir eksen değil, iki zıt eksen olarak görmektir: 1) Gözlemlenebilir Detay (Observable Detail): Kullanıcının ekranda gördüğü veri, sıralama ölçütü, iş mantığı sınırları (28 gün, 5 kitap). Bu detay eksik bırakılırsa AI boşluğu kendi tahminleriyle doldurur ve sistem mimarisi çöker. 2) Mekanizma Detayı (Mechanism Detail): HTTP fiilleri, SQL transaction izolasyon seviyeleri, hash algoritmaları, regex desenleri. Bu detay yazılırsa şartname koda bağımlı hale gelir ve yeniden üretilebilirlik (regenerability) yok olur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Resolving the Precision Paradox: Cockburn warned extensively against over-specification because detail in traditional engineering is expensive to maintain, fragile against refactoring, and strips decisions from experienced developers. However, generative AI inverts this dynamic: an under-specified step like 'System displays books' forces the LLM to invent arbitrary table columns, sorting heuristics, and layout fields. Martinelli resolves this tension via a single axiom: 'Specify everything a stakeholder could observe and a test could verify. Specify nothing about how the system achieves it.' Detail is not one-dimensional; it splits into observable outcomes (required) versus underlying mechanisms (strictly banned).

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Gözlemlenebilir İş Sonucunun Mekanizmadan Ayrılması
package com.maang.aiup.precision;

import java.time.LocalDate;
import java.util.List;

public class PrecisionParadoxDemo {
    // 1. GÖZLEMLENEBİLİR İŞ SONUCU (Paydaş ve Test Bunu Görür!)
    public record BorrowConfirmationView(
        String bookTitle,
        String memberName,
        LocalDate dueDate,
        int remainingLoanQuota
    ) {}

    // 2. MEKANİZMA (Use case'de ASLA geçmez; Spring Data, SQL, Redis buraya aittir)
    public interface LoanRepositoryPort {
        void persistLoanRecord(String memberId, String bookId, LocalDate dueDate);
    }

    // 3. USE CASE İCRA EDİCİSİ (Sadece gözlemlenebilir kuralı işletir)
    public static class BorrowBookInteractor {
        public BorrowConfirmationView execute(String memberName, String title, int currentLoans) {
            if (currentLoans >= 5) { // BR-005 kuralı
                throw new IllegalStateException("Ödünç limiti aşıldı! En fazla 5 kitap.");
            }
            LocalDate dueDate = LocalDate.now().plusDays(28); // 28 gün kuralı
            return new BorrowConfirmationView(title, memberName, dueDate, 5 - (currentLoans + 1));
        }
    }

    void main() {
        var interactor = new BorrowBookInteractor();
        var result = interactor.execute("Alice", "Designing Data-Intensive Apps", 2);
        System.out.println("✅ Gözlemlenebilir Sonuç: " + result);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

'Sistem kullanıcıyı uygun şekilde bilgilendirir' veya 'verileri gösterir' gibi yuvarlak ifadeler kullanmayın. Bu kelimeler AI'a kafasına göre uydurma yetkisi (blank check) verir!

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never use vague escape phrases like 'System notifies user appropriately' or 'System shows relevant data'. Every ambiguous word delegates architecture decisions to AI halluncinations.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Paydaşın görebileceği ve testin doğrulayabileceği her şeyi somut olarak yazın.

• Sistemin bunu nasıl yaptığı (SQL, JWT, regex) use case şartnamesine asla yazılamaz.

• Eksik gözlemlenebilir detay, AI'ın kafasına göre kod uydurmasına davetiye çıkarır.

🎯 Key Architectural Takeaways

• Specify everything a stakeholder can observe and an acceptance test can verify.

• Never specify implementation mechanisms (SQL, JWT, regex) in use case specifications.

• Omitted observable details force the AI to invent business behavior arbitrarily.

38

BÖLÜM: YAPAY ZEKÂ İÇİN HASSASIYET: NE SÖYLENMELI, NE DIŞARIDA BIRAKILMALI? | MODULE: PRECISION FOR AI – WHAT TO SAY AND WHAT TO LEAVE OUT

Zone 1 (Gözlemlenebilir Davranış): Somut Arayüz Yapısı, Kolonlar ve İş Miktarları

Zone 1 (Observable Behavior): Concrete Interface Structure, Columns & Quantities

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              ZONE 1: GÖZLEMLENEBİLİR DAVRANIŞ HARİTASI   │
└──────────────────────────────────────────────────────────┘

❌ YETERSİZ TANIMLAMA (UNDER-SPECIFIED):
1. "Sistem kitapları gösterir."
2. "Sistem kullanıcıyı borçlar hakkında uyarır."
💥 AI Ne Yapar?: Hangi kolonları koysun? ID mi, Basım yılı mı? Uyarıda ne yazsın?

✅ ZONE 1 SOMUT TANIMLAMA (CONCRETE & AUDITABLE):
1. "Sistem kitapları Başlık (artan alfabetik), Yazar, ISBN ve Mevcut Kopya
    kolonlarından oluşan bir tabloda listeler."
2. "Sistem kullanıcıya gecikmiş borçları nedeniyle ödünç alma işleminin
    engellendiğini bildirir ve geciken kitapların listesini gösterir."

📋 ZONE 1 İÇİNDE YER ALANLAR:
├── 1. Görüntülenen alanlar ve sıralama (Örn: 'Title ascending')
├── 2. Kesin iş miktarları (Örn: '28 gün', 'en fazla 5 ödünç')
├── 3. Hata/Onay mesajının içeriği (Widget tipi değil, anlamsal içerik!)
└── 4. Bir kerede girilen veriler (Entity adıyla paketlenmiş)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Anneannenize market alışveriş listesi yazdığınızı düşünün: 'Bana biraz meyve al' derseniz anneanneniz erik yerine greyfurt getirebilir ve suçlayamazsınız! Ama '3 tane Amasya elması, 1 kilo yerli muz' yazarsanız tam istediğiniz gelir. Zone 1, alışveriş listesindeki meyvenin tam adı ve adedidir. Widget'ın poşet mi file mi olduğu önemli değildir; içindeki elmanın cinsi bellidir!

🇬🇧 ENGLISH (ELI5)

Think of writing a grocery list: If you write 'Buy some fruit', you might get sour lemons instead of sweet strawberries, and you can't complain! But if you write '3 Honeycrisp apples and 1 kg of bananas', you get precisely what you crave. Zone 1 is the exact fruit and quantity. Whether carried in a plastic bag or a wicker basket is irrelevant; the fruit itself is unambiguous.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Zone 1'in Derin Mühendisliği (Observable Behavior): AI destekli kod üretiminde şartnamelerin en çok tökezlediği yer Zone 1 eksikliğidir. Geliştirici 'Sistem sipariş detayını gösterir' yazdığında, insan bir yazılımcı şirketin önceki ekranlarına bakarak siparişte hangi alanların olduğunu hatırlar. Ancak bir LLM için bağlam sınırlıdır; 'sipariş detayı' adımından yola çıkarak rastgele DTO'lar, gereksiz kolonlar veya eksik alanlar üretir. Martinelli Zone 1 kurallarını şöyle netleştirir: 1) İsimlendirilmiş Kolonlar ve Sıralama: Tabloda hangi kolonların çıkacağı ve listenin neye göre sıralanacağı yazılmalıdır ('Sistem Grid üzerinde Başlık, Yazar, ISBN ve Stok kolonlarını Başlığa göre artan sırada gösterir'). 2) Kesin İş Miktarları: Asla 'yakında', 'birkaç tane' denemez; 'bugünden itibaren 28 gün sonra', 'en fazla 5 açık ödünç' denmelidir. 3) Hata Mesajının Anlamsal Özü: 'Hata gösterir' denemez; 'Ödünç almanın geciken kitaplar yüzünden engellendiğini bildirir ve geciken kitapları listeler' denmelidir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Zone 1 Precision Engineering: Under-specification in Zone 1 is the single greatest cause of prompt hallucination and code drift. When an engineer specifies 'System displays order details', human developers infer missing fields from adjacent legacy screens. An AI model, having no coffee-machine memory, invents arbitrary fields or omits mission-critical data. Zone 1 demands explicit observable facts: 1) Named Columns and Sort Orders: Define exact fields and orderings ('Grid showing Title, Author, ISBN, Available Copies sorted by Title ascending'). 2) Exact Business Quantities: Never write 'shortly' or 'a few'; declare exact constants ('due date 28 days from today', 'at most 5 loans'). 3) Message Content, Not Widgets: Never prescribe dialog boxes; mandate observable content ('System informs member that borrowing is blocked while loans are overdue and lists the overdue items').

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Zone 1 Gözlemlenebilir Veri Sözleşmesi (Named Columns & Ordering)
package com.maang.aiup.zone1;

import java.util.Comparator;
import java.util.List;

public class CatalogDisplayService {
    // Zone 1 Tanımı: Title, Author, ISBN, AvailableCopies
    public record BookCatalogItem(String title, String author, String isbn, int availableCopies) {}

    public static class CatalogQueryHandler {
        public List<BookCatalogItem> fetchSortedCatalog(List<BookCatalogItem> rawItems) {
            // Şartname Kuralı: "sorted by Title ascending"
            return rawItems.stream()
                .sorted(Comparator.comparing(BookCatalogItem::title, String.CASE_INSENSITIVE_ORDER))
                .toList();
        }
    }

    void main() {
        var items = List.of(
            new BookCatalogItem("The Pragmatic Programmer", "Hunt & Thomas", "978-0201616224", 3),
            new BookCatalogItem("Clean Code", "Robert C. Martin", "978-0132350884", 0),
            new BookCatalogItem("Design Patterns", "GoF", "978-0201633610", 2)
        );

        var service = new CatalogQueryHandler();
        var sorted = service.fetchSortedCatalog(items);

        System.out.println("✅ Zone 1 Uyumu: Başlığa göre sıralı katalog tablosu:");
        sorted.forEach(b -> System.out.println("   -> [" + b.title() + "] by " + b.author() + " (Stok:
" + b.availableCopies() + ")"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Zone 1'de arayüz bileşenlerinin adlarını yazmayın: 'Modal popup açılır', 'Dropdown menü gösterilir' veya 'Toast mesajı fırlar' YAZMAYIN! Bunlar Zone 3 ihlalidir. Sadece içeriği ve aksiyonu söyleyin.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never mandate concrete UI widgets in Zone 1 (e.g., 'A modal pop-up appears' or 'Dropdown renders'). Specify the information conveyed and user actions, not visual presentation widgets.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Tablolardaki kolon isimlerini ve sıralama ölçütünü (sort order) şartnamede açıkça belirtin.

• İş kurallarındaki sayısal sınırları ('28 gün', '5 limit') net olarak yazın; yuvarlak laflardan kaçının.

• Hata mesajlarının biçimini değil, kullanıcıya ilettiği zorunlu bilgi içeriğini tarif edin.

🎯 Key Architectural Takeaways

• Explicitly state displayed column names and sorting orders in the specification.

• Use precise business quantities ('28 days', '5 loans') rather than vague approximations.

• Specify the informative payload of error messages, not the UI container or toast widget.

39

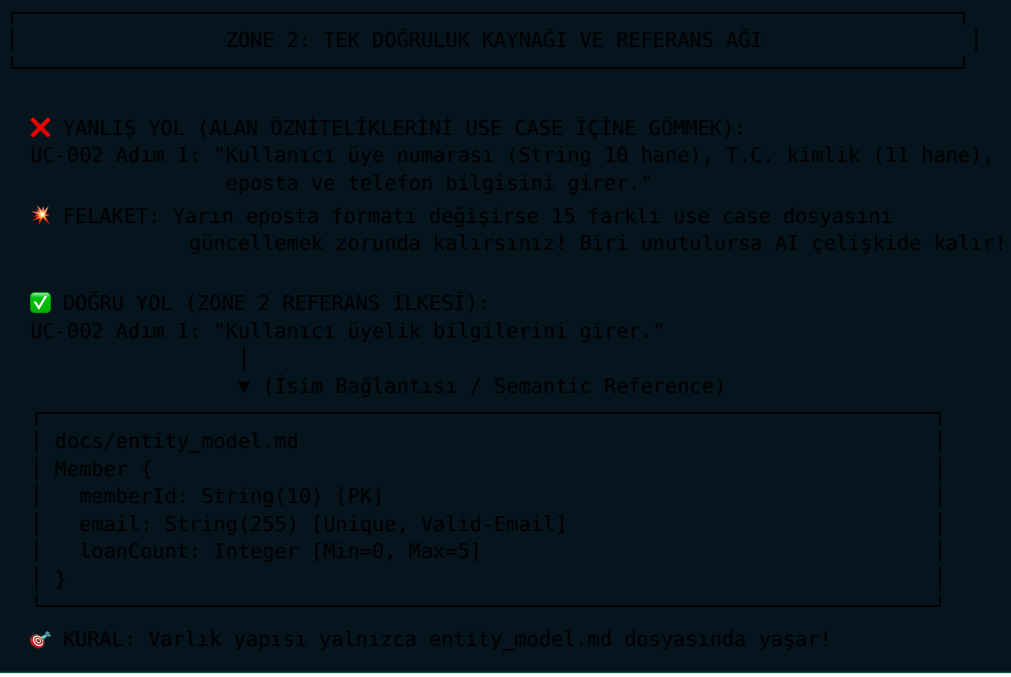
BÖLÜM: YAPAY ZEKÂ İÇİN HASSASIYET: NE SÖYLENMELI, NE DIŞARIDA BIRAKILMALI? | MODULE: PRECISION FOR AI – WHAT TO SAY AND WHAT TO LEAVE OUT

Zone 2 (Alan Modeli): Referans Ver, Asla Tekrarlama (entity_model.md)

Zone 2 (Domain Data): Reference, Never Restate (entity_model.md Truth)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Ev adresinizi düşünün: Arkadaşlarınıza her seferinde 'Dünya gezegeni, Türkiye cumhuriyeti, İstanbul ili, Kadıköy ilçesi, Moda caddesi, No:5 Daire:3' diye 5 sayfa anlatmazsınız. 'Moda'daki evime gel' dersiniz; herkes navigasyona (Harita / Entity Model) bakar. Use case adresi verir, haritanın kendisini içine çizmez!

🇬🇧 ENGLISH (ELI5)

Think of giving your home address: You don't recite: 'Planet Earth, coordinates 41.0082 N, continent Europe, district Kadikoy, apartment 3…' Every time someone asks. You say 'Come to my place in Moda.' They look up the map (Entity Model). The use case supplies the semantic reference; the entity model holds the authoritative schema.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Zone 2'nin DRY (Don't Repeat Yourself) İlkesi: Klasik yazılım mühendisliğinde kodun tekrarlanmaması (DRY) esastır. AI Unified Process bu ilkeyi gereksinim mühendisliğine taşır: 'Tek bir gerçeğin iki farklı kopyası varsa, zamanla bu kopyalar birbirinden uzaklaşır. Yapay zekâ bu çelişkiyi sessizce kendi kafasına göre çözer!' Use case şartnamesi içine veritabanı sütun tipleri, karakter uzunlukları veya DTO alan tabloları ASLA yazılmaz. Use case sadece alan adını (Entity Noun) zikreder: 'Üye', 'Kitap', 'Ödünç', 'Rezervasyon'. Bu isimlerin tüm veri yapısı, tipleri ve alan seviyesindeki doğrulama kuralları (örn: eposta regex'i, ISBN formatı) `docs/entity_model.md` dosyasında merkezi olarak tanımlanır. Böylece veri modeli güncellendiğinde tüm use case'ler otomatik olarak bu güncellemeyle uyumlu kalır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Zone 2 Domain Referencing & Requirements DRY: In classical software engineering, duplication is an operational nuisance. In an AI-driven workflow, duplication between artifacts is fatal. When an entity attribute is described in both `docs/use_cases/UC-002.md` and `docs/entity_model.md`, human maintainers inevitably update one and overlook the other. An LLM encountering contradictory schemas will silently resolve the ambiguity by picking one arbitrary interpretation. Zone 2 dictates: 'Reference domain nouns, never restate attribute tables.' Every domain noun used in a scenario ('Member', 'Loan', 'BookCopy') must exist verbatim in the entity model, which acts as the authoritative dictionary for types, constraints, and relationships.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Zone 2 - Entity Model ile Use Case Ayrımı
package com.maang.aiup.zone2;

// 1. ENTITY MODEL (docs/entity_model.md karşılığı): Tek doğruluk kaynağı
public record Member(String memberId, String email, int activeLoanCount) {
    public Member {
        if (memberId == null || memberId.isBlank()) {
            throw new IllegalArgumentException("memberId boş olamaz!");
        }
        if (activeLoanCount < 0 || activeLoanCount > 5) {
            throw new IllegalArgumentException("Ödünç sayısı 0-5 arasında olmalıdır!");
        }
    }
}

// 2. USE CASE: Varlık özniteliklerini ezberlemez, varlığı bir bütün olarak ele alır
public class BorrowService {
    public boolean canBorrow(Member member) {
        // Use case sadece iş kuralını sorgular; alan tiplerini yeniden tanımlamaz
        return member.activeLoanCount() < 5;
    }

    void main() {
        var member = new Member("M-101", "alice@example.com", 3);
        var service = new BorrowService();
        System.out.println("✅ Üye ödünç alabilir mi? " + service.canBorrow(member));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Use case senaryolarının içerisine tablo çizip veritabanı sütun tiplerini (VARCHAR, INT, NotNull) doldurmayın! Bu tablolar yalnızca entity_model.md dosyasında yaşar.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never embed database schemas, data types (VARCHAR, INT), or attribute definition tables inside use case steps. Those belong exclusively in docs/entity_model.md.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Use case adımlarında geçen tüm isimler (Member, Loan) entity model'de tanımlı olmalıdır.
- Veri özniteliklerini şartnameye kopyalamayın; tek doğruluk kaynağına referans verin.
- Gereksinimlerde DRY ilkesi şarttır: İki farklı dokümandaki çelişkiyi AI kafasına göre çözer.

🎯 Key Architectural Takeaways

- All business nouns used in steps must correspond directly to entities in docs/entity_model.md.
- Never duplicate entity attributes in use cases; reference the single source of truth.
- DRY is a requirements principle: contradictory specs cause silent hallucinations in AI output.

40

BÖLÜM: YAPAY ZEKÂ İÇIN HASSASIYET: NE SÖYLENMELI, NE DIŞARIDA BIRAKILMALI? | MODULE: PRECISION FOR AI – WHAT TO SAY AND WHAT TO LEAVE OUT

Zone 3 (Mekanizma - YASAK!): Kara Liste (JWT, SQL, bcrypt, Regex, SMTP)

Zone 3 (Mechanism - BANNED): The Blocklist (JWT, SQL, bcrypt, Regex, SMTP)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                ZONE 3: ŞARTNAMEDE KESİNLİKLE YASAKLI KELİMELER

🛑 ŞARTNAME DOĞRULAMA MOTORU TARAFINDAN ENGELLENEN BLOKLİST:

 SMTP, email server, JWT, token, bcrypt, hash, salt, SHA, SQL, SELECT,
 INSERT, HTTP verbs (GET, POST), regex patterns, class names, exceptions

❌ YASAKLI YAZIM (ZONE 3 MEKANİZMA):        ✅ DOĞRU YAZIM (ZONE 1 İŞ İRADESİ):

"Sistem şifreyi bcrypt ile hash'ler."       "Sistem parolayı güvenli şekilde saklar."
"Sistem kullanıcıya JWT token verir."       "Sistem kullanıcı oturumunu açar."
"Sistem reservations tablosuna INSERT       "Sistem rezervasyon kaydını oluşturur."
 sorgusu fırlatır."
"Sistem SMTP port 587 üzerinden mail atar"  "Sistem onay e-postası gönderir."
"Sistem ValidationException fırlatır."      "Sistem hata mesajı görüntüler."

🎯 NEDEN?: Bugün bcrypt olan yarın argon2 olur! Şartname teknolojiyi değil
          iş hedefini mühürler; teknoloji değişse de şartname hayatta kalır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Birine mektup gönderirken postacıya 'Mektubu al, dizel motorlu Fiat Doblo marka aracın sağ ön koltuğuna koy, 2. viteste git' demezsiniz! 'Bu mektubu Ankara'daki adrese teslim et' dersiniz. Postacı arabayla mı gider, bisikletle mi gider onun işidir. Zone 3 yasakları, postacının motor markasını use case'e yazmayı yasaklar!

🇬🇧 ENGLISH (ELI5)

When posting a letter, you never instruct the postal carrier: 'Put this envelope in a diesel Fiat van, drive in second gear, and make 3 right turns.' You simply say: 'Deliver this letter to the address in Ankara.' Whether they drive a truck or pedal a bicycle is their operational choice. Zone 3 forbids specifying the carrier's engine model in your contract.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Zone 3 Mekanizma Yasağının Mimari Sebepleri: Simon Martinelli'nin AIUP modelinde şartname doğrulama komutları (`/spec-review`) şartnamelerde geçen belirli teknik terimleri katı bir 'Blacklist' (Kara Liste) ile tarar ve bulduğunda build'i kırar. Bu kelimeler şunlardır: SMTP, JWT, bcrypt, hash, salt, SQL, SELECT, INSERT, UPDATE, HTTP verbs (GET/POST), regex, sınıf adları ve exception isimleri. Bu yasağın arkasında iki hayati neden vardır: 1) İnsan ve AI Ayrımı: İnsan yazılımcı kötü bir şartnamede gördüğü saçma bir SQL tüyosunu görmezden gelip doğru mimariyi kurabilir; ancak yapay zekâ şartnamede gördüğü her kelimeyi kanun kabul eder! 'Şifreyi bcrypt ile hash'le' yazdığınızda, yarın Argon2id standardına geçmek istediğinizde şartname kodla çatışır. 2) Yeniden Üretilebilirlik (Regenerability): Şartname kodun ömründen daha uzun yaşayacaksa (İlke 11), şartname bir teknoloji yığınına (Spring Boot, NestJS, .NET Blazor) göbekten bağlanamaz. Şartname iş hedefini belirtir ('parolayı güvenli şekilde saklar'), mekanizma ise stack plugin'inin sorumluluğundadır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Architectural Rationale for the Zone 3 Blocklist: In the AI Unified Process, `/spec-review` scans use case specifications against a strict lexical blocklist: SMTP, email server, JWT, token, bcrypt, hash, salt, SHA, SQL, SELECT, INSERT, HTTP verbs, regex patterns, class names, exception names. The existence of this blocklist stems from two fundamental realities: 1) Literal AI Execution: Unlike human senior engineers who ignore poor architectural hints in a spec, an LLM adheres slavishly to written text. Specifying 'hash password with bcrypt and salt' prevents the AI from upgrading to Argon2id without spec refactoring. 2) Framework Portability: The specification outlives the implementation (Principle 11). If an organization migrates from Spring Boot to Quarkus, .NET, or Go, a specification free of Zone 3 leaks remains 100% valid and immediately regenerable.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Mekanizmanın (Zone 3) İş Mantığından Soyutlanması (Port & Adapter)
package com.maang.aiup.zone3;

public class MechanismDecouplingDemo {
    // 1. ŞARTNAME TANIMI: "Sistem parolayı güvenli şekilde saklar"
    // Mekanizma bir port arayüzünün arkasına gizlenir; use case mekanizmayı bilmez!
    public interface PasswordHasherPort {
        String secureHash(String rawPassword);
    }

    // 2. MEKANİZMA UYGULAMASI (Bugün PBKDF2/Bcrypt, yarın Argon2)
    public static class ModernPasswordHasher implements PasswordHasherPort {
        @Override
        public String secureHash(String rawPassword) {
            // Şartnamede 'Argon2' veya 'bcrypt' adı ASLA anılmaz!
            return "SECURE_HASH::" + rawPassword.hashCode();
        }
    }

    public static class UserAccountService {
        private final PasswordHasherPort hasher;

        public UserAccountService(PasswordHasherPort hasher) {
            this.hasher = hasher;
        }

        public void registerUser(String username, String rawPassword) {
            // İş mantığı: Güvenli sakla
            String hashed = hasher.secureHash(rawPassword);
            System.out.println("✅ Kullanıcı '" + username + "' kaydedildi. Güvenli özet
oluşturuldu.");
        }
    }

    void main() {
        var service = new UserAccountService(new ModernPasswordHasher());
        service.registerUser("bozkurt", "SuperSecret2026!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

E-posta doğrulama için regex desenini (/^[w]+@[w]+.[a-z]+$/) şartnameye kopyalamayın! 'Sistem e-posta formatını doğrular' yazın; regex'i stack plugin'i veya entity model halleder.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never inline regex validation patterns into use case steps. State 'System validates email format'; let the stack plugin or entity model supply the RFC-compliant pattern.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kara listedeki kelimeler (JWT, SQL, bcrypt, regex, SMTP) use case içinde KESİNLİKLE yasaktır.

• Şartname iş hedefini söyler ('güvenli saklar'); teknolojiyi stack plugin'ine bırakır.

• Mekanizmadan arındırılmış şartnameler, teknoloji yığını değişse bile 10 yıl sonra dahi çalışır.

🎯 Key Architectural Takeaways

• Blocklist keywords (JWT, SQL, bcrypt, regex, SMTP) are strictly prohibited in use cases.

• Specifications state business goals ('securely stores'); technical stacks determine mechanisms.

• Mechanism-free specs survive stack migrations, enabling flawless code regeneration years later.

41

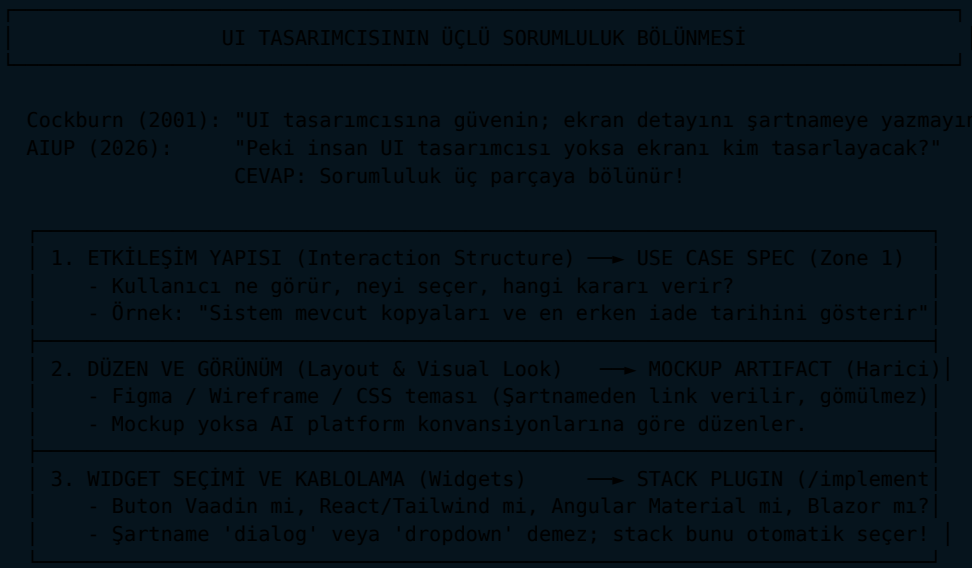
BÖLÜM: YAPAY ZEKÂ İÇİN HASSASIYET: NE SÖYLENMELI, NE DIŞARIDA BIRAKILMALI? | MODULE: PRECISION FOR AI – WHAT TO SAY AND WHAT TO LEAVE OUT

UI Tasarımcısı Nereye Gitti? Üçlü Sorumluluk Bölünmesi

Where Did the UI Designer Go? The Three-Way Responsibility Split

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir tiyatro oyununu düşünün: Senaryo yazarı (Use Case) der ki: 'Romeo sahneye girer, elindeki mektubu Juliet'e uzatır ve aşkını ilan eder.' Senaryo yazarı 'Juliet kırmızı kadife koltuğa 45 derece açıyla otursun, arkadan 40 watt sarı spot yansısın' demez! Koltuğu dekoratör (Mockup), ışığı teknisyen (Stack Plugin) ayarlar!

🇬🇧 ENGLISH (ELI5)

Think of a theater play: The playwright (Use Case) writes: 'Romeo enters stage left, hands the letter to Juliet, and professes his love.' The playwright does not dictate: 'Juliet must sit on a 45-degree angle crimson sofa illuminated by a 40-watt tungsten lamp.' The set designer (Mockup) and stage crew (Stack Plugin) handle layout and fixtures.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

UI Tasarımcısının Üçlü Bölünmesi (The Three-Way Split): Cockburn'ün 'UI tasarımcısına güvenin' öğüdü, şartnameyi okuyup ekran tasarlayan bir insan tasarımcının varlığına dayanıyordu. AI Unified Process'te bu rol üçe ayrılır: 1) Etkileşim Yapısı (Interaction Structure): Use Case şartnamesinin asli görevidir (Zone 1). Kullanıcının her adımda ne yapabileceğini, hangi veriyi gördüğünü ve hangi kararı verdiğini belirtir. 2) Düzen ve Görünüm (Layout & Look): Mockup veya Figma tasarımının görevidir. Eğer harici bir mockup varsa, use case buna link verir (`Mockup: docs/mockups/UC-002.png`). Mockup yoksa AI, platformun modern UX standartlarına göre düzeni kendi kurar. 3) Widget Seçimi ve Kablolama (Widget Choice & Wiring): Stack Plugin'inin görevidir. /implement komutu projenin React mı, Vue mu, Vaadin mi yoksa Flutter mı olduğunu bilir. Use case asla 'dropdown menüden seçer' veya 'modal popup açılır' demez; 'seçer' ve 'görüntüler' der.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Three-Way Split of the UI Designer Role: Cockburn's counsel to trust the UI designer assumed a human specialist interpreting business intent. In generative development, that function partitions across three decoupled layers: 1) Interaction Structure: Owned entirely by the use case specification (Zone 1). It dictates the logical payload—what the actor enters, inspects, and acknowledges. 2) Layout and Aesthetics: Owned by supplementary mockup artifacts (e.g., SVG/Figma files) linked from the spec. In the absence of a mockup, the AI synthesizes layouts adhering to modern platform heuristics. 3) Widget Selection & Component Wiring: Owned by the implementation stack plugin. Whether an input maps to a `Select`, a radio group, or an autocomplete combo-box is the platform stack's prerogative. The spec remains agnostic of widget nomenclature.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Etkileşim Yapısının (Use Case) Widget'tan (UI Framework) Ayrılması
package com.maang.aiup.uisplit;

public class UiSplitDemo {
    // 1. USE CASE: Sadece etkileşim yapısını (veri ve eylem) bilir
    public record BorrowInteractionModel(String selectedBookId, boolean confirmed) {}

    // 2. STACK PLUGIN: Vaadin, React veya Thymeleaf bileşenlerini stack üretir
    // Use case şartnamesi 'Button' veya 'Modal' kelimesini bilmez!
    public static class MockWebComponentRenderer {
        public void renderBookActions(BorrowInteractionModel model) {
            System.out.println("🖥️ UI Renderer: Seçilen Kitap (" + model.selectedBookId() + ") için
buton üretildi.");
            if (model.confirmed()) {
                System.out.println("🖥️ UI Renderer: Onaylandı durumu ekrana yansıtıldı.");
            }
        }
    }

    void main() {
        var interaction = new BorrowInteractionModel("BK-99", true);
        var renderer = new MockWebComponentRenderer();
        renderer.renderBookActions(interaction);
        System.out.println("✅ UI Sorumluluk bölünmesi doğrulandı.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnamede 'Kullanıcı OK butonuna tıklar' veya 'İptal linkine basar' gibi widget isimleri yerine 'Kullanıcı işlemi onaylar' veya 'Kullanıcı işlemi iptal eder' deyin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Replace widget mechanics like 'User clicks OK button' or 'User clicks cancel link' with user intent: 'User confirms operation' or 'User cancels operation'.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Etkileşim yapısı (ne görülür/yapılır) use case'in, düzen mockup'ın, widget seçimi stack'in işidir.
- Use case asla 'dropdown', 'dialog box' veya 'toast' demez; sadece iş niyetini tarif eder.
- Mockup eksikse AI modern tasarım kalıplarına göre düzeni başarıyla oluşturabilir.

🎯 Key Architectural Takeaways

- Interaction structure belongs to specs, layout to mockups, widget choice to stack plugins.
- Never use terms like 'dropdown' or 'dialog box'; specify user intent and information flow.
- When mockups are absent, AI synthesizes layouts from interaction structure and conventions.

42

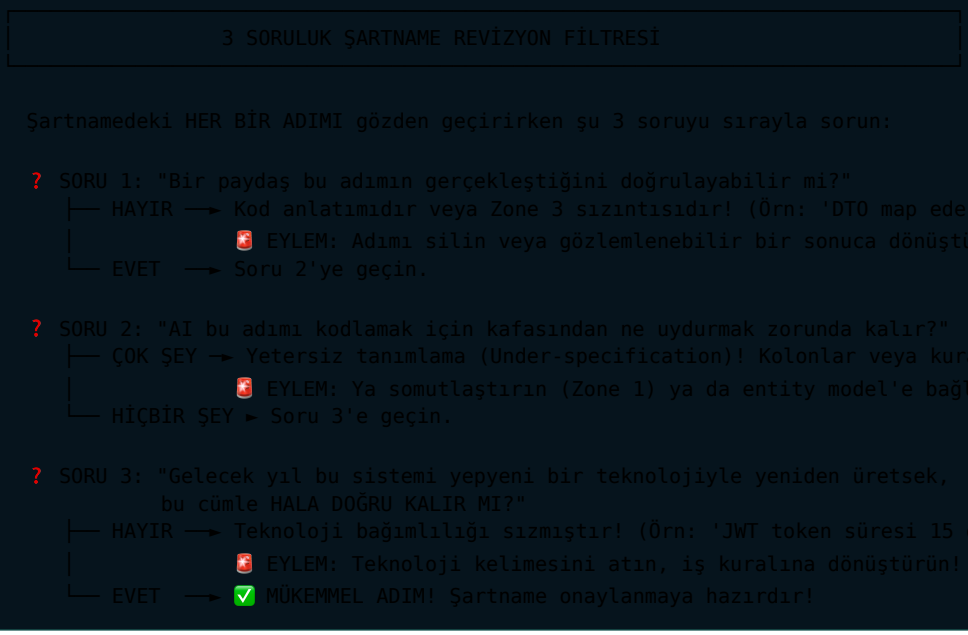
BÖLÜM: YAPAY ZEKÂ İÇİN HASSASIYET: NE SÖYLENMELI, NE DIŞARIDA BIRAKILMALI? | MODULE: PRECISION FOR AI – WHAT TO SAY AND WHAT TO LEAVE OUT

3 Soruluk Revizyon Filtresi: Şartnameyi Yeniden Üretilebilirliğe Karşı Denetleme

The 3-Question Revision Pass: Auditing Specs for True Regeneration

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Evinizin tapusunu düşünün: Tapuda 'Bu arsa Ahmet'indir, sınırları kuzeyde dere, güneyde yoldur' yazar. Tapuda 'Ahmet bu arsaya çimento taşırken sarı renkli 2018 model kamyon kullandı' yazmaz! Çünkü o kamyon 10 yıl sonra hurda olur ama tapu 100 yıl geçerlidir. 3 soruluk filtre, şartnamenizden sarı kamyonları ayıklayıp tapuyu ölümsüz kılar!

🇬🇧 ENGLISH (ELI5)

Think of a land deed: The deed records: 'This parcel belongs to Ahmet, bounded by the creek to the north and the highway to the south.' It never mentions: 'Ahmet hauled concrete using a yellow 2018 dump truck.' The truck will be scrap metal in 10 years, but the deed remains legally binding for a century. The 3-question filter strips away the dump trucks, leaving an immortal contract.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

3 Soruluk Revizyon Filtresinin Metodolojisi: Üretken yapay zekâ tarafından taslak olarak çıkarılan veya bir analist tarafından aceleyle yazılan use case'ler her zaman tahmin edilebilir hatalar barındırır. Bu şartnameleri /implement aşamasına sokmadan önce Martinelli'nin '3 Soruluk Filtresi' işletilir: 1. Paydaş Testi: 'Sistem gelen JSON payload'unu UserDTO nesnesine deserialize eder' gibi bir adımı hiçbir iş paydaşı anlayamaz ve doğrulayamaz. Bu adım derhal silinmelidir. 2. İcat Testi: 'Sistem kullanıcıyı bilgilendirir' adımını gören AI, bir modal mı açacak, SMS mi atacak yoksa konsola log mu basacak? Kararı AI'a bırakmak istemiyorsanız, içeriği ('SMS ile onay kodu gönderir') somutlaştırmalısınız. 3. 1 Yıl Sonra Testi: 'Sistem Redis cache'ini temizler' cümlesi seneye Redis yerine Hazelcast veya doğrudan Postgres kullanıldığında yanlış hale gelecektir. Bu yüzden 'Sistem güncel bakiye bilgisini tazeler' şeklinde teknoloji-bağımsız yazılmalıdır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The 3-Question Spec Revision Methodology: Generated drafts and human drafts fail in predictable archetypes. Martinelli mandates applying three rigorous filter questions to every single step before approval: 1. Stakeholder Verifiability: 'Could a business stakeholder confirm this step occurred?' If an action reads 'System maps entity to response DTO', no stakeholder can confirm it. It is internal narration; delete it. 2. Hallucination Risk: 'What would the AI have to invent to implement this step?' If a step says 'System displays available slots', the AI must invent sorting, pagination, and columns. Either specify them (Zone 1) or consciously delegate. 3. Regeneration Longevity: 'Would this sentence remain 100% true if regenerated on an entirely new platform stack next year?' If the sentence mentions Kafka, SQL, or JWT, it will rot upon migration. Strip the technology; preserve the pure business invariant.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 3 Soruluk Filtreden Geçmiş Ölümsüz Şartname İcra Modeli
package com.maang.aiup.revision;

import java.time.Instant;

public class SpecRevisionDemo {
    // ❌ FİLTREDEN GEÇEMEYEN ŞARTNAME KODU (Teknoloji sızıntısı dolu):
    // public void saveToMongoAndPublishToKafka(UserDTO dto) { ... }

    // ✅ 3 SORULUK FİLTREDEN GEÇMİŞ KOD (Ölümsüz İş Mantığı):
    public record MemberRegistrationOutcome(String memberId, Instant registeredAt, String status) {}

    public static class RegisterMemberUseCase {
        public MemberRegistrationOutcome register(String email) {
            // Soru 1: Paydaş kaydın yapıldığını görebilir mi? -> Evet.
            // Soru 2: AI ne uydurdu? -> Hiçbir şey, outcome alanları net.
            // Soru 3: Seneye Mongo yerine Postgres gelse bu cümle doğru mu? -> Kesinlikle evet!
            return new MemberRegistrationOutcome("MEM-" + System.currentTimeMillis(), Instant.now(),
"ACTIVE");
        }
    }

    void main() {
        var useCase = new RegisterMemberUseCase();
        var outcome = useCase.register("engineer@maang.com");
        System.out.println("✅ 3 Soruluk filtreden geçmiş tertemiz use case sonucu:");
        System.out.println("   -> " + outcome);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartname revizyonunu tek başınıza yapmayın; bir yazılım mühendisi ile bir iş analistinin şartname üzerinde birlikte bu 3 soruyu sorması en kaliteli sonucu verir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never conduct the revision pass in isolation; a software engineer and a business analyst reviewing the 3 questions together guarantees an airtight specification.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Paydaşın gözlemleyemeyeceği adımları ('DTO dönüştürür') şartnameden derhal silin.
- AI'ın doldurmak zorunda kalacağı her boşluğu ya somutlaştırın ya da bilinçli serbest bırakın.
- Gelecek yıl stack değiştiğinde yalan olacak teknik kelimeleri şartnameden temizleyin.

🎯 Key Architectural Takeaways

- Immediately excise internal plumbing steps ('maps DTO') that stakeholders cannot confirm.
- For every gap the AI would have to invent, specify it concretely or consciously leave it free.
- Purge any technological terminology that would become false upon a framework migration.

BÖLÜM 8 / MODULE 8

Dilimler (Slices), Testler ve İterasyon Döngüsü

Slices, Tests, and the Iteration Loop

Use-Case 3.0'ın 10. İlkesi: Sistemi dilimler halinde inşa etmek. Temel akış ilk dilimdir; alternatif akışlar iş önceliğine göre dilimlenir. İlke 14: 'Gerçeği Testler Taşır'. AIUP test piramidi, TC-XXX uçtan uca yolculukları, birebir (literal) test verisi ve BPMN iş süreçlerinden test türetme döngüsü.

Use-Case 3.0's 10th Principle: Build the system in slices. The basic flow is Slice 1; alternative flows are sliced in business-priority order. Principle 14: 'Tests Carry the Truth'. The AIUP test pyramid, TC-XXX end-to-end journeys, literal test data discipline, and deriving test cases from BPMN processes.

43

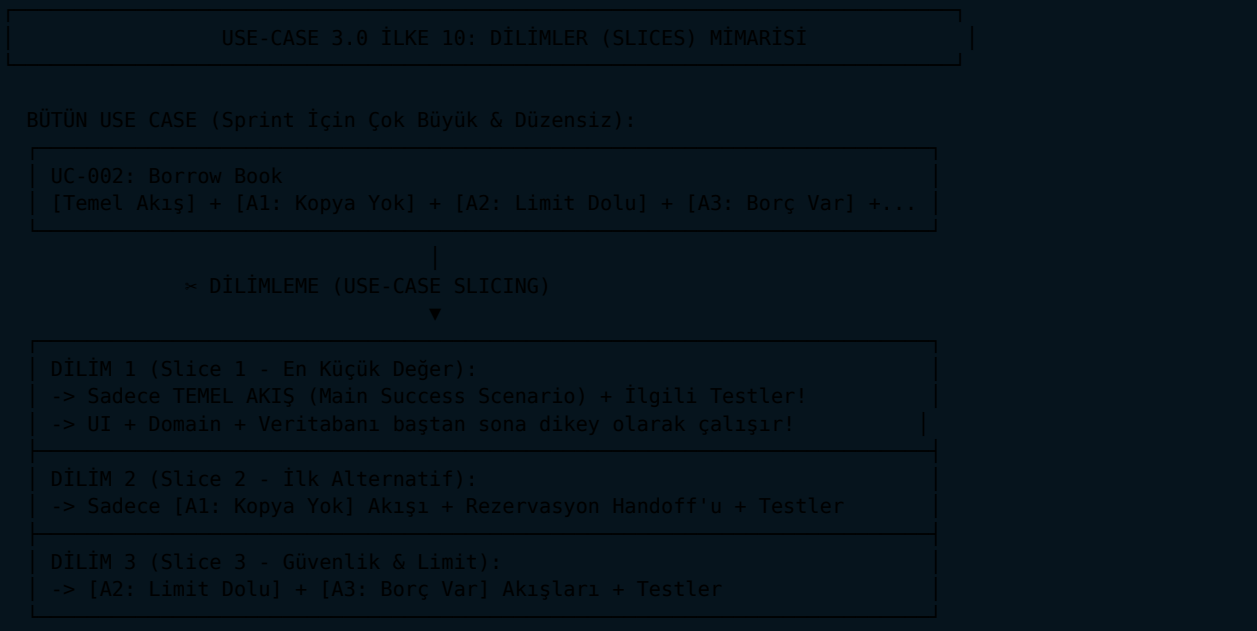
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

10. İlke: Sistemi Dilimlerle İnşa Etmek (Use-Case Slices)

Principle 10: Slices – Slicing the System into Deliverable Units

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kocaman bir doğum günü pastasını bir lokmada ağzınıza atamazsınız, boğulursunuz! Pastayı dilim dilim kesersiniz. İlk dilim sadece kekin ve kremanın tadını verir (Temel akış). Sonraki dilimlerde çilekler ve çikolatalar gelir (Alternatif akışlar). İşte use case pastanın tamamıdır, sprint'e giren şey ise o tek bir lezzetli dilimdir (Slice)!

🇬🇧 ENGLISH (ELI5)

You cannot swallow an entire multi-layered birthday cake in one bite; you'd choke! You slice it cleanly. Slice 1 gives you the foundational sponge cake and frosting (Basic Flow). Subsequent slices bring strawberries and chocolate chips (Alternative Flows). The use case is the whole cake; the actionable backlog ticket for your sprint is the vertical slice!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Use-Case 3.0'ın 10. İlkesi (Build the System in Slices): Ivar Jacobson ve Ian Spence'in klasik use case metodolojisine kattığı en devrimci ilke budur. Bir use case, bir hedefe ulaşmanın TÜM yollarını (başarı, istisnalar, hatalar) içerdiği için tek bir sprint veya tek bir PR için fazla büyüktür ve dengesiz boyutlardadır. Teslimat birimi use case değil, 'Use-Case Slice'dır. Bir dilim; use case içindeki bir veya birkaç yolu, o yolları icra eden tasarımı, kodu ve testleri içeren dikey, bağımsız ve test edilebilir bir pakettir. Bu sayede use case'ler Scrum sprint backlog'larına, Kanban sütunlarına ve SAFe trenlerine mükemmel uyum sağlar. AI Unified Process'te bir şartnamenin Status alanı bu dilimleme yaşam döngüsünü yönetir: Temel akış kodlanıp testten geçtiğinde use case 'Implemented/Tested' aşamasına geçer; kalan alternatif akışlar şartnamede beklerken sistem güvenle çalışır durumda kalır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 10: Building the System in Slices: Jacobson and Spence added the 10th principle to bridge classical use cases with agile delivery. Because a use case captures every permutation of achieving a goal, it is irregularly sized and too monolithic to flow smoothly through a sprint board. The fundamental unit of delivery is the use-case slice: one or more paths through a use case, accompanied by the design, production code, and automated tests that implement and verify them. Slices start at the trigger, conclude at the postconditions, and provide immediate, testable business value. Slices map directly to user stories or sprint backlog items. In the AI Unified Process, this additive structure allows teams to generate Slice 1 via `/implement UC-002`, review the running UI with stakeholders, and then progressively generate slices for alternative flows without architectural churn.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Birinci Dilim (Slice 1: Sadece Temel Akış) Uygulaması
package com.maang.aiup.slices;

import java.time.LocalDate;

public class UseCaseSliceOneDemo {
    // DİLİM 1: Sadece temel akış verileri ve başarısı (Alternatifler henüz yok!)
    public record BorrowSliceOneResult(String bookId, String memberId, LocalDate dueDate) {}

    public static class BorrowBookSliceOneService {
        public BorrowSliceOneResult executeBasicFlow(String bookId, String memberId) {
            // Dilim 1 kapsamında sadece mutlu yol çalışır:
            LocalDate dueDate = LocalDate.now().plusDays(28);
            System.out.println("📦 Slice 1 Başarılı: Kitap (" + bookId + ") üyeye (" + memberId + ") verildi.");
            return new BorrowSliceOneResult(bookId, memberId, dueDate);
        }
    }

    void main() {
        var sliceOne = new BorrowBookSliceOneService();
        var result = sliceOne.executeBasicFlow("BK-42", "MEM-007");
        System.out.println("✅ Dilim 1 Dikey Olarak Çalıştı: " + result);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir use case'in tüm akışlarını (happy path + 8 alternatif akış) tek bir seferde AI'a kodlatmaya çalışmayın! Temel akışı ilk dilim olarak üretip paydaşa gösterin, geri bildirim alın, sonra diğer akışları ekleyin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Resist the urge to generate an entire use case (happy path + 8 branches) in a single massive prompt. Implement Slice 1 first, get visual feedback, then add alternative branches incrementally.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Teslimatın temel birimi use case değil, dikey ve test edilebilir 'use-case slice'dır.
- Dilim 1 daima temel akıştır (happy path); en hızlı iş değerini ve geri bildirimi o sağlar.
- Dilimleme sayesinde use case'ler Scrum, Kanban ve CI/CD akışlarına pürüzsüz entegre olur.

🎯 Key Architectural Takeaways

- The unit of delivery is the use-case slice, not the entire use case document.
- Slice 1 is always the basic flow, delivering immediate feedback and functional baseline.
- Slicing enables seamless integration of formal use cases into Scrum, Kanban, and modern CI/CD.

44

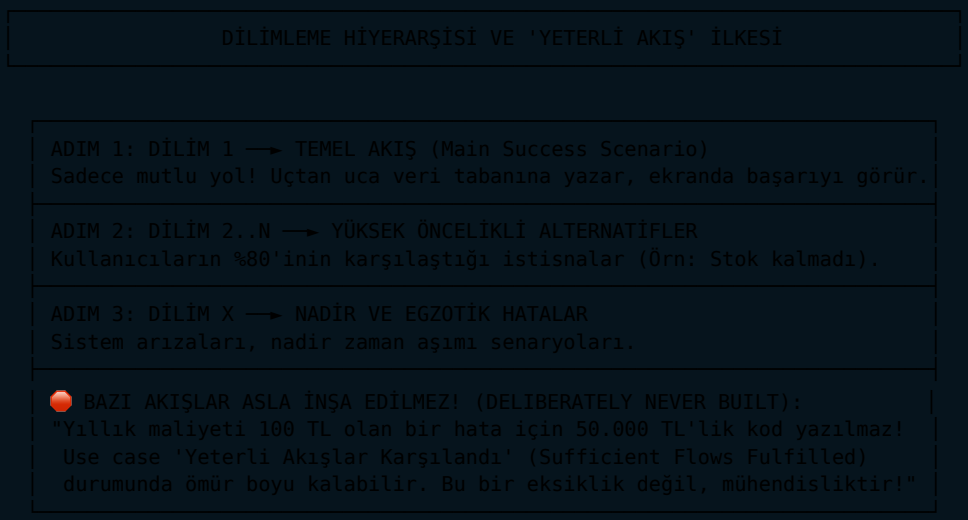
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

Dilimleme Hiyerarşisi: Önce Temel Akış, Sonra Alternatifler (Bazıları Asla Yapılmaz!)

The Slicing Order: Basic Flow First, Alternatives by Priority, Some Never Built

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Yeni açılan bir kahve dükkânını düşünün: İlk gün sadece sıcak kahve satarlar (Temel akış). İkinci hafta vanilyalı ve karamelli şuruplar gelir (Öncelikli alternatifler). 'Organik Madagaskar vanilyalı yulaflı latte'yi ise yılda bir kişi sorar diye menüye hiç koymazlar (Asla inşa edilmeyen akış!). Her şeyi aynı anda yapmaya çalışmak dükkânı batırır!

🇬🇧 ENGLISH (ELI5)

Consider opening a new coffee shop: On Day 1, they only brew classic hot coffee (Basic Flow). By Week 2, they introduce caramel and hazelnut syrups (High-priority alternatives). But 'Organic Madagascar Vanilla Bean Oat Milk Cappuccino' is requested once a year, so they never add it to the menu (Deliberately never built!). Slicing prevents capital exhaustion.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Dilimleme Sırası ve 'Yeterli Akış' Felsefesi: Use-Case 3.0 ve Martinelli'nin metodolojisinde dilimleme sırası katı bir mantığa bağlıdır: 1) Dilim 1 Temel Akış: Sistemin en önemli use case'inin temel akışı, tüm mimari omurgayı (UI, servis, DB, güvenlik) doğrulamak için yeterlidir. 2) Sonraki Dilimler: En sık rastlanan ve iş değeri en yüksek alternatif akışlar sırayla dilimlenir. 3) 'Bazı Akışlar Asla İnşa Edilmez': Geleneksel waterfall zihniyeti şartnamedeki her satırın kodlanmasını şart koşar. Oysa Use-Case 3.0'ın en gerçekçi durumu 'Yeterli Akışlar Karşılandı' (Sufficient Flows Fulfilled) durumudur. Sistem ticari olarak hedefine ulaşmışsa, 5 yılda bir olabilecek çok nadir bir istisna için kod yazmak ve bakım maliyeti üstlenmek israftır. O akış şartnamede analiz edilmiş olarak bekler; manuel operasyonel süreçle idare edilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Slicing Order and the 'Sufficient Flows Fulfilled' State: Slicing follows an immutable cadence derived from the additive spec architecture: 1) Slice 1 is strictly the Basic Flow. Implementing the happy path end-to-end verifies the architectural backbone (REST endpoints, persistence, transaction boundaries, UI state) and yields early stakeholder validation. 2) Successive Slices incorporate alternative flows sorted strictly by business priority and risk mitigation. 3) Deliberately Unbuilt Flows: A vital tenet of Use-Case 3.0 is that not every analyzed flow must be implemented. A use case can intentionally reside in 'Sufficient Flows Fulfilled' indefinitely. Engineering rare edge-case automation whose cost exceeds manual operational fallback is anti-pattern waste. The specification documents the path; the business consciously opts not to cut the slice.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Dilimlerin Öncelik Sırasıyla Eklemeli (Additive) Olarak Büyütülmesi
package com.maang.aiup.slices;

public class SlicingEvolutionDemo {
    // Aşama 1: Sadece Temel Akış (Slice 1)
    public record BorrowRequest(String bookId, int availableCopies, int currentLoans) {}

    public static class BorrowFlowHandler {
        // Slice 1: Temel Başarı
        public String executeSlice1(BorrowRequest req) {
            return "✅ Kitap ödünç verildi.";
        }

        // Slice 2 Eklenir: A1 (Kopya Yok) ve A2 (Limit Doldu)
        public String executeSlice2(BorrowRequest req) {
            if (req.availableCopies() <= 0) {
                return "⚠ Alternatif A1: Kopya yok, rezervasyona yönlendirildi.";
            }
            if (req.currentLoans() >= 5) {
                return "❌ Alternatif A2: Limit dolu (5 kitap), işlem engellendi.";
            }
            return executeSlice1(req);
        }
    }

    void main() {
        var handler = new BorrowFlowHandler();
        var normalReq = new BorrowRequest("BK-1", 3, 2);
        var noStockReq = new BorrowRequest("BK-2", 0, 1);

        System.out.println("1. İstek: " + handler.executeSlice2(normalReq));
        System.out.println("2. İstek: " + handler.executeSlice2(noStockReq));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Temel akışı canlıya almadan veya paydaşa onaylatmadan alternatif akışları kodlamaya başlamayın; temel akışta yapılacak bir UI değişikliği tüm alternatifleri çöp edebilir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never begin coding alternative flows before the basic flow is reviewed by stakeholders; a fundamental change in happy path UI invalidates all downstream branches.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Dilimleme sırası sabittir: Önce temel akış, sonra en sık rastlanan alternatifler.
- Her alternatif akışı kodlamak zorunda değilsiniz; bazıları operasyonel olarak çözülür.
- 'Yeterli Akışlar Karşılandı' durumu bir use case için tamamen geçerli bir bitiş durumudur.

🎯 Key Architectural Takeaways

- The slicing order is fixed: basic flow first, followed by frequent alternatives.
- Not all alternative flows must be coded; manual workarounds are often economically sound.
- 'Sufficient Flows Fulfilled' is a valid, mature end-state for a production use case.

45

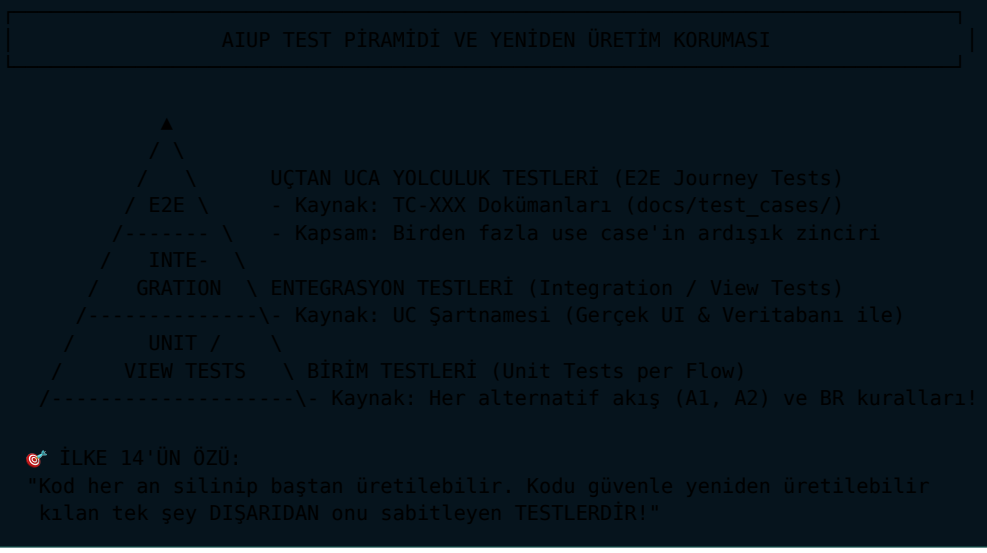
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

14. İlke: Gerçeği Testler Taşır (AIUP Test Piramidi)

Principle 14: Tests Carry the Truth – The AIUP Test Pyramid

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Lunaparktaki hız trenini (rollercoaster) düşünün: Trenin vagonunu istediğiniz renge boyayabilir, koltuklarını değiştirebilirsiniz. Ama vagonun raydan çıkmasını engelleyen şey altındaki çelik kilitlerdir! Testler o çelik kilitlerdir. AI kodu baştan üretse bile testler trenin raydan çıkıp uçuruma yuvarlanmasını engeller!

🇬🇧 ENGLISH (ELI5)

Think of a roller coaster: You can repaint the coaster cars or upgrade the upholstery anytime. But what keeps the car from flying off the tracks at 100 mph are the steel under-friction safety wheels! Automated tests are those steel clamps. Even if an AI completely regenerates the engine, the tests guarantee the car stays on track.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

14. İlke: Gerçeği Testler Taşır (Tests Carry the Truth): Use-Case 3.0'daki en çarpıcı cümle şudur: 'Test senaryoları, bir use case'in anlatısından bile daha önemli olan en kritik parçasıdır; çünkü use case'in ne zaman başarıyla tamamlandığını sadece testler tanımlar!' AI Unified Process bu tavsiyeyi yapısal bir zorunluluğa dönüştürür. 4. İlke 'Test-Protected' (Testle Korunan): Kod her an baştan üretilebilen geçici bir derleme çıktısıdır. Eğer bir sistemin davranışı kodun dışında bir mekanizma tarafından sabitlenmemişse, o sistemi AI ile yeniden üretmek kumar oynamaktır! Test piramidi doğrudan şartname zincirine bağlanır: 1) Birim/Akış Testleri: Şartnamedeki her bir doğrulama adımını ve her bir alternatif akışı (A1, A2) tek tek sınar. 2) Entegrasyon Testleri: Bir use case'in tamamını gerçek arayüz ve servislerle test eder. 3) Yolculuk Testleri (TC-XXX): Birden fazla use case'i bağlayan iş süreçlerini test eder.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 14: Tests Carry the Truth & The AIUP Test Pyramid: Use-Case 3.0 asserts boldly: 'The test cases are the most important part of a use case's description, more important even than the narrative.' In the AI Unified Process, this transitions from agile guidance to absolute architectural necessity. Principle 4 (Test-Protected) establishes that because code can be generated, refactored, or discarded at will, intent must be pinned by artifacts outside the code. The tests freeze observable behavior. The test hierarchy aligns cleanly with the artifact chain: 1) Unit/Flow Tests: Derived directly from the use case validations and alternative flows (BR-XXX). 2) Integration Tests: Drive an entire use case through the presentation layer. 3) Journey Tests (TC-XXX): Verify multi-use-case lifecycles where state transitions cross use case boundaries.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Alternatif Akışları Doğrulayan Test-Protected Mimari
package com.maang.aiup.testing;

import java.util.Optional;

public class TestCarriesTruthDemo {
    public record BorrowResult(boolean success, String message) {}

    public static class BorrowValidator {
        public static BorrowResult validate(int availableCopies, int currentLoans, boolean hasOverdue)
{
            // BR-004: Kopya kontrolü -> Alternatif A1
            if (availableCopies <= 0) return new BorrowResult(false, "A1: Kopya kalmadı.");
            // BR-005: Limit kontrolü -> Alternatif A2
            if (currentLoans >= 5) return new BorrowResult(false, "A2: Limit aşıldı.");
            // BR-006: Gecikmiş borç -> Alternatif A3
            if (hasOverdue) return new BorrowResult(false, "A3: Gecikmiş borç var.");

            return new BorrowResult(true, "Ödünç başarılı.");
        }
    }

    // TEST MOTORU (Şartnamenin bekçisi)
    void main() {
        // Test 1: A1 Akışını sına
        var resA1 = BorrowValidator.validate(0, 1, false);
        assert !resA1.success() && resA1.message().contains("A1") : "A1 Testi Başarısız!";

        // Test 2: A2 Akışını sına
        var resA2 = BorrowValidator.validate(2, 5, false);
        assert !resA2.success() && resA2.message().contains("A2") : "A2 Testi Başarısız!";

        // Test 3: Başarı Yolunu sına
        var resOk = BorrowValidator.validate(2, 2, false);
        assert resOk.success() : "Mutlu Yol Testi Başarısız!";

        System.out.println("✅ Tüm akış testleri geçti! Şartname koruma altındadır.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Testi yazılmamış bir use case şartnamesi yalnızca bir temenniden ibarettir! Her alternatif akışın en az bir otomatik testi olmak zorundadır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

A use case without automated test cases is merely a wish. Every alternative flow must have at least one automated assertion guarding it against regression.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kod geçicidir, test kalıcıdır: Sistemi güvenle yeniden üretilebilir kılan testlerdir.

• Şartnamedeki her bir doğrulama (validates) ve alternatif akış (A1, A2) için test yazılır.

• Yolculuk testleri (TC-XXX) use case'ler arasındaki durum ve veri akışını denetler.

🎯 Key Architectural Takeaways

• Code is ephemeral; tests are enduring. Automated tests make regeneration safe.

• Every validation step and alternative flow in the spec maps to a specific test case.

• Journey tests (TC-XXX) verify cross-use-case boundaries and continuous state progression.

46

BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

Test Senaryoları (TC-XXX): Use Case'leri Birbirine Bağlayan Uçtan Uca Yolculuklar

Test Cases (TC-XXX): End-to-End Journeys Chaining Multiple Use Cases

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                TC-XXX: UÇTAN UCA İŞ YOLCULUĞU TABLOSU

TC-001: Yeni Üye Olma ve İlk Kitabı Ödünç Alma Yolculuğu

| #  | Adım Açıklaması          | Kullanılan Test Verisi | İlgili Use Case |
|----|--------------------------|------------------------|-----------------|
| 1  | Kullanıcı üye kaydı açar | "bozkurt@maang.com"    | UC-001 Register |
| 2  | E-posta onayını tamamlar | Onay Linki / Token     | UC-001 Register |
| 3  | Katalogda kitap arar     | "Clean Architecture"   | UC-003 Search   |
| 4  | Kitabı ödünç alır        | Kitap ID: "BK-101"     | UC-002 Borrow   |
| 5  | Ödünç listesini denetler | Vade: 28 gün sonra     | UC-004 My Loans |

🎯 MİMARİ İLKE:
- Tek tek use case'ler detayları (kolonlar, hata mesajları) test eder.
- TC-XXX ise DİKİŞ YERLERİNİ (Seams) test eder: Durumun bir use case'den
  diğer use case'e pürüzsüz aktarıldığını doğrular!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bayrak yarışını (atletizm) düşünün: 4 farklı koşucu vardır. Her koşucu kendi 100 metresini harika koşabilir (Bireysel Use Case). Ama en kritik an, sopayı bir sonraki koşucunun eline düşürmeden vermektir (Dikiş Noktası / Seams)! TC-XXX testleri, sopanın elden ele düşmeden geçtiğini kontrol eden başhakemdir!

🇬🇧 ENGLISH (ELI5)

Think of a 4x100m track relay race: Four runners sprint their own legs (Individual Use Cases). Each runner may run exceptionally well alone. But the most catastrophic failure happens at the baton handoff (The Seam)! A TC-XXX test case acts as the head referee, ensuring the baton passes flawlessly from runner to runner across the entire lap.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

TC-XXX Uçtan Uca Yolculuk Dokümanlarının Mimarisi: Bir use case tek bir aktörün tek bir oturumda ulaştığı deniz seviyesi hedefidir (Sea-level). Ancak gerçek hayatta müşteriler ardışık birçok hedefi zincirleme icra eder: 'Kayıt Ol -> Doğrula -> Ara -> Sepete Ekle -> Öde -> Kargo Takip Et'. Cockburn'ün 'Uçurtma Seviyesi' (Kite-level) dediği bu süreçler AIUP içinde `docs/test_cases/TC-XXX-\<name>.md` dosyaları olarak formalize edilir. TC dokümanı tek bir akış tablosudur. Her satır bir adımı, o adımda kullanılan somut test verisini ve zincirlenen use case dosyasını (`UC-001`, `UC-002`) barındırır. Bu ayrım katı bir iş bölümü sağlar: 1) Use case seviyesindeki testler iç detaylara (kolonlar, validasyonlar, hata mesajları) odaklanır. 2) TC yolculuk testleri ise dikiş yerlerini (seams) denetler; yani UC-001'in oluşturduğu üye ID'sinin UC-002 tarafından hatasız şekilde tanınıp kullanılabildiğini garanti eder.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Architecture of TC-XXX End-to-End Journey Specifications: While a use case encapsulates a single sea-level user goal achieved in one sitting, business workflows span cohesive multi-step journeys (e.g., onboarding, checkout, fulfillment). Cockburn designated these 'kite-level' summary goals. In the AI Unified Process, kite-level workflows are codified as test journey artifacts in `docs/test_cases/TC-XXX-\<name>.md`. A TC document consists of a structured tabular progression where each row declares: the step description, literal input/output test data, and a hyperlink to the executed use case specification. This establishes a clean demarcation of testing concerns: individual use case specs own flow-level minutiae (validation messages, column sort orders), while journey tests own system seams—ensuring data and state flow seamlessly across use case boundaries.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Use Case'ler Arası Durum Aktarımını Doğrulayan Yolculuk Testi
package com.maang.aiup.journeys;

public class CrossUseCaseJourneyTest {
    // Adım 1: Kayıt Use Case Çıktısı
    public record RegisteredMember(String id, String email, boolean active) {}

    // Adım 2: Ödünç Use Case Çıktısı
    public record BorrowedReceipt(String loanId, String memberId, String bookId) {}

    public static class RegistrationService {
        public RegisteredMember register(String email) {
            return new RegisteredMember("MEM-99", email, true);
        }
    }

    public static class BorrowService {
        public BorrowedReceipt borrow(RegisteredMember member, String bookId) {
            if (!member.active()) throw new IllegalStateException("Üye aktif değil!");
            return new BorrowedReceipt("LN-1001", member.id(), bookId);
        }
    }

    void main() {
        // TC-001 Yolculuk İcrası:
        var regService = new RegistrationService();
        var borrowService = new BorrowService();

        // 1. Dikiş Noktası: Kayıt ol
        var member = regService.register("bozkurt@maang.com");
        System.out.println("TC-001 Adım 1 Başarılı: " + member);

        // 2. Dikiş Noktası: Kayıtlı üyeyi ödünç servisine geçir
        var receipt = borrowService.borrow(member, "BK-Clean-Arch");
        System.out.println("TC-001 Adım 2 Başarılı: " + receipt);

        assert receipt.memberId().equals("MEM-99") : "Dikiş hatası! Üye ID aktarılamadı.";
        System.out.println("✅ TC-001 Uçtan Uca Yolculuk Başarıyla Doğrulandı!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Yolculuk testlerinde (TC-XXX) tablo kolonlarını veya arayüz renklerini yeniden test etmeye kalkmayın! Yolculuk testinin tek görevi use case'ler arasındaki durum aktarımını sınamaktır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never re-assert table columns or UI styling inside end-to-end journey tests. Journey tests own the seams between use cases, not internal UI presentation details.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• TC-XXX dokümanları, birden fazla use case'i birleştiren uçtan uca yolculukları tanımlar.

• Yolculuk testleri 'dikiş yerlerini' (seams) denetler; durumun use case'ler arası geçişini doğrular.

• Her satırda somut test verisi ve tetiklenen use case referansı yer alır.

🎯 Key Architectural Takeaways

• TC-XXX documents specify end-to-end user journeys that chain multiple use cases.

• Journey tests verify system seams, ensuring state propagates across use cases flawlessly.

• Each journey step declares literal test data and links to the invoked use case specification.

47

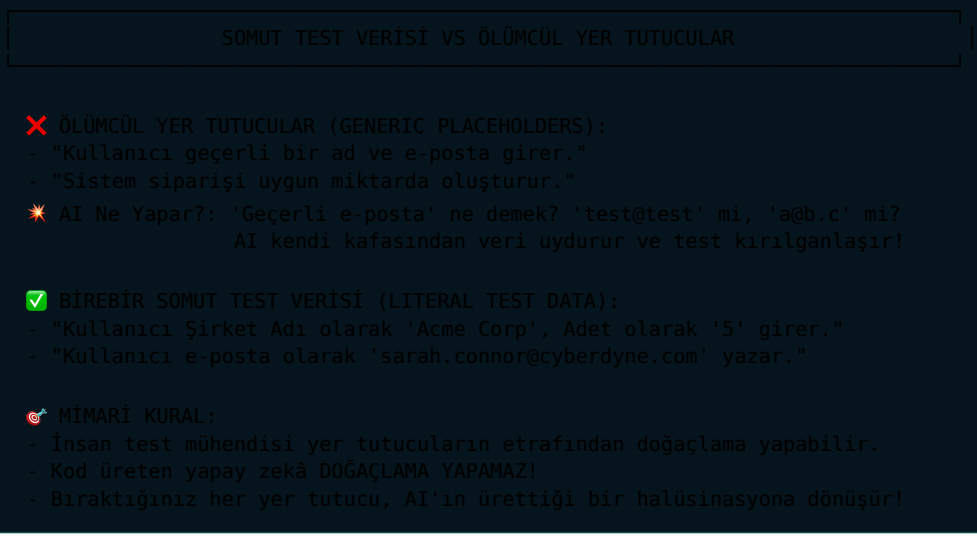
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

Test Verisi Disiplini: Somut Veri ('Acme Corp', '5') vs Ölümcül Yer Tutucular

Test Data Discipline: Literal Test Data vs Deadly Generic Placeholders

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir terziye takım elbise diktireceğinizi düşünün: Terziye 'Bana uygun boyda bir pantolon dik' derseniz (Yer tutucu!), terzi boyunuzu tahmin eder ve pantolon ya dizinizde kalır ya yerde sürünür! Ama 'Bel 82 cm, paça boyu 104 cm' derseniz (Somut veri!), pantolon üstünüze tam oturur. Testte de 'geçerli veri' değil, tam santimetreleri vermelisiniz!

🇬🇧 ENGLISH (ELI5)

Imagine ordering a custom tailored suit: If you tell the tailor 'Sew me a properly sized jacket' (Generic placeholder!), they guess your dimensions and the sleeves hang down past your knees! But if you declare 'Chest 40 inches, sleeve 33 inches' (Literal data!), it fits like a glove. Automated tests cannot guess; they demand exact dimensions.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Test Verisinde Birebirlik (Literal Test Data) İlkesi: Şartname ve test dokümanlarında mühendislerin düştüğü en büyük tuzaklardan biri soyut yer tutucudır ('geçerli bir müşteri', 'örnek bir ürün', 'uygun bir tarih'). İnsan test uzmanı bu metni okuduğunda zihninde 'John Doe', 'Laptop', '2026-05-01' gibi veriler uydurabilir. Ancak yapay zekâ destekli üretim motorları (`/test-case`) soyut kavramları rastgele doldurur. Bir çalıştırmada boşluk 'user1' ile doldurulurken diğerinde 'test@invalid' ile doldurulabilir ve testler tutarsızlaşır (flaky tests). Martinelli'nin altın kuralı: 'Test verisi harfi harfine somut (literal) olmalıdır.' Test tablosunda müşteri adı 'Acme Corp', ürün adı 'Widget-X', miktar '5', tarih '2026-10-20' olarak açıkça yazılmalıdır. Bu sayede üretilen test kodları deterministik, tekrarlanabilir ve sıfır varsayımla çalışır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Literal Test Data Mandate: In traditional testing, human QA engineers tolerate placeholder abstractions like 'a valid customer' or 'sufficient balance' because human intuition bridges the gap with impromptu mock values. Generative AI agents possess no such intuition. When presented with placeholder phrasing, an AI implementer invents arbitrary mock data—varying between runs, introducing edge-case boundary mutations, and creating flaky test suites. Martinelli insists upon literalism: 'Acme Corp, Widget, 5, never a valid customer.' Specifying explicit string literals, deterministic integers, and concrete RFC-compliant dates ensures that generated unit and integration tests compile with identical, deterministic assertions across every single regeneration cycle.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Deterministik Somut Test Verisi ile Güvenli Test İcrası
package com.maang.aiup.testdata;

public class LiteralTestDataDemo {
    // ❌ YANLIŞ: Soyut yer tutucu (AI kafasına göre mock nesnesi üretir)
    // var customer = createAnyValidCustomer();

    // ✅ DOĞRU: Birebir Somut Veri (Literal Test Data)
    public record CustomerOrder(String companyName, String productSku, int quantity) {}

    public static class OrderProcessor {
        public static boolean process(CustomerOrder order) {
            return "Acme Corp".equals(order.companyName())
                && "WDG-99".equals(order.productSku())
                && order.quantity() == 5;
        }
    }

    void main() {
        // Harfi harfine şartnamedeki somut veri:
        var order = new CustomerOrder("Acme Corp", "WDG-99", 5);
        boolean success = OrderProcessor.process(order);

        assert success : "Somut veri doğrulaması başarısız!";
        System.out.println("✅ Somut veriyle çalışan deterministik test başarıyla geçti: " + order);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Test senaryolarında asla 'geçerli bir kullanıcı adı' veya 'yeterli bakiye' gibi soyut tabirler yazmayın; her veriyi 'sarah.connor' ve '500.00 TL' şeklinde harfi harfine yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never write abstract descriptors like 'a valid user' or 'an active subscription'. Specify literal values ('sarah.connor', '$500.00') to guarantee deterministic tests.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Test senaryolarındaki veriler harfi harfine somut olmalıdır ('Acme Corp', '5 adet').
- Yer tutucular ('geçerli müşteri') AI'ın kafasına göre veri uydurmasına ve testlerin bozulmasına yol açar.
- Deterministik test verisi, şartnameden üretilen kodun her çalıştırmada aynı sonucu vermesini sağlar.

🎯 Key Architectural Takeaways

- Test data must be strictly literal ('Acme Corp', '5 items') with zero placeholders.
- Placeholders cause AI to improvise test values, introducing flaky and non-reproducible tests.
- Deterministic test fixtures guarantee identical outcomes across all code regenerations.

48

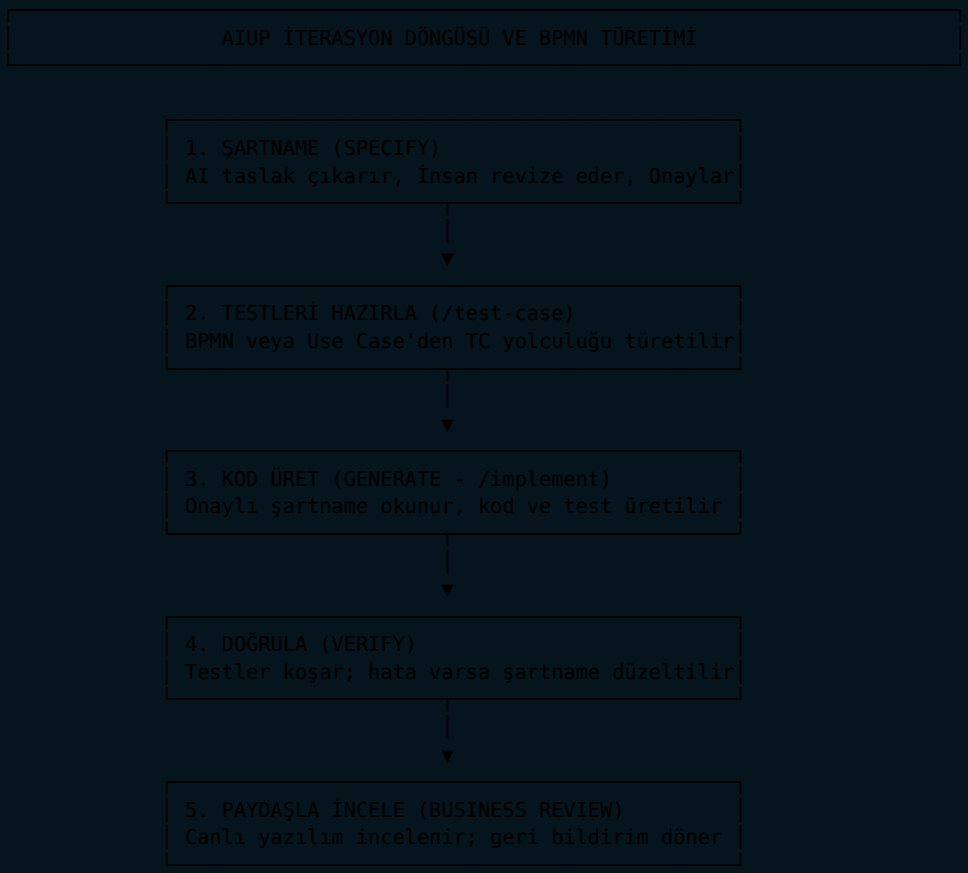
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

İterasyon Döngüsü ve BPMN Süreçlerinden Test Yolculuğu Türetme

The Iteration Loop & Deriving Journey Tests from BPMN Processes

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir film stüdyosunu düşünün: Önce senaryo yazılır (Şartname), sonra prova sahneleri planlanır (Testler), sonra aktörler sahneyi oynar (Kod üretimi), yönetmen sahneyi izler (Doğrulama), en son yapımcıya izletilir (Paydaş İncelemesi). Eğer yapımcı bir sahneyi beğenmezse aktör sette kafasına göre rol değiştirmez; senaryo yazarı senaryoyu günceller ve döngü baştan başlar!

🇬🇧 ENGLISH (ELI5)

Think of a film production studio: First the screenplay is polished (Spec), next the stunt sequences are choreographed (Tests), then actors perform on camera (Code Generation), the director reviews the footage (Verification), and finally the producer screens the cut (Business Review). If the producer wants a change, actors don't ad-lib on set; the writer updates the script and the loop re-executes cleanly!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

AIUP İterasyon Döngüsü ve BPMN Entegrasyonu: Her AI Unified Process iterasyonu 5 adımlı katı bir döngüde koşar: 1) Belirt (Specify): Analist şartnameyi hazırlar, paydaş onaylar (Approved). 2) Testleri Hazırla: `/test-case` komutu çalıştırılır. Eğer Inception aşamasında bir BPMN iş süreci çizilmişse, bu süreçteki her bir aktivite otomatik olarak bir use case çağrısına, her bir karar dalı ise ayrı bir TC yolculuk testine dönüştürülür! 3) Üret (Generate): `/implement UC-XXX` komutu mevcut kodları yerinde uzlaştırarak (reconcile) yeni kodları üretir. 4) Doğrula (Verify): Otomatik testler çalışır. Bir test başarısız olursa sorun %90 şartnamedeki bir belirsizlikten kaynaklanmaktadır; şartname düzeltilir ve tekrar üretilir. 5) Paydaşla İncele: Çalışan yazılım paydaşa sunulur; geri bildirimler doğrudan şartnameye işlenir. Bu döngüde 'şartnameyi atlayıp koda yama yapmak' (hotfix bypass) kesinlikle yasaktır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The 5-Step AIUP Iteration Loop & BPMN Derivation: Every engineering increment in the AI Unified Process executes a disciplined cadence: 1) Specify: The requirements engineer revises the use case to 'Approved'. 2) Prepare Tests: The `/test-case` engine ingests BPMN 2.0 process models from the Inception phase; every task activity maps to a use case invocation, and each gateway decision path synthesizes an independent TC journey file. 3) Generate: `/implement UC-XXX` executes, reconciling existing repositories and controllers in place without generating conflicting duplicates. 4) Verify: All tests execute automatically. Failures are traced back to spec ambiguities and rectified at the specification source. 5) Business Review: Working software is validated against the behavioral spec story. No code change ever bypasses this cycle.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 5 Adımlı İterasyon Döngüsünün Programatik Temsili
package com.maang.aiup.iteration;

public class IterationLoopDemo {
    public enum SpecStatus { DRAFT, APPROVED, IMPLEMENTED, VERIFIED, DONE }

    public record SpecLifecycle(String useCaseId, SpecStatus status, boolean testsPassing) {
        public SpecLifecycle advance() {
            return switch (status) {
                case DRAFT -> new SpecLifecycle(useCaseId, SpecStatus.APPROVED, false);
                case APPROVED -> new SpecLifecycle(useCaseId, SpecStatus.IMPLEMENTED, false);
                case IMPLEMENTED -> new SpecLifecycle(useCaseId, SpecStatus.VERIFIED, true);
                case VERIFIED -> testsPassing
                    ? new SpecLifecycle(useCaseId, SpecStatus.DONE, true)
                    : new SpecLifecycle(useCaseId, SpecStatus.APPROVED, false);
                case DONE -> this;
            };
        }
    }

    void main() {
        var cycle = new SpecLifecycle("UC-002", SpecStatus.DRAFT, false);
        while (cycle.status() != SpecStatus.DONE) {
            System.out.println("🔄 Döngü Adımı: " + cycle.status());
            cycle = cycle.advance();
        }
        System.out.println("🎯 İterasyon Tamamlandı: Use Case DONE durumuna ulaştı!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Üretim baskısı altında bile 'kodu hemen düzeltelim, şartnameyi sonra güncelleriz' tuzağına düşmeyin! Şartnameyi atlayan her müdahale şartnamenin güvenilirliğini öldürür.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never fall into the trap of patching code directly under production pressure with promises to update the spec later. Bypassing the specification starts the rot.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- AIUP iterasyon döngüsü 5 adımdan oluşur: Specify -> Test -> Generate -> Verify -> Review.
- Inception aşamasındaki BPMN süreçleri otomatik olarak TC-XXX yolculuk testlerine dönüştürülür.
- Test başarısızlığı öncelikle şartnamedeki bir eksiklik olarak ele alınır ve şartnamede düzeltilir.

🎯 Key Architectural Takeaways

- The AIUP iteration loop runs 5 steps: Specify -> Test -> Generate -> Verify -> Review.
- BPMN business process models directly synthesize executable TC-XXX test journeys.
- Test failures indicate spec ambiguities first and are rectified at the specification source.

BÖLÜM 9 / MODULE 9

Değişiklik Talepleri, Hatalar (Bugs) ve Mevcut (Brownfield) Sistemler

Change Requests, Bugs, and Brownfield Systems

15. İlke: 'Tüm İşler Bir Use Case Değişikliğidir'. İmza testi (Sign-off test) ile hata vs geliştirme ayrımı, neden /bug komutunun olmadığı, mevcut (brownfield) sistemlerin niyetini tersine mühendislikle kurtarma, 40 REST ucunu 4-8 use case'e indirgeme ve SE-RE ortak taban çizgisi (baseline) incelemesi.

Principle 15: 'All Work is a Change to a Use Case'. The sign-off test to classify bugs vs enhancements, why there is no /bug command, recovering specifications from legacy brownfield systems, collapsing 40 REST endpoints into 4-8 use cases, and the joint SE-RE baseline review.

49

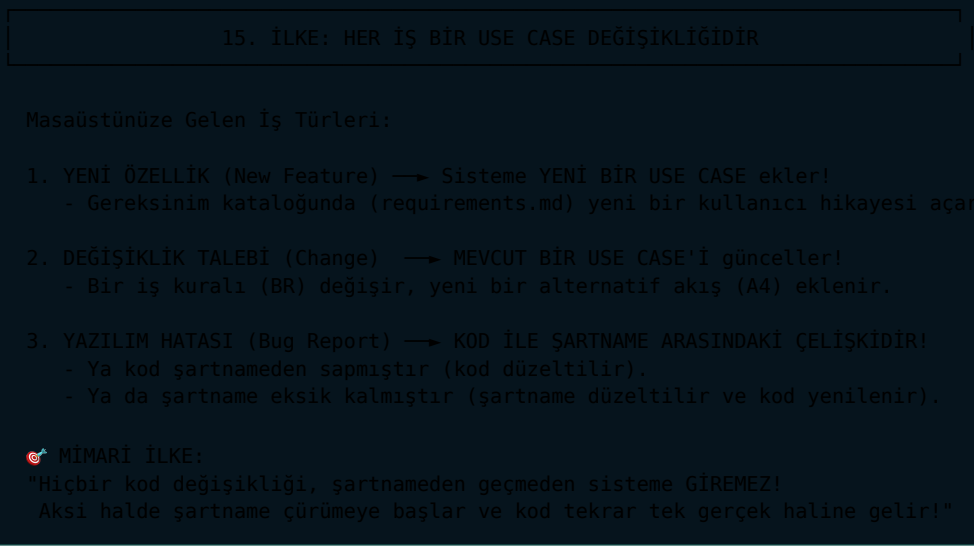
BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

15. İlke: Tüm İşler Bir Use Case Değişikliğidir (Özellik, Değişiklik, Hata)

Principle 15: All Work is a Change to a Use Case (Features, Changes, and Bugs)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphaneyi düşünün: Kütüphaneye yeni bir kitaplık eklemek (Yeni Özellik), mevcut kitaplığın sırasını değiştirmek (Değişiklik) veya yanlış rafa konmuş bir kitabı doğru rafa koymak (Hata)... Hepsi kütüphane kataloğuna (Use Case Şartnamesi) işlenmek zorundadır! Eğer rafa yeni kitap koyup kataloğa yazmazsanız, o kütüphane 1 hafta sonra çöplüğe döner!

🇬🇧 ENGLISH (ELI5)

Think of a grand library: Adding a new science section (New Feature), rearranging the history shelves (Change Request), or reshelving a misplaced book (Bug fix)... All three require updating the master library catalog (Use Case Spec)! If you move books without updating the catalog, the library descends into chaos within a week.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

15. İlke: Tüm İşler Bir Use Case Değişikliğidir (All Work is a Change to a Use Case): Yazılım projelerinde işler geliştiricilerin masasına tertemiz sıfır kilometre use case'ler halinde gelmez; Pazartesi bir değişiklik talebi, Salı günü ise bir bug raporu olarak düşer. AI Unified Process'te her üç iş türü de temelde aynı şeydir: Bir use case değişikliği! 1) Yeni Özellik: Sisteme yeni bir use case (UC-XXX) ekler veya mevcut olanı yeni aktör ve akışlarla genişletir. 2) Değişiklik Talebi: Var olan use case'i revize eder; örneğin 28 günlük ödünç süresini 14 güne düşürür (BR-004 güncellemesi). 3) Bug: Şartname ile kod arasındaki uyuşmazlıktır. Çoğunlukla kod yanlıştır; bazen de şartname paydaşın gerçek iradesini eksik yansıtmıştır. Çözüm her halükarda önce şartnamenin incelenmesi ve güncellenmesiyle başlar. Bu disiplin korunmazsa 'Principle 11' (Şartname koddan daha uzun yaşar) anında çöker.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 15: All Work is a Change to a Use Case: Software engineering reality rarely delivers pristine greenfield specifications. Work arrives as frantic Jira tickets, customer change requests, or production incidents. In the AI Unified Process, all three are structurally identical: a change to a use case specification. 1) New Features add use case specifications or attach new actors/flows. 2) Change Requests modify existing preconditions, business rules, or alternative flows. 3) Bugs represent disagreements between specification and reality. If a side-channel exists where developers patch production code directly without updating specifications, the specification rots and code reverts to being the fragile truth. By enforcing that every change flows specification-first, the architecture retains continuous regenerability.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Tüm İş Değişikliklerinin Şartname Tabanlı Mühürlenmesi
package com.maang.aiup.allwork;

public class WorkDeltaArchitecture {
    // İş Kuralı BR-004 Değişikliği: 28 günden 14 güne indirildi
    public record LoanPolicy(int maxDaysAllowed) {
        // Eski sürüm: maxDaysAllowed = 28
        // Değişiklik sürümü (CR-402): maxDaysAllowed = 14
        public static LoanPolicy defaultPolicy() {
            return new LoanPolicy(14); // Şartname güncellendi, kod yenilendi
        }
    }

    public static class LoanService {
        private final LoanPolicy policy;

        public LoanService(LoanPolicy policy) {
            this.policy = policy;
        }

        public int calculateDueOffset() {
            return policy.maxDaysAllowed();
        }
    }

    void main() {
        var service = new LoanService(LoanPolicy.defaultPolicy());
        System.out.println("✅ Şartname Değişikliği (CR-402) Koda Yansıdı: Yeni Vade = " +
service.calculateDueOffset() + " gün.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Acil bir bug çıktığında doğrudan koda girip 'quick fix' yapmayın! Önce şartnamedeki kuralı netleştirin, sonra /implement komutunu çalıştırın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never apply an unrecorded hotfix directly to production source code. Rectify the specification first, then execute generation to maintain single-truth traceability.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Yeni özellik, değişiklik veya bug fark etmez: Hepsi bir use case değişikliğidir.
- Şartnameyi atlayan her kod yaması, şartnamenin çürümesine ve projenin çökmesine yol açar.
- Değişiklik önce şartnamede onaylanır; ardından kod ve testler tek komutla yenilenir.

🎯 Key Architectural Takeaways

- Features, change requests, and bugs are all structurally a change to a use case.
- Any patch that bypasses the spec accelerates documentation rot and destroys regenerability.
- Changes are approved in the specification first; code and tests are regenerated deterministically.

50

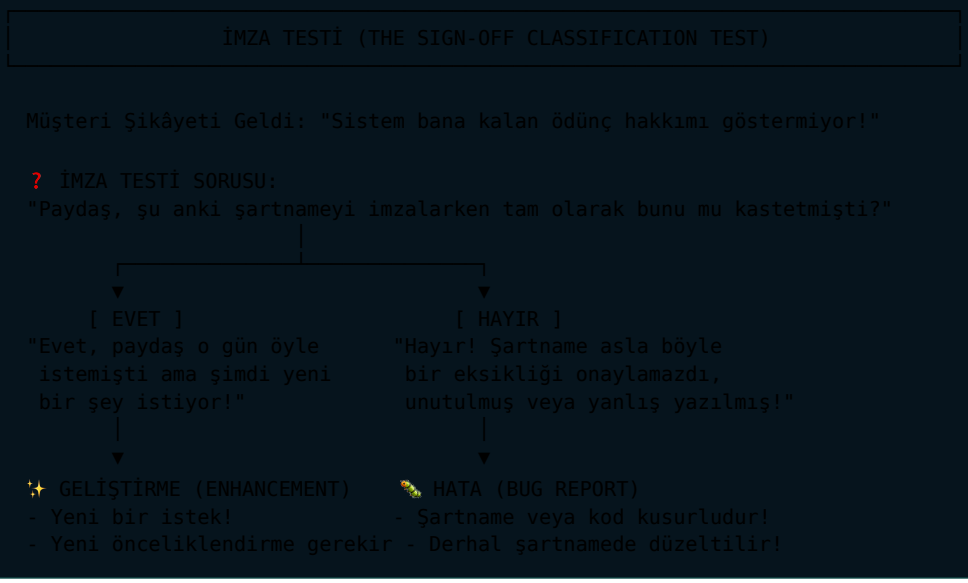
BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

İmza Testi (Sign-Off Test): Hata (Bug) ile Geliştirme (Enhancement) Ayrımı

The Sign-Off Test: Distinguishing Genuine Bugs from Scope Enhancements

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir marangoza masa yaptırdınız. Masayı teslim alırken 'Ahşap ayaklı 4 kişilik meşe masa' sözleşmesini imzaladınız. 2 ay sonra 'Ben bu masanın mermer olmasını istiyordum' derseniz marangoza kızamazsınız; bu yeni bir istektir (Enhancement)! Ama masanın bir bacağı kısaysa ve sallanıyorsa bu bir hatadır (Bug)! İmza testi, sözleşmeye bakıp kimin haklı olduğunu saniyeler içinde anlar!

🇬🇧 ENGLISH (ELI5)

You commission a carpenter to build a dining table and sign a contract for 'Solid Oak Table, 4 seats'. Two months later you declare: 'I actually wanted white Italian marble!' The carpenter didn't make a mistake; that is an Enhancement request! But if the table arrived with 3 legs and wobbles, that is a Bug! The sign-off test inspects original intent to classify the discrepancy instantly.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İmza Testinin (The Sign-Off Test) Hukuki ve Mimari Gücü: Kurumsal yazılım geliştirmede geliştiriciler ile ürün sahipleri arasındaki en büyük sürtünme 'Bu bir bug mı yoksa yeni bir geliştirme mi?' tartışmasıdır. Hatalar garanti kapsamında bedelsiz ve derhal çözülürken, geliştirmeler yeni bütçe ve sprint planlaması gerektirir. Martinelli bu karmaşayı tek bir zarif soruyla çözer: 'Bir paydaş, şu an ne istediğini biliyorken, eski şartnameyi imzalar mıydı?' Eğer cevap 'Evet, o tarihte bunu istiyorduk ama şimdi fikrimiz değişti' ise, bu açıkça bir Değişiklik/Geliştirmedir (Enhancement). Eğer cevap 'Hayır, o şartname zaten bizim asıl niyetimizi yanlış ifade etmiş veya eksik bırakmış' ise, bu tartışmasız bir Hatadır (Bug). Bu test sınıflandırmayı duygu ve polemikten çıkarıp objektif bir sözleşme analizine dönüştürür.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Sign-Off Classification Test: The eternal debate between product owners and engineering leads centers on classifying discrepancies as either defects or scope changes. Defects typically carry contractual SLA penalties and zero additional billing, whereas scope changes require backlog refinement. Martinelli resolves this dispute via the Sign-Off Test: 'Would a stakeholder have signed off the old specification, knowing what they want now?' If yes, the specification captured true historical consensus, and the customer has since evolved their requirements. That is an Enhancement. If no, the specification either misstated stakeholder intent or omitted a crucial invariant. That is a Bug. This test converts an adversarial dispute into an objective specification reconciliation.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: İmza Testine Göre İş Akışının Otomatik Yönlendirilmesi
package com.maang.aiup.signoff;

public class SignOffClassificationDemo {
    public enum WorkType { BUG, ENHANCEMENT }

    public record DiscrepancyReport(String description, boolean wouldStakeholderHaveSignedOldSpec) {
        public WorkType classify() {
            // İmza Testi Mantığı
            return wouldStakeholderHaveSignedOldSpec ? WorkType.ENHANCEMENT : WorkType.BUG;
        }
    }

    void main() {
        var report1 = new DiscrepancyReport("Üye ödünç geçmişinde kitap ISBN'i görünmüyor", true);
        var report2 = new DiscrepancyReport("Ödünç limiti 5 olmasına rağmen sistem 6. kitaba izin
veriyor", false);

        System.out.println("Rapor 1 Sınıflandırması: " + report1.classify() + " (Yeni istek)");
        System.out.println("Rapor 2 Sınıflandırması: " + report2.classify() + " (Kritik iş kuralı
hatası)");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Müşteri 'bu bir bug' dedi diye şartnameye bakmadan hemen koda müdahale etmeyin; imza testini uygulayın, işin geliştirme mi hata mı olduğunu tespit edin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not rush to patch code simply because a client labels an issue as a 'critical bug'. Apply the sign-off test to classify intent accurately before touching specifications.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- İmza testi: Paydaş şu anki aklıyla eski şartnameyi imzalar mıydı?
- Cevap 'Evet' ise bu yeni bir geliştirmedir (Enhancement); 'Hayır' ise bir hatadır (Bug).
- Sınıflandırma testi mühendisler ile ürün sahipleri arasındaki polemiği tamamen bitirir.

🎯 Key Architectural Takeaways

- The sign-off test: Would a stakeholder have signed off the old spec, knowing what they want now?
- If yes, it represents an enhancement; if no, it is a genuine defect.
- The test eliminates subjective arguments between engineering and product management.

51

BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Tek Bir Döngü, Özel Komut Yok: Neden /bug Komutu Bulunmaz?

One Loop, No Special Commands: Why There is No /bug Command in AIUP

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                NEDEN /BUG VEYA /CHANGE KOMUTU YOKTUR?

Geleneksel Yanılgı:
- "Yeni özellik için /create-feature çalıştır!"
- "Bug için /bug-fix çalıştır!"
- "Değişiklik için /change-request çalıştır!"

❌ BU NEDEN YANLIŞTIR?:
Çünkü sistemin kod üretim motoru (/implement) ne tür bir iş yaptığınızı
umursamaz! Motor sadece GÜNCEL ŞARTNAMEYİ ve TESTLERİ okur!

✅ AIUP TEK DÖNGÜSÜ (ONE LOOP TO RULE THEM ALL):
┌──────────────────────────────────────────────────────────┐
│ 1. Niyet Belirleme: İmza testiyle durumu analiz edin.    │
│ 2. Şartnameyi Değiştir: UC-XXX dosyasında kuralı veya akışı güncelleyin! │
│ 3. Aynı Komutu Koş: `/implement UC-XXX`                  │
│ 4. Testler Güvenceye Alır: Değişen testler güncellenir, eski testler │
│    yeşil kalmaya devam eder!                             │
└──────────────────────────────────────────────────────────┘

🎯 NETİCE: Özel bir sihirli komut yoktur; şartname değişir, kod hizalanır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir 3D yazıcıyı düşünün: 3D yazıcıya 'Bu bir heykel', 'Bu kırık bir dişli tamiri' veya 'Bu yeni bir vazo' diye ayrı komutlar vermezsiniz. 3D yazıcıya sadece güncel 3D çizim dosyasını (CAD / Şartname) verirsiniz ve 'Yazdır' (Print / Implement) dersiniz! Yazıcı dosyadaki farkı kendisi basar. AIUP'ta da tek komut 'Hizala ve Üret'tir!

🇬🇧 ENGLISH (ELI5)

Consider an industrial 3D printer: You don't possess separate buttons for 'Print art', 'Print replacement gear', or 'Print vase'. You simply feed it the updated 3D CAD blueprint (The Specification) and press 'Print' (/implement)! The printer materializes the exact delta. In AIUP, there is only one universal generation loop.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Tek Bir Döngü İlkesi (One Loop, No Special Commands): Birçok AI destekli araç 'Fix Bug', 'Refactor', 'Add Feature' gibi onlarca farklı özel prompt veya komut icat etmeye çalışır. Simon Martinelli bu yaklaşımı reddeder: AI Unified Process'te ne bir `/bug` komutu ne de bir `/change-request` komutu vardır! İşin türü hangi komutu çalıştıracağınıza değil, şartnamenin neresini düzenleyeceğinize karar verir. 1) Sınıflandırma yapılır. 2) Gereksinim mühendisi şartnameyi günceller (örneğin kural metnini veya alternatif akış adımını düzeltir). 3) Her zamanki `/implement UC-XXX` komutu yeniden çalıştırılır. Üretim motoru mevcut Java sınıflarını (Controller, Entity, Repository, Test) okur ve şartnamedeki değişikliği yerinde uzlaştırır (reconciliation). 4) Testler güncellenen beklentiyi karşılar; kaldırılan senaryoların testleri silinir. Bu sayede hata çözümü geçici bir yama üretmek yerine, sistemin en değerli varlığı olan şartnameyi zenginleştiren kalıcı bir yatırıma dönüşür.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Single Uniform Loop Principle: Fragmented tooling often introduces divergent commands for defects versus enhancements (`/fix-bug`, `/patch`, `/refactor`). Martinelli explicitly rejects this fragmentation: there is no `/bug` or `/change-request` command in the AI Unified Process. The nature of the work dictates what you edit in the specification, never which CLI command you invoke. 1) Classify by intent via the sign-off test. 2) Update the specification first (correcting a business rule or refining a flow trigger). 3) Rerun the identical `/implement UC-XXX` command. The generator parses existing source files, inspects Git diffs, and reconciles implementations in place without duplicate scaffolding. 4) Tests carry the delta. Existing invariants remain green while modified assertions guard the new behavior. Fixing a defect becomes an asset enhancement rather than an unrecorded runtime patch.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Yerinde Uzlaştırma (In-Place Reconciliation) Mantığı
package com.maang.aiup.reconcile;

public class SpecReconciliationDemo {
    // Şartname: "Sistem parolanın en az 12 karakter olduğunu doğrular (BR-009)"
    // Eski Kod: 8 karakterdi (Hata tespit edildi ve şartnamede 12 olarak düzeltildi)
    public static class PasswordSecurityPolicy {
        public static boolean isPasswordStrong(String password) {
            // /implement komutu şartnamedeki BR-009'u okur ve sabiti 12 yapar:
            int minimumLength = 12; // Reconciled from spec!
            return password != null && password.length() >= minimumLength;
        }
    }

    void main() {
        System.out.println("8 Karakterli Şifre Geçerli mi?: " +
PasswordSecurityPolicy.isPasswordStrong("Pass1234"));
        System.out.println("14 Karakterli Şifre Geçerli mi?: " +
PasswordSecurityPolicy.isPasswordStrong("SuperStrongPass2026!"));
        System.out.println("✅ /implement şartnameyi okudu ve kodu yerinde güncelledi.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Hata düzeltirken test dosyasındaki assertion'ı doğrudan değiştirmeyin; assertion şartnameden türer. Önce use case şartnamesini değiştirin, testi şartname güncellesin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never alter an automated assertion directly during a bug fix; the assertion derives from the spec. Rectify the specification first, then reconcile the test suite.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- AIUP'ta /bug veya /patch komutu yoktur; tek bir evrensel üretim döngüsü vardır.
- İşin türü çalıştırılan komutu değil, şartnamede düzenlenen alanı belirler.
- Hata çözümleri şartnameye işlendiği için aynı hatanın tekrar oluşması imkansızdır.

🎯 Key Architectural Takeaways

- There is no /bug command in AIUP; all changes traverse the single specification loop.
- Work classification dictates which spec section to edit, not which command to run.
- Defects fixed through specs permanently inoculate the codebase against regressions.

52

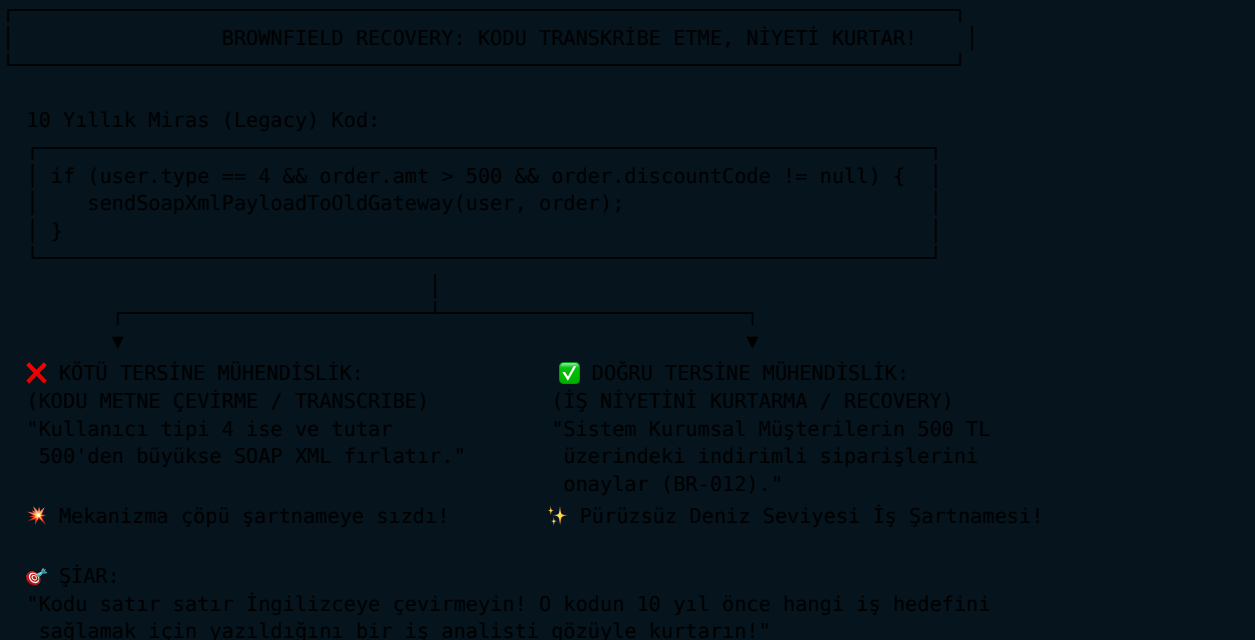
BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Mevcut Sistemleri Kurtarma (Brownfield): Kodu Değil, Niyeti Kurtarın!

Brownfield Recovery: Reverse-Engineering Intent, Not Transcribing Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Toprağın altından kırık bir antik vazo çıkaran bir arkeolog düşünün: Arkeolog vazonun üstündeki çamuru ve tozu kopyalayıp müzeye koymaz! Çamuru dikkatlice temizler ve altındaki paha biçilmez altın deseni (Niyeti) ortaya çıkarır. Brownfield recovery, 10 yıllık eski kodun üstündeki SOAP ve XML çamurlarını temizleyip saf iş mantığını kurtarmaktır!

🇬🇧 ENGLISH (ELI5)

Think of an archaeologist excavating an ancient vase: The archaeologist doesn't duplicate the dirt and mud crust into museum showcases! They delicately brush off the grime to reveal the exquisite gold inlay beneath (The Intent). Brownfield recovery washes away 10 years of legacy XML and SOAP grime to rescue pure business logic!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Mevcut (Brownfield) Sistemlerde Tersine Mühendisliğin Felsefesi: Dünyadaki en kritik yazılımların %90'ı sıfırdan yazılmaz; yıllardır çalışan brownfield sistemlerdir. Martinelli'nin 'reverse-engineer' becerisi şu manifesto ile başlar: 'Siz kodu transkribe etmiyorsunuz. Siz o kodun tatmin etmek için inşa edildiği İŞ NİYETİNİ kurtarıyorsunuz ve bunu bir iş analistinin kod yazılmadan önce yazacağı seviyede kâğıda döküyorsunuz!' Concretely: 1) Controller ve View'lar use case değildir; use case'in kanıtlarıdır. 2) Koddaki her bir `if (x > y)` kontrolü muhtemel bir iş kuralı (BR-XXX) adayıdır. 3) Veritabanı şeması doğrudan Entity Model'e dönüştürülür. 4) Eğer bir use case adımı yalnızca o kodu okumuş birinin anlayacağı terimler içeriyorsa derhal silinmeli ve iş diline çevrilmelidir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Philosophy of Brownfield Reverse-Engineering: Greenfield green-pasture projects are rare; enterprise software is overwhelmingly brownfield. The AIUP reverse-engineering discipline is guided by an uncompromising stance: 'You are not transcribing the code. You are recovering the intent the code was built to satisfy, and writing it down at the level a business analyst would have written it before implementation.' Concretely: 1) Endpoints, controllers, and screens are not use cases; they are circumstantial evidence of use cases. 2) Guard clauses and validation branches are candidate business rules (BR-XXX), requiring translation into domain vernacular. 3) Database tables are evidence of entity model entities. 4) If a recovered step only makes sense to an engineer who has examined the repository source, it violates Zone 3 and must be rewritten into observable stakeholder behavior.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Legacy Spagettiden Kurtarılan Saf İş Niyeti
package com.maang.aiup.brownfield;

public class LegacyRecoveryDemo {
    // ❌ ESKİ MİRAS KOD (10 Yıllık Spagetti):
    // public void proc_99(int t, double a) { if (t == 4 && a > 500) ... }

    // ✅ KURTARILAN İŞ MODELİ (Clean & Expressive):
    public record CorporateOrder(String companyId, double amount, boolean hasDiscount) {}

    public static class RecoveredCorporateOrderPolicy {
        public static boolean isEligibleForVipProcessing(CorporateOrder order) {
            // Koddaki 't == 4' ifadesinin kurumsal müşteri olduğunu kurtardık!
            return order.amount() > 500.0 && order.hasDiscount();
        }
    }

    void main() {
        var order = new CorporateOrder("CORP-101", 750.0, true);
        System.out.println("✅ Miras Koddan Kurtarılan İş Kuralı Sonucu: "
            + RecoveredCorporateOrderPolicy.isEligibleForVipProcessing(order));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Eski sistemi kurtarırken 'hazır görmüşken bu hatayı da şartnamede düzelteyim' demeyin! Kurtarma (Discovery) ile Karar Verme (Deciding) iki ayrı aşamadır. Önce ne olduğunu yazın, sonra incelemede düzeltin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not attempt to fix business bugs while reverse-engineering legacy code. Recovery is discovery; decision-making is a subsequent stakeholder activity. Document what exists first.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Miras sistemlerde kod transkribe edilmez; kodun çözdüğü iş niyeti kurtarılır.
- Koddaki if-else guard blokları şartnamede BR-XXX iş kurallarına dönüştürülür.
- Kurtarma (keşif) ile karar verme (düzeltme) süreçleri birbirinden kesin olarak ayrılmalıdır.

🎯 Key Architectural Takeaways

- In brownfield systems, do not transcribe code; recover the underlying business intent.
- Guard clauses and validations in legacy code translate into candidate BR-XXX business rules.
- Discovery must remain separate from decision-making during legacy specification recovery.

53

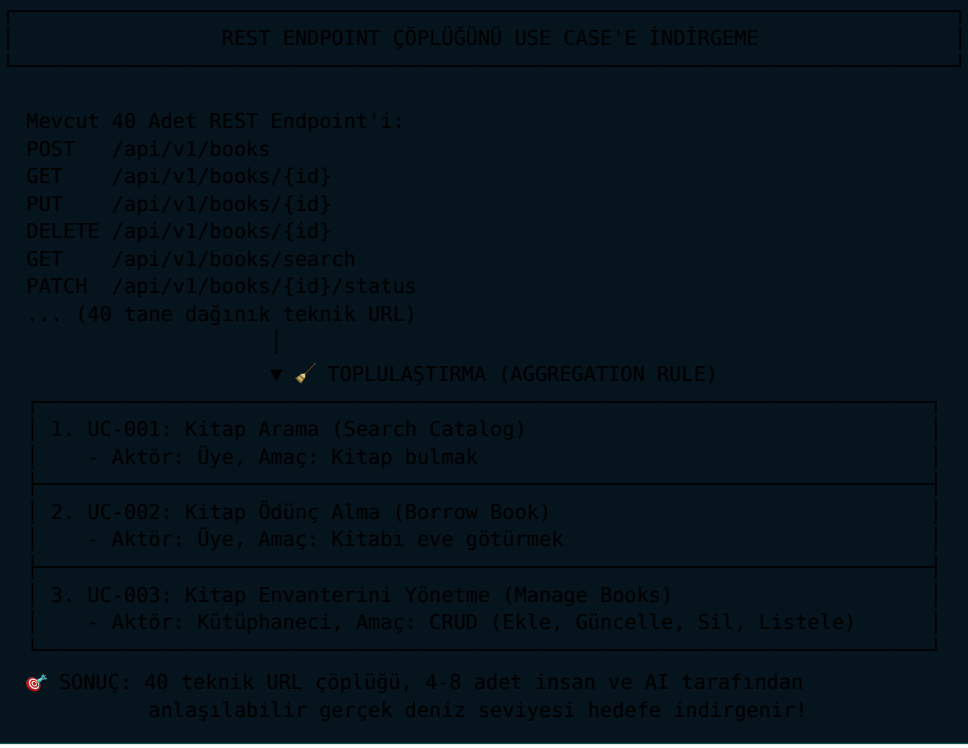
BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Uç Noktaları Birleştirme: 40 REST Endpoint'ini 4-8 Use Case'e İndirgeme

Aggregating Endpoints: Collapsing 40 REST Endpoints into 4-8 Use Cases

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir akıllı telefonun içini açarsanız 500 tane lehimli kablo ve devre görürsünüz. Ama ekranı açtığınızda sadece 4 ikon görürsünüz: Telefon, Mesajlar, Kamera, Ayarlar! Kimse lehim kablosuyla uğraşmak istemez. İşte 40 REST endpoint'i o lehim kablolarıdır; 4-8 use case ise ekrandaki sade ve kullanışlı uygulama ikonlarıdır!

🇬🇧 ENGLISH (ELI5)

If you pry open an iPhone, you find 500 soldered micro-circuits and copper ribbons. But on the screen, you only interact with 4 icons: Phone, Messages, Camera, Settings! Nobody wants to touch raw solder. Forty REST endpoints are internal wiring; 4 to 8 use cases are the clean, intuitive app icons on the home screen!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Endpoint Toplulaştırma ve 'Manage X' Kuralı: Brownfield projelerde yapılan en yaygın hata, her bir Controller metodunu veya REST endpoint'ini bir use case zannetmektir. Bir projede 40 endpoint varsa, 40 tane use case yazmaya kalkan bir ekip şartname çöplüğünde boğulur. Martinelli'nin altın kuralı: 'Controller'lar, rotalar ve ekranlar use case değildir; use case'in delilleridir. Onları toplulaştırın!' 5 farklı CRUD endpoint'i (Create, Read, Update, Delete, List) aynı aktöre ve aynı veri akışına hizmet ettiği sürece tek bir 'Manage Books' use case'idir. Ancak ne zaman bir işlem kendi bağımsız iş kurallarına ve güvenlik rollerine sahip olursa (örn: 'Block Customer'), o zaman o işlem 'Manage X' çatısından ayrılıp kendi eylem seviyesi (action-level) use case'ine dönüşür. Tipik bir mikroservis bu yöntemle 4 ila 8 arası derli toplu use case'e indirgenir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Endpoint Aggregation and the 'Manage X' Heuristic: The fatal pitfall in legacy recovery is mistaking technical controllers or routes for use cases. Documenting 40 use cases for 40 HTTP endpoints yields bureaucratic paralysis. Martinelli's rule: 'Controllers, routes, and views are evidence of use cases, not use cases. Aggregate them.' Five REST endpoints (`POST /books`, `PUT /books/{id}`, `GET /books`, `DELETE /books/{id}`) are consolidated into a single 'Manage Books' use case so long as they share an identical actor and shape. Only when a sub-operation acquires unique preconditions, distinct role-guards, or diverging state transitions (e.g., 'Archive Book' requiring audit approval) does it split out as an action-level use case. A production microservice typically condenses into 4 to 8 user-goal specifications.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 4 REST Endpoint'inin Tek Bir 'ManageBooks' Servisinde Toplulaştırılması
package com.maang.aiup.aggregation;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class EndpointAggregationDemo {
    public record BookRecord(String id, String title, String author) {}

    // Tek bir 'UC-003: Manage Books' use case'i altındaki CRUD operasyonları
    public static class ManageBooksUseCase {
        private final Map<String, BookRecord> db = new ConcurrentHashMap<>();

        public void createBook(BookRecord book) { db.put(book.id(), book); }
        public BookRecord readBook(String id) { return db.get(id); }
        public void updateBook(BookRecord book) { db.put(book.id(), book); }
        public void deleteBook(String id) { db.remove(id); }
    }

    void main() {
        var uc = new ManageBooksUseCase();
        uc.createBook(new BookRecord("B1", "Effective Java", "Joshua Bloch"));
        System.out.println("✅ 4 REST endpoint'i tek bir 'Manage Books' use case'inde
toplulaştırıldı.");
        System.out.println("   -> Okunan Kitap: " + uc.readBook("B1"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Her HTTP GET veya POST isteği için ayrı bir use case dosyası açmayın! CRUD işlemlerini tek bir 'Manage X' altında toplayın; yalnızca kuralları ayrışan işlemleri split edin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not generate individual use case documents for every HTTP GET or POST route. Aggregate standard CRUD into 'Manage X'; split only when business policies diverge.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- REST uçları use case değildir; use case'in varlığına işaret eden teknik izlerdir.
- Aynı aktör ve akışı paylaşan CRUD işlemleri tek bir 'Manage X' altında toplanır.
- Tipik bir kurumsal mikroservis 40 endpoint'ten 4-8 anlamlı use case'e indirgenir.

🎯 Key Architectural Takeaways

- REST endpoints are not use cases; they are technical evidence of underlying use cases.
- CRUD operations sharing the same actor and flow aggregate into a single 'Manage X' spec.
- A typical microservice condenses from 40 low-level routes into 4 to 8 meaningful use cases.

54

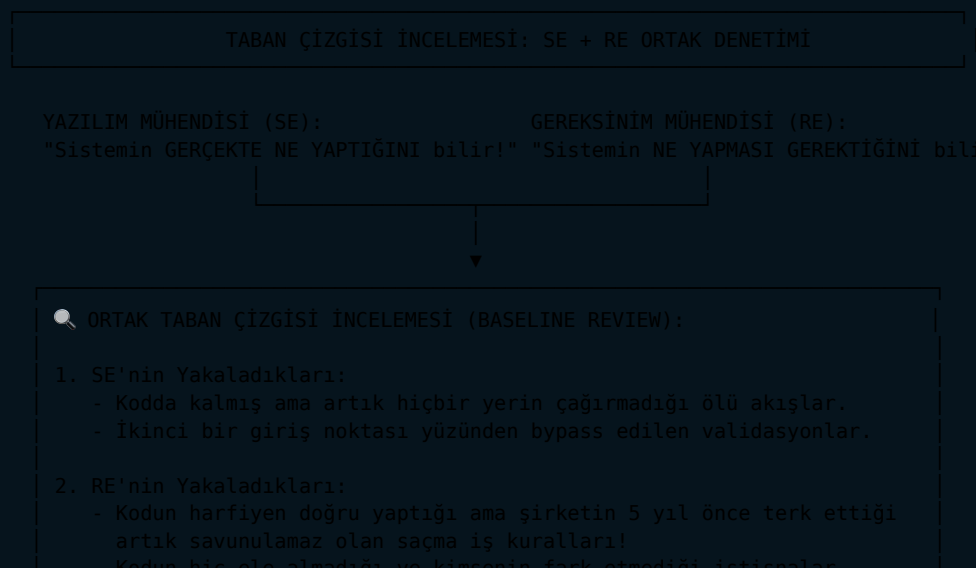
BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Taban Çizgisi İncelemesi (Baseline Review): SE ve RE Ortak Eşleşmesi

The Baseline Review: Software Engineer & Requirements Engineer Joint Pairing

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski bir evi tadilata sokacağınızı düşünün: Tesisat ustası (Yazılım Mühendisi) der ki: 'Bu boru patlak, bu kablo hiçbir yere bağlanmıyor!' İç mimar (Gereksinim Mühendisi) der ki: 'Bu duvarı yıkıp salonu büyütmemiz lazım, o oda artık kullanılmıyor!' Tesisatçı tek başına evi anlayamaz, mimar tek başına boruları bilemez. İkisi el ele verince ev saraya dönüşür!

🇬🇧 ENGLISH (ELI5)

Imagine renovating a 100-year-old mansion: The plumber (Software Engineer) notes: 'This pipe is rusted, and this wire leads nowhere!' The interior architect (Requirements Engineer) says: 'We need to knock down this wall to expand the living room; that pantry is obsolete!' Neither can restore the estate alone. Together, they create a master renovation baseline!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Taban Çizgisi İncelemesinin (The Baseline Review) Kritik Rolü: Mevcut bir sistemden çıkarılan tersine mühendislik taslakları, koda bakarak yazıldığı için süreçteki 'en az güvenilir' belgelerdir. Bu taslakları güvenilir bir 'Taban Çizgisi' (Spec Baseline) haline getiren şey, iki farklı disiplinden uzmanın bir odaya kapanıp yaptığı ortak incelemedir: 1) Yazılım Mühendisi (SE): Kodun gerçekte ne yaptığını bilir. Artık hiçbir ekranın tetiklemediği ölü kod yollarını, arka kapılardan bypass edilen güvenlik açıklarını ve eski bir sabitten türetilmiş gereksiz kuralları yakalar. 2) Gereksinim Mühendisi (RE): Sistemin ne yapması gerektiğini veya kime soracağını bilir. Kodun harfiyen doğru icra ettiği ama günümüz iş mantığına tamamen aykırı düşen mantıksız akışları, kodun hiç yakalamadığı istisnaları ve şirketin 5 yıl önce bıraktığı terimleri temizler. İnceleme sonucunda uzlaşılamayan her konu (belgelenmemiş ama kullanılan davranışlar, bozulmuş fonksiyonlar) bir Değişiklik Günlüğü'ne (Change Log) yazılır ve paydaş onayına sunulur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Baseline Review Pairing: A recovered specification draft written from source code inspection is inherently untrustworthy—it describes what was compiled, not what was intended. Elevating these drafts into an authoritative 'Spec Baseline' requires a rigorous joint pairing between two complementary roles: 1) The Software Engineer (SE) understands actual system behavior. They catch obsolete paths nothing invokes, validations bypassed by secondary routes, and fake business rules inferred from legacy constants. 2) The Requirements Engineer (RE) understands business domain intent. They catch flows that are mathematically accurate to the code but indefensible to customers, unhandled failure modes that went unnoticed for years, and deprecated terminology. Neither role can succeed in isolation: an SE alone produces formatted code transcription; an RE alone produces an ungrounded wish list. The reconciled output establishes the spec baseline from which all future development proceeds.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Baseline Review Sonrası Temizlenen Taban Çizgisi Modeli
package com.maang.aiup.baseline;

public class BaselineReviewReconciliation {
    // SE ve RE İncelemesinden Geçmiş Uzlaşılmış Varlık
    public record ReconciledLoanRule(
        boolean deadCodeEliminated,
        boolean businessIntentVerified,
        String approvedPolicy
    ) {}

    public static class BaselineReviewAuditor {
        public static ReconciledLoanRule audit(boolean seApproved, boolean reApproved) {
            if (seApproved && reApproved) {
                return new ReconciledLoanRule(true, true, "✅ APPROVED BASELINE: Vade 28 gün, Maksimum
5 kitap.");
            }
            throw new IllegalStateException("❌ İnceleme tamamlanmadı! Her iki uzmanın onayı
şarttır.");
        }
    }

    void main() {
        var baseline = BaselineReviewAuditor.audit(true, true);
        System.out.println("Taban Çizgisi İnceleme Sonucu: " + baseline.approvedPolicy());
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Taban çizgisi incelemesini tek bir uzmana (sadece yazılımcıya veya sadece analiste) emanet etmeyin; yazılımcı koda aşık olur, analist ise gerçeklerden kopar.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never assign the baseline review to a single person. An engineer transcribes code habits; an analyst invents an ungrounded wish list. Pairing is indispensable.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Kurtarılan taslak şartnameler, SE ve RE ortak incelemesinden geçerek 'Baseline' olur.
- Yazılım mühendisi kod gerçekliğini, gereksinim mühendisi ise iş hedeflerini korur.
- Uzlaşılamayan davranışlar doğrudan Değişiklik Günlüğü'ne (Change Log) aktarılır.

🎯 Key Architectural Takeaways

- Recovered specification drafts become an authoritative 'Baseline' via joint SE-RE review.
- The software engineer guards code truth; the requirements engineer guards business intent.
- Unresolved ambiguities populate the initial change log for formal stakeholder arbitration.

BÖLÜM 10 / MODULE 10

Kontrol Listeleri, Yaygın Hatalar ve MAANG Mimar Seviyesi Ustalık

Checklists, Common Mistakes & MAANG Architect Mastery

24 maddelik şartname kontrol listesi, 10 yaygın hata (5 klasik + 5 AI çağı tuzağı), monitör kenarı adım yazma kartı (Desk Card), MAANG sistem tasarımı mülakatlarında AIUP, otonom çoklu ajan (Multi-Agent) iş akışları ve Staff+ Seviye Gereksinim Mimarlığı yönetişimi.

The 24-point specification checklist, the ten common mistakes (5 classical + 5 AI-era traps), the split-pane step-writing desk card, AIUP in MAANG system design interviews, autonomous multi-agent code generation workflows, and Staff+ Requirements Architect governance.

55

BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

24 Maddelik Şartname Kontrol Listesi: Reviewed ve Approved Eşiği

The 24-Point Specification Checklist: Gatekeeper for Reviewed & Approved States

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

24 MADDELİK ŞARTNAME KONTROL LİSTESİ TABLOSU

KATEGORİ	MADDELER
1. Kimlik ve Yapı (Identity & Structure) - Aktif fiil öbeği, kebab-case dosya adı, Tetikleyici vs Ön Koşul ayrımı	Maddeler 1 - 6 Sea-level hedefi, requirements.md bağı
2. Ön ve Son Koşullar (Pre & Postconditions) - Sistemce zorlanabilir ön koşul, asla akış içinde tekrar test edilmez	Maddeler 7 - 9 Başarı ve Hata minimum garantileri dolu!
3. Ana Başarı Senaryosu (Main Flow) - 3-9 adım, 'Top Kimde' ilk kelime, UI mekaniği (widget) temizlenmiş!	Maddeler 10 - 14 'Validates' kullanımı, 3 tür adımın dengesi
4. Alternatif Akışlar (Alternative Flows) - Her validasyona 1 akış, adıma çıpalı, kesin bitiş ('continues' / 'ends')	Maddeler 15 - 18 Açık uçlu akış YASAK! Alt hatalar birleştirildi
5. İş Kuralları ve Referanslar - BR-XXX eşleşmesi, entity_model bağı,	Maddeler 19 - 21 NFR ve Mockup linkleri
6. Üç Okuyucu Testi (The Three-Reader Test) - Zone 1 somut veriler, Kara Liste sıfır! Paydaş onaylayabilir mi?	Maddeler 22 - 24 Seneye stack değişse bu cümle doğru kalır mı?

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir uzay mekiği fırlatılmadan önce astronotların ve mühendislerin elindeki kontrol listesini (checklist) düşünün: 'Oksijen vanası açık mı? Basınç tamam mı? Yakıt kapağı kilitli mi?' Tek bir madde bile işaretlenmezse roket fırlatılmaz! 24 maddelik use case kontrol listesi, kod üretim butonuna basmadan önceki o fırlatma kontrol listesidir!

🇬🇧 ENGLISH (ELI5)

Think of the pre-launch checklist used by NASA astronauts and flight controllers before a space shuttle liftoff: 'Oxygen valves open? Thruster pressure nominal? Fuel hatch sealed?' If a single checkbox is missed, the launch is aborted! The 24-point specification checklist is the mandatory pre-flight audit before triggering AI code generation!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

24 Maddelik Şartname Kontrol Listesinin Mimarisi: Cockburn'ün klasik geçiş/kalış (pass/fail) testleri ile AI Unified Process'in katı doğrulama kurallarının birleşiminden oluşan bu liste, bir şartname 'Reviewed' ve ardından 'Approved' statüsüne geçmeden önce iki kez koşturulur. Liste 6 ana alana ayrılır: 1) Kimlik ve Yapı: İsim kullanıcının dilinde bir eylem öbeğidir, dosya adı `UC-XXX-\<kebab-name>.md` formatındadır, tek bir deniz seviyesi hedefi anlatır. 2) Ön ve Son Koşullar: Ön koşullar sistem tarafından zorlanabilir durumdadır ve akış içinde asla tekrar `if` ile test edilmez. Başarı ve hata son koşulları boşa çıkarılamaz bir sözleşmedir (no partial data). 3) Ana Akış: 3 ila 9 adımdır, her adımda topun sahibi ilk kelimedir, 'validates' kelimesi başarıyı mühürler, UI tıklama mekanikleri ayıklanmıştır. 4) Alternatif Akışlar: Her doğrulama adımının kesin bir hata akışı vardır; her akış bir adıma çıpalanır (`step 4`) ve mutlaka kesin bir dille sonlanır (`Use case continues at step N.` veya `Use case ends.`). 5) Kurallar ve Referanslar: Her BR kuralı en az bir adımdan referans alır; varlıklar `entity_model.md`'den gelir. 6) Üç Okuyucu Testi: Kara liste (JWT, SQL, bcrypt vb.) sıfır toleransla taranır; paydaş onayı ve 1 yıl sonraki teknoloji bağımsızlığı doğrulanır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The 24-Point Specification Checklist Architecture: Merging Cockburn's classical pass/fail heuristics with the algorithmic verification rules of the AI Unified Process, this 24-point checklist gates specification promotion: executed once prior to 'Reviewed', and once more prior to 'Approved'. It spans six cohesive evaluation domains: 1) Identity & Structure: User-goal naming, strict kebab-case file paths matching `docs/use_cases.puml`, single-sitting sea-level scope, and requirements catalog linkages. 2) Boundary Guarantees: Preconditions must be enforceable assertions established upstream (never re-tested in-flow); failure postconditions must enforce transactional rollbacks (zero orphan state). 3) Scenario Discipline: 3-9 steps, active voice, ball-holder primacy, strict usage of 'validates', and zero UI widget micro-actions. 4) Deviation Robustness: 1-to-1 parity between validations and alternative flows, anchored triggers, and strictly banned open-ended terminations. 5) Referential Integrity: Business rules bi-directionally linked, domain nouns rooted in `docs/entity_model.md`. 6) Three-Reader Audit: Complete absence of Zone 3 blocklist terms, verifiable stakeholder outcomes, and enduring stack portability.

⚡ Modern Java 25 & Clean Architecture Uygulaması / ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: 24 Maddelik Şartname Denetim Motoru (Spec Linter Pipeline)
package com.maang.aiup.checklist;

import java.util.List;

public class SpecChecklistAuditor {
    public record SpecCheckResult(int passedCount, int totalCount, List<String> violations) {
        public boolean isApproved() { return violations.isEmpty() && passedCount == totalCount; }
    }

    public static class AuditorEngine {
        public SpecCheckResult auditSpec(String specContent) {
            var violations = new java.util.ArrayList<String>();

            // Kontrol 1: Kara liste kelimeleri (Zone 3)
            List<String> blocklist = List.of("JWT", "SQL", "bcrypt", "SELECT", "INSERT", "SMTP");
            for (String forbidden : blocklist) {
                if (specContent.contains(forbidden)) {
                    violations.add("Kural 22 İhlali: Yasaklı Zone 3 kelimesi tespit edildi -> " +
forbidden);
                }
            }

            // Kontrol 2: 'checks whether' kullanımı (Kural 13)
            if (specContent.contains("checks whether")) {
                violations.add("Kural 13 İhlali: 'checks whether' yerine 'validates that'
kullanılmalıdır.");
            }

            int total = 24;
            int passed = total - violations.size();
            return new SpecCheckResult(passed, total, violations);
        }
    }

    void main() {
        var engine = new AuditorEngine();
        String sampleSpec = "Step 4: System validates that user account is active (BR-001).";
        var result = engine.auditSpec(sampleSpec);

        System.out.println("✅ Şartname Denetim Skoru: " + result.passedCount() + " / " +
result.totalCount());
        System.out.println("   Onay Durumu: " + (result.isApproved() ? "APPROVED" : "REJECTED"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı / ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir şartnameyi 'küçük bir hata var ama kod üretirken düzeltiriz' diyerek Approved statüsüne geçirmeyin! 24 kuralın tamamı yeşil olmadan kod üretim komutunu (/implement) çalıştırmayın.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never promote a spec to 'Approved' with known checklist violations under the assumption that AI will compensate. All 24 criteria must pass cleanly before generation.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• 24 maddelik liste şartnameyi Reviewed ve Approved aşamalarına taşıyan kapı bekçisidir.

• Liste kimlikten ön koşullara, senaryo ritminden kara liste denetimine 6 alanı kapsar.

• Tek bir ihlal bile AI'ın mimariyi yanlış üretmesine yol açabileceği için sıfır toleranslıdır.

🎯 Key Architectural Takeaways

• The 24-point checklist acts as the definitive gatekeeper for Reviewed and Approved states.

• It evaluates 6 domains: identity, pre/postconditions, scenario, branches, rules, and 3 readers.

• Zero tolerance: a single checklist defect invites prompt hallucination and layer misallocation.

56

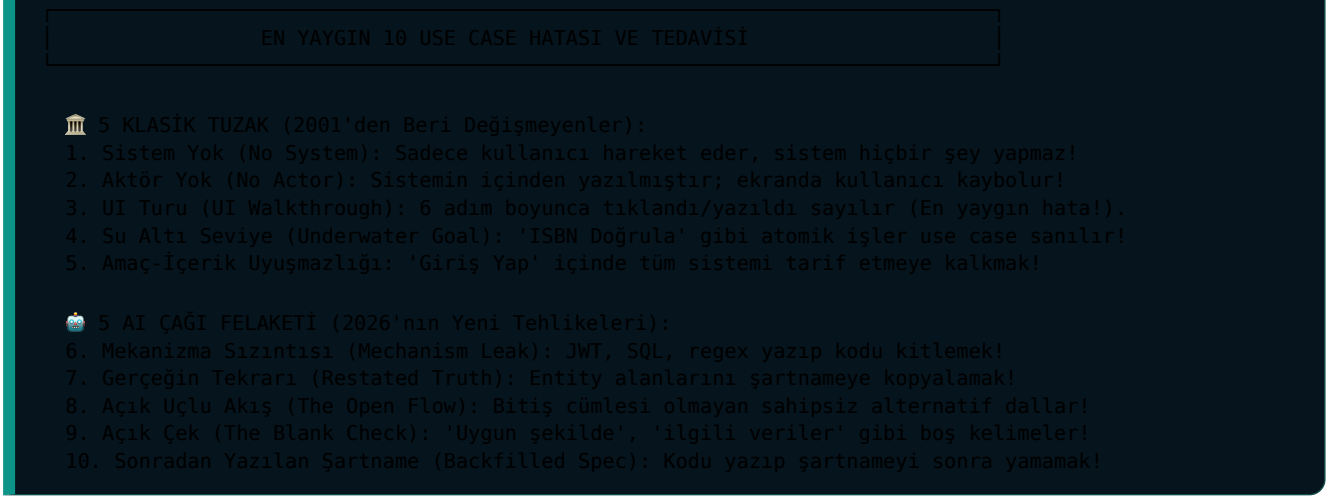
BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

En Yaygın 10 Hata: 5 Klasik Tuzak vs 5 AI Çağı Felaketi

The Ten Common Mistakes: 5 Classical Pitfalls vs 5 AI-Era Disasters

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir pastacı düşünün: 5 tane klasik hata vardır (Fırını açmayı unutmak, unu döküp şekeri unutmak, yumurtayı kabuğuyla kırmak gibi). Bir de yeni nesil akıllı elektrikli fırınların getirdiği 5 hata vardır (Fırının wifi şifresini kek harcına karıştırmak, derece yerine fırının kodunu değiştirmek gibi). İşte bu 10 hata hem 20 yıldır yapılan hem de AI ile yeni başlayan en tehlikeli 10 tuzaktır!

🇬🇧 ENGLISH (ELI5)

Think of baking: There are 5 classical amateur blunders (forgetting to preheat, dumping uncracked eggs, confusing salt with sugar). And there are 5 brand-new modern digital hazards (spilling liquid on the microchip panel, overriding the firmware safety valve). The 10 common mistakes split evenly between eternal human blunders and fatal AI-era traps!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

10 Yaygın Hatanın Analizi ve Önleme Stratejileri: Martinelli'nin use case analizinde hatalar iki eşit kampa ayrılır: 1) Klasik Hatalar: Cockburn'ün 2001'de uyardığı tuzaklar bugün de devam etmektedir. 'Sistem Yok' hatasında kullanıcı her şeyi yapar ama sistem tepki vermez, sonuçta AI eylemsiz bir UI üretir. 'UI Walkthrough' hatasında 6 adımda form doldurma anlatılır, oysa 'Kullanıcı üye bilgilerini girer' denmelidir. 'Su Altı Hedef' hatasında tek bir buton kontrolü use case yapılır. 2) AI Çağı Hataları: 'Mekanizma Sızıntısı' (Zone 3 ihlali), şartnamenin teknoloji göçlerinde çöp olmasına yol açar. 'Gerçeğin Tekrarı' (Restated Truth), entity_model.md ile use case arasında çelişki yaratır ve AI sessizce uydurma yapar. 'Açık Uçlu Akış' (Open Flow), sonu belirtilmemiş bir alternatif daldır; insan bunu görmezden gelir ama AI kafasından bir bitiş uydurarak müşteriyi yanlış bir ekrana fırlatır. 'Açık Çek' (Blank Check), 'uygun şekilde işler' diyerek kararı AI'a bırakmaktır. 'Sonradan Yazılan Şartname' (Backfilled Spec) ise şartname tabanlı yazılımı tamamen öldüren en ölümcül günahtır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Analysis of the Ten Common Mistakes: Martinelli categorizes specification failures into five classical human biases and five novel AI-era failure modes: 1) Classical Pitfalls: 'No System' leaves out automated responses, yielding inert UI forms; 'No Actor' specifies internal batch routines, discarding presentation layers; 'UI Walkthrough' recites keystrokes rather than intent; 'Underwater Goals' elevate atomic validation routines to full use cases; 'Purpose-Content Mismatch' overloads auxiliary flows. 2) AI-Era Disasters: 'Mechanism Leakage' bakes ephemeral protocol jargon (JWT, bcrypt, SQL) into living assets; 'Restated Truth' creates duplicate schemas that drift and cause silent LLM reconciliation bugs; 'The Open Flow' omits explicit terminations, causing AI to hallucinate arbitrary navigation jumps; 'The Blank Check' uses lazy vocabulary ('handles appropriately', 'etc.') delegating core business decisions; 'Backfilled Specs' updates documentation retroactively, destroying spec-first authority.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: 10 Yaygın Hatanın Önlenmesi (Açık Çek ve Açık Uçlu Akış Tedavisi)
package com.maang.aiup.mistakes;

public class CommonMistakesRemedyDemo {
    // ❌ HATA 8: Açık Uçlu Akış (Ne olacağı belirsiz):
    // if (!valid) return; // Ne döndü? Log mu attı? Ekrana ne dedi?

    // ✅ TEDAVİ: Kesin Bitiş ve Minimum Garanti (Failure Postcondition)
    public sealed interface FlowResult permits FlowResult.Success, FlowResult.Halted {
        record Success(String orderId) implements FlowResult {}
        record Halted(String errorCode, String userMessage) implements FlowResult {}
    }

    public static class SafeOrderHandler {
        public static FlowResult execute(int stock) {
            if (stock <= 0) {
                // Alternatif Akış A1: Açık uçlu değil! Kesin bitiş ve mesaj var:
                return new FlowResult.Halted("ERR-01", "A1: Stok tükenmiştir. Use case sonlanır.");
            }
            return new FlowResult.Success("ORD-999");
        }
    }

    void main() {
        var result = SafeOrderHandler.execute(0);
        switch (result) {
            case FlowResult.Success s -> System.out.println("Başarı: " + s.orderId());
            case FlowResult.Halted h -> System.out.println("✅ Hata 8 ve 9 Önlendi: " +
h.userMessage());
        }
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnamenizde 'vb.', 've benzeri', 'uygun şekilde işler' kelimelerini gördüğünüz anda silin! Her biri AI'a kafasına göre uydurma yetkisi veren birer açık çektir.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Search your specification for 'etc.', 'and so on', or 'appropriately'. Erase them immediately. Every vague qualifier delegates architectural control to LLM randomness.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- 5 klasik hata (UI walkthrough, su altı hedefi) 20 yıldır şartnamelerin en büyük derdidir.
- 5 AI çağı hatası (mekanizma sızıntısı, açık uçlu akış, açık çek) üretimi doğrudan zehirler.
- Sonradan şartname yazmak (backfilling) şartname tabanlı kalkınmanın ölüm fermanıdır.

🎯 Key Architectural Takeaways

- 5 classical blunders (UI walkthrough, underwater goals) remain persistent pitfalls.
- 5 AI-era disasters (mechanism leaks, open flows, blank checks) directly corrupt code generation.
- Backfilling specifications after coding completely destroys spec-driven development.

57

BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

Masaüstü Adım Yazma Kartı (Step-Writing Desk Card): Bölünmüş Ekranda Hızlı Rehber

The Step-Writing Desk Card: The Split-Pane Reference for Engineers

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────┐
│              MASAÜSTÜ ADIM YAZMA KARTI (STEP-WRITING CARD)              │
└──────────────────────────────────────────────────────────┘

 Mühendisler İçin Monitörün Sağında Açık Tutulacak Özet Kart:

┌──────────────────────────────────────────────────────────┐
│ 📝 BİR ADIM YAZARKEN (FOR MAIN SCENARIO STEPS):          │
│ 1. Tek bir cümle. Geniş / Şimdiki zaman. Etken çatı (Active Voice).  │
│ 2. İLK KELİME = Top kimdeyse o aktörün adı!              │
│ 3. Formül: Özne → Yüklem → Nesne → Edat Öbeği.           │
│ 4. Hareketleri değil niyeti yaz; tek yöndeki verileri tek adımda topla! │
│ 5. ASLA "checks whether" deme; "validates that..." de!   │
│ 6. Gözlemlenebilir alanda SOMUT ol (Kolon adları, net sayılar, mesaj). │
│ 7. Mekanizmada TAMAMEN SESSİZ kal (Protokol, şema, DB, kod adı YOK!). │
│ 8. 3 ila 9 adım arası yaz. Son adım = Hedefin teslim edildiği andır! │
├──────────────────────────────────────────────────────────┤
│ 🔀 ALTERNATİF AKIŞ YAZARKEN (FOR ALTERNATIVE FLOWS):     │
│ 1. Başlık: ### A#: <Durumun Adı>                         │
│ 2. Tetikleyici: **Trigger:** <Sistemin yakaladığı durum> (step N) │
│ 3. 1..N Adım: Yukarıdaki adım kurallarına harfiyen uyan adımlar. │
│ 4. Kesin Bitiş: "Use case continues at step N." VEYA "Use case ends." │
│ 5. "ends" ile biten her akış için Failure Postcondition garantisi şart! │
└──────────────────────────────────────────────────────────┘
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Uçak pilotlarının kokpitindeki küçük acil durum kartlarını düşünün: Uçakta motor arızası çıktığında pilot 500 sayfalık kalın kitabı açıp okumaz; hemen direksiyonun yanındaki 8 maddelik küçük karta bakar! Adım yazma kartı, şartname yazarken monitörünüzün sağ köşesine yapıştıracağınız o hayat kurtaran kokpit kartıdır!

🇬🇧 ENGLISH (ELI5)

Think of the quick-reference checklist clipped directly to an airline pilot's yoke: In severe turbulence, pilots don't flip through a 600-page manual; they scan the laminated 8-point card right in front of them! The Step-Writing Desk Card is that vital cockpit card, pinned to your split-pane IDE window while drafting specs!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Masaüstü Adım Yazma Kartının Pratik Değeri: Simon Martinelli kitabın 10.3 bölümünde bu kartı özel olarak tek sayfalık bir referans olarak sunar. Çünkü kimse şartname yazarken 60 sayfalık metodoloji kitabını baştan sona okumaz; mühendisler ikiye bölünmüş monitörde (split-pane) bir tarafta use case yazarken diğer tarafta hızlı kurallara bakmak ister. Bu kart iki kompakt bloktan oluşur: 1) Senaryo Adımları: 8 altın kural. Özne-yüklem uyumu, topun sahibinin ilk kelime olması, 'validates that' kalıbı, Zone 1 somutluğu ve Zone 3 sessizliği. 2) Alternatif Akışlar: 5 altın kural. Adıma çıpalama (`step N`), 1..N adımlı işleme ve mutlaka iki yasal bitişten birisi (`continues at step N` veya `ends`). Bu kartı takip eden bir mühendisin şartnamesi `/spec-review` linter'ından ilk seferde %100 geçer not alır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Practical Utility of the Step-Writing Desk Card: In Section 10.3, Martinelli crystallizes the entire writing discipline into a desk card designed specifically for split-pane engineering environments. Software engineers and business analysts drafting specifications in an IDE do not consult a full-length book during authoring; they need an ambient, unencumbered reference visible on their secondary monitor or split editor. The card establishes two airtight syntactic blocks: 1) Scenario Step Rules: 8 constraints enforcing active voice, ball-holder primacy, intent encapsulation, strict 'validates that' grammar, Zone 1 observability, Zone 3 silence, and 3-9 step bounds. 2) Alternative Flow Rules: 5 constraints governing condition titling, anchored triggers (`step N`), and explicit binary exits. Adhering to the desk card guarantees zero lint violations during `/spec-review` passes.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Masaüstü Kartı Kurallarına %100 Uyumlu Temiz Use Case Modeli
package com.maang.aiup.deskcard;

public class DeskCardComplianceDemo {
    // Kural 1-3: Aktör net (Üye, Sistem). Kural 5: Validates kuralı.
    public record BorrowContext(String memberId, String bookId, boolean isAvailable) {}

    public static class StepExecutionEngine {
        // Adım 4: Sistem kopyanın mevcut olduğunu doğrular (BR-004)
        public static String step4ValidateAvailability(BorrowContext ctx) {
            if (!ctx.isAvailable()) {
                // A1 Akışı: Tetikleyici step 4'e bağlı. Bitiş: Use case ends.
                return "A1: Kopya mevcut değil. En erken iade tarihi gösterildi. Use case ends.";
            }
            // Başarı yolu:
            return "Adım 4 Başarılı: Kopya mevcut. Use case continues at step 5.";
        }
    }

    void main() {
        var availableCtx = new BorrowContext("M1", "BK-9", true);
        var unavailableCtx = new BorrowContext("M1", "BK-9", false);

        System.out.println(StepExecutionEngine.step4ValidateAvailability(availableCtx));
        System.out.println(StepExecutionEngine.step4ValidateAvailability(unavailableCtx));
        System.out.println("✅ Adım Yazma Kartı kuralları koda kusursuz yansıdı.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Desk kartındaki formülü bozup 'Kitap mevcut ise sisteme kaydet' şeklinde şartlı (if-then) cümleler kurmayın! Şartları daima alternatif akışlara taşıyın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never violate the desk card syntax by embedding conditional 'if-then' clauses within a step. Main steps assert success; deviations belong strictly in alternative flows.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Masaüstü kartı, çift ekranlı çalışırken split-pane'de açık tutulacak en pratik kaynaktır.
- Adım kuralları (özne başta, validates, 3-9 adım) hatasız bir ana akış kurar.
- Akış kuralları (adıma çıpalama, net bitiş) açık uçlu hataları tamamen engeller.

🎯 Key Architectural Takeaways

- The desk card is the definitive split-pane reference for daily specification authoring.
- Step rules (ball-holder first, validates, 3-9 steps) construct an unbranching happy path.
- Flow rules (anchored triggers, explicit exits) eliminate dangling failure paths entirely.

58

BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

MAANG Sistem Tasarımı Mülakatı: Yüksek Seviye Mimariden AIUP Şartnamesine

MAANG System Design Interview: Translating High-Level Architecture into AIUP Specs

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                MAANG SYSTEM DESIGN MÜLAKATINDA AIUP DÖNÜŞÜMÜ                │
└──────────────────────────────────────────────────────────────┘

 MÜLAKAT SORUSU (Meta / Google / Amazon):
 "Bize ölçeklenebilir bir Twitter / X veya Uber rezervasyon sistemi tasarlayın."

 GELENEKSEL ADAY (Bocalayan):
 - Hemen beyaz tahtaya 'Redis', 'Kafka', 'Cassandra' kutuları çizer.
 - Hangi aktörlerin ne yaptığını, hangi iş kurallarının olduğunu unutur.
 - Mülakatçı sorar: "Peki bakiye yetersizse ne olacak?" Aday tıkanır!

 🌟 MAANG STAFF+ ADAY (AIUP ile Kazanan):
 1. ADIM: Aktörleri ve Deniz Seviyesi Hedefleri Belirler (UC-001, UC-002).
 2. ADIM: Kapsamı In/Out Tablosu ile Netleştirir (Scope Boundary).
 3. ADIM: 3-9 Adımlı Ana Senaryoyu ve Alternatif Akışları (A1, A2) Yazar.
 4. ADIM: Mekanizmayı (Kafka, Redis) Zone 3 olarak şartnameden soyutlar;
          mimaride bu bileşenleri iş kurallarının (BR) altına birer Adapter olarak bağlar!

 🎯 MÜLAKATÇININ YORUMU:
 "Bu aday sadece kutu çizmeyi değil; bir sistemi yaşayan, test edilebilir ve
  otonom ekiplerce üretilebilir bir şartname zincirine oturtmayı biliyor!"
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir inşaat mühendisliği sınavını düşünün: Acemi aday hemen 'Ben 50 tonluk çimento mikseri getireceğim' diye aletleri anlatmaya başlar. Usta mimar ise önce binada kimlerin yaşayacağını, kapıların nereye açılacağını, deprem anında nereye kaçılacağını (Use Case) çizer! Mikser zaten çağrılır. MAANG mülakatında önce hayatı ve kuralları çizmek sizi birinci yapar!

🇬🇧 ENGLISH (ELI5)

Think of an architectural board exam: An amateur candidate immediately brags: 'I will order 50 tons of hydraulic concrete mixers!' The master architect first presents the residential floor plan, primary fire escape egress routes, and earthquake load specs (Use Cases). The concrete mixer is just plumbing. In a MAANG interview, framing user-goal invariants first earns the Staff+ offer!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

MAANG Sistem Tasarımı Mülakatlarında AIUP Yaklaşımı: Meta, Amazon, Apple, Netflix ve Google (MAANG) mülakatlarında Staff ve Principal seviyesindeki adaylardan sadece dağınık kutular ve 'Redis cache koyalım' demeleri beklenmez. Mülakatçıların aradığı en kritik yetkinlik 'Sistemik Netlik' (Systemic Clarity) ve 'Sınırları Doğru Çizebilme' (Bounded Contexts) kabiliyetidir. AIUP disiplini adaya şu muazzam avantajları sağlar: 1) Gereksinim Netleştirme: Mülakatın ilk 5 dakikasında aktörleri ve deniz seviyesi hedefleri belirleyip bir In/Out tablosu çizen aday, kontrolü eline alır. 2) Senaryo ve Hata Hakimiyeti: Ana akışı 'validates' kalıbıyla kurup alternatif akışları (stok yetersiz, ödeme zaman aşımı) numaralı adımlara bağlayan aday, mülakatçının en sevdiği 'Peki ya sistem çökerse?' sorularının tamamını önceden yanıtlamış olur. 3) Temiz Mimari Ayrımı: Mekanizmayı (Zone 3: Kafka, DynamoDB) iş kurallarından (Zone 1) ayrı bir katmanda sunarak, sistemin teknoloji bağımsızlığını ve test edilebilirliğini kanıtlar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

AIUP Strategy in MAANG System Design Interviews: In Staff- and Principal-level design loops at Google, Meta, or Amazon, whiteboarding arbitrary distributed components (Kafka, Cassandra, Redis) without behavioral boundaries signals architectural immaturity. Interviewers assess an architect's ability to tame ambiguity, define bounded contexts, and enforce operational resilience. Applying the AI Unified Process establishes immediate command of the interview: 1) Scoping with In/Out Lists: The candidate partitions the problem space within 5 minutes, eliminating scope creep. 2) Explicit Behavioral Workflows: Presenting sea-level use cases where each validation is paired with an alternative failure flow demonstrates failure-domain mastery. 3) Strict Separation of Zones: By cleanly isolating business rules (BR-XXX) and observable outcomes (Zone 1) from underlying infrastructure adapters (Zone 3: distributed cache, write-ahead log), the candidate demonstrates clean architectural rigor.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: MAANG Seviyesi Ölçeklenebilir Sistem Tasarımı Use Case Mimarisi
package com.maang.aiup.maangdesign;

import java.util.concurrent.CompletableFuture;

public class MaangSystemDesignDemo {
    // 1. USE CASE MODELİ (Teknoloji bağımsız iş mantığı)
    public record RideRequest(String riderId, double pickupLat, double pickupLon) {}
    public record RideMatched(String rideId, String driverId, double estimatedFare) {}

    public interface DriverMatchingPort {
        CompletableFuture<String> matchNearestDriver(double lat, double lon);
    }

    public static class RequestRideUseCase {
        private final DriverMatchingPort matchingPort;

        public RequestRideUseCase(DriverMatchingPort matchingPort) {
            this.matchingPort = matchingPort;
        }

        public CompletableFuture<RideMatched> requestRide(RideRequest request) {
            // Şartname Kuralı: En yakın sürücüyü bul, eşleşirse başarı döndür
            return matchingPort.matchNearestDriver(request.pickupLat(), request.pickupLon())
                .thenApply(driverId -> new RideMatched("RIDE-101", driverId, 45.50));
        }
    }

    void main() {
        DriverMatchingPort mockKafkaPort = (lat, lon) -> CompletableFuture.completedFuture("DRIVER-
ALICE");
        var useCase = new RequestRideUseCase(mockKafkaPort);

        useCase.requestRide(new RideRequest("RIDER-1", 41.0, 29.0))
            .thenAccept(res -> System.out.println("✅ MAANG Mülakat Mimarisi Çalıştı: " + res));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Sistem tasarımı mülakatında hemen veritabanı şeması veya Kafka topic'i çizmeye başlamayın; önce aktörleri, hedefleri ve In/Out kapsamını netleştirin.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not dive straight into Kafka partitions or database schemas during a system design interview. Pin down actors, user goals, and the In/Out boundary first.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• MAANG mülakatlarında kutu çizmeden önce aktörler, hedefler ve In/Out listesi sunulur.

• Ana akış ve alternatif akış disiplini, mülakatçının 'Peki ya hata olursa?' sorularını peşinen çözer.

• Zone 1 iş mantığını Zone 3 altyapısından ayırmak adayı Staff+ mimar seviyesine taşır.

🎯 Key Architectural Takeaways

• Present actors, user goals, and an In/Out boundary before drawing architecture boxes.

• Scenario and alternative flow discipline preemptively resolves failure-mode questions.

• Decoupling Zone 1 business invariants from Zone 3 plumbing signals Staff+ design maturity.

59

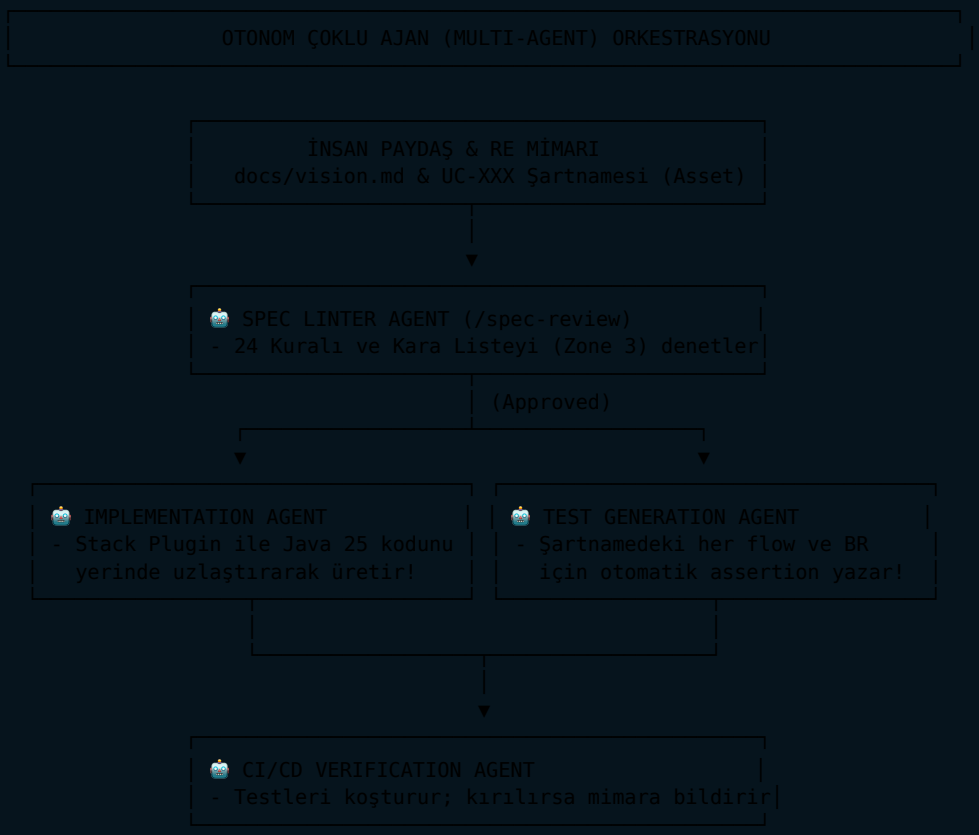
BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

Otonom Çoklu Ajan (Multi-Agent) İş Akışları ve Şartname Tabanlı Kod Üretimi

Autonomous Multi-Agent AI Workflows: Orchestrating Spec-Driven Code Generation

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir inşaat şantiyesindeki robot işçileri düşünün: Mimar (İnsan) elindeki kesin mimari planı (Use Case Şartnamesi) robot şefe verir. Robot şef planı kontrol eder. Sonra bir robot duvarları örerken (Kod Ajanı), diğer robot betonun sağlamlığını lazerle ölçer (Test Ajanı). Kimse kafasına göre bina yapmaz; herkes mimarın tek bir çizimine bakar!

🇬🇧 ENGLISH (ELI5)

Imagine a high-tech robotic construction site: The human architect provides the structural CAD blueprint (The Use Case Spec). A supervisor robot audits the blueprint. Then, while one robotic arm lays the precision steel beams (Coding Agent), a companion drone fires lasers to verify structural integrity (Testing Agent). All autonomous agents align around a single blueprint!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Otonom Çoklu Ajan Orkestrasyonu (Multi-Agent Workflows): Yapay zekâ yazılım mühendisliği artık tek bir 'Chat GPT' penceresine prompt yazmaktan çıkmış, uzmanlaşmış çoklu ajanların (Multi-Agent Systems) işbirliğine dönüşmüştür. Ancak kontrolsüz ajanlar birbirlerinin kodunu bozabilir ve halüsinasyon sarmalına girebilir. AI Unified Process bu sorunu şartnameyi 'ortak koordinasyon düzlemi' (shared coordination substrate) yaparak çözer: 1) Spec Linter Ajanı: Şartnamenin sözdizimini, 24 kuralını ve Zone 3 temizliğini denetler. 2) Implementasyon Ajanı: Şartnameden sadece onaylanmış adımları okuyarak Java kodunu üretir. 3) Test Üretim Ajanı: Doğrulama adımlarını ve BR kurallarını bağımsız olarak test koduna dönüştürür. Kod üreten ajan ile test üreten ajanın birbirlerinden habersiz olarak sadece ortak şartnameyi okuması, 'kendi yazdığı kodu test etme' önyargısını tamamen ortadan kaldırır. 4) Doğrulama Ajanı: Kod ve testleri derleyip CI/CD ortamında koşturur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Orchestrating Spec-Driven Multi-Agent Workflows: Autonomous AI software development has outgrown monolithic chat interfaces in favor of specialized multi-agent architectures. Without rigorous guardrails, multi-agent pipelines succumb to hallucination feedback loops and code drift. The AI Unified Process leverages the specification artifact chain as the shared immutable state substrate across agent topologies: 1) Spec Review Agent: Executes deterministic static analysis and advisory reviews on `UC-XXX.md` artifacts. 2) Code Implementation Agent: Ingests approved specs and stack plugins to synthesize domain services and presentation controllers. 3) Test Generation Agent: Reads failure postconditions and business rules to synthesize orthogonal test suites. Because the test agent works independently from the specification rather than inspecting generated production code, it prevents the classic bias of AI tests confirming AI mistakes. 4) Verification Agent: Runs test suites within sandboxed CI environments, providing deterministic feedback loops.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Çoklu Ajan Koordinasyon Arayüzü (Agent Orchestration Substrate)
package com.maang.aiup.multiagent;

public class MultiAgentOrchestratorDemo {
    public record SpecArtifact(String id, String content, boolean isApproved) {}
    public record GeneratedCode(String sourceFile, String testFile) {}

    // Ajan 1: Şartname Denetleyicisi
    public static class SpecLinterAgent {
        public boolean verify(SpecArtifact spec) {
            return spec.isApproved() && !spec.content().contains("SQL");
        }
    }

    // Ajan 2: Kod Üreticisi
    public static class CodeGeneratorAgent {
        public GeneratedCode generate(SpecArtifact spec) {
            return new GeneratedCode("LoanService.java", "LoanServiceTest.java");
        }
    }

    void main() {
        var spec = new SpecArtifact("UC-002", "System validates that copy is available (BR-004).",
true);
        var linter = new SpecLinterAgent();
        var generator = new CodeGeneratorAgent();

        if (linter.verify(spec)) {
            var code = generator.generate(spec);
            System.out.println("🤖 Çoklu Ajan İş Akışı Başarılı!");
            System.out.println("   -> Üretilen Kod: " + code.sourceFile());
            System.out.println("   -> Üretilen Bağımsız Test: " + code.testFile());
        }
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Kod üreten ajan ile test üreten ajanı aynı prompt zincirinde çalıştırmayın! Test ajanı koda bakarak değil, sadece şartnameye bakarak test yazmalıdır.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never permit the test-generation agent to inspect generated production source code. Tests must be synthesized strictly from the specification to prevent self-affirming hallucinations.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Şartname, çoklu yapay zekâ ajanlarının etrafında toplandığı tek ortak gerçektir.

• Kod üreten ajan ile test üreten ajan birbirinden bağımsız çalışarak tarafsızlığı korur.

• Ajanlar arası iletişim serbest sohbetle değil, yapısal şartname zinciriyle yönetilir.

🎯 Key Architectural Takeaways

• The specification serves as the shared coordination substrate across autonomous AI agents.

• Code and test generation agents operate orthogonally to eliminate confirmation bias.

• Inter-agent workflows are governed by the structured artifact chain, not conversational chat.

60

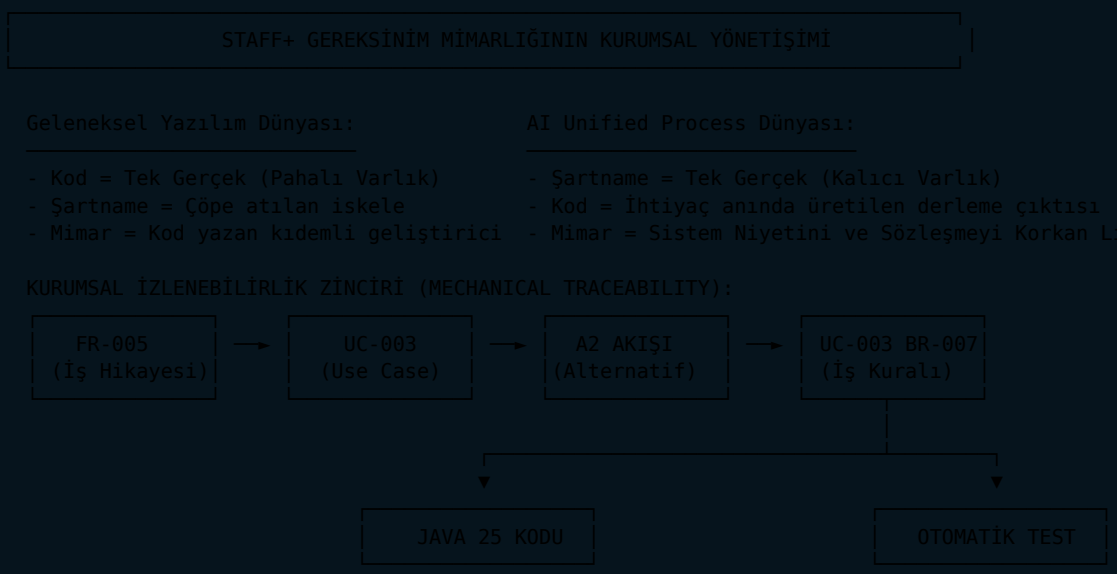
BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

Staff+ Seviye Gereksinim Mimarlığı: Yönetişim, İzlenebilirlik ve AIUP Çağı

The Staff+ Requirements Architect: Governance, Traceability & The AIUP Future

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Dünyanın en büyük orkestrasını düşünün: 100 tane kemancı, 50 tane trompetçi vardır. Trompetçi değişebilir, kemancı emekli olabilir, hatta enstrümanlar ahşaptan elektroniğe geçebilir. Ama Mozart'ın yazdığı nota kâğıdı (Partisyon / Şartname) 300 yıldır aynı ihtişamla çalınır! Staff+ Mimar, o ölümsüz partisyonu yazan ve tüm orkestrayı yöneten baş şeftir!

🇬🇧 ENGLISH (ELI5)

Think of the world's greatest philharmonic orchestra: 100 violinists, 50 brass players. Musicians retire, violinists are replaced, instruments may transition from acoustic to carbon-fiber. But Mozart's symphonic score (The Specification) endures across centuries with flawless fidelity! The Staff+ Architect is the master conductor who preserves the eternal score.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Staff+ Seviye Gereksinim Mimarlığı ve Kurumsal Yönetişim: Yazılım dünyasında 40 yıl boyunca 'Kod tek gerçektir' yalanıyla yaşadık. Bu yaklaşım, sistemlerin 5 yılda bir teknik borç bataklığına saplanmasına ve milyonlarca dolarlık 'legacy rewrite' facialarına yol açtı. AI Unified Process ve Simon Martinelli'nin metodolojisi yazılım mühendisliğini gerçek bir mühendislik disiplinine kavuşturur: Kod artık derlenip atılabilen geçici bir ikincil çıktıdır; asıl şirket varlığı (asset) ise gereksinimlerden use case'lere, oradan entity model ve testlere uzanan 'Şartname Zinciri'dir (Artifact Chain). Staff+ Requirements Architect; bu zincirin mekanik izlenebilirliğini (Traceability: `FR -> UC -> Flow -> BR -> Code -> Test`), kara liste kontrollerini ve alan modellerini yöneten kurumsal stratejisttir. Bir regülatör veya müşteri 'Neden bu işlem reddedildi?' diye sorduğunda, kod aramak yerine saniyeler içinde kural kimliğini gösterir. Şirketin teknoloji yığını yarın Spring Boot'tan başka bir platforma geçse dahi, şartname zinciri sayesinde tek bir iş kaybı olmadan tüm sistem saatler içinde yeniden üretilebilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Staff+ Requirements Architecture and Enterprise Governance: For four decades, the software industry operated under the economic premise that 'the code is the truth'—an assumption that produced unmaintainable monoliths, trillion-dollar technical debt, and recurrent failed re-writes. The AI Unified Process permanently inverts this paradigm. Code is demoted to an ephemeral, regenerable compilation artifact. The durable enterprise asset is the Specification Artifact Chain: Vision -> Requirements Catalog -> Entity Model -> Use Case Diagram -> Use Case Specifications -> Test Journeys. The Staff+ Requirements Architect is the strategic custodian of this ecosystem, enforcing mechanical traceability (`FR-XXX -> UC-XXX -> Flow -> BR-XXX -> Code -> Test`), lexical hygiene, and architectural boundaries. When auditors or clients query why a transaction was declined, the answer is a deterministic chain of identifiers rather than code archaeology. When infrastructure stacks evolve, the enterprise regenerates its entire portfolio with zero business friction.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Staff+ Seviye Mekanik İzlenebilirlik Zinciri (Traceability Metadata)
package com.maang.aiup.staffarchitect;

import java.lang.annotation.*;

public class EnterpriseTraceabilityDemo {
    // Kurumsal İzlenebilirlik Notasyonu
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.METHOD)
    public @interface Traceability {
        String requirementId();
        String useCaseId();
        String flowId();
        String businessRule();
    }

    public static class EnterpriseLoanEngine {
        @Traceability(
            requirementId = "FR-012",
            useCaseId = "UC-002",
            flowId = "A2",
            businessRule = "BR-005: Loan Limit"
        )
        public boolean evaluateLoanQuota(int currentLoans) {
            // Mekanik zincir: Koddaki her satır doğrudan iş kuralına bağlanır!
            return currentLoans < 5;
        }
    }

    void main() throws NoSuchMethodException {
        var method = EnterpriseLoanEngine.class.getMethod("evaluateLoanQuota", int.class);
        var trace = method.getAnnotation(Traceability.class);

        System.out.println("🏛️ Staff+ Kurumsal İzlenebilirlik Raporu:");
        System.out.println("   -> İlgili Gereksinim: " + trace.requirementId());
        System.out.println("   -> İlgili Use Case  : " + trace.useCaseId());
        System.out.println("   -> İlgili Akış      : " + trace.flowId());
        System.out.println("   -> Uygulanan Kural  : " + trace.businessRule());
        System.out.println("✅ Kod doğrudan iş şartnamesine bağlandı. AIUP Tamamlandı!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartname zincirini sadece dokümantasyon olarak görmeyin; o zincir şirketin yazılımsal DNA'sıdır. Kod ölür, şartname yaşar!

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never treat the artifact chain as mere bureaucratic documentation. The specification is the living genetic code of the enterprise. Code is ephemeral; specifications endure.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• AI Unified Process ile yazılım mühendisliği gerçek bir mühendislik disiplinine kavuşur.

• Mekanik izlenebilirlik (FR -> UC -> Flow -> BR -> Code -> Test) kurumsal şeffaflığı sağlar.

• Şartname tek kalıcı varlıktır; kod yarın başka bir dile veya stack'e saniyeler içinde taşınabilir.

🎯 Key Architectural Takeaways

• The AI Unified Process elevates software development into a true engineering discipline.

• Mechanical traceability connects every line of code to a specific business requirement.

• Specifications are the sole durable asset; code can be regenerated onto new stacks effortlessly.

🎓 TEBRİKLER! / CONGRATULATIONS!

60 Derslik AIUP & Use Case Mimarlık Yolculuğunu Başarıyla Tamamladınız!

You have successfully mastered all 60 lessons of the Writing Use Cases for AI Curriculum!

Spec-Driven Development • 3 Readers • 3 Zones • Mechanical Traceability • Java 25 LTS